CAS PCBA 8: Adaptive Cognitive Circuit Engineering with Typed, Fail-Closed Design Rules

Little Henderson

Research monograph v0.3, 2026-08. Artifact status: DRAFT / OPEN / RING 4 / NOT LICENSED / HELD. This monograph expands the corresponding v0.2 research paper over the same Project Coherence Gate 1 / Gate 4 source artifacts; every status label, proof status, hash identity, and open obligation of those sources governs this text. Drafting this monograph promotes no claim; independent review remains open.

Contents

Chapter 1. Introduction 1
1. Motivation and setting 1
2. The name of the discipline 2
3. The research question 2
4. What "circuit" means here — and what it does not 3
5. Three load-bearing separations 4
6. The six contributions 8
7. The thesis: a qualified affirmative at the software-contract level 9
8. What is gained over informal practice 10
9. Claim discipline and artifact status 11
10. Reading guide 12

Chapter 2. Background: Hardware Description, State Formalisms, and Verification 13
1. Purpose and reading discipline of this survey 13
2. Hardware description languages 13
3. Statecharts and explicit state semantics 15
4. Electronic design automation practice: netlists and rule checking 17
5. Graph algorithms for circuit analysis 19
6. Simulation verification versus operational validation 20
7. Property-based testing 21
8. Provenance 22
9. Arithmetic control: floating point and decimal contexts 23
10. Information geometry on probability simplices: a primer 23
11. A note on causal claims 25
12. What was deliberately not borrowed 26
13. Summary: the borrowings and their limits 26

Chapter 3. The Versioned Source Basis 29
1. Why a source chapter exists 29
2. The engineering baseline 29
3. Why not a semantic version 30
4. Source-identity methodology: content addressing for large design documents 31
5. The baseline's own disclaimer and the Phase-0 authority matrix 32
6. The controlling repository artifacts 33
7. The publication lineage of this monograph 35
8. The audit's method, disclosed at the source level 36
9. Downstream read-only boundaries 36
10. The Gate-5 preregistration kernel as read-only context 37
11. What the source basis excludes 38
12. Summary 38

Chapter 4. The Type System 39
1. Design stance 39
2. The four semantic factors 39
3. State literals 40
4. ND-cells and the forgetful projection 41
5. Simplex addresses 43
6. Ports: direction, signal kind, unit 45
7. Component kinds and typed parameter contracts 46
8. Nets, endpoints, and driver/consumer discipline 46
9. Test points and calibration references 47
10. Deterministic trace and measurement identity 47
11. Research boundary and source provenance types 48
12. Tagged exact values 49
13. The single immutable decimal context 50
14. Canonical identity is injective 51
15. Measurement kinds as a claim taxonomy 51
16. Worked type-level fragments 52
17. Invariant summary 53
18. Claim boundary 53

Chapter 5. The Board Model 54
1. The board as an evidence object 54
2. The immutability discipline 54
3. Components 55
4. Typed ports 57
5. Parameters and finite-state annotations 58
6. Nets and endpoints 59
7. Test points and calibration references 59
8. Declared feedback loops 60
9. Source and research-boundary metadata 60
10. Deterministic identity material 61
11. Boards as values 63
12. Worked example fragments 64
13. Invariants and boundary 66

Chapter 6. Geometry Profiles 68
1. Why geometry, and under what discipline 68
2. Primer: information geometry on the open simplex 68
3. The implemented baseline: a disclosed numerical pseudometric 71
4. Projector geometry: a separate optional research profile 73
5. The Gram-kernel overlap result and its limits 74
6. The capacity-transfer proof gap 75
7. Two arithmetic regimes, and why they differ 76
8. Geometry values in the wiring and the trace 77
9. Separations between the two profiles 77
10. Claim boundary 78

Chapter 7. The Design-Rule Checker 79
1. Architecture: one deterministic report 79

2. Model integrity: the MDL family 82
3. Semantic resolution: the SEM family 83
4. Typed parameters: the PAR family 84
5. Typed state bindings: the STATE family 86
6. Bias coherence: PCBA8-BIAS-001 87
7. Typed connectivity: the NET family 88
8. Probe integrity: the TP family 92
9. Calibration integrity: the CAL family 93
10. Feedback analysis: the FBK family 95
11. Safety analysis: the SAF family 98
12. The adapter boundary: PCBA8-TKS-001 100
13. The five check classes 101
14. Claim boundary 102

Chapter 8. Deterministic Reference Simulation 103
1. Admission rules 103
2. Execution order and feedback breaking 105
3. A worked illustrative run 107
4. The canonical trace schema 108
5. Measurement and result validation 111
6. Reading the artifact: three audit walkthroughs 112
7. Determinism and replayability 113
8. Locally asserted calibration is structural 114
9. Unsupported behavior fails closed 114
10. Relation to PROV, without a conformance claim 115
11. The green-fixture interpretation rule 116
12. Simulation is not validation 116

Chapter 9. The TKS v8 Adapter and the Noetic Topos Boundary 118
1. Certification conditions for adaptation 118
2. Chains and powers: paths and endomaps 119
3. The adapter certificate 120
4. A worked illustrative adaptation 122
5. Ambient module substitution is rejected 123
6. Certificate lifecycle and staleness 123
7. PCBA8-TKS-001: the DRC face of the boundary 124
8. No automatic semantic theorem 125
9. What stays outside the finite arrow 126
10. The Noetic Topos evidence boundary 127

Chapter 10. The CCDL Grammar Audit 130
1. Why a grammar audit, and why here 130
2. The audit method 131
3. A formal statement of the coverage question 133
4. The formal-language evidence 134
5. Board CCB-1-CSB: the Cognitive Safety Board listing 136
6. Board CCB-2-COB: the bounded non-counterexample 137
7. Board CCB-3-CDAB: the mirror declaration 138
8. Board CCB-4-CMB: the memory-discipline statements 139

9. Board CCB-5-CPMB: the power-management vocabulary 140
10. A taxonomy of the counterexamples 140
11. The corpus conclusion 141
12. The candidate grammar rule 142
13. The coverage record as a machine-readable artifact 143
14. Claim boundary of the record 144
15. Lessons for language-bearing engineering documents 145

Chapter 11. The Cognitive Safety Board: A Known-Fault Analysis 146
1. Introduction: a safety exemplar that cannot perform its function 146
2. The published values 147
3. Unreachability under the declared semantics 147
4. Plateau, not latch: the missing fail-closed state machine 150
5. Disposition: a known-fault fixture, permanently flagged 153
6. The negative-evidence philosophy 154
7. Requirements on any repaired design 156
8. Lessons for safety-mechanism specification in cognitive-system engineering 157

Chapter 12. Evidence, Threats to Validity, and the Research Program 160
1. The role of this chapter 160
2. An evidence type system 160
3. The Gate-4 receipt 161
4. Interpreting the receipt 163
5. Threats to validity 164
6. The dependency-ordered research program 167
7. The claim quarantine 170
8. Related methodological work 171
9. Conclusion 172

Appendix A. Catalogs 174
1. The diagnostic-code catalog 174
2. Check class to code mapping 175
3. Canonical trace schema 175
4. Adapter certificate fields 175

Appendix B. A Worked Illustrative Board 177
1. A clean pass 177
2. Representative failures 177
3. A simulation narrative 178

Appendix C. Glossary 179

Bibliography 181

CHAPTER 1

Introduction

1. Motivation and setting

Every research community that models complex adaptive behavior eventually confronts the same methodological hazard: the informal diagram. A block sketch of a hypothesized cognitive pathway — boxes for stages, arrows for influence, annotations for thresholds and gains — is cheap to draw, easy to circulate, and almost impossible to audit. Two readers of the same sketch can disagree about whether an arrow denotes causation, correlation, data flow, or mere suggestion; whether a threshold is a design intention or a measured property; whether a feedback loop is stable, unstable, or simply unanalyzed; and whether the sketch as a whole is a claim about the world or a claim about a model of the world. The sketch has no type system, no versioning, no diagnostics, no replayable execution semantics, and no evidence discipline. When such sketches accumulate into a large body of design work — in the present case, a 743-page engineering baseline — the hazard compounds: assertions of very different epistemic standing sit side by side on the page, typeset with equal confidence, and nothing in the medium itself distinguishes a tested software behavior from a hypothesis, an idealization, or an error.

Hardware engineering solved a structurally similar problem decades ago, not by proving anything about physics that physicists did not already know, but by changing the medium of design description. Machine-readable hardware description languages, explicit state formalisms, netlist-centered design databases, design-rule checkers that fail closed, deterministic simulators with archived traces, and sign-off disciplines that separate verification from validation together converted circuit design from a craft of annotated drawings into an auditable engineering practice. Chapter 2 surveys that heritage in detail. The essential observation for this monograph is that the value of those tools was largely *methodological*: they made design intent explicit, mechanically checkable, and reproducible, and they made whole categories of error observable before fabrication.

CAS PCBA 8 — the subject of this monograph — asks whether the same methodological move is available to cognitive-system modeling. It proposes a typed, provenance-bearing research architecture in which a candidate cognitive-system hypothesis is written down as a *circuit-shaped model*: a directed, typed network of components with declared ports, signal kinds, units, parameters, state annotations, calibration references, feedback declarations, and evidence limits, subject to a deterministic design-rule checker and, where supported, a narrow deterministic simulator that emits canonical traces. The proposal is emphatically *not* that cognition is electronics. The proposal is that the engineering *organization* of electronics — its description discipline, its checking discipline, and its evidence discipline — can be borrowed as a modeling grammar while every physical, causal, safety, and product claim is explicitly withheld.

This introductory chapter states the research question precisely, explains what "circuit" does and does not mean in this work, develops the three load-bearing separations that keep

the analogy from decaying into an identity claim, enumerates the six contributions, states the qualified-affirmative thesis defended in the remainder of the monograph, fixes the claim discipline under which every subsequent chapter operates, and closes with a reading guide.

2. The name of the discipline

The monograph's subtitle names its subject *adaptive cognitive circuit engineering*, and each word of the name is a scoped commitment worth fixing before the technical development begins.

Engineering commits the work to the practices of that word: declared interfaces, versioned artifacts, deterministic tools, catalogs of checks, archived evidence, and an explicit boundary between what has been tested and what has been asserted. It deliberately does not commit the work to science: no chapter of this monograph reports an experiment, and the word "engineering" is chosen partly to keep that fact in the reader's view. What is engineered here is a modeling and checking apparatus — a language, a checker, a simulator, adapters, and an evidence discipline — and the apparatus is the deliverable.

Circuit commits the work to the specific engineering organization of Definition 1.1 and Boundary 1.2: typed components, ports, nets, state annotations, calibration references, deterministic traces. The commitment is organizational, never physical.

Cognitive commits the work to a subject matter — hypotheses about cognitive systems, drawn from the CAS framework whose semantic identities the typed model carries — while committing it to no position on whether any such hypothesis is true. The cognitive content enters the architecture only through typed semantic references with their own namespaces, sources, and definition hashes, and the architecture's machinery is deliberately indifferent to whether those references denote anything real: it checks structure, connectivity, calibration shape, and reachability identically whether the board models a mind, a control loop, or nothing at all. That indifference is a feature. A checker that needed the cognitive hypotheses to be true in order to run would have smuggled the conclusion into the tool.

Adaptive commits the work, prospectively, to systems whose parameters, structure, or behavior change over time — the eventual subject matter includes feedback, state, drift, and recalibration — while the honest present tense is narrower: the current candidate supports declared feedback with unit delays and conservative gain bounds, and treats stateful, stochastic, and richer adaptive semantics as explicitly open, failing closed wherever they are encountered. The word marks the direction of the research program, not an achieved capability.

The workflow the name describes is a loop the later chapters will make precise. A hypothesis is written as a board (design); the board is checked against the closed diagnostic catalog (rule check); supported boards are simulated under declared profiles, emitting canonical traces (simulate); boards, reports, and traces are audited against their sources and against the claim discipline (audit); and the resulting artifacts enter the evidence ledgers typed as what they are — software receipts, never more (evidence). Every stage is deterministic, identity-bound, and fail-closed, and the loop's product is not a validated model of cognition but a falsifiable, replayable, honestly bounded description of one.

3. The research question

The engineering question is not whether cognition *is* electronics. It is:

> Can a circuit-shaped modeling language make cognitive-system hypotheses more explicit, type-safe, replayable, and falsifiable without converting an analogy into a physical, causal, safety, or product claim?

Each adjective in the question carries weight, and it is worth unpacking them before anything else is asserted.

Explicit means that every element of a hypothesis that the modeler relies on — the components, their interfaces, the values they may carry, the transfer behavior they are assumed to exhibit, their statefulness, their time semantics, their calibration status, and the boundary of what the model does not claim — appears in the model text as a declared, typed object rather than as an unstated assumption. An informal diagram is explicit about topology and silent about almost everything else; a circuit-shaped model in the sense of this monograph is required to be explicit about all of it, and a model that omits a required declaration does not degrade gracefully into a vaguer model — it fails the checker.

Type-safe means that the modeling language distinguishes kinds of things that must not be silently interchanged, and that the distinction is enforced mechanically. A semantic state literal is not a probability profile over states; a probability profile is not an element of a finite kernel; a design parameter is not a measurement; a locally asserted calibration label is not authenticated calibration evidence; an integer 1, a Boolean *true*, and a string `"true"` are three distinct values everywhere. Type safety in this sense is the primary technical instrument by which the analogy is prevented from leaking: the places where an analogy would like to become an identity are exactly the places where the type system refuses a coercion.

Replayable means that every executable claim is bound to enough identity material — source hashes, profile identities, decimal-context identity, component-catalog identity, input samples, and a canonical trace — that an independent party holding the same artifacts can rerun the exact computation and compare results value for value. Replayability is a property of the software contract, not of the world: replaying a simulation demonstrates that the software is deterministic under its declared profile, and nothing more.

Falsifiable means that the model text exposes surfaces on which it can fail, and that failures are preserved rather than repaired away. The design-rule checker can reject a board; the threshold-reachability analysis can prove that a published safety stage cannot activate; a grammar-coverage audit can show that a published formal grammar does not cover its own published corpus. The monograph's source audit (summarized in Chapter 3 and developed in full in later chapters) records precisely such failures against the engineering baseline itself, and the architecture retains them as first-class findings and permanent regression fixtures.

The final clause of the question — *without converting an analogy into a physical, causal, safety, or product claim* — is the constraint that makes the question nontrivial. It is easy to build an evocative circuit metaphor; the literature of every era is full of cognitive models dressed in the technological costume of their day — hydraulic, telegraphic, telephonic, computational. The historical failure mode of such metaphors is not that they are useless but that they are boundless: the costume is mistaken for anatomy. The question this monograph addresses is whether the costume can be worn with a mechanically enforced boundary — whether the analogy can be made load-bearing for design, audit, and falsification while remaining, checkably and permanently, an analogy.

4. What "circuit" means here — and what it does not

Definition 1.1 (Circuit-shaped model, informal). In this monograph, a *circuit-shaped model* is a directed, typed network whose elements must declare: (i) interfaces (ports with

direction, signal kind, and unit); (ii) allowed values (typed parameters and finite-state annotations); (iii) transfer behavior (a declared component profile, where one is supported); (iv) statefulness and time semantics; (v) calibration references for any point at which observation is claimed; and (vi) evidence limits (a fixed research boundary carried in the model's own identity material). "Circuit" names this engineering organization and nothing else.

Claim Boundary 1.2 (What "circuit" does not assert). The word "circuit" in CAS PCBA 8 does not assert a physical electrical implementation. It does not identify any model quantity with voltage, current, resistance, charge, flux, or power in the SI sense. It does not assert that any cognitive system — human, artificial, or biological — contains components corresponding to the model's components, nets corresponding to its nets, or thresholds corresponding to its thresholds. It does not assert that a simulated behavior of the model is a behavior of any system outside the model. Any statement in this monograph that appears to approach such an assertion is governed by this boundary and by the claim quarantine of Section 9.

The choice of the circuit organization, rather than some other engineering organization, is motivated by a specific list of disciplines that circuit practice developed and that informal cognitive diagrams lack. Six are worth naming here, because each reappears as a concrete mechanism in later chapters.

First, *interface discipline*: in circuit practice a component is usable only through declared ports, and connecting incompatible ports is a detectable error rather than a silent reinterpretation. Second, *connectivity discipline*: a netlist makes the complete connection structure of a design a single queryable object, so that floating inputs, contending drivers, and dangling outputs are mechanical findings. Third, *rule discipline*: design-rule checking established, industrially, that a design is not done when it looks right but when it passes a versioned, deterministic body of checks — and that the checks fail closed. Fourth, *trace discipline*: simulation practice archives waveforms against exact design versions, so that a claimed behavior is inseparable from the identity of the thing that behaved. Fifth, *calibration discipline*: a probe reading means nothing without an instrument identity, a protocol, and an uncertainty; hardware measurement culture keeps design values and measured values in separate ledgers. Sixth, *sign-off discipline*: verification (does the artifact implement its specification?) and validation (is the specification adequate for an external purpose?) are distinct activities with distinct evidence, and passing the first is never silently promoted into the second.

Each of these is an organizational and methodological achievement, not a physical one. None of them depends on electrons. That is precisely why they are candidates for transfer: a modeling language for cognitive-system hypotheses can adopt interface, connectivity, rule, trace, calibration, and sign-off discipline wholesale, without adopting a single claim about physics. The engineering literature surveyed in Chapter 2 supports this transfer at the level of *method* — machine-readable formal design description [18], explicit state semantics [19], verification as distinct from validation [23] — while validating nothing about CAS itself.

5. Three load-bearing separations

The architecture's claim discipline rests on three separations. They are called *load-bearing* because every mechanism described in this monograph — the type system, the design-rule checker, the simulator's trace bindings, the calibration rules, the adapter boundaries — exists to enforce at least one of them, and because the historical failure modes of circuit-flavored cognitive modeling are, on inspection, violations of exactly these three. Each separation

is stated as a rule and then developed through an extended examination of the failure it prevents.

5.1. Separation I: model versus world.

Rule 1.3 (Model versus world). A simulated transfer rule is behavior of the declared software profile, not evidence that a cognitive or physical system obeys it. No output of the architecture — no green test, no deterministic trace, no diagnostic-free design-rule report — is admissible as evidence about any system other than the software that produced it.

The failure this separation prevents is the quiet migration of a claim from the model column to the world column. The migration rarely happens in a single dishonest step; it happens through a chain of individually plausible restatements. Consider a concrete, deliberately detailed example. A modeler constructs a board containing a component intended to represent an aversion-triggered refusal stage: an input carrying an activation-like quantity, a comparator against a threshold parameter, an output that gates a downstream path. The modeler writes a simulation fixture in which the input ramps past the threshold and the gate closes, runs it, and obtains a green result with a canonical trace. So far every statement is safe: *the software performed the declared typed transition and produced the declared trace for the exact fixture under the exact profile.*

Now watch the restatements. In a status report, the green fixture becomes "the refusal mechanism works." In a slide, "the refusal mechanism works" becomes "the model refuses under overload." In a discussion section, "the model refuses under overload" becomes "this architecture exhibits the refusal behavior observed in deployed systems," and in a grant abstract the architecture has become evidence about cognition. Not one of the restatements was flagged as a new claim; each merely dropped a qualifier. The end state is a world-claim resting entirely on a software fixture that was never designed, powered, or entitled to support it. The history of simulation-based fields shows this slope is not hypothetical: wherever simulation output is cheap and empirical access is expensive, the simulated column bleeds into the empirical column unless the medium itself resists.

A second example shows the same failure in the opposite emotional register — not enthusiasm but reassurance. Suppose a board's design-rule report comes back clean: no errors, no warnings, every net driven, every threshold reachable, every declared feedback loop bounded. The clean report is a real and useful fact about the model text. The migration begins when "the design checks out" is heard as "the design is safe," and completes when a clean DRC report on a refusal-pathway model is offered as comfort about the behavior of a deployed system that the model was never validated against. Here the world-claim is not even about the model's subject; it is about a different artifact entirely, connected to the model by nothing but shared vocabulary. The checker's own fail-closed design anticipates the first half of this slope — unresolved analyses yield unresolved diagnostics, never a pass — but no tool can prevent a reader from over-reading a clean report about A as evidence about B. Only the evidence discipline can, by typing the report as a statement about the exact board digest it names and nothing else.

CAS PCBA 8 makes the medium resist in three ways. First, every simulation result carries its research boundary in its own identity material: the trace binds a fixed Ring-4 boundary object, so the artifact that would be quoted as evidence physically contains the statement of what it is not evidence for. Second, the interpretation rule for a green fixture is stated normatively and verbatim in the architecture's own documents and is repeated in this monograph wherever a result is reported: the software performed the declared transition;

nothing else. Third, the evidence taxonomy (developed later in the monograph) assigns software receipts, experiments, measurement validation, safety evidence, and authority to disjoint categories, so that a receipt cannot be counted in a category it does not belong to without a detectable category error. The separation costs something — it forbids the architecture from ever advertising its most exciting-sounding interpretations — and that cost is accepted deliberately.

5.2. Separation II: design value versus measurement.

RULE 1.4 (Design value versus measurement). A locally supplied parameter or calibration label remains structural metadata until authenticated measurement evidence and uncertainty are supplied. The architecture never promotes a value from the design ledger to the measurement ledger on the strength of anything the model author alone can construct.

The failure this separation prevents is *self-certification*: the laundering of an assumption into a datum by passing it through official-looking machinery. Again an extended example makes the mechanism concrete. A modeler declares a test point on a net and attaches a calibration reference: an instrument identity, a protocol name, an uncertainty figure, a measurement kind. All of these fields are well-formed; the design-rule checker verifies their shape, their unit compatibility with the net, and their presence where safety-marked paths require probes. A simulation run then emits test-point observations, each carrying measurement identifiers with parent/child derivations and the calibration reference.

Every artifact in this chain looks like measurement. There are instrument fields, protocol fields, uncertainty figures, measurement IDs, and derivation graphs. And yet nothing was measured: every field was typed in by the same author who typed in the model, and the "observation" is a value computed by the author's own declared transfer rules. If the architecture allowed these records to flow into its evidence stream, it would have built an automated assumption-laundering machine — more dangerous than an informal diagram, because the informal diagram at least looks informal. The hardware world knows this failure intimately: a datasheet's typical values are design targets until a characterization lab, with traceable instruments and stated uncertainty, turns them into measurements, and no amount of typesetting the target in the measurement column makes it one.

A second form of the same failure operates on parameters rather than observations. A transfer-stage gain of 0.85, a threshold of 0.9, a clamp of 0.1 — every such number in a board is a *design value*: a quantity the modeler chose, perhaps thoughtfully, perhaps arbitrarily, and wrote into the model text. Design values are indispensable; nothing can be checked or simulated without them. The failure is the citation loop: the model is simulated with the chosen parameters, the simulation exhibits some behavior, the behavior is written up, and in the write-up the parameters have become findings — "the refusal stage saturates at 0.1" reported in the tone of a discovered property rather than a typed-in choice. Six months later the write-up is a source, and the design value has completed its unearned journey to datum. The architecture's countermeasure is ledger-keeping at the type level: parameters are typed as parameters everywhere — in the board, in the DRC report, in the trace — and no representation exists in which a parameter's provenance as a supplied value is absent. The number can be quoted, but it cannot be quoted *as a measurement* without the quotation being checkably false against the artifact it cites.

The architecture therefore enforces the separation structurally. Locally constructed calibration records are typed as structural, test-only metadata: they satisfy the checker's shape requirements, they enable simulation fixtures, and they are permanently ineligible to confer

authenticated evidence status. Nothing the modeler can write in the model text — no field, no flag, no self-hash — crosses the ledger boundary. Crossing it requires an authenticated evidence pathway that the model author alone cannot fabricate, and at the bound state of the work described in this monograph, that pathway has not been exercised at all: there is no live calibrated model, no instrument chain, and no measurement claim anywhere in the admitted evidence. The measurement ledger is not merely separate; it is, at present, deliberately empty.

5.3. Separation III: analogy versus identity.

Rule 1.5 (Analogy versus identity). Names borrowed from circuits do not make CAS quantities SI electrical quantities. Physical voltage, current, resistance, charge, flux, and power are outside the candidate unless a separately typed, instrumented physical profile is proposed and validated. No such profile exists at the state of work described in this monograph.

The failure this separation prevents is *nominal capture*: the drift by which a borrowed name gradually acquires the semantics of its origin. The mechanism is linguistic before it is technical. A model component is called a *fuse* because its declared behavior — open the path when a threshold is crossed — is shaped like a fuse's behavior. The name is apt, memorable, and dangerous. Within a few pages of informal writing, the fuse "blows," "protects the downstream stages," and "guarantees" something; within a few documents, the model containing it is a "safety architecture"; and at the end of the slope someone is asserting, on the strength of a noun, that a cognitive system has an overcurrent-protection property that no one has defined, let alone measured. The same slope exists for every borrowed term: gain, impedance, saturation, rail, ground. Each is a compressed description of declared software behavior, and each invites its physical semantics to follow the name through the door.

The engineering baseline that this monograph audits provides a vivid demonstration of why the separation must be mechanical rather than rhetorical. The baseline's published Cognitive Safety Board — its flagship safety-flavored design — contains a red-stage saturation clamp of 0.1 upstream of a fuse threshold of 0.9. Under the declared clamp semantics the fuse input can never exceed 0.1, and the protective crossing is unreachable: the fuse, as published, cannot trip. (The finding is stated precisely, with page anchors and a formalized unreachability argument, later in the monograph; Chapter 3 locates it in the source basis.) A reader relying on the name *fuse* and the label *safety* would have attributed a protective property to a design whose own parameters forbid it. The design-rule checker, which propagates conservative bounds through the declared path and flags unreachable safety thresholds, catches the contradiction mechanically — because it reasons about declared semantics and never about names.

The separation also governs the quantitative vocabulary. CAS quantities in this architecture carry their own units and signal kinds, checked under a closed admissibility table; none of them is an SI electrical quantity, and the checker treats an attempt to wire incompatible kinds together as an error rather than a metaphor. If future work ever proposes that some CAS quantity has a physical correlate, the architecture's position is that this is a new, separately typed, instrumented physical profile requiring its own validation program — not a renaming of an existing model quantity. Until such a profile exists and is validated, every electrical-sounding word in this monograph is a typed pun, and the type system knows it.

5.4. The separations jointly. The three separations are stated independently but function as a system. Model-versus-world polices the *output* direction: what a computed

result may be said to show. Design-value-versus-measurement polices the *input* direction: what a supplied number may be said to be. Analogy-versus-identity polices the *vocabulary*: what a name may be said to mean. A modeling practice that enforced only two of the three would leak through the third: with the first two but not the third, names would smuggle physics; with the first and third but not the second, self-certified "measurements" would seed honest simulations with counterfeit data; with the second and third but not the first, honest inputs and honest names would still be spent on world-claims the software cannot support. The monograph will return to this triangulation repeatedly, because almost every design decision in the architecture — from tagged exact values to trace-bound research boundaries to the adapter's refusal to coerce — is most easily explained as the enforcement of one of these separations at a specific interface.

6. The six contributions

Against this background, the monograph develops six contributions. They are the contributions of the current candidate — a software research artifact at a bound source state — and each is claimed at the software-contract level only.

(1) **A typed board model** separating semantic state, geometry, signal kind, unit, component contract, calibration metadata, and evidence provenance. The model distinguishes state literals from ND-cells, keeps the strictly positive simplex address of a mixed profile disjoint from both, uses tagged exact type-plus-value identity for finite carrier values, and runs all decimal arithmetic under one immutable, versioned context. Board identity includes the complete canonical structure, so any structural edit changes the board's digest.

(2) **A deterministic design-rule checker (DRC) with fail-closed diagnostics.** The DRC computes one deterministic report under a closed 35-code `PCBA8-*` catalog spanning model, semantics, parameters, states, bias, nets, test points, calibration, feedback, safety, and TKS-adapter classes. Errors fail the board; warnings never become an implicit clean result; unsupported or resource-bounded analysis yields an unresolved diagnostic, never a pass.

(3) **A narrow trace-emitting reference simulator** for declared deterministic component profiles. Supported components declare ports, transfer rules, clipping, statefulness, and time semantics; every run emits a canonical trace binding board, component, DRC-report, catalog, decimal-context, profile, and source identities together with typed samples, events, observations, measurement derivations, and the fixed research boundary.

(4) **A source- and profile-bound adapter** from certified finite component tables to the TKS v8 finite reference kernel. Only a certified total, finite, deterministic, stateless, non-stochastic transition may cross; the adapter returns an immutable certificate and proves at most type-correct representation, carrying no automatic semantic theorem.

(5) **A source audit that preserves, rather than silently repairs, adverse findings.** The audit records that the baseline's consolidated grammar does not cover its own published five-board corpus (four direct syntactic counterexamples) and that the published Cognitive Safety Board's protective crossing is unreachable under its declared parameters; both findings are retained with page anchors as permanent fixtures.

(6) **An evidence discipline** distinguishing green software tests from experiments, measurement validation, safety evidence, and authority, with every claim record at the bound state classified Ring 4 and NOT LICENSED.

Two remarks about this list. First, none of the six is an empirical result about cognition; the list is deliberately and entirely architectural. Second, the fifth contribution deserves emphasis because it is the one most often absent from projects of this kind: the architecture treats the discovery that its own source material is defective — syntactically incomplete in its grammar, self-contradictory in its flagship safety design — as a result to be formalized and guarded, not an embarrassment to be patched. A checker that has flagged its own canon is a checker whose fail-closed behavior has been demonstrated on a case that mattered.

7. The thesis: a qualified affirmative at the software-contract level

REGISTERED HYPOTHESIS 1.6 (Thesis of the monograph). The research question of Section 3 admits a qualified affirmative answer at the software-contract level: a circuit-shaped modeling language, equipped with the type system, design-rule checker, trace-bound simulator, adapter boundaries, source audit, and evidence discipline developed in this monograph, makes cognitive-system hypotheses more explicit, type-safe, replayable, and falsifiable than their informal-diagram equivalents, while mechanically withholding physical, causal, safety, and product claims.

Every word of the qualification is intended. *Qualified*: the affirmative is supported by bounded software evidence — at the bound source state, an integrated receipt records three green focused suites totaling 208 test methods and 272 parameterized subtests, of which 82 methods belong to CAS PCBA v8 itself — and by two formalized adverse findings against the source; it is not supported by any experiment, measurement, replication, or external review, all of which remain open. *At the software-contract level*: the affirmative asserts that the software honors its declared contracts on its tested surface, and asserts nothing about any system outside the software. The thesis would be falsified by, for example, a demonstration that the checker passes a board violating a cataloged rule, that the simulator's traces are not deterministic under a declared profile, that a coercion crosses a boundary the type system claims to close, or that the preserved adverse findings misstate the source at the cited anchors. The thesis is *not* strengthened by any accumulation of green fixtures beyond its tested surface, and the monograph will not argue as if it were.

7.1. Falsifiability surfaces of the thesis. Because the monograph asks its models to expose failure surfaces, it owes the same of its thesis. The qualified affirmative of Hypothesis 1.6 is falsifiable on at least five distinct surfaces, each checkable by a party holding the bound artifacts, and the monograph invites exactly this scrutiny:

(1) **Checker soundness.** Exhibit a board that violates a rule of the closed diagnostic catalog and nonetheless receives a clean report from the checker at the bound implementation. One such witness defeats the fail-closed claim for the violated rule class.
(2) **Determinism.** Exhibit two runs of the checker or the simulator, on identical inputs under identical declared profiles and contexts, producing non-identical reports or traces. One such pair defeats the replayability claim.
(3) **Boundary leakage.** Exhibit a coercion the type system claims to forbid — a state literal accepted where a simplex address is required, an untagged value crossing the

adapter, a locally constructed calibration record acquiring authenticated evidence status — actually succeeding in the bound implementation.

(4) **Audit fidelity.** Show that a preserved adverse finding misstates the source at its cited anchors — that the consolidated grammar in fact contains an alternative producing a cited surface form, or that the safety-board listing does not carry the cited clamp and threshold values at the cited pages of the hashed build.

(5) **Evidence typing.** Show that the architecture's own artifacts, read as typed, license an inference the claim quarantine prohibits — that some receipt, certificate, or report, under its declared semantics rather than a careless reading, constitutes a physical, causal, safety, or product claim.

Symmetrically, the thesis is *not* falsified by demonstrations that belong to the withheld levels: showing that no board in the candidate corresponds to any real cognitive mechanism would defeat a validity claim the thesis does not make, and showing that the checker's rules encode contestable modeling taste would defeat an objectivity claim it does not make either. The thesis lives and dies at the software contract, on the five surfaces above.

It is worth stating plainly what would be required to answer the research question affirmatively at any level beyond the software contract, because the distance is part of the honest content of this work: a complete versioned grammar with round-trip conformance; stateful and stochastic component semantics; live calibration with authenticated instruments and uncertainty; preregistered held-out validation; independent replication in a hermetic toolchain; and separate formal, systems, experimental, safety, and claim-language review. That program is laid out, dependency-ordered, at the end of the monograph. None of it has begun in the admitted evidence, and nothing in the intervening chapters will pretend otherwise.

8. What is gained over informal practice

Because the thesis withholds so much, it is fair to ask what, concretely, is left — what a modeler gains by writing a cognitive-system hypothesis as a checked board rather than as a diagram. The answer, developed mechanism by mechanism in the later chapters, is that whole categories of error become *observable earlier*, mechanically, at design time rather than at argument time. Eight categories recur throughout the monograph and are worth previewing as a list, since together they are the practical content of the qualified affirmative:

(1) **Scale and unit substitution.** Quantities of different declared kinds or units cannot be wired together; the admissibility table is closed, and a mismatch is a distinct diagnostic rather than a silent reinterpretation.

(2) **Ambiguous semantic crosswalks.** Semantic references live in disjoint namespaces with source and definition hashes; two objects sharing a label or a position never silently become the same object, so an equivocation must be written down as an explicit (and checkable) translation to exist at all.

(3) **Incomplete grammar claims.** A language that claims to describe a corpus can be audited, construct by construct, against that corpus — and when the monograph's own source canon failed such an audit, the failure was recordable at page-anchor resolution precisely because the claim had been made mechanical.

(4) **Multiple-driver and unresolved-port defects.** The netlist discipline makes contention, floating inputs, and dangling references findings rather than latent ambiguities.

(5) **Unreachable threshold paths.** Conservative bound propagation can prove that a declared protective crossing cannot occur — the category of defect that condemns the source's own safety board.
(6) **Undisciplined feedback.** Cycles must be declared, delayed, calibrated, and bounded; an undeclared cycle is an error, and an unanalyzable one is unresolved, never clean.
(7) **Fabricated or disconnected measurement lineage.** Measurement records must form acyclic, existence-checked derivation chains bound to calibration references, and locally asserted calibration can never self-authenticate.
(8) **Receipts misrepresented as authority.** Software receipts, claim records, and review ledgers are typed artifacts; a green suite, a self-hash, or a generated receipt is checkably *not* a license, a certification, or a review.

None of these categories requires the cognitive hypotheses to be true. That is the design: the gains are gains in the hygiene of hypothesizing, available in full even if every hypothesis so expressed turns out false. An informal diagram can be wrong invisibly in all eight ways at once; a checked board can be wrong too — about the world, richly and perhaps irreparably — but not invisibly in these ways, and the difference is what the word "falsifiable" in the research question is doing.

9. Claim discipline and artifact status

Claim Boundary 1.7 (Standing claim boundary for the monograph). The work described here is a research candidate with artifact status DRAFT / OPEN / RING 4 / NOT LICENSED / HELD. Ring 4 designates the outermost research ring: no live, scientific, canon, product, license, or disclosure authority exists or is created by this text. All claim records at the bound source state are Ring 4 and NOT LICENSED. Drafting, compiling, or circulating this monograph promotes no claim; independent systems, safety, regulatory, clinical, and formal review remains open.

The following claim quarantine, inherited verbatim in substance from the governing research paper and the Gate 4 audit, applies to every chapter. The items below remain hypotheses or prohibited product inferences, and no sentence in this monograph asserts any of them: exact thought/transistor isomorphism; shared four-region identity; overload-induced semantic involution; Fisher–Rao causal gain; physical electrical identity for CAS quantities; semantic correctness inferred from a response-region label; safety effectiveness, certification, regulatory compliance, or clinical use; complete compiler conformance; production readiness; and disclosure or license authority derived from a self-hash or generated receipt. Historical AI-authored, self-hashed licensing receipts are not an authenticated reviewer or legal authority, and none is treated as one here.

Three further conventions govern the text. First, *background is background*: Chapter 2 surveys established engineering and mathematical literature, and nothing in that survey is presented as a novelty of this project or as validation of CAS. Second, *sources are bound by identity*: every fact about the engineering baseline and the controlling repository artifacts is cited against the exact content hashes recorded in Chapter 3, and where the sources do not specify a fact, the monograph says so rather than estimating. Third, *negative results are first-class*: adverse findings are stated in their own environments, at full evidential resolution, and are never softened in summary.

10. Reading guide

The monograph is organized so that a reader can locate the boundary of every claim without reading linearly, but a linear reading is the intended one.

Chapter 2 is a tutorial survey of the background this work stands on: hardware description languages and what VHDL demonstrated about formal, machine-readable design description; Statecharts and the explicit semantics of state, hierarchy, and concurrency; the industrial practice of netlists, design-rule checking, and electrical-rule-style connectivity checks; graph algorithms for circuit analysis; the verification/validation distinction; property-based testing; provenance modeling; floating-point versus decimal arithmetic control; and a primer on information geometry over probability simplices, including the Fisher–Rao metric and its closed-form geodesic distance. The chapter is deliberately self-contained so that readers from cognitive-science, software, and hardware backgrounds can each acquire the vocabulary the later chapters assume. Nothing in it is new, and it is careful to say so.

Chapter 3 fixes the versioned source basis: the 743-page engineering baseline with its source ID and SHA-256 hash; the reasons the work identifies sources by content hash rather than by an invented public version; the baseline's own disclaimer and the Phase-0 authority matrix; the controlling repository artifacts and their hashes; the downstream read-only boundaries into the TKS v8 finite kernel and the Noetic Topos 2.0 evidence discipline; and the read-only status of the Gate-5 preregistration kernel. Everything later in the monograph that mentions a source fact resolves through this chapter.

The chapters that follow the source basis develop the architecture in dependency order: the typed semantic and board model, including state types, tagged values, the single decimal context, and board identity; the design-rule checker and its closed diagnostic catalog, check class by check class; the narrow deterministic simulator and its canonical trace bindings; the geometry profiles — the Fisher–Rao baseline and the optional projector profile — with their disclosed numerical limits; and the certified-transition adapter into the TKS v8 finite kernel, together with the Noetic Topos ambient/resolved boundary. After the architecture, the monograph presents the adverse source audit at full resolution — the grammar-coverage finding with its board-by-board evidence table, and the Cognitive Safety Board finding with the formalized unreachability argument — followed by the bounded software evidence at the bound commit, the threats to validity, and the dependency-ordered research program. A reader interested only in what this work does *not* claim can read Section 9, the boundary environments of each chapter, and the threats-to-validity chapter, and will have an accurate picture.

A note on tone. The monograph is written in the declarative voice of an engineering document, and long stretches of it describe machinery that works as declared. The reader is asked to hold that voice together with the standing boundary of Section 9 at all times: "works" always means "performs its declared software contract on its tested surface," the tested surface is finite and disclosed, and the most important results in the monograph are the two places where the machinery, applied to its own source canon, correctly said *no*.

CHAPTER 2

Background: Hardware Description, State Formalisms, and Verification

1. Purpose and reading discipline of this survey

This chapter assembles, at tutorial depth, the established engineering and mathematical background on which the rest of the monograph stands: hardware description languages; explicit state formalisms; the industrial practice of netlists and rule checking; graph algorithms for circuit analysis; the distinction between simulation verification and operational validation; property-based testing; provenance modeling; arithmetic control; and information geometry on probability simplices. The survey is self-contained because the monograph's readership spans communities — cognitive-science readers who have never read a netlist, software readers who have never derived a Fisher metric, hardware readers who have never needed a formal provenance vocabulary — and the later chapters assume all of this vocabulary at once.

Two disciplines govern the reading of every section below.

Claim Boundary 2.1 (Background is background). Nothing in this chapter is a contribution of the CAS PCBA 8 project. Every concept, formalism, algorithm, standard, and theorem surveyed here is prior art of its own community, cited where a citation exists in this monograph's reference list and described in general terms where the material is standard industrial practice. Conversely, nothing in this chapter validates CAS: the fact that VHDL, Statecharts, design-rule checking, Tarjan's algorithm, verification-and-validation discipline, QuickCheck, PROV, IEEE 754, or the Fisher–Rao metric are sound and useful in their home domains is evidence about *those* domains and about engineering method. It is not evidence that the CAS ontology, its cognitive interpretation, or any CAS PCBA 8 design choice is empirically correct. The support this literature lends is at the level of method only.

The second discipline is a map. For each background item, the section states (i) what the item is, (ii) what its home community learned from it, and (iii) which mechanism of the CAS PCBA 8 architecture borrows the lesson — always as an organizational borrowing, never as a transferred validity claim. Section 13 collects the map in one table.

2. Hardware description languages

2.1. From annotated drawings to machine-readable design text. Before the mid-1980s, the dominant medium of digital-hardware design description was the schematic drawing supplemented by prose. The medium had the same epistemic weaknesses that Chapter 1 attributed to informal cognitive diagrams: a drawing fixes topology but leaves timing, interface contracts, allowed values, and intended behavior to convention and caption; two engineering teams could read the same drawing into different circuits; and there was no mechanical way to ask a drawing whether it was even self-consistent. As designs grew from

hundreds to millions of gates, the medium failed to scale in three independent ways: description (the drawing could not be complete), communication (the drawing could not be unambiguous), and verification (the drawing could not be executed or checked).

Hardware description languages (HDLs) answered all three failures at once by making the design text itself formal, machine-readable, and executable. An HDL description is not documentation *about* a design; under the language's semantics, it *is* the design, and every downstream activity — simulation, synthesis, rule checking, equivalence checking, archiving — consumes the same text. This is the single most consequential idea the present monograph borrows: the move from diagram-plus-prose to a typed design language with defined semantics is a methodological act, independent of subject matter, whose payoff is that design intent becomes checkable.

2.2. What VHDL demonstrated. VHDL, standardized by the IEEE as Std 1076 and maintained through successive revisions to the current language reference manual [18], is the canonical demonstration of this move, and several of its specific design decisions are instructive far beyond hardware.

First, VHDL separates *interface* from *implementation* as a matter of language structure: an entity declaration fixes the ports — names, directions, and types — through which a design unit may be used, while one or more architecture bodies supply behavior. A component is usable only through its declared interface, and the language makes it impossible to connect to a port that was not declared. The lesson is interface discipline: contracts live in the text, not in the reader's head.

Second, VHDL is strongly and expressively typed. Signals and variables carry declared types; operations between incompatible types are rejected at analysis time rather than misinterpreted at run time; and the language's package mechanism lets a community standardize shared types with resolved semantics. Decades of industrial experience taught that the type checker's refusals are not bureaucratic friction but early defect detection: a large class of wiring and width errors that would otherwise surface as silent simulation misbehavior surface instead as analysis errors with source locations. The lesson is that type safety is a defect-prevention technology.

Third, VHDL has defined simulation semantics — an event-driven model with delta cycles, in which the meaning of a description is the trace of signal transactions it produces under a defined scheduling discipline. Because the semantics is part of the standard, two conforming simulators must agree on what a description means, and a simulation result is a property of the description rather than of one vendor's tool. The lesson is that executability with defined semantics converts design text into a replayable object.

Fourth, VHDL demonstrated the value of *standardization of description itself*. The language emerged from a documentation requirement — the need for a uniform, tool-independent way to describe hardware across contractors and decades — and its longevity shows that a stable, versioned, publicly specified description language is an archival technology: a design written down in it remains interpretable when the tools, companies, and engineers of its era are gone.

Fifth — a point easily missed from outside the field — VHDL culture developed the *testbench* as a first-class design artifact: a description, written in the same language, whose sole purpose is to exercise another description and check its responses. The design under test and the apparatus that tests it share a type system, a simulator, and an archive, so a verification claim ("this entity passed this testbench") is a statement about two identified artifacts and can be rerun by anyone holding both. The testbench idea generalizes far beyond

hardware: it is the recognition that checking apparatus deserves the same rigor of description, identity, and versioning as the thing checked. CAS PCBA 8's fixtures — including, notably, its known-fault fixtures, which encode designs that *must* fail the checker in a declared way — are testbench culture applied to a rule checker.

Sixth, the industry's experience with the *synthesis subset* carries a cautionary lesson in the other direction. Not every legal VHDL description can be synthesized into hardware; synthesizable VHDL is a subset, and for years the boundary of that subset was tool-dependent and imperfectly documented, so that a description could simulate one behavior and synthesize another — the simulation/synthesis mismatch, a notorious class of defect. The lesson is that when a language is richer than the machinery that consumes it, the gap between what can be *written* and what is actually *supported* must be explicit, versioned, and checked, or it becomes a source of silent divergence. This lesson bears directly on the monograph's central adverse finding: the source baseline's published grammar covers only a fraction of the constructs its own reference designs use, an unmanaged written/supported gap of exactly the kind synthesis history warns about. The candidate's response — declare the supported surface exactly, fail closed outside it, and demand that any future grammar publish a construct-coverage matrix — is the hardware community's remedy, transplanted.

For this monograph, VHDL's demonstration is cited strictly at the level of method [18]: formal machine-readable design description with typed interfaces and defined execution semantics is feasible, scalable, and industrially transformative. CAS PCBA 8's typed board model, closed component catalog, and deterministic simulator borrow the organization. The borrowing carries no validity: a cognitive-system hypothesis written in a typed board model is exactly as conjectural as the same hypothesis on a whiteboard — it is merely explicit, checkable, and replayable in a way the whiteboard version is not.

2.3. Description versus realization. One further HDL lesson deserves its own subsection because it prefigures the monograph's first load-bearing separation. An HDL description that simulates correctly is not thereby a working chip. Between description and silicon lie synthesis, technology mapping, layout, parasitics, process variation, and physical verification, and hardware culture maintains a strict vocabulary boundary between what simulation shows (the description's behavior under a model) and what characterization of fabricated parts shows (the artifact's behavior in the world). The industry learned, sometimes expensively, that eliding this boundary — treating simulation sign-off as silicon sign-off — is a category error with recall-sized consequences. The model-versus-world separation of Section 5.1 is this hardware lesson, imported whole.

3. Statecharts and explicit state semantics

3.1. The problem: state explosion and implicit dynamics. Finite-state machines are the classical formalism for reactive behavior, but flat state machines fail on realistic systems for a structural reason: the number of global states is the product of the states of the system's independently varying parts, so a handful of concurrent features multiplies into an unreadable and unmaintainable diagram. Practitioners of the 1970s and early 1980s responded by leaving most of the state structure implicit — in prose, in code, in convention — which reproduced, for dynamics, exactly the auditability failure that schematics-plus-prose produced for structure.

3.2. Harel's contribution. Harel's Statecharts [19] attacked the problem by enriching the visual formalism itself with three orthogonal mechanisms, each with defined semantics.

Hierarchy (depth): states may contain substates, and a transition from a superstate abstracts a bundle of transitions from all its substates, so the diagram supports abstraction and refinement without losing precision. *Orthogonality* (concurrency): a state may be the AND-composition of independent regions, so the diagram represents a product of n regions in size proportional to their sum, taming the exponential blowup syntactically. *Broadcast communication*: events raised in one region are visible to others, giving a defined mechanism for coupling concurrent parts. Harel's summary formula — statecharts = state-diagrams + depth + orthogonality + broadcast-communication — names precisely the three additions.

The deeper lesson of the Statecharts program, however, is not any single mechanism but the demand it institutionalized: *a visual formalism must have a semantics.* A diagram whose boxes and arrows lack defined meaning is not a specification, however suggestive; once depth, concurrency, and communication have defined semantics, the diagram becomes a mathematical object that tools can execute, check, and transform. The subsequent absorption of statechart variants into mainstream modeling practice testifies that the demand was answerable at industrial scale.

3.3. The aftermath: semantics is where the work is. The years after the original paper taught a second lesson that the first had made possible. Once statecharts were executable, the community discovered that the notation admitted many defensible but different semantics: whether transitions taken in one step can trigger further transitions in the same step; how conflicts between a superstate's transition and a substate's are resolved; whether the effects of simultaneous events are perceived instantaneously or in the next step; what history connectors restore, exactly, on re-entry. Dozens of published variants differed on such points, and two tools could disagree about the behavior of the same chart while both plausibly claiming to implement "statecharts." The lesson is not that the formalism failed — it is that formalization *surfaced* ambiguities which informal boxes and arrows had contained invisibly all along, and that the surfacing is the value. A notation without semantics cannot even disagree with itself; executable semantics converts hidden ambiguity into visible, decidable, versionable choice. For the present work the corollary is direct: a modeling language for cognitive hypotheses must expect that its early versions under-specify, must version its semantic choices explicitly, and must treat a discovered ambiguity (such as a grammar that does not say what its own corpus means) as a first-class finding to record rather than a nuance to gloss.

3.4. The lesson borrowed. CAS PCBA 8 borrows the demand, not the notation. Its board model requires every component to declare statefulness and time semantics explicitly; components without declared state semantics are unsupported by the simulator and fail closed rather than defaulting to an implicit dynamics; feedback — the circuit-shaped analog of coupled concurrent regions — must be declared, delayed, and bounded rather than left emergent. Where the architecture currently supports only a narrow deterministic profile (stateless transfer with declared unit delays), it says so, and richer state semantics are recorded as open obligations rather than assumed. The Statecharts literature [19] is cited for the methodological point that explicit state, hierarchy, concurrency, and communication semantics are what turn behavioral diagrams into checkable objects; it neither knows of nor validates any cognitive interpretation.

4. Electronic design automation practice: netlists and rule checking

The material in this section is standard industrial practice, described in general terms; it is the accumulated craft of the electronic design automation (EDA) industry rather than the contribution of any single citable source.

4.1. The netlist as central data structure. At the center of every EDA flow sits the netlist: a complete, queryable enumeration of a design's components (with their kinds and parameters), their ports, and the nets — the connection equivalence classes — that join port to port. The netlist is the design's connectivity reduced to a data structure, and its centrality is a methodological fact: once connectivity is a data structure rather than a drawing, connectivity questions become computations. Which inputs are unconnected? Which nets have two drivers? Which component instances share a reference designator? What is the fan-out of this output? Every such question, undecidable against a pile of schematic sheets by anything but human inspection, is a traversal over the netlist.

The netlist also established an identity discipline. Each component instance carries a unique reference designator; each net a unique name; and the design database treats collisions as errors. This sounds trivial and is not: unique, stable identity is what makes diff, review, archive, and cross-reference possible across a design's lifetime, and its absence is what makes informal diagrams unauditable. CAS PCBA 8's board structure — immutable, identity-bearing records for components, ports, nets, test points, calibrations, and feedback declarations, with uniqueness enforced by its checker — is the netlist discipline, borrowed whole.

4.2. Design-rule checking as an industrial discipline. Design-rule checking (DRC) arose in physical design: fabrication processes publish geometric rules (minimum widths, spacings, enclosures), and a DRC tool checks a layout against the rule deck exhaustively, emitting a report of violations with locations. Over decades this practice hardened into a sign-off culture with several transferable characteristics.

First, *the rule deck is versioned and closed*: checks come from a defined catalog with stable identifiers, so that "DRC clean" is a statement relative to an exact deck, and two runs against the same deck and the same design are comparable. Second, *checking is exhaustive and deterministic*: the tool checks every instance of every rule, and its report is a function of its inputs, not of operator attention. Third, *clean is a gate, not a grade*: a design with violations does not proceed, and waivers — deliberate acceptances of specific flagged violations — are themselves recorded artifacts with identities and justifications, never silent. Fourth, *the report is the product*: what sign-off consumes is not the designer's assurance but the tool's report bound to the design version it checked.

Electrical rule checking (ERC) extended the same discipline from geometry to connectivity semantics, and its standard check families are worth enumerating because they map so directly onto the monograph's subject matter: outputs shorted to outputs (multiple drivers on one net); inputs left floating (undriven loads); ports connected across incompatible signal families (a power-kind pin into a signal-kind pin); fan-out and loading violations; and unconnected required pins. ERC checks are shallow individually and powerful collectively: each encodes a semantic expectation about connectivity that a drawing cannot enforce, and industrial experience is that they catch, mechanically and early, a class of defect that no amount of careful drawing eliminates.

CAS PCBA 8's design-rule checker is a deliberate transplant of this discipline: a closed, versioned diagnostic catalog (`PCBA8-*`); deterministic single-report semantics; errors that

fail the board; warnings that never decay into an implicit pass; and net-class checks — unknown endpoint owner, invalid driver count, missing net load, signal-kind mismatch, unit mismatch, multiply-connected ports, required ports unconnected — that any ERC engineer would recognize at a glance. The transplant is organizational. That DRC/ERC discipline works for electronics establishes that fail-closed cataloged checking is a sound engineering practice; it establishes nothing about whether any particular CAS rule expresses a true constraint on cognition, and the monograph never argues otherwise.

4.3. Consistency checking between representations. A third pillar of EDA sign-off practice, alongside DRC and ERC, deserves mention because it names a check *shape* the monograph reuses: layout-versus-schematic (LVS) checking, which extracts the connectivity actually present in a physical layout and compares it, device by device and net by net, against the schematic netlist that the layout purports to implement. LVS institutionalized the insight that when the same design exists in two representations, their agreement is not to be assumed but computed — and that the comparison must be canonical (both sides reduced to a normal form before matching) for the verdict to be meaningful. The general shape — two representations of one design, a canonicalization, a mechanical equivalence check, and a fail-closed verdict — is exactly the shape of the round-trip obligations the candidate imposes on any future CCDL grammar (parse the corpus, serialize canonically, and match), and of the audit's comparison between the source's published grammar and the source's published boards. In LVS terms, the monograph's grammar-coverage finding is a failed cross-representation consistency check on the source canon itself, executed at the level of surface syntax under a disclosed, bounded method.

Waiver practice completes the sign-off picture. Real designs sometimes carry deliberate rule violations — a spacing waived for a matched analog pair, a floating pin documented as intentionally unused — and mature flows do not handle these by relaxing the rule or hand-editing the report. They record a *waiver*: an identified, justified, reviewed artifact that acknowledges the specific violation instance and travels with the design. The report stays true (the violation is real); the disposition is explicit; and an auditor can enumerate every place where human judgment overrode the deck. The candidate's analog is its explicit exception discipline — for instance, declared low-gain exceptions in feedback analysis — in which every departure from the conservative default is itself a typed, identified declaration rather than a silent tolerance.

4.4. Fail-closed as a cultural achievement. One aspect of EDA practice deserves separate emphasis because it is a culture, not an algorithm. Mature checking tools treat *inability to check* as failure: a rule the tool cannot evaluate on a given structure produces an error or an explicit unresolved status, never a silent pass. The alternative — treating unevaluated as clean — is seductive (reports look better) and catastrophic (the report's silence becomes uninterpretable). The fail-closed convention is what makes a clean report mean something. CAS PCBA 8 adopts it globally: unsupported feedback analysis yields an unresolved-safety diagnostic; unsupported simulation behavior fails closed; an unresolved reachability bound is not treated as reachable. The convention costs green checkmarks and buys interpretability, and the monograph regards the trade as settled by several decades of industrial precedent.

5. Graph algorithms for circuit analysis

5.1. Circuits as directed graphs. A netlist induces graphs, and most connectivity analyses are graph algorithms. For the analyses relevant to this monograph, the induced object is a directed graph on components: an edge runs from component u to component v when some net is driven by a port of u and loads a port of v. Questions about signal flow become questions about this graph: acyclicity determines whether a design can be evaluated in one forward pass; cycles locate feedback; reachability delimits the influence cone of an input; and topological order, when it exists, is an evaluation schedule.

5.2. Depth-first search and strongly connected components. The classical toolkit here is depth-first search, and its crown result for present purposes is Tarjan's algorithm [20], which computes the strongly connected components (SCCs) of a directed graph in a single depth-first traversal, in time linear in the number of vertices and edges. A strongly connected component is a maximal vertex set within which every vertex reaches every other; contracting each SCC to a single node yields the condensation of the graph, which is always acyclic. The decomposition is exactly the right primitive for feedback analysis: every directed cycle lies within one SCC, so the SCCs of size greater than one (together with trivial SCCs bearing self-loops) enumerate the regions of a design where feedback exists, and the acyclic condensation orders everything else for straight-line evaluation.

The algorithm itself repays a paragraph of description, both because its economy is instructive and because the monograph's checker relies on its completeness. A depth-first search from each unvisited vertex assigns every vertex a discovery index and maintains a stack of vertices whose component is not yet determined. For each vertex v the algorithm computes a *lowlink*: the smallest discovery index reachable from v by following tree edges and at most one back or cross edge into the stack. When the search finishes a vertex whose lowlink equals its own discovery index, that vertex is the root of a strongly connected component, and the component is exactly the set of vertices above it on the stack, which are popped together. Every vertex is pushed once and popped once, and every edge is examined a constant number of times — hence linearity — and the components are emitted in reverse topological order of the condensation, so a single pass yields both the feedback regions and an evaluation order for the acyclic remainder [20]. That last property is quietly load-bearing for the architecture: the same classical computation that finds every cycle also schedules everything that is not a cycle, which is why a checker and a simulator can share one graph analysis.

Two properties of this result matter methodologically. First, it is *complete*: linear-time SCC computation finds all feedback regions, so a checker built on it can assert the absence of undeclared cycles, not merely the presence of found ones. Second, it is *cheap*: linearity means the analysis scales to any design a database can hold, so there is no engineering pressure to sample or skip. CAS PCBA 8's feedback check class builds the directed component graph, computes SCCs by the classical method [20], and then applies its own conservative policy to each realizable declared cycle: an exact feedback declaration, a declared unit delay, an existing dimensionless calibration, and a sub-unity gain bound, with explicit declared exceptions and with stateful or stochastic loops unsupported and failing closed. The graph theory is classical; the policy layered on it is the candidate's own contract, and its conservatism is a design choice, not a theorem.

5.3. Bound propagation on acyclic structure. The condensation's acyclicity also underwrites a second analysis pattern used later in the monograph: conservative bound propagation. On an acyclic evaluation order, an upper bound on each component's output

can be computed from upper bounds on its inputs and the component's declared behavior (a clamp caps a bound; a monotone non-amplifying stage preserves it; a gain stage scales it), and the composition of per-stage bounds along a path yields a sound bound at the path's end. Interval-style propagation of this kind is standard static-analysis practice; its soundness depends on each per-stage bound being sound and on the traversal respecting dependency order. The design-rule checker's threshold-reachability class — which flags a safety-critical threshold that provably exceeds the best upstream bound — is an application of this standard pattern, and its verdict on the source's Cognitive Safety Board is developed in later chapters.

6. Simulation verification versus operational validation

6.1. Two questions that must not be merged. The modeling-and-simulation community maintains a foundational distinction, surveyed authoritatively by Sargent [23], between *verification* and *validation*. Verification asks: does the implemented model correctly implement its conceptual specification — is the code a faithful realization of the model the developers intended? Validation asks: is the conceptual model an adequate representation of the problem entity for the model's intended purpose — does the model, however faithfully implemented, track the world well enough to use? The two questions have different subjects (software versus world), different evidence (testing, tracing, and code analysis versus comparison against system data, expert scrutiny of behavior, and predictive trials), and different failure modes (bugs versus wrong models), and Sargent's framework further distinguishes conceptual-model validity, data validity, and operational validity as separate obligations within the overall modeling process, each requiring its own evidence at its own stage.

Sargent's survey is valuable to this monograph not only for the distinction but for its catalog of *validation techniques*, because the catalog shows concretely how different the two evidence regimes are. Verification draws on software-engineering technique: structured walkthroughs, traces of program logic, testing against the conceptual model's specification. Validation draws on a different toolbox entirely: comparison of model output against data from the real system where such data exist; comparison against other validated models; degenerate-case and extreme-condition tests (does the model behave sensibly when inputs are driven to zero or to their limits?); internal-validity checks on stochastic replications; face validation by domain experts examining behavior; historical data validation with held-out samples; predictive validation against outcomes not used in construction; and animation or operational graphics reviewed by people who know the modeled system. Almost every item on the validation side presupposes access to the modeled system, to its data, or to experts on it — which is exactly why validation evidence cannot be manufactured inside the development loop, and why a project that possesses only development-loop evidence must say so in so many words. Sargent's framework also emphasizes that validity is always relative to an *intended purpose* and required accuracy: a model is not valid in the abstract but adequate-for-use, and a validity claim without a stated purpose is not yet a claim.

The community codified this distinction because merging the questions produces a specific, recurring error: a model that reproduces its own specification perfectly is treated as if it had been shown to represent reality. Verification evidence is comparatively cheap — it can be manufactured entirely inside the development loop — while operational validation requires access to the modeled system, experimental design, and statistical care. Wherever the expensive evidence is scarce, the cheap evidence expands to fill the vacuum unless the vocabulary itself resists.

6.2. Position of the present work. In Sargent's vocabulary [23], everything green in CAS PCBA 8 at the bound source state is verification evidence: bounded regression suites showing that the checker, simulator, and adapters implement their declared contracts on their tested surfaces. Operational validation — evidence that any circuit-shaped cognitive model tracks any cognitive system — has not been attempted, is not claimed, and appears in the monograph only as an open obligation in the research program. The model-versus-world separation of Chapter 1 is the verification/validation distinction enforced mechanically: by binding every trace to a research boundary and typing every receipt as software evidence, the architecture makes the cheap-evidence-expansion error a detectable category violation rather than a rhetorical drift. The distinction itself is the simulation community's, not this project's; the project's contribution, if any, is only the mechanical enforcement.

7. Property-based testing

7.1. From examples to properties. Conventional unit testing checks a program against hand-chosen input/output examples. Its blind spot is structural: the examples test the cases the author thought of, and defects live disproportionately in the cases nobody thought of. Property-based testing, introduced to practice by Claessen and Hughes's QuickCheck [24], inverts the economy: the author writes a *property* — a predicate that should hold for all inputs of a type — and the framework generates random inputs from that type's distribution, checks the property on each, and reports the first counterexample. Two further ideas complete the method. *Generators* are first-class: the author controls the distribution of random inputs, composing generators for structured data so that random boards, random expressions, or random traces are constructible. *Shrinking* makes counterexamples usable: on failure, the framework searches for a smaller input that still fails, typically presenting a near-minimal witness rather than the sprawling random structure that first triggered the fault.

Concreteness helps here, because the properties relevant to this monograph are of a recognizable family. A round-trip property says that serialization inverts parsing: for every generated board b, $\mathrm{parse}(\mathrm{serialize}(b)) = b$ under canonical equality. A determinism property says that a computation is a function of its declared inputs: for every generated board, two runs of the checker yield byte-identical reports. An invariance property says that an equivalence the design intends is real: for every board and every permutation of its component declaration order, the canonical digest is unchanged. A metamorphic property relates outputs across related inputs without knowing either output in advance: adding an undriven net to any clean board must produce exactly one new missing-load diagnostic. Each property is a few lines to state; each quantifies over a space no example suite can cover; and each, under random generation with shrinking, yields on failure a small witness board that a developer can read. The QuickCheck lesson [24] is that specifications of this shape are cheap to write and disproportionately effective to check — and the corresponding honesty obligation is that until such a campaign is run, invariants of this shape are contracts with example-level evidence only.

The method's track record established a general lesson: for software whose specification is expressible as invariants — parsers that must round-trip, checkers that must be deterministic, orders that must be total, encodings that must invert — randomized property checking finds defects that example suites systematically miss, at low authoring cost. The lesson comes with an honest converse: property testing samples the input space and therefore establishes confidence, not proof; a passed property is evidence proportional to the generator's coverage of the space, and a biased generator can starve the very region where the defect lives.

7.2. Role in this work. CAS PCBA 8's contracts are invariant-shaped throughout — one deterministic DRC report per board; trace determinism under a declared profile; digest stability under canonical serialization; admissibility tables that must be closed under their own composition rules — and are therefore natural property-testing targets. The monograph is explicit about status: at the bound source state, the architecture's evidence is bounded example-style regression suites, and a property-based/random generation campaign over DRC and simulation invariants is a scheduled step of the research program [24], not an accomplished one. The citation is to the method; no property-testing result is claimed, and QuickCheck is not authority for the separately listed mutation-testing obligation, which requires its own sources and specification.

8. Provenance

8.1. The provenance vocabulary. When computational results feed decisions, the question "where did this number come from?" needs a machine-readable answer. The W3C PROV family of specifications, with PROV-O as its ontology realization [22], standardizes a small core vocabulary for exactly this: *entities* (things with fixed state — documents, datasets, results), *activities* (processes that occur over time and act on entities), and *agents* (things bearing responsibility), connected by typed relations — an entity *was generated by* an activity, an activity *used* entities, an entity *was derived from* another entity, an entity or activity *was attributed to* or *associated with* an agent. The apparent modesty of the vocabulary is its strength: with generation, use, derivation, and attribution made explicit and typed, lineage questions become graph queries — what did this result depend on, transitively? which results does this defective input contaminate? — and independent systems can exchange lineage without bilateral format agreements.

The vocabulary scales in both directions. Downward, PROV supports specialization and qualified relations, so that a bare "was derived from" can be refined — which activity performed the derivation, using which entity, in which role — when finer lineage matters. Upward, PROV supports *bundles*: provenance descriptions that are themselves entities, so that one can record the provenance of a provenance record — who asserted this lineage, generated by what process, when. The bundle idea matters for any architecture whose receipts might be mistaken for authority: it makes "this receipt was generated by the same process it vouches for" an expressible, queryable fact rather than a buried circumstance, which is precisely the fact one needs surfaced to refuse self-certification.

Two design points of PROV matter for the borrowing at hand. First, derivations form a directed structure whose acyclicity within a record set is a checkable well-formedness condition: nothing is its own ancestor. Second, provenance assertions are claims by agents, not oracles; PROV records who says a derivation held, and reconciling or trusting such claims is explicitly out of the vocabulary's scope. The vocabulary makes lineage sayable and queryable; it does not make it true.

8.2. Role in this work. The canonical traces and measurement records of CAS PCBA 8 keep the same separations the PROV vocabulary keeps — results (entity-like) apart from runs (activity-like) apart from derivations, with parent/child measurement links validated for existence and acyclicity, and every record bound to identities rather than descriptions. The monograph's claim here is deliberately minimal and matches the source documents: the record discipline *resembles* the entity/activity/derivation separation of W3C PROV [22], and no PROV-O conformance is claimed. The resemblance is cited to locate the design in a known methodological tradition; conformance would be a checkable claim the project has not earned

and therefore does not make. The second PROV design point — provenance as claim, not oracle — reappears in the architecture's rule that locally asserted calibration metadata cannot authenticate itself: recording lineage in valid shape never converts the recorded content into evidence.

9. Arithmetic control: floating point and decimal contexts

9.1. Why arithmetic is a reproducibility concern. A deterministic, replayable computation contract must specify its arithmetic, because "the same computation" is underdetermined without it. IEEE Std 754 [21] is the standard that made binary floating-point arithmetic specifiable: it defines the binary formats (binary64 being the ubiquitous double precision), the rounding-direction attributes, the required correctly rounded basic operations, exceptional values and their propagation, and the status flags that record inexact, overflow, underflow, and invalid events. The standard's achievement is frequently misstated, and the accurate statement matters here: IEEE 754 makes each *individual operation* exactly specified, but a *computation* composed of many operations remains sensitive to evaluation order, compiler transformations, intermediate precisions, library function variation, and parallel reduction order. Bitwise reproducibility across toolchains is achievable but is an engineering discipline built on the standard, not a free consequence of it. Floating point is also, famously, non-associative in the small: rounding makes $(a+b)+c$ and $a+(b+c)$ potentially different, which is exactly why evaluation order is part of a computation's identity.

Decimal arithmetic under an explicit context object is the other established control regime the monograph's subject draws on. In a context-based decimal system, every operation executes under a context that fixes precision, rounding mode, and exceptional-condition handling; the context is data, so a computation can construct, version, and record the context it ran under, and two parties holding the same context and the same operation sequence obtain identical digit strings. The trade-offs between the regimes are conventional: binary64 buys hardware speed and a vast numerical literature at the price of representation error for decimal quantities and order sensitivity; explicit decimal contexts buy exact decimal representation and recordable, versioned arithmetic identity at the price of speed.

9.2. Role in this work. CAS PCBA 8 uses both regimes, each disclosed. Candidate arithmetic runs under one immutable, versioned decimal context created from primitive constants for each operation, with ambient caller precision made irrelevant and the context identity bound into reports and results — an application of the context-control pattern, with IEEE 754 [21] cited as the external reference point for explicit arithmetic control while the candidate's decimal profile remains its own software contract. Separately, the disclosed binary64 implementation of the Fisher–Rao distance (Section 10) is characterized in the source documents as a numerical pseudometric at its resolution — an honest floating-point disclosure of exactly the kind this section's background predicts is necessary: near-identical profiles may be numerically indistinguishable, so the implementation does not claim to separate all distinct points, and independent numerical audit remains an open obligation.

10. Information geometry on probability simplices: a primer

This section develops, at primer depth, the geometry that the architecture's mixed-profile state addresses inhabit. The material is classical; modern book-length treatments are Amari [25] and Ay et al. [26], and the founding papers are Fisher's on the information quantity [42] and Rao's on the induced distance [43].

10.1. The open probability simplex. Fix n mutually exclusive categories. A probability distribution over them is a point of the closed simplex $\Delta_{n-1} = \{p \in \mathbb{R}^n : p_k \geq 0,\ \sum_k p_k = 1\}$; the *open* simplex Δ°_{n-1} requires strict positivity, $p_k > 0$ for all k. The open simplex is an $(n-1)$-dimensional smooth manifold, and restricting to it is not pedantry: the geometric structure below degenerates on the boundary (where some $p_k = 0$), and a state space defined on the open simplex builds "every category retains nonzero mass" into the type. A tangent vector u at p is a direction of infinitesimal reweighting and satisfies $\sum_k u_k = 0$, since motion must preserve total mass.

10.2. The Fisher–Rao metric. A Riemannian metric assigns each point an inner product on its tangent space, thereby defining lengths of curves and distances between points. The Euclidean metric inherited from $\mathbb{R}^n$ is available but statistically arbitrary: it treats a mass shift of 0.01 the same whether it moves between categories holding 0.5 each or between categories holding 0.011 and 0.989, though the statistical distinguishability of the two changes is utterly different. Fisher's information quantity [42] supplies the statistically motivated alternative: for a parametric family, the Fisher information measures the expected curvature of the log-likelihood — how sharply data discriminate nearby parameter values. Taking the categorical family with the probabilities themselves as coordinates, the Fisher information matrix induces the *Fisher–Rao metric* on Δ°_{n-1}:

$$g^{\mathrm{FR}}_p(u, v) \;=\; \sum_{k=1}^{n} \frac{u_k\, v_k}{p_k},$$

for tangent vectors u, v at p. The $1/p_k$ weighting encodes the statistical intuition exactly: reweighting a rare category is a longer move, per unit of mass, than reweighting a common one. Rao's 1945 paper [43] was the first to treat this induced Riemannian structure as a geometry on the space of distributions and to propose its geodesic distance as a measure of dissimilarity between populations — the founding act of the field now called information geometry [25, 26].

A two-category example makes the weighting tangible. On Δ°_1, parameterize points by $p = (t, 1-t)$ with $0 < t < 1$; a tangent vector is $u = (s, -s)$. The Fisher–Rao squared length of u at p is

$$g^{\mathrm{FR}}_p(u, u) = \frac{s^2}{t} + \frac{s^2}{1-t} = \frac{s^2}{t(1-t)},$$

which is minimized at $t = \frac{1}{2}$ and diverges as $t \to 0$ or $t \to 1$: the same nominal mass shift s is geometrically short between balanced categories and arbitrarily long near the boundary. The divergence at the boundary is the concrete face of the earlier remark that the geometry degenerates when any $p_k = 0$, and it is the mathematical reason a state space built on this metric is typed over the *open* simplex: the metric itself refuses the boundary.

10.3. The sphere map and the closed-form geodesic distance. The categorical Fisher–Rao geometry admits a classical closed form via the square-root embedding. Map $p \mapsto \xi(p) = 2(\sqrt{p_1}, \ldots, \sqrt{p_n})$. Since $\sum_k (\sqrt{p_k})^2 = 1$, the image $\xi(p)$ lies on the sphere of radius 2 in $\mathbb{R}^n$, in its strictly positive orthant; and a short computation shows the map is an isometry from $(\Delta^\circ_{n-1}, g^{\mathrm{FR}})$ onto its image with the sphere's round metric. The computation is one line of calculus: since $\xi_k = 2\sqrt{p_k}$, an infinitesimal displacement satisfies $d\xi_k = dp_k/\sqrt{p_k}$, so the Euclidean squared line element of the image is $\sum_k d\xi_k^2 = \sum_k dp_k^2/p_k$ — which is exactly the Fisher–Rao squared line element on the simplex. The curved statistical geometry is, up to this change of variables, a piece of a round sphere. Geodesic distance on a sphere of radius

r is r times the angle between unit vectors, so the Fisher–Rao geodesic distance pulls back to

$$d(p,q) \;=\; 2\arccos\Big(\sum_{k=1}^{n}\sqrt{p_k\,q_k}\Big).$$

The argument of the arccosine, $\sum_k \sqrt{p_k q_k}$, is the Bhattacharyya coefficient of p and q; it equals 1 exactly when $p = q$ (giving distance 0) and decreases as the distributions separate. Because the positive orthant of the sphere has angular diameter $\pi/2$ between its axis points, distances are bounded: $0 \le d(p,q) \le \pi$ on the closed simplex, with the maximum attained only between distributions of disjoint support — which the open simplex excludes, so on Δ°_{n-1} the distance is always strictly less than π. The formula makes the earlier metric intuition quantitative and gives implementations a two-line evaluation: an inner product of square roots and an arccosine.

Numerically, the formula has a disclosed sharp edge that Section 9 predicts: for very close p and q the Bhattacharyya sum approaches 1, where the arccosine's derivative diverges, so binary64 evaluation loses relative accuracy exactly in the near-coincident regime. A binary64 implementation is therefore honestly described as a numerical pseudometric at its resolution — it may fail to separate distinct but extremely close profiles — and that is precisely the characterization the architecture's source documents adopt.

10.4. Invariance, and what the geometry does and does not say. The Fisher–Rao metric is not merely one defensible choice among many; classical results in information geometry (associated with Chentsov and treated in the modern monographs [**25**, **26**]) single it out, up to scale, as the Riemannian metric on these spaces invariant under the natural statistical maps between categorical models. Invariance is the principled reason a system committed to representing distributional states might adopt this metric rather than the Euclidean one: the geometry then reflects statistical structure rather than an arbitrary coordinate convention.

The boundary statement for this monograph is the same as every other section's. That the Fisher–Rao geometry is well-founded mathematics [**42**, **43**, **25**, **26**] makes it a sound *setting* for strictly positive normalized profiles over a finite cell set — which is how CAS PCBA 8 uses it, as the named metric profile for its 40-dimensional mixed-state addresses. The mathematics does not identify a learned model's internal state with a CAS state, does not make a profile over ND-cells an empirical measurement of anything, and does not convert geodesic distance between two software-constructed profiles into a cognitive quantity. The geometry constrains the model's internal bookkeeping; it asserts nothing about the world.

11. A note on causal claims

One further piece of background belongs in this survey because its *absence* from the architecture's claims is load-bearing. The modern mathematics of causal inference, systematized by Pearl [**27**], draws a sharp line between associational quantities (computable from observed joint distributions) and causal quantities (defined by interventions, requiring a causal model and identification assumptions to connect to data). A directed graph whose edges depict influence is, by itself, on the associational side of the line: drawing an arrow, propagating a bound along it, or simulating a transfer rule across it does not establish that intervening on the source would move the target. CAS PCBA 8's boards are directed graphs with declared transfer semantics, and the temptation to read them causally is therefore structural. The architecture's position, developed with the threats to validity later in the monograph,

is Pearl's line enforced as a boundary: graph connectivity, gain composition, prediction, and residual localization do not establish causation; causal claims would require explicit causal models, identification assumptions, and interventions [27]; and none is admitted.

12. What was deliberately not borrowed

A survey of borrowings is incomplete without its negative space: several prominent instruments of the surveyed fields were considered by the architecture's design lineage and *not* adopted, and the reasons are themselves methodological content.

From hardware practice, the architecture does not borrow *physical modeling*. Electronic simulation's deepest layer — device physics, parasitic extraction, timing derived from geometry and process — presupposes that the modeled quantities are physical quantities with instruments that measure them. Importing any of it would contradict Boundary 2.1's companion discipline in Chapter 1: there is no physical profile in the candidate, so there is nothing for device physics to be about. The borrowing stops, deliberately, at the design-database layer — description, connectivity, rules, traces — which is the layer that is physics-free.

From formal methods, the architecture does not (yet) borrow *proof*. Model checking, theorem proving, and verified compilation all offer stronger guarantees than cataloged checking and bounded suites, and nothing in the architecture's design forecloses them. They are absent because the honest sequencing runs the other way: proof obligations are worth their cost when the specification is stable and complete, and the monograph's own source audit shows the specification layer (the CCDL grammar) is neither. Proving theorems about a grammar that does not cover its corpus would harden the wrong artifact. The research program places grammar completion and round-trip conformance before any formal escalation, and until then the candidate claims exactly what bounded suites support.

From statistics and machine learning, the architecture does not borrow *fitting*. Nothing in the candidate estimates parameters from data, and this is a boundary, not a gap: a fitted parameter is a measurement claim in disguise, and the design-value-versus-measurement separation requires that the measurement ledger stay empty until an authenticated, preregistered pathway fills it. The information geometry of Section 10 is adopted as a metric structure for typed addresses, not as an estimation apparatus.

Each refusal could be reversed by future work with the appropriate evidence discipline — a physical profile with instruments, a proof layer over a completed grammar, a preregistered estimation pathway — and the research program names the preconditions. The refusals are recorded here so that their reversal, if it comes, is a visible event with a visible warrant rather than a drift.

13. Summary: the borrowings and their limits

Table 1 condenses the chapter. Each row names a background item, the methodological lesson its home community established, the CAS PCBA 8 mechanism that borrows the lesson, and — the column this monograph exists to keep honest — what the borrowing does not license.

Table 1: Background borrowings and their limits.

Background	Established lesson	Borrowing mechanism	Not licensed
VHDL [18]	Formal machine-readable design text with typed interfaces and defined execution semantics	Typed board model; closed catalogs; deterministic simulator	Any validity of CAS content written in the language
Statecharts [19]	Behavioral diagrams need explicit state, hierarchy, concurrency, communication semantics	Declared statefulness and time semantics; declared feedback; fail-closed on undeclared dynamics	Any cognitive interpretation of the states
EDA practice (general)	Netlist identity discipline; versioned closed rule decks; fail-closed sign-off	Board identity records; `PCBA8-*` catalog; errors fail the board	Truth of any specific CAS rule about cognition
Tarjan [20]	Linear-time complete SCC decomposition	Feedback region enumeration; acyclic evaluation order; bound propagation	Soundness of the candidate's feedback policy beyond its stated conservatism
Sargent [23]	Verification and operational validation are distinct obligations	Evidence taxonomy; trace-bound research boundary	Any operational validation of any board
QuickCheck [24]	Randomized property checking finds defects example suites miss	Scheduled research-program campaign over DRC/simulation invariants	Any completed property-testing result; mutation-testing authority
PROV [22]	Typed generation/use/derivation vocabulary makes lineage queryable	Trace and measurement record discipline; derivation acyclicity checks	PROV-O conformance; truth of recorded content
IEEE 754 [21]	Arithmetic must be explicitly specified for reproducibility	One immutable decimal context; disclosed binary64 pseudometric caveat	Bitwise reproducibility beyond the declared contract
Fisher, Rao, Amari, Ay [42, 43, 25, 26]	Invariant Riemannian geometry on the open simplex with closed-form geodesic distance	Named Fisher–Rao profile for 40-cell mixed addresses	Identification of model states with CAS states; empirical meaning of distances

Background	Established lesson	Borrowing mechanism	Not licensed
Pearl [27]	Association and intervention are distinct; causal claims need models and assumptions	Causal-claim quarantine in the threat model	Any causal reading of board structure or simulation

The chapter closes where it began. Everything above is the accomplishment of other communities, surveyed because the architecture of this monograph is assembled from their methods. The assembly is the subject of the chapters that follow; its evidence begins with the versioned source basis fixed in Chapter 3; and at no point does the strength of the surveyed foundations transfer, by association, to the cognitive content modeled on top of them.

CHAPTER 3

The Versioned Source Basis

1. Why a source chapter exists

A monograph whose central discipline is that every claim must be traceable to an exact artifact owes its reader, early and in one place, a complete account of what its artifacts *are*: which documents govern, how each is identified, what authority each does and does not carry, and where the read-only boundaries lie. This chapter is that account. Every later chapter that states a fact about the engineering baseline, the audit, the implementation, or the downstream formal kernels resolves its citation through the identities fixed here; and where the sources do not specify a fact — a page count, a hash, a status — the monograph's rule is to say "unspecified" rather than to estimate. The chapter also develops, as methodology rather than housekeeping, the identification scheme itself: why the work identifies its principal source by a source ID plus a content hash instead of a public semantic version, and what a content hash does and does not attest.

The reader should hold one asymmetry in mind throughout. The source basis described here is *versioned* in the strong sense — every controlling artifact is bound by a SHA-256 digest, and the publication delta is bound to an exact repository commit — but it is not thereby *validated* in any sense. Identity discipline establishes that all parties are talking about the same bytes. It establishes nothing about whether those bytes are correct, safe, complete, or authoritative; indeed, two of the monograph's central results are demonstrations that the principal source is internally defective at precisely identified locations. Fixing identity is what makes such demonstrations meaningful: an adverse finding against an unidentified document is an anecdote, while an adverse finding against SRC-PCBA-ENGINEER-20260726 at a cited page anchor is a reproducible check.

2. The engineering baseline

2.1. The artifact. The immediate engineering baseline of CAS PCBA 8 is a single large document: the owner-supplied CAS PCBA Engineer build, `CAS_PCBA_Engineer.pdf`, 743 pages, identified as

Source ID	SRC-PCBA-ENGINEER-20260726
SHA-256	`294320B39FA521EAC086E99E2D9DB913E2B6CFF0754FF82DB1EF966424566274`
Extent	743 pages (PDF)
Role	Engineering reference baseline

The document is an engineering reference: it contains the CAS circuit description language (CCDL) exposition and its consolidated grammar, the five published reference boards, the component and validator prose, the appendix material, and the design narratives that the candidate implementation and the source audit take as their object. Two properties of the baseline shape everything downstream. First, it is *large*: at 743 pages, no reader or reviewer holds it whole in mind, so page-anchored citation and mechanical audit are not conveniences

but the only available rigor. Second, it is *closed as supplied*: the candidate treats the build as a fixed input identified by its hash, and does not amend, patch, or re-render it. Where the audit finds the baseline internally inconsistent — and it does — the inconsistency is preserved and formalized against the fixed bytes, never repaired in place. A repaired design would be a new artifact with a new identity, a rule stated in the audit and honored throughout this monograph.

2.2. The dual-key identification. The baseline is identified by a *pair*: a human-legible source ID, SRC-PCBA-ENGINEER-20260726, and a machine-checkable content hash. The two keys do different work. The source ID is a stable name for discourse — it appears in claim records, review matrices, and prose, and it encodes the supply date of the build (2026-07-26) for human orientation. The hash is the actual identity: any party holding a file alleged to be the baseline can compute its SHA-256 and compare all sixty-four hex digits against the value above, and agreement identifies the bytes with overwhelming confidence while any disagreement — substitution, truncation, re-rendering, silent edit — is certain to change the digest. The source ID without the hash would be a label anyone could reuse for different bytes; the hash without the source ID would be unreadable in prose. The pair is the identification scheme, and the monograph never cites the baseline by either key alone in a load-bearing context.

2.3. The dual page-anchor convention. Citations into the baseline use a dual anchor, and the convention is fixed here because every audit finding depends on it. The baseline, like most large typeset books, has two pagination systems: the *PDF page*, counted from the first sheet of the file, and the *book page*, the printed folio, which lags the PDF count by the extent of the front matter. The audit records both wherever both are meaningful — for example, the reserved Part-IV statement leaders appear at PDF p. 578, book p. 562; the erratum language for missing appendix forms at PDF pp. 591–592, book pp. 575–576; the consolidated structural EBNF at PDF pp. 598–599, book pp. 582–583; the Chapter 18 safety-board listing at PDF p. 342, book p. 326; its Appendix-D repetition at PDF p. 608, book p. 592; and the Appendix-D note on the infinite-gain idealization at PDF p. 610, book p. 594. The monograph's own citations use the PDF anchor as primary — it is the anchor a party holding the hashed file can check without interpreting folios — and repeat the book anchor where the audit supplies it. Both anchors, again, are defined only against the hashed build: a revised edition would renumber both systems independently, which is one more reason the pin of Section 2.2 precedes everything else.

3. Why not a semantic version

3.1. The internal-label problem. The natural bureaucratic move for a 743-page evolving engineering book would be to assign it a public semantic version — "v8.1," "2026.3" — and cite that. The candidate deliberately refuses this move, for a reason recorded in the Gate 4 audit: *the baseline's own latest internal changelog label predates the supplied 2026-07-26 build.* The document's self-description of its version history, in other words, does not extend to the state actually supplied; content exists in the build that postdates the last internally recorded version label. Under that condition, any public semantic version the candidate might attach would be an invention — a version fact fabricated by the citing party and attributed, implicitly, to the source. There would then be two authorities for the baseline's version (its own changelog and the candidate's label), disagreeing silently, with the disagreement invisible to every downstream reader.

The refusal generalizes into a rule the monograph applies to all source facts:

RULE 3.1 (No invented source facts). The candidate does not assert, about a source, version facts, page facts, count facts, or status facts that the source does not itself carry. Where a needed fact is absent or stale in the source — as the version label is here — the candidate substitutes an identification it can itself warrant (a supply-dated source ID plus a computed content hash) and discloses the substitution. Unspecified means unspecified.

3.2. What is lost and what is gained. The cost of refusing a semantic version is real and admitted: semantic versions carry ordering and compatibility conventions — readers infer from "8.1 versus 8.2" a direction of change and a rough promise about its scope — and a hash carries neither. Two hashes are merely different; nothing about `294320B3...` orders it against any other build. The compensation is that everything a hash *does* say is true and checkable, whereas the ordering a fabricated version would say is unwarranted. For a research discipline whose failure mode is precisely the silent promotion of unwarranted statements (Chapter 1), the trade is not close. Ordering among builds, where the work needs it, is recovered honestly: by supply dates in source IDs (which claim only chronology of supply, a fact the candidate can warrant) and by the bound repository history, whose commits are themselves content-addressed and totally ordered by their ancestry.

4. Source-identity methodology: content addressing for large design documents

4.1. The general practice. Identifying an artifact by a cryptographic digest of its content — *content addressing* — is established practice wherever computation must be accountable to exact inputs. Version-control systems address every stored object by content hash, so that a commit identifier fixes an entire tree of sources; package ecosystems pin dependencies by digest so that a build consumes exactly the audited bytes; and the reproducible-builds movement in software distribution (an effort across compiler, packaging, and distribution communities, named here inline as context rather than cited) pushed the discipline end to end, defining a build as reproducible precisely when independent parties, from identified inputs, produce bit-identical outputs — with the comparison performed, of course, by hash. The common structure in all three practices is the one this chapter has already used: a digest converts "the same artifact" from a social claim into a computation, and thereby converts "I checked artifact X" into a statement a third party can re-ground without trusting the checker.

Applying the practice to a large *design document* — rather than to code — is less common and worth a methodological remark, because documents are usually cited loosely (by title, edition, or version label) on the assumption that editions are stable and public. The baseline here is neither: it is a privately supplied, actively evolving build whose internal version labeling has already been shown to lag its content. In that setting the loose-citation assumption fails, and the document must be treated the way a build input is treated: pinned by digest, with every page-anchored citation in the audit and in this monograph implicitly indexed by that digest. A page anchor such as "PDF p. 342" is meaningful *only* against `294320B3...`; against any other build of the book, the anchor is undefined. The audit's page anchors and the monograph's repetitions of them inherit their precision entirely from the pin.

4.2. Hash conservatism. Content addressing is deliberately *over*-sensitive, and the over-sensitivity should be owned rather than hidden. A SHA-256 digest changes under any byte change whatsoever, including changes with no semantic content: re-rendering the same

manuscript through a different PDF producer, updating an embedded timestamp, recompressing an image stream. Two files could, in principle, present identical pages to a reader and carry different digests, and the scheme would treat them as different artifacts. This is the right failure direction for the present purpose. The alternative — any scheme that tries to hash "the content" rather than the bytes — requires a canonicalization step whose own correctness then silently joins the trusted base, and for a 743-page rendered document no such canonicalization is trustworthy. The scheme therefore accepts false distinctions (semantically identical builds counted as distinct) in exchange for excluding false identifications (distinct builds counted as identical), because for an audit trail the second error is fatal and the first is merely inconvenient: a party holding a semantically identical but differently-rendered build simply does not hold the audited artifact, and must obtain it before checking anchors. Conservatism here is a choice with a direction, and the direction is fail-closed.

The same conservatism explains why the basis pins *artifacts* rather than *claims*: a digest over the audit document fixes every finding, anchor, and status literal in it at once, so the monograph never needs to enumerate which sentences of a source it relies on — reliance on any of them is reliance on the digest, and a revised audit would be a visibly new artifact rather than a silently shifted authority.

4.3. What a hash attests — and does not.

Claim Boundary 3.2 (Limits of content addressing). A content hash attests identity of bytes and nothing else. Agreement with a published digest establishes that a party holds the same artifact; it does not establish that the artifact is correct, complete, internally consistent, safe, authorized, or authoritative, and it does not establish who authored it or with what review. In particular, no self-hash — no digest computed by an artifact's own machinery over its own content — confers evidence status, license status, or authority on that artifact; a hash-bearing receipt is exactly as authoritative as the process that produced it, which for locally generated receipts is not at all. This monograph's claim discipline (Section 9) already quarantines authority-from-receipt inferences; this boundary restates the quarantine at the level of the identification scheme itself.

The point is not hypothetical hygiene. The project's history includes historical AI-authored, self-hashed "licensing receipts," and the governing documents are explicit that these are not an authenticated reviewer or legal authority. The identification scheme of this chapter is therefore deliberately *austere*: hashes bind discourse to bytes; all authority questions are answered elsewhere, by the authority matrix (Section 5) and the claim records, and the answer at the bound state is uniformly negative.

5. The baseline's own disclaimer and the Phase-0 authority matrix

Identification says what the baseline is; authority says what it may be used to support. On authority, three independent instruments agree, and their agreement is a fact of the source basis rather than an interpretation of this monograph.

First, *the book's own disclaimer*: the baseline presents itself as an engineering reference and disclaims, in its own front matter, the statuses a careless reader might project onto a confident 743-page technical book. Second, *the Phase-0 authority matrix*: the project's authority-mapping instrument, which assigns to each artifact class the authorities it carries, denies to the baseline both *safety certification* and *physical telemetry* — the book certifies no safety property of anything, and it reports no measurement of any physical system. Third, *the candidate itself*: the Gate 4 audit and specification, in constituting the baseline as their

engineering input, restate both denials as conditions of use. The alignment is deliberate: the baseline is admitted *as design text* — as the object the grammar audit parses, the board listings quote, and the known-fault fixture is drawn from — and under no instrument is it admitted as evidence of safety, measurement, certification, or empirical validity.

It is worth pausing on why the denials must be *institutional* rather than merely sincere. A 743-page book with circuit diagrams, validator prose, and a chapter titled around safety generates, by its sheer weight and genre, a presumption of authority in casual readers that no front-matter sentence reliably dispels; and the documented history of this project includes exactly the failure the matrix guards against — generated artifacts (self-hashed receipts) whose form imitated authority their process never had. An authority matrix answers the genre problem structurally: authority becomes a typed property assigned per artifact class by an instrument outside the artifact, so the question "may this document support a safety claim?" has a lookup, not a vibe, for an answer. For the baseline the lookup answers no, twice — no safety certification, no physical telemetry — and the candidate's conditions of use repeat the answer so that no chain of citation, however long, can launder the book's confidence into a warrant it never carried.

This three-way denial does load-bearing work later in the monograph. The Cognitive Safety Board finding (previewed in Section 5.3 and developed in the audit chapters) would be impossible to state cleanly if the baseline carried safety authority: a certified design with an unreachable protective threshold would be a certification failure, a different and graver genre of result. Because the authority matrix already places the baseline outside safety authority, the finding is what it should be: an adverse *source-quality* result about design text, preserved as a known-fault fixture, with no certified system anywhere in the blast radius — and, equally, with no safety assurance anywhere in the candidate, since the same matrix denies that authority to the candidate's own artifacts.

6. The controlling repository artifacts

6.1. The publication delta and its base commit. The monograph describes a candidate whose executable and evidentiary content lives in a repository, and the publication lane is bound to an exact state of it: the Project Coherence Paper 6 delta at base commit `a1cd990109e7561dfd03949bf0e1653461c7ae44` (cited in short form as `a1cd9901`). "Delta" has a precise meaning here: the publication artifacts organize and cite the existing Gate 4 candidate; they do not modify the Gate 4 specifications, registries, or manifests. The implementation surface at the bound state is the `src/cas_pcba_v8/` package, comprising 14 modules; its tested behavior is summarized by the suite identities in the Gate 4 artifact manifest and reported later in the monograph.

6.2. The artifact set. Seven artifacts control the source basis beyond the baseline itself. Their identities and roles, as recorded in the governing seed and its underlying publication delta, are:

Table 1: Controlling repository artifacts at base commit `a1cd9901`.

Artifact and role	SHA-256
`CAS_CORE_V8_CANDIDATE_SPEC_v05.tex` — semantic identity, evidence, and research-boundary kernel	`1E6E91D005C7268D32C3943C2851138B30E2492C7D73877A04B30685CFCA1FC`
`CAS_PCBA_V8_SOURCE_AUDIT_AND_SPEC_v04.tex` — normative Gate 4 engineering candidate: source audit, adverse findings, type system, DRC contract, simulation contract	`3FEC7B921304A3B55DB51132C30B90A6004D15A611EC759906335F2E41450F0`
`PCBA_CCDL_CONSTRUCT_COVERAGE_v0.1.jsonl` — machine-readable grammar/corpus adverse evidence: construct coverage, four direct counterexamples, one bounded listing	`5B797EC67CF289B4CBE8DF1256F07425C3DA13AA8CF2DA03E47DFCE1CB46808`
`GATE4_REVIEW_MATRIX_v0.1.json` — implemented/open decision ledger: 53 review criteria with 20 open at the bound state	`44DD99A847DA742C3EBEBC06D64452A6868421EE12129E789D1CC790DAE41B74`
`GATE4_ARTIFACT_MANIFEST_v0.1.json` — exact artifact and test-suite identities: 54 hash-bound files, three focused-suite test-ID digests	`49102F9BCBAA2345804F5EE3CA708FD6A84E83F8CF5ABB51E6612893A090D57B`
TKS v8 core formal specification — downstream finite-arrow boundary (read-only)	`F740BA05AB7ADB2DCDF55E271016D127E45073B015497DE8982D44A49F0EA2C4`
Noetic Topos 2.0 specification — downstream ambient/resolved evidence boundary (read-only)	`156A5D8C5320F36E81E3B46473BA8A038B35672DCD28839A404D96A3DB1A763A`

6.3. The Gate-4 receipt as a source fact. The artifact manifest's content is itself part of the source basis, and its headline quantities are cited so often in later chapters that they are fixed here once, with their status stated in the same breath. At the bound commit, the integrated Gate 4 receipt (dated 2026-08-04 in the underlying publication delta) records 54 hash-bound artifacts and three green focused suites, whose suite-level test-ID digests are:

Suite	Methods	Suite test-ID SHA-256
CAS Core v8	61	`E2AF5014DE28DD0702211EF32A2757B99B869A13FAFBB0B8D1B18A50A9F8FEF5`
CAS PCBA v8	82	`D5F776C5F15758CD390470B5A542CCA110784DC5EBB9807B6BE41B2C80453470`
CAS-Ohm v1	65	`2E81A685961286D944894A22EB93345B97211B4433B6C71EAF628D84CD8EB1EF`

The combined focused run comprises 208 test methods and 272 parameterized subtests; the receipt further records 11 claim records, all Ring 4 and NOT LICENSED; 53 review criteria with 20 open; four direct EBNF counterexamples; eight CAS-Ohm source conflicts; and an integrated Gate 1 pass. One negative fact in the receipt is as controlling as the positive ones and is repeated wherever the suites are: the repository-wide collector was *not* clean at the bound receipt — it reported four pre-existing package-path namespace collisions after 1,200 collected items — so the focused suites cannot support a repository-wide verification claim, and no chapter of this monograph makes one. All of these numbers are historical, source-bound software receipts consumed by this monograph as source facts; they are not experiments, and this publication lane adds no new runs to them.

6.4. A further read-only mathematical authority. One additional identity belongs in the basis because a later chapter's geometry discussion depends on knowing where its authority does *not* lie. The projector-geometry theorem statements referenced by the candidate are governed solely by the Formal Mathematics Paper 3 record at its own commit, `3c8491db6f5559cf090909d9de0efa03d924f53f`, with source-draft SHA-256 `679E1119CB084D9A429F6FCC2637`
The engineering artifacts of Table 1 carry read-only dependency summaries of that record, not the mathematics itself; in particular, the registered normalized Gram-kernel result requires common rank $d > 0$, and the stronger CAS capacity transfer remains, at that record's own status, a proof gap in review at Ring 4. Nothing in this monograph re-derives, strengthens, or promotes any of it; the identity is recorded here so that the later citation is, like every other, a pointer to exact bytes.

Three remarks on the set as a whole. First, the artifacts form a *layered* basis: the Core v8 specification supplies the semantic and boundary kernel that everything else presupposes; the PCBA audit and specification is the normative engineering document of the candidate itself; the coverage record is the machine-readable half of the grammar finding, deliberately separated from prose so that the adverse evidence is directly consumable by tools; and the review matrix and artifact manifest are the ledgers — what was decided and what exists — against which "at the bound state" statements resolve. Second, the set is *closed for this monograph*: a claim that cannot be grounded in the baseline, these seven artifacts, the governing research paper, or the bound commit's receipts is not made. Third, the two downstream specifications appear in the same table as the candidate's own artifacts but with a different mode of use, developed next.

7. The publication lineage of this monograph

The monograph itself has a documented lineage within the source basis, and stating it closes the loop of self-description. The governing seed is the research paper *CAS PCBA 8: adaptive cognitive circuit engineering with typed, fail-closed design rules and a preserved adverse source audit*, version 0.2, dated 2026-08-07, which supersedes the white paper v0.1; beneath it sits the Project Coherence charter Paper 6 engineering draft v0.1, source-bound to base commit `a1cd990109e7561dfd03949bf0e1653461c7ae44`. The monograph expands the seed to book length; it introduces no fact, hash, page anchor, count, or status not present in the seed, the Paper 6 draft, or the Gate 4 artifacts of Table 1, and where the seed is silent the monograph is silent.

The lineage's status fields are themselves source facts, recorded at the Paper 6 draft and unchanged by anything in this text: authorship is OPEN, with no authorship or submission decision recorded; internal cross-review is OPEN; external specialist review is OPEN; and the submission state is NOT READY FOR SUBMISSION. The Paper 6 draft additionally binds its

assertion discipline to two machine-readable companions — a claim-assertion matrix and an evidence map, with citation identifiers resolving through a citation ledger — and declares that the publication lane modifies no Gate 4 specification, registry, or manifest. The monograph inherits all of it. In particular, the existence of this book-length treatment changes no status: a longer exposition of an unlicensed, unreviewed, Ring-4 candidate is a longer unlicensed, unreviewed, Ring-4 candidate.

8. The audit's method, disclosed at the source level

The adverse findings that later chapters present rest on an audit whose method is bounded, and the bounds are source-basis facts because they determine what kind of evidence the findings are. As recorded in the seed: text extraction over the hashed PDF located candidate pages; every cited page was then rendered from the source PDF and visually inspected, so that no anchor rests on extraction alone; and the grammar comparison is between *published surface syntax* and the *consolidated EBNF* — it is not a parser implementation and not a semantic-validation result. The machine-readable coverage record (`PCBA_CCDL_CONSTRUCT_COVERAGE_v0.1.json`, Table 1) fixes the outcome shape the method can support: four boards are direct syntactic counterexamples to the published grammar; one board is a bounded listing for which no direct syntactic counterexample was established; and the corpus-level conclusion is that the formal EBNF does not cover the published five-board corpus.

Two consequences of the bounds are worth drawing here rather than in the audit chapters, because they are consequences about *evidence class*. First, the method's verdicts are asymmetric in strength: a counterexample is strong (an observed surface form with no producing grammar alternative is a definite non-coverage witness at the cited anchor), while the single non-counterexample verdict is weak by construction ("no direct counterexample was established under a bounded audit" is not "this board conforms"), and the source documents are careful to claim only the weaker form — the corpus-level conformance claim fails *without* overstating the result for the one bounded listing. Second, the method's ceiling is syntactic: even a hypothetical five-for-five conformance result would have said nothing about semantic validity, and the audit accordingly never uses the word. The candidate's own obligations mirror the ceiling: what it demands of any future grammar — an exact construct-coverage matrix, rejection of unsupported forms, canonical round-trips — is precisely the machinery that would raise audits of this kind from bounded surface comparison to checked conformance.

9. Downstream read-only boundaries

Two formal frameworks sit downstream of the candidate, and the source basis fixes the candidate's relationship to each as strictly *read-only*: the candidate consumes their published boundaries and conforms to them; it does not modify them, extend them, or acquire any theorem from them by adjacency.

9.1. TKS v8: the finite-arrow boundary. The TKS v8 core formal specification (hash in Table 1) defines the finite reference kernel into which, under narrow conditions, a PCBA component's behavior may be adapted. The boundary the candidate reads off that specification is the *finite-arrow* discipline: only a certified total, finite, deterministic, stateless, non-stochastic component transition may be represented as a finite arrow of the kernel; typed sequences become semigroupoid paths; only an endomap has powers, so iterated feedback requires a certified endomap. Everything richer — continuous simplex profiles, response curves, instrumented observations, stateful devices, stochastic kernels, memristive

history variables — remains outside the finite arrow and must be handled by distinct types and comparison maps rather than coercion. The adapter that enforces this boundary, and the immutable certificate it issues (binding the exact TKS kernel bytes and profile, the board, component, and transition identities, the tagged carrier, the semantic profile, and the fixed research boundary), are described in their own chapter; what belongs to the source basis is the direction of authority. The kernel's specification is an input the candidate must satisfy, and satisfaction buys exactly one thing: type-correct representation. The adapter carries no automatic CAS semantic theorem, and no property of the kernel — its finiteness, its determinism, its formal pedigree — transfers to the cognitive content of any adapted component.

9.2. Noetic Topos 2.0: the ambient/resolved evidence boundary. The Noetic Topos 2.0 specification (hash in Table 1) supplies the second read-only boundary, this one about evidence. The distinction the candidate inherits is between *ambient* and *resolved* standing for records: raw or partially resolved records may be treated as external evidence associated with the ambient construction, but a marked cover does not make a measurement true, and records do not enter resolved semantics until restriction and gluing under one calibration profile are defined and tested. For the source basis, the inherited discipline lands in one sentence: every observation-like record the candidate produces — test-point observations, measurement chains, trace events — is ambient-side material, and nothing in the candidate performs, or claims to perform, the restriction-and-gluing obligations that would move any record to resolved standing. The boundary is read off the downstream specification and honored; it is not implemented, extended, or discharged here.

10. The Gate-5 preregistration kernel as read-only context

One further document sits in the source basis and requires the most careful statement of all, because its entire function is to *not* be used yet. The Gate-5 preregistration specification is the project's control kernel for future preregistered experimentation — the instrument through which, if the research program ever reaches empirical work, calibration and held-out validation would be registered in advance. Its status at the bound commit is:

Instrument	Gate-5 preregistration specification
Status at bound commit	OPEN / NOT_STARTED / RING 4 / NOT LICENSED / HELD / NOT FROZEN
Authority created	none (no live, scientific, canon, product, license, or disclosure authority)
Relation to this monograph	read-only context; supplies no experiment

Claim Boundary 3.3 (Nothing here promotes Gate 5). The Gate-5 specification supplies no experiment to this monograph, and this monograph performs no act with respect to it. Describing the kernel, citing its status literals, or scheduling future preregistered work through it in the research program does not start it, freeze it, license it, or advance any of its status fields. Each literal in its status line is load-bearing and is reproduced without softening: OPEN (the gate is not passed), NOT_STARTED (no preregistered activity has begun), RING 4 (outermost research ring), NOT LICENSED (no claim licensing exists), HELD (the artifact is held, not released), and NOT FROZEN (the specification itself is not yet fixed — even the instrument's final form remains open). A reader who encounters, anywhere downstream of this monograph, a suggestion that Gate-5 work is underway or that its kernel validates anything in these chapters is encountering a claim this source basis contradicts.

The kernel nonetheless belongs in the source basis rather than in a footnote, for a structural reason. The research program with which the monograph closes is dependency-ordered, and its empirical steps route through Gate 5 by design: the candidate's position is that when calibration and held-out validation happen, they must happen preregistered, under a control instrument that existed — with a known identity and an honest NOT_STARTED status — long before any data did. Recording the kernel now, in its unstarted state, is what makes that future discipline checkable: the preregistration instrument is itself provenance-bound, and no later account can quietly backdate an experimental plan into it. Read-only context today is the commitment device for licensed evidence tomorrow; the asymmetry between those two dates is the entire point.

11. What the source basis excludes

The discipline of Rule 3.1 has a complement: it is equally a source-basis fact that certain things are *not* in the basis, and later chapters lean on these exclusions as often as on the inclusions. Four exclusions bear listing.

First, *no physical artifact*: the basis contains no fabricated board, no instrument chain, no telemetry, and no physical measurement of any kind; the word "board" throughout refers to a typed software object or to a published design listing in the baseline. Second, *no live model or empirical dataset*: no calibrated live system, no held-out evaluation data, and no experimental records exist in the admitted basis; the only quantitative receipts are software-test receipts bound in the Gate 4 manifest. Third, *no external review*: the basis records internal cross-review as open and external specialist review (systems, safety, regulatory, clinical, formal) as open; no artifact in Table 1 has passed independent review, and the review matrix says so explicitly, with 20 of its 53 criteria open at the bound state. Fourth, *no authority*: as Sections 4.3 and 5 establish from two directions, nothing in the basis carries safety certification, measurement authority, claim license, canon status, or disclosure authorization, and every claim record at the bound state is Ring 4 and NOT LICENSED.

12. Summary

The source basis of this monograph is: one 743-page engineering baseline, pinned as SRC-PCBA-ENGINEER-20260726 with SHA-256 `294320B39FA521EAC086E99E2D9DB913E2B6CFF0754FF82DB1EF96642` admitted as design text under a three-way denial of safety and telemetry authority; seven controlling repository artifacts, each hash-bound, layering a semantic kernel, a normative engineering candidate, machine-readable adverse evidence, and two ledgers beneath two read-only downstream boundaries; a publication delta at base commit `a1cd9901` that organizes but does not modify the Gate 4 state; and one deliberately unstarted preregistration kernel recorded in its entirety of status literals precisely so that nothing can later be smuggled into it. The identification scheme is content addressing with disclosed limits: hashes make every citation in the chapters that follow a checkable act, and confer authority on none of them. The baseline is identified, not endorsed; the audit chapters will show, at page-anchored resolution, that the distinction was necessary.

CHAPTER 4

The Type System

1. Design stance

Every later chapter of this monograph — the board model, the design-rule checker, the simulator, the geometry profiles, the TKS adapter, and the adverse source audit — rests on a single decision made at the lowest layer of the candidate: *semantic distinctions are enforced by types, not by convention.* The CAS PCBA 8 type system is the place where that decision is made concrete. It fixes, once, which values may exist at all; everything downstream can then reason about which values may be *connected*, *checked*, *simulated*, or *adapted*, without re-litigating what the values are.

The stance has four components, each of which recurs throughout this chapter.

RULE 4.1 (Exact-type discipline). Every typed record in the candidate accepts only its exact runtime type. Subclasses are rejected at construction and again at every revalidation. A value that merely *behaves like* a state literal, a port specification, or a calibration reference is not one; the candidate refuses to consume it.

RULE 4.2 (Fail-closed construction). Construction validates; revalidation re-validates. A record that cannot prove its own well-formedness at the moment it is touched raises a typed error rather than degrading into a permissive default. No field is ever filled in silently: an absent Drive is absent, an unknown unit is an error, an unsupported value type has no canonical identity and therefore cannot enter a trace.

RULE 4.3 (Identity from structure). Wherever a value participates in evidence — board digests, trace events, measurement records, geometry profiles — its identity is a canonical, versioned, deterministic serialization of its complete typed structure, hashed with SHA-256. Two values are the same exactly when their canonical payloads are byte-identical. There is no ambient notion of equality that can be substituted for this one.

RULE 4.4 (Boundary in the types). The research boundary — Ring 4, OPEN, NOT LICENSED, HELD — is itself a type. Artifacts do not carry a mutable status field that a caller could upgrade; they carry a value of a type whose only inhabitant *is* the fixed boundary, rederived and rechecked by every public operation.

Nothing in this chapter is a claim about cognition, about any deployed model, or about physical electronics. The type system is a software contract. Where a type carries a CAS name (Noetic, Domain, Drive, Register), the name records which CAS coordinate the type is *intended* to represent inside the research candidate; it does not assert that the coordinate is realized in any external system. That boundary is restated formally in Section 18.

2. The four semantic factors

The candidate models the CAS state space as a product of finite coordinate sets. Four enumerated factor types are declared, and — deliberately — nothing else is assumed about

their interaction. In particular, the implementation's own module documentation is explicit that the factors are *not* called geometrically orthogonal: no orthogonality theorem is assumed by the type layer, and none is imported from the source book.

Definition 4.5 (Noetic factor). Noetic is a ten-element enumeration carried on the integers 0 through 9: IDEA (0), MIND (1), POSITIVE (2), NEGATIVE (3), VIBRATION (4), FEMALE (5), MALE (6), RHYTHM (7), ABOVE-CAUSE (8), BELOW-EFFECT (9).

Definition 4.6 (Domain factor). Domain is a four-element enumeration carried on the strings `P`, `C`, `E`, `O` (Principled, Cognitive, Evaluative, Operational). Each domain exposes a total precedence rank $P \mapsto 0$, $C \mapsto 1$, $E \mapsto 2$, $O \mapsto 3$, encoding the structural order $P \succ C \succ E \succ O$ as data available to the checker; the type layer itself attaches no descent semantics to the rank.

Definition 4.7 (Drive factor). Drive is a seven-element enumeration carried on the strings `F1` through `F7`.

Definition 4.8 (Register factor). Register is a four-element enumeration carried on the strings `T` (Transcendent), `A` (Analytical), `f` (Affective), and `E` (Embodied). The lowercase `f` is deliberate and canonical; it prevents a case-folding collision with the Embodied register `E` and with the Evaluative domain `E`, which lives in a different enumeration entirely.

Remark 4.9 (Domain `E` versus Register `E`). The string `E` appears as a carrier value in both Domain and Register. Under the exact-type discipline of Rule 4.1 this is harmless: the two values inhabit disjoint enumeration types and no operation in the candidate accepts a bare string where a factor is required. The collision would be hazardous only in a stringly-typed design — which is precisely the design the candidate refuses. This is a small, early illustration of why the type system exists.

3. State literals

Definition 4.10 (State literal). A state literal is an element of

$$\text{StateLiteral} = \text{Noetic} \times \text{Domain} \times \text{Drive}_{\text{opt}} \times \text{Register}_{\text{opt}},$$

where $\text{Drive}_{\text{opt}} = \text{Drive} \sqcup \{\bot\}$ and $\text{Register}_{\text{opt}} = \text{Register} \sqcup \{\bot\}$ adjoin an explicit absence marker. In the implementation the type is the frozen record `CasState` with fields `noetic`, `domain`, `drive` (optional), and `register` (optional).

Three points about this definition carry analytical weight.

First, *optionality is explicit, not defaulted.* The implementation's own docstring states the contract: Drive and Register are optional because the inherited notation permits partial literals, and "their absence is explicit and never filled by a default or inferred from another state." A literal with no Drive is a different mathematical object from a literal with any particular Drive, and the candidate never manufactures the latter from the former. This matters because the source notation (`^n^D`, `^n^D_{Fk}`, `^n^D_{Fk}|R`) treats the shorter forms as legitimate partial specifications, not as abbreviations of some canonical completion. A type system that auto-completed them would be inventing semantics the source does not license.

Second, *construction and revalidation enforce factor exactness.* Each field must be an exact member of its factor enumeration; `None` is admitted only for the two optional fields. A string `"P"` in the domain position, an integer `3` in the noetic position, or a subclass of any factor type is rejected with a typed error.

Third, *the literal has a canonical rendering.* The record exposes a rendering of the form `^3^P`, `^3^P_{F2}`, or `^3^P_{F2}|A`, produced only after revalidation. The rendering is derived output, never an alternative identity: equality and hashing are structural, over the typed fields, and the canonical trace identity of a state literal (Section 10) serializes the fields individually rather than the rendered string. A rendered string can be regenerated from the fields; the fields cannot be safely regenerated from an arbitrary string without a parser, and the v0.1 candidate deliberately declares no parser (Chapter 10 records why).

4. ND-cells and the forgetful projection

Definition 4.11 (ND-cell). An ND-cell is an element of

$$\mathrm{NDCell} = \mathrm{Noetic} \times \mathrm{Domain},$$

implemented as the frozen record `NdCell` with typed fields `domain` and `noetic` and a derived label of the form `P0`, `C7`, `O9`. The forty canonical cells are fixed in the order $P0, \ldots, P9$, $C0, \ldots, C9$, $E0, \ldots, E9$, O exported as an immutable tuple together with the parallel tuple of canonical labels.

The canonical order is load-bearing: it is the coordinate order of every simplex address (Section 5), the row discipline of every geometry identity payload, and the order against which the mixed-simplex validator checks incoming cell tuples. A permutation of the forty cells is not "the same cells in a different order"; it is a malformed address. Table 1 fixes the correspondence between coordinate index, label, and factor pair once, for reference throughout the monograph.

Table 1: The forty canonical ND-cells in canonical order. Index k is the coordinate position used by every simplex address and geometry payload.

k	Label	Factors	k	Label	Factors
1	`P0`	(Idea, Principled)	21	`E0`	(Idea, Evaluative)
2	`P1`	(Mind, Principled)	22	`E1`	(Mind, Evaluative)
3	`P2`	(Positive, Principled)	23	`E2`	(Positive, Evaluative)
4	`P3`	(Negative, Principled)	24	`E3`	(Negative, Evaluative)
5	`P4`	(Vibration, Principled)	25	`E4`	(Vibration, Evaluative)
6	`P5`	(Female, Principled)	26	`E5`	(Female, Evaluative)
7	`P6`	(Male, Principled)	27	`E6`	(Male, Evaluative)
8	`P7`	(Rhythm, Principled)	28	`E7`	(Rhythm, Evaluative)
9	`P8`	(Above/Cause, Principled)	29	`E8`	(Above/Cause, Evaluative)
10	`P9`	(Below/Effect, Principled)	30	`E9`	(Below/Effect, Evaluative)
11	`C0`	(Idea, Cognitive)	31	`O0`	(Idea, Operational)
12	`C1`	(Mind, Cognitive)	32	`O1`	(Mind, Operational)
13	`C2`	(Positive, Cognitive)	33	`O2`	(Positive, Operational)
14	`C3`	(Negative, Cognitive)	34	`O3`	(Negative, Operational)
15	`C4`	(Vibration, Cognitive)	35	`O4`	(Vibration, Operational)
16	`C5`	(Female, Cognitive)	36	`O5`	(Female, Operational)
17	`C6`	(Male, Cognitive)	37	`O6`	(Male, Operational)
18	`C7`	(Rhythm, Cognitive)	38	`O7`	(Rhythm, Operational)
19	`C8`	(Above/Cause, Cognitive)	39	`O8`	(Above/Cause, Operational)

20	C9	(Below/Effect, Cognitive)	40	O9	(Below/Effect, Operational)

DEFINITION 4.12 (Forgetful projection). The projection

$$\pi : \text{StateLiteral} \longrightarrow \text{NDCell}, \qquad \pi(n, d, f, r) = (n, d)$$

forgets the Drive and Register coordinates. In the implementation it is the method `to_nd_cell`, whose docstring states that it projects "explicitly forgetting two axes."

PROPOSITION 4.13 (The projection is many-to-one and not an isomorphism). *Each fiber* $\pi^{-1}(c)$ *over a canonical cell* c *has cardinality* $(7+1)\cdot(4+1) = 40$*, so*

$$|\text{StateLiteral}| = 40 \cdot 40 = 1600, \qquad |\text{NDCell}| = 40.$$

In particular π *is surjective but not injective, and no map* $s : \text{NDCell} \to \text{StateLiteral}$ *satisfies* $s \circ \pi = \mathrm{id}_{\text{StateLiteral}}$.

PROOF. Fix a cell $c = (n, d)$. A literal in the fiber is determined by its Drive coordinate, ranging over the seven drives together with the explicit absence $\perp$, and its Register coordinate, ranging over the four registers together with $\perp$: $8 \cdot 5 = 40$ choices, all realized and pairwise distinct by construction of the record type. Surjectivity is witnessed by the partial literal $(n, d, \perp, \perp)$. Injectivity fails because any fiber contains more than one element; a left inverse s with $s \circ \pi = \mathrm{id}$ would force injectivity of π. □

4.1. Why forgetting Drive and Register matters analytically. Proposition 4.13 is elementary as counting; its consequences are not. The candidate keeps both types — and keeps the projection explicit — because three distinct analytical hazards arise when the collapse is performed silently.

First hazard: aggregation is a modeling decision, not a formality. A simplex address (next section) distributes mass over the forty ND-cells. When a state literal carrying a Drive and a Register is scored into that address, forty distinct literals land in the same coordinate. Whatever distinguished analytical work the Drive coordinate was doing — which motivational architecture the state participates in — and whatever the Register coordinate was doing — which operational character the state manifests under — is unrecoverable from the address. Under Correction 2 of the framework's own mathematical discipline (prerequisite satisfaction is one-directional; a retraction is not an inverse), this is exactly the situation of a section-retraction failure: one can embed a cell into a canonical partial literal and project back to the same cell, but one cannot project a full literal to a cell and re-embed without loss. The type system makes the loss visible as a typed operation (`to_nd_cell`) rather than an implicit coercion, so that any analysis pipeline that crossed the projection can be audited for whether it was entitled to.

Second hazard: distinctness claims silently weaken. Two literals $\ell_1 = (\wedge 3, P, F2, A)$ and $\ell_2 = (\wedge 3, P, F6, f)$ are distinct objects with, on the framework's own account, different diagnostic content. Their projections coincide. A report that compared board state bindings at cell resolution and found "the same state on both ports" would be making a claim that is true of the projections and false of the literals. Because the candidate's canonical trace identity for a state value serializes all four fields (Section 10), identity-level evidence never passes through the projection; only an analysis that explicitly chose cell resolution can conflate the two, and that choice is then legible in the analysis, not buried in the types.

Third hazard: the inverse direction invites fabrication. Since no section of π exists (Proposition 4.13), any procedure that produces a full literal from cell-resolution data has *added*

information — a Drive attribution and a Register attribution — that the data did not contain. In an evidence-bearing pipeline that addition must be declared as a modeling step with its own provenance, or it is fabrication. The type system cannot prevent an analyst from writing such a procedure, but it can and does refuse to provide one: there is no "default drive," no "neutral register," and no constructor that lifts a cell to a literal by fiat. The only canonical object available at cell resolution is the cell itself.

REMARK 4.14 (Relation to the register-multiplicity discipline). The v7.7 framework layer distinguishes register-set specification, co-presence, and cross-register integration precisely because collapsing them destroys diagnostic content. The type layer of this candidate sits below that discipline — a single `CasState` carries at most one register — but the same conservative instinct governs both: where the notation distinguishes, the types distinguish, and any collapse is an explicit, auditable act. Multi-register expressions are the business of the notation and its (future) parser, not of the state-literal record; the record's single optional register is the correct target for one register-specified primitive, not a claim that richer expressions do not exist.

5. Simplex addresses

DEFINITION 4.15 (SimplexAddress40). A $\text{SimplexAddress}_{40}$ is a strictly positive normalized profile over the forty canonical ND-cells:

$$p \in \mathbb{R}^{40}, \qquad p_k > 0 \text{ for all } k, \qquad \sum_{k=1}^{40} p_k = 1.$$

Equivalently, p is a point of the *open* 39-dimensional probability simplex

$$\Delta^{\circ}_{39} = \Big\{ p \in \mathbb{R}^{40} \ : \ p_k > 0, \ \textstyle\sum_k p_k = 1 \Big\},$$

with coordinates indexed in the canonical cell order of Definition 4.11. The implementation is the frozen record `MixedSimplexAddress` with exactly forty finite, strictly positive `Decimal` weights whose *exact* sum is 1, cells fixed to the canonical tuple, and a fixed profile discriminator `CANONICAL_MIXED_SIMPLEX_BASELINE` under the versioned profile name `cas-pcba-v8.mixed-simplex.v1`.

5.1. Why strict positivity. The open simplex — rather than the closed simplex $\overline{\Delta}_{39}$, which admits zero coordinates — is a deliberate and threefold choice.

Geometric reason. The Fisher–Rao metric, which Chapter 6 develops as the baseline geometry, has coordinate expression

$$g_p(u, v) = \sum_k \frac{u_k v_k}{p_k}$$

on the tangent space at p. The expression degenerates as any $p_k \to 0$: the metric coefficients blow up, the closed simplex is not a Riemannian manifold under this metric, and geodesics can reach the boundary only in a limiting sense. Restricting the type to the open simplex is what makes "the geometry profile" a coherent phrase at all: every value the type admits is a point where the metric is defined and smooth [**25, 26**]. The type system thus enforces, at construction time, the precondition of every downstream geometric computation, instead of letting each computation rediscover boundary pathologies at run time.

Semantic reason. A zero coordinate is not a small quantity; it is a categorical assertion — *this cell carries no mass whatsoever.* That is a different and much stronger claim type

than a profile that concentrates mass elsewhere, and it is a claim the candidate's evidence discipline has no basis to admit: no measurement in the admitted evidence distinguishes "vanishingly small" from "structurally absent." By excluding zeros the type refuses to let an address smuggle in an impossibility claim. Analysts who need absence claims must make them in a different vocabulary (the framework's lacunarity analysis), where they can carry their own evidence burden.

Numerical reason. Downstream computations divide by coordinates and take logarithms and square roots of them. Strict positivity, validated strictly at construction and at every revalidation, means no downstream profile computation needs a guard against division by zero on data that passed the type. The validation is exact, not tolerance-based: weights are `Decimal` values, positivity is exact comparison, and the sum constraint is checked with an exact aligned-integer summation (Section 13) rather than a floating tolerance. Distinct questions — "is this a well-formed address?" and "are these two addresses numerically distinguishable?" — get distinct answers from distinct machinery; the second is the pseudometric disclosure of Chapter 6.

5.2. Exact normalization. The type provides one constructor from unnormalized data: `from_positive_weights` accepts exactly forty finite, strictly positive `Decimal` values, divides by their exact sum under the fixed decimal context, and then repairs the residual: the exact sum of the normalized weights is recomputed, the deviation from exactly 1 is computed by exact arithmetic, and that residual is added to a deterministically chosen anchor coordinate (the largest input weight, ties broken by lowest index). If the repair would drive the anchor coordinate nonpositive, construction fails rather than admitting a boundary point. The result is an address whose weights sum to `Decimal("1")` *exactly* — not to within an epsilon — so the sum constraint in Definition 4.15 is a checkable identity, not an approximation, and the constructor is deterministic: the same input tuple yields the same address and the same identity digest on every run.

5.3. What a simplex address is not.

Rule 4.16 (Not one-hot). A one-hot vector — mass 1 on one cell, 0 elsewhere — is a vertex of the closed simplex and is excluded from Δ°_{39} by strict positivity. A simplex address therefore can never assert "the system is in cell c." The object that names a single cell is `NdCell` itself; the object that names a single full state is `CasState`. Conflating a sharply concentrated address with a definite cell is a category error the type system makes unrepresentable rather than merely inadvisable.

Rule 4.17 (Not a finite TKS kernel element). The TKS v8 finite reference kernel traffics in finite carriers: tagged finite values, finite transition tables, finite arrows. A simplex address has an uncountable ambient type and continuous structure; it is not an element of any finite carrier, and the adapter boundary (Chapter 9, and the boundary discipline restated in the seed specification) rejects any attempt to coerce one into a finite arrow. The signal-kind vocabulary of Section 6 keeps the two worlds apart at the wiring level: `MIXED_SIMPLEX_ADDRESS` and `FINITE_STATE` are distinct signal kinds with disjoint admissible units, so a net cannot silently deliver a continuous profile to a port that expects a finite state.

Remark 4.18 (Identity of an address). Every address exposes a canonical identity payload — profile name, profile kind, the decimal-context digest, the forty canonical labels in order, and the forty weights as tagged decimal strings — and a SHA-256 over that payload. Two addresses are evidentially the same exactly when these digests agree. This is the

exact-identity discipline that Chapter 6 contrasts with the binary64 pseudometric: the digest separates all distinct addresses; the floating-point distance does not promise to.

6. Ports: direction, signal kind, unit

Ports are where the type system meets wiring. A port specification is a frozen record with five typed fields.

Definition 4.19 (Port specification). A `PortSpec` consists of: a reference string `ref` conforming to the safe-reference profile (Section 11); a `direction` in the two-element enumeration {INPUT, OUTPUT}; a `signal_kind` in a nine-element enumeration; a `unit` in a nine-element enumeration; and a boolean `required` (default true).

The signal kinds are: ACTIVATION, DECISION, CAS_STATE, FINITE_STATE, MIXED_SIMPLEX_ADDRESS, PROJECTOR_PROFILE, TEXT, CLOCK, and SCALAR. The units are: NONE, ACTIVATION_0_1, BOOLEAN, PROBABILITY, NORMALIZED_0_10, CAS_OHM_SCORE, VOLT, SECOND, and HERTZ.

Rule 4.20 (Closed signal-kind/unit admissibility). Admissible pairings of signal kind and unit form a closed, immutable table (Table 2). The admissibility predicate returns false — it does not raise, and it does not guess — for any pair involving a value that is not an exact member of the respective enumeration.

Table 2: Closed signal-kind/unit admissibility table.

Signal kind	Admissible units
ACTIVATION	ACTIVATION_0_1
DECISION	BOOLEAN
CAS_STATE	NONE
FINITE_STATE	NONE
MIXED_SIMPLEX_ADDRESS	PROBABILITY
PROJECTOR_PROFILE	NONE
TEXT	NONE
CLOCK	HERTZ
SCALAR	NONE, PROBABILITY, NORMALIZED_0_10, CAS_OHM_SCORE, VOLT, SECOND, HERTZ

The table encodes a semantic commitment per row. An activation is a bounded intensity in $[0, 1]$ and nothing else; a decision is a boolean and nothing else; the four "carrier-typed" kinds (CAS state, finite state, projector profile, text) are unitless because their values are structured identities rather than magnitudes; a mixed-simplex address is probability-typed because its coordinates are exactly a normalized profile; a clock is a rate. Only the deliberately generic SCALAR kind admits a menu of units, and it is correspondingly excluded from every semantically loaded role: no reference component contract in the closed catalog exposes a scalar port where an activation is meant, so a scalar can never impersonate an activation merely by carrying a compatible number.

Two remarks on this table. First, the presence of VOLT among the scalar units is a naming admission, not a physics claim: the unit exists so that source-derived example boards can be transcribed faithfully, and the research boundary (Section 18) denies physical telemetry

regardless of unit names. A scalar labeled volts is a scalar labeled volts; nothing in the candidate measures electric potential. Second, the table is deliberately closed: an unsupported pairing is not a warning but a structural impossibility that the DRC reports under its net diagnostics. Extending the table is a versioned change to the engine identity, visible in every downstream digest.

7. Component kinds and typed parameter contracts

A component on a board names a *kind*; the kind resolves, through a closed catalog, to a *contract* that fixes what the component is allowed to look like.

DEFINITION 4.21 (Parameter rule). A `ParameterRule` declares, for one named parameter: a value type (DECIMAL or STRING); whether the parameter is required; optional finite `Decimal` bounds `minimum` and `maximum` (with an exclusive-maximum flag); and, for string parameters, a closed set of admissible choices, each of which must itself conform to the safe-reference profile. Construction rejects a minimum exceeding a maximum, non-finite bounds, and untyped flags.

DEFINITION 4.22 (Component contract). A `ComponentContract` binds: a kind name and a versioned semantic profile string; a tuple of `PortSpec` values; an immutable mapping from parameter names to `ParameterRule` values; a set of required state-binding names; and three booleans — `deterministic`, `stateful`, and `tks_finite_adapter_eligible` — which downstream stages consult before simulating or adapting the component.

The default catalog in the v0.1 implementation is small and closed. Its kinds are: `pass` (identity on one activation), `scale` (activation scaling with a nonnegative decimal `factor` and an activation-level `ceiling`), `cjt` (the cognitive-junction reference: two activation inputs `cause` and `bias`, one activation output `effect`; parameters `beta` (nonnegative finite decimal), `saturation`, and `cutoff` (both activation-level); required state bindings `cause`, `bias`, `effect`), `decision_threshold` (activation in, boolean decision out, with an activation-level `threshold`), `mux` (two activation inputs selected by a decision input), `fuse` (activation in and out, with an activation-level `threshold` and a string `action` restricted to the closed choice set `permanent_block` or `warn`; stateful, because a permanently blocking fuse carries run-local open/closed state), `delay` (the unit delay, stateful, with an activation-level `initial`), and `finite_transition` (finite state in and out, the only kind eligible for the TKS finite adapter). The catalog as a whole has a canonical digest that enters the engine identity and hence every board identity (Chapter 5); adding a kind, changing a bound, or widening a choice set is a visible engineering event, not a quiet edit.

Two consequences deserve emphasis. The `beta` parameter of the reference junction is typed as a *finite* nonnegative decimal: the source book's idealized infinite gain (the $\beta = \infty$ of the published Cognitive Safety Board, Chapter 11) is unrepresentable in the candidate's parameter types, which converts a documented source inconsistency into a construction-time impossibility. And the `fuse` kind's `action` parameter is a closed string choice, not free text: there is no way to declare a fuse whose action the simulator would have to guess at.

8. Nets, endpoints, and driver/consumer discipline

DEFINITION 4.23 (Endpoint and net). An `Endpoint` is a pair of reference strings (`owner_ref`, `port_ref`); the distinguished owner sentinel `$board` denotes the board's own external port surface and is the only reference exempt from the leading-alphanumeric rule of the safe-reference profile. A `Net` is a reference string, a tuple of endpoints, and a boolean `safety_critical` flag.

At the type layer a net is deliberately dumb: it is a set of typed attachment points, nothing more. The intelligence lives in the design-rule checker (Chapter 7), which resolves every endpoint to a declared owner and port, requires exactly one driver per net, and enforces signal-kind and unit agreement between the driver and every consumer under the closed table of Rule 4.20. The type layer's contribution is to make the checker's job well-posed: because endpoints are exact typed records with validated references, the checker's questions ("does this owner exist?", "does this port exist on that owner?", "what kind and unit does it carry?") have deterministic answers, and because nets are immutable, the answers cannot change between the check and the use of its report. The driver/consumer rules themselves — *one* driving output per net, at least one load, direction compatibility, no multiply-connected port, required inputs connected or defaulted in the selected simulator profile — are stated and itemized as `NET`-class diagnostics in Chapter 7; here they are named only to record that every rule is expressible as a predicate over the exact types of this chapter, with no reflection, no string parsing, and no appeal to convention.

9. Test points and calibration references

DEFINITION 4.24 (Test point). A `TestPoint` names a reference, the net it observes (`net_ref`), an optional `calibration_ref`, and a boolean `safety_critical` flag.

DEFINITION 4.25 (Calibration reference). A `CalibrationRef` carries: reference and version strings; a `profile_sha256` in exact uppercase 64-hex form; a typed `measurement_kind` drawn from the six-element enumeration DESIGN_VALUE, BEHAVIORAL_PROXY, INSTRUMENTED, HUMAN_SCORE, LLM_SCORE, MODEL_DERIVED; a typed `unit`; instrument and protocol identity strings; a finite `Decimal` `uncertainty`; and a boolean `packet_eligible`, default false.

The design intent is to make the *shape* of measurement honesty a type. A test point is a declared observation site, not an observation; a calibration reference is a declared pointer to calibration metadata, not evidence that calibration occurred. The fields force every calibration claim to name its instrument identity, protocol identity, uncertainty, and measurement kind at declaration time — so the later question "what would it take for this observation to count?" has a typed answer — while the `packet_eligible` flag records the candidate's standing rule that locally constructed calibration records are structural, test-only objects: nothing a board author writes can make a calibration reference Evidence-Packet eligible, because eligibility is granted by an authentication process outside the type system, never by the record itself. The measurement-kind enumeration also encodes one substantive prohibition enforced at simulation intake: a board *input* sample may not claim MODEL_DERIVED provenance, which blocks the circular move of feeding a model-derived quantity back in as if it were fresh external data.

10. Deterministic trace and measurement identity

Everything the simulator emits is identity-bearing. The type layer provides the vocabulary in three parts.

Canonical value identities. Every value that can flow through a port or appear in a trace has exactly one canonical tagged JSON identity under the versioned profile `cas-pcba-v8.trace-value.v1`: finite decimals (tagged with the decimal-context digest), tagged finite atoms (Section 12), full CAS state literals (all four fields serialized, including explicit nulls for absent Drive or Register), mixed-simplex addresses (by their complete identity payload and its digest), and projector profiles (likewise, with the matrix digest linked). Any other runtime type has *no*

canonical identity and is rejected; "unsupported value" is an error, never a best-effort string. A canonical identity round-trips: the serialized text must re-canonicalize to itself byte-for-byte, and its digest is the SHA-256 of that canonical text. Port validation is performed against the serialized identity, per signal kind: an activation port's value must be a decimal identity in $[0, 1]$; a decision port's must be an exact boolean atom; a mixed-simplex port's must be a mixed-simplex identity; and so on — so a value's fitness for a port is checked on the same bytes that enter the evidence, not on a live object that could differ from what was recorded.

Measurement records. A `MeasurementRecord` (profile `cas-pcba-v8.measurement-record.v2`) binds: an identity-scope digest tying the record to one exact run scope; a non-negative integer tick; producer and port references; the canonical value text and its digest; typed unit and measurement kind; an optional calibration reference; the `packet_eligible` flag, fixed false at creation; an ordered tuple of *child measurement identities*; and the fixed research boundary. The record's own `measurement_id` is the SHA-256 of its complete canonical payload — so a measurement cannot be edited, re-parented, re-ticked, or re-valued without changing its identity, and the child tuple gives each derived measurement an explicit, acyclic ancestry that the result constructor validates (children must exist; derivation must be acyclic; ticks and trace linkage must agree). The discipline resembles the entity/activity/derivation separation of the W3C provenance model [22], and the candidate is explicit that the resemblance is methodological only: no PROV-O conformance is claimed.

Trace events and results. Trace events and simulation results carry their own versioned profiles (`cas-pcba-v8.trace-event.v2`, `cas-pcba-v8.simulation-result.v2`) and bind the exact DRC report identity, engine and simulator profiles, component catalog and decimal-context digests, source-bundle digest, and research boundary, as Chapter 8 details. What matters at the type layer is uniformity: every one of those bindings is a typed field holding a validated digest or a validated record, so "the trace says X" is always reducible to "these exact bytes, under this exact profile, hash to this exact identity."

11. Research boundary and source provenance types

Definition 4.26 (Research boundary type). `ResearchBoundary` is a class with *no instance storage* (empty slots). Its constructor validates that every argument equals the fixed research posture — admission ring 4, status OPEN, license status NOT LICENSED, disclosure status HELD, safety certification false, telemetry authorization false, regulatory authority false, and the fixed research claim sentence — and raises otherwise. All accessors are constant properties. Equality holds exactly between values of the exact type.

The empty-slots design is the point: because instances store nothing, even low-level attribute forcing cannot mutate a boundary value into an "accepted," "licensed," "disclosed," or "certified" state — there is nowhere for the forged field to live. Every public operation that consumes a boundary calls the revalidation function, which checks the exact type and re-reads every property against the fixed posture. The boundary travels inside measurement records, projector identities, and simulation results, so the Ring-4 / NOT LICENSED posture is not a banner on the paper; it is a field in the evidence.

Definition 4.27 (Safe-reference profile). Every reference string in the candidate (component, port, net, test-point, calibration, feedback, profile, basis, and source identifiers) must match the versioned profile `cas-pcba-v8.safe-reference-ascii.v1`: ASCII only, first character alphanumeric, up to 128 characters drawn from alphanumerics, dot, underscore, colon,

and hyphen. The validator rejects control characters, bidirectional and formatting characters, and confusable non-ASCII lookalikes.

This is a human-review defense encoded as a type rule: an identifier that will appear in audit reports, diffs, and evidence packets must not be able to spoof another identifier under visual review or reorder text under bidirectional rendering. The sole sanctioned exception is the `$board` endpoint sentinel of Definition 4.23, which is permitted only where an endpoint owner is expected and only by an explicit flag at the call site.

Source provenance. The candidate binds its own implementation identity into everything it produces: a source-bundle digest over the package's Python sources (profile `cas-pcba-v8.python-source-bur` an engine identity payload (profile `cas-pcba-v8.reference-engine.v1`) that nests the source identity, the component-catalog digest, and the complete decimal-context payload with its digest. Board identities embed the engine identity; traces embed both; so every artifact answers not only "what was modeled?" but "which exact software, under which exact arithmetic and catalog, said so?"

12. Tagged exact values

Definition 4.28 (Tagged finite value). A finite carrier atom is exactly one of: a boolean, an integer, or a string. Its canonical identity is a tagged payload under the versioned profile `cas-pcba-v8.tagged-finite-value.v1` carrying the type tag (`bool`, `int`, or `string`) and a canonical value representation — booleans as JSON booleans, integers as their exact decimal string, strings as themselves. Equality of finite values is equality of tagged payloads: type tag *and* value.

Rule 4.29 (Type-pun exclusion). Integer 1, Boolean *true*, and string `"true"` are three distinct values everywhere in the candidate: in DRC evaluation, in execution, in traces, in measurement identities, and across the TKS adapter. No implicit conversion between them exists at any layer.

The hazards this rule blocks are concrete, and each corresponds to a failure mode observed in mainstream ecosystems.

The bool/int pun. In the implementation language, booleans are a subtype of integers: `True == 1` evaluates true, the two hash identically, and a dictionary keyed by `1` silently swallows a later entry keyed by `True`. A finite transition table keyed by untagged atoms could therefore conflate a numeric state 1 with a logical state *true* — merging two rows of the table, changing its size, and corrupting its certificate — without any error being raised. The tagged payload assigns the two atoms different type tags, hence different canonical texts, hence different digests; the kernel-facing `TaggedFiniteValue` token exists precisely so that hash-based containers "whose native equality conflates types" (the implementation's own words) can key on the tag pair instead of the raw atom.

The string-coercion pun. Configuration formats, spreadsheets, and lenient parsers routinely deliver `"true"` where a boolean was meant, or `"1"` where an integer was meant. A permissive layer that coerced these would make a board's semantics depend on which serialization path a parameter happened to travel. In the candidate, a string parameter is a string parameter: the tagged identity of `"true"` shares nothing with the tagged identity of *true*, canonical validation rejects a payload whose tag and value disagree (an `int` tag whose value string is not the exact canonical decimal form; a `bool` tag whose value is not a JSON boolean), and the round-trip check of Section 10 ensures the distinction survives serialization.

The evidence pun. The deepest reason is evidential. Identities are hashes of canonical bytes; if two semantically distinct values could share canonical bytes, the identity system would licence false equalities in the permanent record. Tagging makes the value space injective into the payload space by construction: distinct (tag, value) pairs yield distinct canonical texts. Where floating-point values must enter a canonical payload (board metadata admits them), they are serialized by exact hexadecimal representation with explicit `NaN` and infinity markers — again an injective encoding, chosen so that "same digest" can never paper over "different value."

13. The single immutable decimal context

All decimal arithmetic in the candidate runs under one versioned context profile, `cas-pcba-v8.decimal-con` precision 60; rounding half-even; exponent range $E_{\min} = -999{,}999$ to $E_{\max} = 999{,}999$; clamping off; capitals on; and traps armed for division by zero, invalid operation, and overflow.

Rule 4.30 (Per-operation context creation). The context is created *from primitive constants* for each public operation, used for exactly that operation's dynamic extent, and discarded. No caller-supplied or ambient (thread-local) decimal context is ever consulted; a caller running under precision 3, or under a context with traps disarmed, obtains bit-identical results to a caller running under defaults. Public arithmetic entry points are wrapped so that the fixed context is installed before the operation begins and uninstalled after it completes.

The rule has an evidence corollary. The complete context payload is hashed once, at import, into the constant `DECIMAL_CONTEXT_SHA256`, and that digest enters: every decimal value identity in a trace; every mixed-simplex identity payload; every projector identity payload; every component identity payload; every board identity payload; and the engine identity itself. A future reader of any artifact can therefore verify not only *what* was computed but *under which arithmetic* — and any change to the arithmetic profile, however small, changes every downstream digest, making silent arithmetic drift structurally impossible.

Three trapped conditions replace three silent failure modes: division by zero raises instead of yielding an infinity; invalid operations raise instead of yielding `NaN`; overflow raises instead of saturating. Since the type layer separately excludes non-finite decimals from every record field (weights, uncertainties, bounds, gains must be finite), the combination gives a closed world: no `NaN` or infinity can be constructed by candidate arithmetic, and none can be smuggled in through a field.

Alongside context-governed arithmetic, the candidate provides *context-independent* exact summation: finite decimals are summed by aligning exponents and adding integer coefficients, with no rounding at any precision. This is the operation behind the exact sum-to-one constraint of Definition 4.15 and the exact residual computation of Section 5.2: a constraint checked by exact summation is an identity of integers, not a tolerance judgment.

Remark 4.31 (Relation to IEEE 754). IEEE 754 [**21**] is cited here as the external reference point for explicitly controlled computer arithmetic: reproducible results require a fully specified format, rounding rule, and exception discipline, and the candidate adopts exactly that posture. But the candidate's decimal profile is its *own* software contract, not an instance of an IEEE interchange format: precision 60 is not an IEEE decimal format parameter, the exponent range is the candidate's, and conformance claims to the standard are neither made nor needed. The standard supplies the engineering lesson (specify the arithmetic, or lose determinism); the profile supplies the specification; the digest supplies the proof of which

specification was in force. Where binary64 floating point *is* used — solely inside the disclosed Fisher–Rao pseudometric of Chapter 6 — that use is named, profiled, and bounded, and its limits are part of the disclosure rather than an ambient assumption.

14. Canonical identity is injective

The tagged-value discipline of Section 12 and the canonical serializations of Section 10 exist to support one property, which is worth stating and proving at the level of the declared encodings.

PROPOSITION 4.32 (Injectivity of the finite-atom encoding). *Let $V = \mathbb{B} \sqcup \mathbb{Z} \sqcup \Sigma^*$ be the disjoint union of booleans, integers, and finite strings, and let $e : V \to \Sigma^*$ be the canonical tagged encoding: $e(v)$ is the canonical JSON text of the payload carrying the profile constant, the type tag of the summand v inhabits, and the canonical value representation (JSON boolean; exact decimal integer string; the string itself). Then e is injective: $e(v) = e(w)$ implies $v = w$,* including *across summands.*

PROOF. Suppose $e(v) = e(w)$. Canonical JSON serialization with sorted keys and fixed separators is injective on payload objects, so the payloads of v and w are equal as objects; in particular their type tags agree, so v and w inhabit the same summand. Within a summand, the value representation is injective: JSON booleans distinguish the two booleans; the exact decimal string of an integer determines the integer (the canonical form forbids leading zeros and non-canonical signs, and the validator rejects any value string that does not round-trip through integer parsing to itself); a string represents itself. Hence $v = w$. The cross-summand cases that defeat untagged encodings — integer 1 versus Boolean *true* versus string `"true"`, or integer 1 versus string `"1"` — are separated at the first step, by the type tag. □

COROLLARY 4.33 (Digest separation for finite atoms). *Distinct finite atoms have distinct canonical texts, hence distinct SHA-256 identities up to hash collision. The candidate treats SHA-256 collision resistance as an external assumption, disclosed as such: no claim in this monograph depends on it beyond the standard evidentiary use of content digests.*

The corollary marks a contrast that Chapter 6 develops at length: the *identity* layer of the candidate separates all distinct values by construction (modulo the disclosed hash assumption), while the *numerical geometry* layer explicitly does not promise separation at binary64 resolution. Claims of sameness belong to the identity layer; claims of closeness belong to the geometry layer; the type system keeps the two vocabularies apart so that neither can borrow the other's strength.

15. Measurement kinds as a claim taxonomy

The six-element `MeasurementKind` enumeration of Definition 4.25 is small, but it is the type-level skeleton of the candidate's entire evidence discipline, so its semantics deserve explicit statement.

design_value.: A quantity asserted by the board author as part of the design: a parameter, a fixture input, a declared constant. It carries no measurement claim at all; it is structural metadata. The default measurement kind of every input sample is DESIGN_VALUE, so a fixture that says nothing claims nothing.

behavioral_proxy.: A quantity derived from observed behavior of some system under a declared protocol, offered as a proxy for a modeled quantity. The kind records that the link from observation to quantity is itself a modeling assumption.

instrumented.: A quantity attributed to an instrument chain with declared identity, protocol, and uncertainty. In the v0.1 candidate this kind can be *declared* but never *authenticated*: no instrument chain exists in the admitted evidence, and locally declared instrumented records remain structural/test-only.

human_score, llm_score.: Quantities produced by a human rater or a language-model rater under a declared protocol. The kinds exist so that rater-derived numbers can never be conflated with instrumented ones by type.

model_derived.: A quantity computed by the candidate itself from other quantities. Board *inputs* may not carry this kind (Section 9); it exists for derived, child measurements inside a trace, where the parent linkage documents the derivation.

The enumeration is thus a closed taxonomy of *how a number came to be*, attached at the type level to every number that enters the evidence. The DRC and simulator consume it (calibration compatibility, input admissibility); the Evidence-Packet discipline consumes it (eligibility by kind); and the audit consumes it (a report can be filtered, by type, to exactly the claims of a given evidentiary strength).

16. Worked type-level fragments

The fragments below are *illustrative only*: they are not normative source code, they elide imports and error text, and they exist to make the failure modes of this chapter concrete. Chapter 5 gives the corresponding board-level constructions.

Example 4.34 (Legal state literals). The partial literal $(\wedge 3, P, \bot, \bot)$ — constructed as `CasState(noetic=Noetic.NEGATIVE, domain=Domain.PRINCIPLED)` — is legal, renders as `^3^P`, and projects to the cell `P3` (index 4 in Table 1). The full literal `CasState(Noetic.NEGATIVE, Domain.PRINCIPLED, Drive.F2, Register.ANALYTICAL)` is equally legal, renders as `^3^P_{F2}|A`, and projects to the *same* cell — the worked instance of Proposition 4.13.

Example 4.35 (Illegal state literals). Each of the following is rejected at construction with a typed error, not repaired: `CasState(3, Domain.PRINCIPLED)` (bare integer where a `Noetic` is required); `CasState(Noetic.NEGATIVE, "P")` (bare string where a `Domain` is required); `CasState(Noetic.NEGATIVE, Register.EMBODIED)` (a Register in the Domain position — rejected even though both carry the string `E` family, because the enumerations are disjoint types); and any instance of a subclass of `CasState`, however faithful its fields (Rule 4.1).

Example 4.36 (Simplex boundary rejection). The uniform profile with all forty weights `Decimal("0.025")` is a legal address: weights are strictly positive and their exact sum is `Decimal("1")`. The "one-hot" tuple with weight `1` on `P0` and `0` elsewhere is rejected (zero coordinates); the tuple of forty weights `0.02` and `0.03` alternating is rejected if its exact sum differs from one; a tuple of thirty-nine weights is rejected on arity; and a tuple containing `float 0.025` rather than `Decimal("0.025")` is rejected on value type, because binary64 atoms have no place in an exact address.

Example 4.37 (Type-pun rejection). A finite transition table over carrier $\{0, 1\}$ and a table over carrier $\{false, true\}$ have disjoint tagged carriers and distinct certificates, even though the underlying language would happily evaluate `True == 1` as true. A fixture that attempts to feed the string `"true"` to a decision port fails port validation: the serialized identity has tag `string`, and a decision port requires tag `bool` (Section 10).

Example 4.38 (Ambient-context irrelevance). A caller that sets its thread-local decimal precision to 3 and then normalizes the weight tuple $(1, 1, 1, \ldots, 1)$ (forty ones) obtains the same address, with the same forty weights of `Decimal("0.025")` and the same profile digest, as a caller running under default precision: the normalization runs under the fixed sixty-digit context of Rule 4.30 regardless of ambient state, and the exact-sum check is context-free integer arithmetic. The ambient precision is not consulted, not restored incorrectly, and not recorded — because it never mattered.

17. Invariant summary

Invariant 4.39 (Type-layer invariants). For every value admitted by the type system: (i) its runtime type is exact — no subclass, no coercion; (ii) every factor field is an exact enumeration member, with absence explicit where permitted and nowhere defaulted; (iii) every reference string satisfies the safe-reference profile (with the single audited `$board` exception); (iv) every decimal field is finite, and all decimal arithmetic occurred under the fixed versioned context, whose digest accompanies the value; (v) every finite atom carries its type tag, and no two distinct tagged atoms share a canonical identity; (vi) every simplex address has exactly forty strictly positive weights in canonical cell order with exact sum one; (vii) every evidence-bearing record binds the fixed research boundary and is identified by the SHA-256 of its complete canonical payload; and (viii) revalidation of any record at any later time either reconfirms all of the above or raises — there is no degraded acceptance path.

Threat 4.40 (What the type system does not do). The type system validates form, not truth. A well-typed calibration reference does not evidence calibration; a well-typed state binding does not evidence that any system occupies that state; a well-typed simplex address does not evidence that forty numbers describe anything outside the software. Type-level guarantees are also bounded by the implementation's own correctness: the invariants above are enforced by tested code, not proved code, and the bounded-evidence posture of Chapter 12 applies to them in full.

18. Claim boundary

Claim Boundary 4.41. This chapter specifies software contracts of the CAS PCBA 8 v0.1 research candidate, at Ring 4, NOT LICENSED. The types named after CAS coordinates represent those coordinates inside the candidate only; no claim is made that any human, biological, or artificial system realizes them. Strict positivity, exact summation, tagged identity, and the fixed decimal context are properties of the candidate's arithmetic and records, not of any external quantity. No physical, causal, safety, clinical, regulatory, or product claim is made or licensed by anything in this chapter, and the projection, positivity, and identity results stated here (Propositions and Rules 4.13, 4.16, 4.17, 4.29, 4.30) are statements about declared types and tested code paths, offered for independent review.

CHAPTER 5

The Board Model

1. The board as an evidence object

A *board* is the candidate's unit of design: the complete, immutable, identity-bearing description of one circuit-shaped cognitive-system model. Everything the candidate can do — check, simulate, trace, adapt, audit — it does *to a board*, and everything it reports, it reports *about a board identified by digest*. This chapter walks through every record type that a board contains, field by field, and then through the two disciplines that make the walk worthwhile: the immutability discipline, which guarantees that a board cannot change between the moment it was checked and the moment its check report is relied upon; and the identity discipline, which guarantees that any structural edit — any edit at all — produces a detectably different board.

The design target is worth stating before the field lists. In an evidence-bearing research program, the dangerous artifacts are not the wrong ones but the *ambiguous* ones: a design whose parameters might have been edited after review, a netlist whose meaning depends on element order, a "safety" annotation that might have been added by a later hand. The board model removes ambiguity by construction. A board is a value, not an object with a lifecycle; its identity is a deterministic function of its complete structure; and its research boundary is rederived — not stored, not trusted, not inherited — by every public operation that touches it.

As everywhere in this monograph, the boundary comes first: a board is a model description. Nothing in this chapter asserts that any board describes a real cognitive system, a deployed model, or a physical circuit. Section **??** closes the chapter with the formal statement.

2. The immutability discipline

Every record in the board model obeys the same five mechanical rules, inherited from the type layer of Chapter 4 and worth restating here because the board is where they compound.

Rule 5.1 (Frozen records). Every board-model record is a frozen dataclass with slots: fields are assigned once, at construction, and attribute assignment thereafter raises. There are no setters, no builders that return partially initialized values, and no mutable default fields.

Rule 5.2 (Exact types, no subclasses). Construction and every revalidation check the exact runtime type of the record itself and of every typed field. A subclass of `Net`, `ComponentInstance`, or `Board` — however faithful-looking — is rejected. This closes the classic loophole in which a subclass overrides equality, hashing, or a property to lie about its contents while passing type checks.

Rule 5.3 (Recursive freezing of open-ended containers). The two open-ended containers a component carries — its parameter mapping and its state-binding mapping — are recursively frozen at construction: mappings become immutable mapping proxies over detached

copies, sequences become tuples, sets become frozen sets, and every leaf must be one of the canonically encodable value types (Section 5). A metadata value that cannot be frozen canonically is a construction error, not a tolerated opaque blob. Detachment matters as much as freezing: the frozen copy shares no structure with the caller's original, so a caller who retains and mutates the dictionary it passed in mutates only its own copy.

RULE 5.4 (Revalidation on touch). Every record exposes a revalidation method that rechecks the full well-formedness of the record and, transitively, of every child record. Public operations call it on entry. Immutability makes revalidation *normally* redundant — that is the point — but the candidate does not rely on the language to enforce what it can cheaply re-verify: low-level attribute forcing, pickling round-trips, and hostile or buggy callers are all caught by the re-check rather than trusted away.

RULE 5.5 (Safe references everywhere). Every identifier field in every record — board, component, port, net, test point, calibration, feedback loop — must satisfy the versioned safe-reference profile (ASCII, leading alphanumeric, at most 128 characters of alphanumerics and `. _ : -`), with the single audited exception of the `$board` endpoint sentinel. Identifiers appear in audit reports and evidence packets; the profile exists so that no identifier can visually impersonate another under human review.

3. Components

DEFINITION 5.6 (Component instance). A `ComponentInstance` consists of seven fields: `ref`, `kind`, `semantic_profile`, `parameters`, `state_bindings`, `safety_critical`, and `finite_transition`.

`ref` is the component's board-unique identifier, a safe reference. Uniqueness is not enforced by the record — a lone component cannot know its siblings — but by the design-rule checker, which fails a board with duplicated references under its model-class diagnostics. The division of labor is deliberate: records enforce everything locally checkable; the DRC enforces everything relational.

`kind` names the component's contract in the closed catalog of Chapter 4 (in v0.1: `pass`, `scale`, `cjt`, `decision_threshold`, `mux`, `fuse`, `delay`, `finite_transition`). The record stores the name, not the contract: resolution happens at check time against the exact catalog whose digest is bound into the board identity, so a board checked under one catalog cannot silently be re-interpreted under another. An unknown kind is a semantics-class diagnostic, never a guess.

`semantic_profile` is the versioned name of the component's declared meaning (for example `pcba8.cjt.reference.v0`). It exists so that a later revision of a component's transfer semantics is a *different profile string* — visible in every payload — rather than a silent behavioral change under an unchanged name. The DRC checks that the instance's profile matches the catalog contract's profile for its kind; a mismatch means the board was authored against a different semantics than the engine implements, and that is an error, not a warning.

`parameters` is the recursively frozen mapping of Section 5. The record freezes and detaches it; the DRC later checks it against the kind's typed parameter contract — exact key set, value types, bounds, and choice sets.

`state_bindings` is a frozen mapping from binding names to exact `CasState` literals (Chapter 4). Bindings are the board's semantic annotations: a junction may declare which CAS states its cause, bias, and effect roles are intended to model. The record validates that every bound value is an exact revalidated state literal; the DRC validates the relational part

— that required binding names for the kind are present, that no unknown binding names appear, and that the bias-domain coherence rule holds where the kind demands it.

safety_critical is an exact boolean marking the component as participating in a safety-relevant path. The flag has consequences downstream — safety-marked paths require marked test points and calibration references, and unreachable-threshold analysis attaches to them — but the flag itself is a *declaration*. The candidate's standing rule applies: marking a component safety-critical creates obligations, never credit. A board with safety markings and without the corresponding probes and calibrations fails; a board with them merely has its declarations checked for internal consistency. No configuration of flags constitutes evidence that anything is safe.

finite_transition is either absent or an exact `FiniteTransitionSemantics` record, described next. Only components whose kind is adapter-eligible meaningfully carry one; the TKS adapter of Chapter 9 refuses everything else.

3.1. Finite transition semantics.

Definition 5.7 (Finite transition certificate). A `FiniteTransitionSemantics` record binds: a versioned profile string; a named source carrier (name plus tuple of tagged finite elements); a named target carrier (likewise); a total mapping, given as a tuple of integer indices into the target carrier, one per source element; a `certificate_sha256` over the canonical payload of all of the above; and four booleans fixing the declared character of the transition — `total`, `deterministic`, `stateful`, `stochastic` — of which the first two default true and the last two default false.

Field-level points. The carrier elements are *tagged* finite values, so a source carrier $\{0, 1\}$ and a source carrier $\{\mathit{false}, \mathit{true}\}$ are distinct carriers with distinct certificates (Chapter 4, Rule 4.29); the mapping is by index, so the transition table is meaningful only relative to the exact element order serialized into the certificate; and the certificate is computed by the same canonical-JSON-then-SHA-256 discipline as every other identity in the candidate.

The record's docstring states its own limit, and the audit language of this monograph adopts it verbatim in substance: the digest establishes *exact transition-table identity only*. It is not an external review, not a safety license, and not a claim that the carrier labels have CAS meaning. A certificate answers one question — "is this exactly the table that was certified?" — and no other.

One method deserves special mention because it is a piece of honest test infrastructure: the record offers a `with_mapping` operation that returns a *deliberately uncertified* mutation — the mapping changes, the certificate does not — documented as "useful for negative tests." Its existence lets the test suite verify that every downstream consumer actually recomputes and rejects a stale certificate, rather than trusting the stored digest. A discipline that cannot exhibit its own forgeries cannot test its forgery detection.

3.2. The catalog viewed from the board. Because every component field ultimately answers to a catalog contract, the closed v0.1 catalog deserves a board's-eye reference here; Table 1 summarizes it. The catalog is published under the versioned profile `cas-pcba-v8.closed-component-ca` and its pinned digest is itself revalidated: the catalog payload constructor re-checks, on every computation, that the catalog container has not been substituted, that every entry is an exact contract, and that each catalog key equals its contract's declared kind. A substituted or diverged catalog is a runtime error, not a silently different engine.

Table 1: The closed v0.1 component catalog, as consumed by boards. All activation ports carry signal kind ACTIVATION with unit ACTIVATION_0_1; decision ports carry DECISION with BOOLEAN; finite-state ports carry FINITE_STATE with NONE. "S" marks stateful kinds; `finite_transition` is the sole TKS-adapter-eligible kind.

Kind	Ports (in → out)	Parameters	Flags
`pass`	`input` → `output`	(none)	
`scale`	`input` → `output`	`factor` (decimal ≥ 0); `ceiling` (decimal in $[0, 1]$)	
`cjt`	`cause`, `bias` → `effect`	`beta` (decimal ≥ 0); `saturation`, `cutoff` (decimal in $[0, 1]$); required state bindings `cause`, `bias`, `effect`	
`decision_threshold`	`input` → `decision` (boolean)	`threshold` (decimal in $[0, 1]$)	
`mux`	`when_false`, `when_true`, `select` (boolean) → `output`	(none)	
`fuse`	`input` → `output`	`threshold` (decimal in $[0, 1]$); `action` (string in {`permanent_block`, `warn`})	S
`delay`	`input` → `output`	`initial` (decimal in $[0, 1]$)	S
`finite_transition`	`source` → `target` (finite state)	(none)	TKS

Reading the table board-side yields several observations that recur in the audit chapters. The junction's required state bindings are the only place the catalog *demands* CAS annotation — a board may not declare a `cjt` without saying which states its three roles are intended to model, which is what makes the bias-domain coherence check well-posed. The two stateful kinds are exactly the two whose semantics involve memory: the unit delay (one tick of activation history, seeded by `initial`) and the fuse (run-local open/closed state under `permanent_block`); statefulness is declared in the contract so that the DRC's feedback analysis and the simulator's scheduling can refuse configurations they do not support rather than mis-simulate them. And the finite-transition kind is the single, narrow doorway to the TKS adapter — eligibility is a catalog flag, not a property a board can assert about an arbitrary component.

4. Typed ports

A board carries a tuple of `PortSpec` records (Chapter 4, Definition 4.19) as its external surface: the points at which fixtures inject inputs and observe outputs. Each port declares its direction, signal kind, unit, and required flag; board ports are addressed by nets through the `$board` endpoint sentinel, so "the board's own port `x`" and "component `c`'s port `x`" are structurally different endpoints and can never be conflated by a name collision.

Two board-level remarks. First, the `required` flag on an input port is the hook for the DRC's connectivity rule: a required input must be either connected to a driving net or covered by an executable default in the selected simulator profile; an unconnected required input with no default is a net-class error. Second, board ports are the *only* sanctioned crossing of the board's evidential boundary. Fixtures deliver typed, canonically serialized input samples at input ports, and every observation in a trace is attributed either to a declared port or to a declared test point — there is no side channel by which a value can enter or leave a simulation unrecorded.

5. Parameters and finite-state annotations

The parameter and state-binding mappings are where board authors have the most freedom, so they are where the model is most careful. Freedom of *content* is preserved: a parameter mapping may nest mappings, sequences, and sets, and may carry decimals, integers, booleans, strings, state literals, enumeration members, floats, and explicit nulls. Freedom of *form* is not: every leaf must be one of the canonically encodable types, and the encoding is injective and type-tagged.

Construction 5.8 (Canonical value encoding). The canonical encoding maps: null to a tagged none; booleans to a tagged boolean; integers to a tagged exact decimal string; floats to a tagged binary64 encoding by exact hexadecimal representation, with explicit markers for `NaN` and signed infinities; decimals to a tagged canonical string; state literals to a tagged record of all four factor fields with explicit nulls for absent Drive and Register; enumeration members to a tagged record of their class path and encoded value; strings to a tagged string; mappings to a tagged, key-sorted item list (string keys only); sequences to a tagged item list preserving order; and sets to a tagged item list sorted by each item's canonical JSON text, so that set encoding is independent of iteration order. Any other type raises a board-identity error naming the offending type.

Three properties of this encoding do real work. *Injectivity across types*: the tags separate integer `1` from boolean *true* from string `"1"` from decimal `Decimal("1")` from binary64 `1.0` — five different tagged payloads, five different digests (Chapter 4, Proposition 4.32). *Order discipline*: mapping items are sorted by key and set items by canonical text, so authoring order cannot leak into identity, while sequence order — which is semantically meaningful — is preserved exactly. *Totality by refusal*: the encoding is total on the declared value universe and undefined — loudly — outside it, so no board can carry an unserializable payload whose identity would then depend on ambient object state.

State bindings ride the same machinery with one addition: every bound `CasState` is revalidated during freezing and again during canonical encoding, so a binding that was somehow corrupted after construction cannot reach the identity computation intact.

The phrase *finite-state annotations* in the chapter contract deserves a precise gloss. Bindings and parameters are *annotations*: declared, structural, design-value metadata in the sense of the measurement-kind taxonomy (Chapter 4, Section 15). They assert what the author *intends* a component to model, and the DRC checks their internal and relational coherence. They are not observations, they carry no measurement kind other than design value, and no accumulation of annotations upgrades itself into evidence about any system outside the board.

6. Nets and endpoints

Definition 5.9 (Net). A `Net` consists of a safe reference `ref`, a tuple of `Endpoint` records, and an exact boolean `safety_critical`. An `Endpoint` is an (`owner_ref`, `port_ref`) pair, where `owner_ref` is either a component reference or the board sentinel `$board`.

The record level enforces locality: endpoints are exact typed pairs, the tuple is frozen, and every reference is profile-checked. Everything relational belongs to the DRC's net-class diagnostics, and the board model is designed so that each such rule is decidable from the frozen structure alone: every endpoint must resolve to an existing owner and an existing port on that owner; every net must have exactly one driving (output-direction) endpoint; every net must have at least one load; the driver's signal kind and unit must agree with every consumer's under the closed admissibility table; no port may appear on two nets; and the signal-kind/unit pair itself must be admissible. None of these checks consults anything outside the board and the versioned catalog — which is what makes the check report a durable artifact rather than a statement about a moment.

The `safety_critical` flag on a net mirrors the component flag: it marks the net as part of a declared safety path, obligating a marked test point on that net (a test-point-class diagnostic if absent) and enrolling the net's path in threshold-reachability analysis. As with components, the flag is obligation, not credit.

7. Test points and calibration references

Definition 5.10 (Test point, restated at board level). A `TestPoint` binds a safe reference, the reference of the net it observes, an optional calibration reference, and an exact boolean `safety_critical`.

A test point is a *declared observation site*: a commitment, made in the design, that a particular net is observable and that observations of it will be attributed to this named point. During simulation, test-point observations enter the trace as measurement records attributed to a producer reference derived from the test point, so the provenance of every observed value includes *where in the design* it was observed, not merely what its value was.

The attribution vocabulary is itself typed. A measurement's producer reference admits exactly three forms: a component reference (a safe reference naming a declared component); the board sentinel `$board` (for values entering or leaving the board surface); or a prefixed test-point form, `test-point:`*ref*, whose suffix must itself be a safe reference naming a declared test point. Any other producer string is rejected when the measurement is constructed or revalidated. The three forms partition the possible origins of a value in a run — a component computed it, the fixture supplied it at the surface, or a probe observed it — and the partition is machine-checkable, so an audit can classify every number in a trace by origin without heuristics. Nothing else can produce a value; in particular, there is no "anonymous" or "synthetic" producer form, which is another face of the no-side-channel rule of Section 4.

Definition 5.11 (Calibration reference, restated at board level). A `CalibrationRef` binds: a safe reference and a version string; a `profile_sha256` (exact uppercase 64-hex); a typed measurement kind; a typed unit; instrument and protocol identity strings; a finite decimal uncertainty; and a boolean `packet_eligible` that defaults to — and in every locally constructed record remains — false.

The field set is the candidate's answer to a specific failure mode: calibration claims that are vague enough to be unfalsifiable. Here a calibration reference must say *which* profile

(by digest), *which* instrument identity, *which* protocol identity, *what* uncertainty, and *what kind* of measurement it purports to describe — or it cannot be constructed at all. The DRC then enforces the relational rules: an executable test point on a checked board needs a calibration reference that exists, is well-formed, and is unit-compatible with the observed net; safety-marked paths need their probes; and a missing or mismatched calibration is a distinct diagnostic, never a silent downgrade.

The `packet_eligible` flag encodes the non-self-certification rule at the type level: a board author can declare calibration metadata, and the declaration is useful — it makes the design checkable and the fixture executable — but nothing the author writes can mark the record as eligible for an authenticated Evidence Packet. Eligibility is granted, if ever, by an authentication process outside the board, against the digest the record already carries.

8. Declared feedback loops

Definition 5.12 (Feedback declaration). A `FeedbackLoop` binds: a safe reference; a tuple of component references constituting the declared cycle; the reference of the delay component that breaks the cycle; a finite decimal `max_round_trip_gain`; and a calibration reference for the gain bound.

The board model treats feedback the way it treats safety: as something that must be *declared to exist at all.* The DRC computes the strongly connected components of the board's directed component graph; every realizable cycle must be covered by an exact feedback declaration naming its components; the declared delay component must be of the stateful unit-delay kind and must lie on the cycle; the declared gain bound must be conservative and sub-unity (with explicit declared exceptions where the source discipline admits them); and the bound must carry a dimensionless calibration reference. An undeclared cycle, a cycle without delay, an unsafe gain, and a malformed declaration are four distinct feedback-class diagnostics.

Field-level notes. The gain is a *finite* decimal by type: the idealized infinite gain that the published source uses in its safety board (Chapter 11) is unrepresentable in a feedback declaration, exactly as it is unrepresentable in the junction parameter contract. The calibration reference means that even the *bound* — a design value — must name what kind of quantity it is and with what uncertainty it is asserted, keeping the gain bound inside the same measurement-kind taxonomy as every other number. And the declaration is by component references, not by object pointers: the declared cycle is checkable against the frozen structure, and the declaration survives serialization unchanged.

The limits are as important as the checks, and Chapter 7 states them in full: stateful-beyond-unit-delay loops and stochastic loops are unsupported, and unsupported or resource-bounded analysis yields an *unresolved* safety diagnostic — never a pass. A feedback declaration is a claim the checker can test, not a permission slip.

9. Source and research-boundary metadata

A board's canonical payload does not begin with its components; it begins with *who is speaking.* Four blocks of metadata bind every board identity to the exact software and arithmetic that produced and would interpret it.

Source identity. The versioned source-bundle digest — a SHA-256 over the candidate package's own Python sources under the profile `cas-pcba-v8.python-source-bundle.v1` — enters the payload. Two boards authored identically under different implementations

have different identities, because "the same board" under different interpreting code is not evidentially the same object.

Engine identity. The engine payload (profile `cas-pcba-v8.reference-engine.v1`) nests the source identity, the component-catalog digest, and the complete decimal-context payload with its digest. The catalog digest is computed over every contract in the closed catalog — kinds, ports, parameter rules, bounds, choice sets, flags — so a catalog edit anywhere is an engine-identity change everywhere.

Decimal context. The full context payload (precision, rounding, exponent range, clamp, capitals, trap list) and its digest appear directly in the board payload as well, per Chapter 4, Rule 4.30: an artifact must be interpretable without consulting any ambient arithmetic state.

Research boundary. The fixed Ring-4 boundary travels inside the evidence-bearing records that reference boards (measurement records, projector identities, simulation results). The board payload itself does not need a boundary field to be safe, because of the operational rule of the next section — but every artifact *derived* from a board carries the boundary explicitly, revalidated at construction.

Rule 5.13 (Boundary rederivation). Public operations never accept a caller-supplied authority object and never trust a stored one. Wherever a research boundary is needed, it is derived fresh from the boundary type's fixed constructor and revalidated; wherever one is received (inside a record), it is revalidated against the fixed posture before use. Because the boundary type has no instance storage (Chapter 4, Definition 4.26), there is no mutable object whose corruption could propagate: the check is against constants, every time.

This rule is the board model's answer to a subtle attack surface: shared mutable "status" singletons. A design that stored its boundary in a shared object would make every artifact's claim posture only as strong as the weakest code path that could reach that object. The candidate's design makes the posture a mathematical constant.

10. Deterministic identity material

Construction 5.14 (Canonical board payload). The canonical payload of a board, under the versioned identity schema `cas-pcba-v8.board-identity.v2`, consists of: the identity schema name; the source identity; the engine identity; the catalog digest; the complete decimal-context payload and digest; the board's `ref` and `version`; its ports, sorted by reference, each with direction, signal kind, unit, and required flag canonically encoded; its components, sorted by reference, each as a complete canonical component payload (itself under the versioned schema `cas-pcba-v8.component-identity.v1`, including parameters, state bindings, safety flag, finite-transition payload, and the decimal-context digest); its nets, sorted by reference, with endpoints sorted by owner then port; its test points, calibrations, and feedback loops, each sorted by reference with all fields canonically encoded (feedback component lists sorted for order-insensitivity of the declared set). The board digest is the uppercase SHA-256 of the canonical JSON serialization of this payload; a component digest is defined analogously over its own payload.

Proposition 5.15 (Structural edits change the digest). *Let B and B' be boards accepted by the model. If B and B' differ in any structural respect — any field of any port, component, parameter, state binding, net, endpoint, test point, calibration, or feedback declaration; the board reference or version; or the interpreting source, catalog, or decimal-context identity — then their canonical payloads differ, and hence their digests differ up to SHA-256 collision. Conversely, if B and B' differ only in authoring order of ports, components, nets, test points,*

calibrations, feedback declarations, endpoint lists, mapping insertion order, or set iteration order, their payloads are identical and they receive the same digest.

PROOF. For the forward direction: the canonical payload embeds every structural field through the injective tagged encoding of Construction 5.8 (injectivity per Chapter 4, Proposition 4.32, extended structurally to the composite encodings, whose tags separate the container types and whose item lists preserve or canonically sort content), so distinct structures serialize to distinct canonical texts; SHA-256 then separates them absent collision. For the converse: every collection in Construction 5.14 is either sorted by a key that the sort leaves invariant under reordering, or is a genuine sequence whose order is itself structural (and thus out of scope of the converse). Canonical JSON fixes key order and separators, so equal payload objects yield byte-equal texts and equal digests. □

REMARK 5.16 (Evidence consequences). Proposition 5.15 is the board model's contribution to the audit trail. Every DRC report binds the digest of the exact board it checked; every simulation result binds the exact DRC report identity and board identity; every measurement's identity scope chains back to them. Consequently: a post-review parameter tweak, a quietly added endpoint, a relaxed gain bound, a re-pointed calibration — each produces a board whose digest matches no existing report, and every artifact that claimed to be "about" the reviewed board is visibly about something else. The converse direction is equally practical: because authoring order does not enter identity, two independent transcriptions of the same design — or a serialization round trip — agree on digest, so digest agreement is usable as a transcription check. Both directions rest on the disclosed external assumption of SHA-256 collision resistance (Chapter 4, Corollary 4.33).

REMARK 5.17 (What identity does not claim). A digest identifies structure; it does not validate it. A board whose digest is stable is not thereby checked (that is the DRC's report), let alone correct, safe, or meaningful. Identity answers "which board?" — exactly and only that question — so that every other question can be asked *of the right board*.

10.1. The digest chain. The board digest is the root of a chain that runs through every downstream artifact. It is worth laying the chain out end to end, because its integrity argument is compositional: each link binds the identity of the previous link into its own canonical payload, so tampering anywhere breaks verification everywhere downstream.

PROTOCOL 5.18 (Board-rooted evidence chain).

(1) *Arithmetic and engine.* The decimal-context digest is a constant of the interpreting software; the catalog digest and source-bundle digest combine with it into the engine identity (Section 9).
(2) *Board.* The canonical board payload embeds the engine identity and the complete structure; the board digest identifies both at once (Construction 5.14).
(3) *Check report.* The DRC report's identity is computed over the exact board digest it checked, the DRC profile version, and the complete ordered diagnostic list; "this board passed" is never separable from *which* board and *which* checker.
(4) *Simulation result.* A result binds the board digest, the exact DRC report identity, the simulator profile, the component catalog and decimal-context digests, the source baseline, and the fixed research boundary; its identity scope digest seeds every measurement.
(5) *Measurements and trace events.* Each measurement identity is a digest over its identity scope, tick, producer, port, canonical value text and value digest, unit, kind, calibration reference, and child-measurement identities; trace events link measurements to ticks under their own versioned profile.

Verification replays the chain: recompute each canonical payload from its stored fields, recompute each digest, and compare. A single edited parameter at step 2 invalidates steps 3–5 wholesale; an edited measurement value at step 5 invalidates exactly that record and every record naming it as a child ancestor.

Example 5.19 (Digest agreement as a transcription check). Illustrative: two collaborators independently transcribe the same published reference design into board records — one authoring components alphabetically, the other in schematic order; one building nets from a netlist file, the other by hand; one inserting parameters in declaration order, the other sorted. If both transcriptions are faithful, Proposition 5.15's converse direction says their board digests agree, because none of those orderings enters the canonical payload. If the digests *disagree*, at least one transcription differs structurally — a parameter value, a missing endpoint, a flag — and the canonical payloads can be diffed field-by-field to locate it. The digest thus doubles as a cheap, deterministic cross-transcription audit, which the five-board corpus program of Chapter 12 exploits.

11. Boards as values

It is instructive to contrast the board model with the mutable document model that mainstream design tools — electronic CAD suites, diagram editors, notebook environments — share. In the mutable model, a design is an object with a lifecycle: it is opened, edited in place, saved, and versioned (if at all) by timestamps or by an external version-control system whose snapshots are taken at operator-chosen moments. Analyses run against "the current state," and the binding between an analysis report and the design state it analyzed is maintained by workflow discipline: the operator is trusted not to edit between analysis and use.

The board model replaces every element of that picture. A board is a *value*: it does not open, it is constructed; it does not change, it is superseded by a different value; it is not saved into an ambient current state, it is referenced by digest. The binding between a report and its subject is not workflow discipline but arithmetic (Protocol 5.18). "Editing" is the pure-functional idiom: construct a new board sharing whatever substructure is unchanged — shared safely, because everything is immutable — and receive a new identity for the new whole.

Three practical consequences follow. *Concurrency is free*: any number of checks, simulations, and audits can run against the same board value without locks, because nothing can mutate under them. *Caching is sound*: a DRC report keyed by board digest can be reused for as long as the digest matches, with no invalidation protocol beyond digest comparison — and the report itself records the checker version, so an engine upgrade naturally misses the cache. *History is cheap and honest*: an archive of board values is an archive of every design state that ever mattered, each self-identifying, none overwritable. The candidate's known-fault fixtures — boards that must fail specific checks forever (Chapter 11) — are simply archived values whose failing reports are regression-tested against their unchanging digests.

The cost, equally practical, is that authoring convenience must be rebuilt above the model: builders that accumulate records before the single validating construction, and diff tools that compare canonical payloads rather than object graphs. The candidate accepts the cost deliberately. Authoring convenience above a sound core is engineering; evidential soundness above a convenient core is, on the record of the source audit this monograph preserves, how grammars come to not cover their corpora and safety fuses come to not trip.

12. Worked example fragments

Everything in this section is *illustrative only*: fragments are schematic, elide imports and full argument lists, and are chosen to make the model's acceptance boundary concrete. They are not normative source code, and no fragment asserts anything about any system outside the candidate.

12.1. A legal minimal board.

EXAMPLE 5.20 (Pass-through board). The smallest interesting legal board wires one `pass` component between the board surface's input and output:

- Board ports: `stimulus_in` (INPUT, ACTIVATION, ACTIVATION_0_1) and `response_out` (OUTPUT, ACTIVATION, ACTIVATION_0_1).
- Component: `u1` of kind `pass`, semantic profile `pcba8.pass.v0`, empty parameters, empty state bindings.
- Nets: `n_in` with endpoints (`$board`, `stimulus_in`) and (`u1`, `input`); `n_out` with endpoints (`u1`, `output`) and (`$board`, `response_out`).

Construction succeeds; the board has a stable digest; and the DRC accepts it: every endpoint resolves, each net has exactly one driver (the board's input surface drives `n_in`; `u1`'s output drives `n_out`), kinds and units agree, and no safety, feedback, or calibration obligations are triggered because none are declared.

EXAMPLE 5.21 (Legal annotated junction). A more representative fragment declares a junction with semantic annotations: component `j1` of kind `cjt`, with a finite decimal parameter `beta` of `Decimal("4")`, and state bindings mapping `cause`, `bias`, and `effect` binding names to exact `CasState` literals — say $(\wedge 8, C, \bot, \bot)$, $(\wedge 2, E, \bot, \bot)$, and $(\wedge 9, C, \bot, \bot)$. The record freezes both mappings; the DRC checks `beta` against the kind's nonnegative-decimal rule, checks the binding names against the kind's required set, and applies the bias-domain coherence rule to the bound literals. Note what has and has not happened: the board now *says* something in CAS vocabulary, and the checker has verified that what it says is well-typed and coherent — and nothing has been evidenced about any actual cognitive process.

EXAMPLE 5.22 (Annotated safety path, declared correctly). Illustrative: a fragment that declares a monitored refusal-shaped path — the shape whose published source version Chapter 11 dissects — with every obligation the model imposes made explicit:

- Components: `stage` of kind `cjt` (`beta` = `Decimal("4")`, `saturation` = `Decimal("0.95")`, `cutoff` = `Decimal("0.05")`, bindings on all three roles); `guard` of kind `fuse` (`threshold` = `Decimal("0.9")`, `action` = `"permanent_block"`), marked `safety_critical`.
- Net `n_guard` from `stage.effect` to `guard.input`, marked `safety_critical`.
- Test point `tp_guard` on `n_guard`, marked `safety_critical`, with calibration reference `cal_act_v1`.
- Calibration `cal_act_v1`: version, profile digest, measurement kind DESIGN_VALUE, unit ACTIVATION_0_1, instrument and protocol identities, finite uncertainty — and `packet_eligible` false, as always for a locally declared record.

The board is record-legal, and — with the remaining wiring in place — passes the test-point and calibration classes of the DRC. Whether it passes *threshold reachability* depends on the numbers: with `saturation` = `0.95` above `threshold` = `0.9`, the conservative bound propagation can resolve the crossing as reachable; had `saturation` been `Decimal("0.1")` — the published source's own configuration — the checker must flag the unreachable threshold. The example is the board model doing its job: the difference between a checkable safety

declaration and an uncheckable safety *story* is exactly the difference between these typed fields and prose. No outcome of the check, in either direction, is evidence that any real refusal mechanism exists or works; it is evidence about the declared design.

12.2. Reading a canonical payload. Auditors meet boards mostly as canonical payloads, so a reading guide belongs in this chapter. A canonical board payload is a JSON object with sorted keys and fixed separators; its top level always contains, in the model's v2 schema: `identity_schema` (the constant `cas-pcba-v8.board-identity.v2`); `source` and `engine` (the interpreting-software identities of Section 9); `catalog_sha256`; `decimal_context` and `decimal_context_sha256`; `ref` and `version`; and the six sorted collections (`ports`, `components`, `nets`, `test_points`, `calibrations`, `feedback_loops`).

Practical audit heuristics follow directly from the schema. To confirm two payloads describe the same design under different engines, strip the `source`, `engine`, `catalog_sha256`, and decimal-context members and diff the remainder — the model deliberately makes "same structure, different interpreter" a well-defined, mechanically checkable relation, while the full digests remain distinct (Proposition 5.15). To locate a structural difference flagged by digest mismatch, diff the sorted collections pairwise by `ref`; because every element is reference-sorted and canonically encoded, the first differing byte lies inside the first genuinely differing field. And to audit a parameter's exact type — the difference between `1`, `true`, `"1"`, and `Decimal("1")` — read its tag: the payload preserves the type tags of Construction 5.8 verbatim, so type puns that would be invisible in a pretty-printed value listing are explicit in the canonical form.

12.3. Illegal constructions at the record layer.

EXAMPLE 5.23 (Rejected at construction). Each fragment below raises a typed error before any board exists:

- *Subclassed record.* `class SneakyNet(Net): ...` — any instance is rejected by the exact-type check (Rule 5.2), regardless of field content.
- *Unsafe reference.* A component with `ref="u1\u202e"` (a bidirectional control character) or `ref="_hidden"` (leading non-alphanumeric) fails the safe-reference profile.
- *Untyped gain.* A feedback declaration with `max_round_trip_gain=0.8` (binary64 float) is rejected: the field requires an exact finite `Decimal`. So is `Decimal("Infinity")` — finiteness is checked, which is precisely what makes the source's idealized infinite gain unrepresentable.
- *Unfreezable metadata.* A parameter mapping containing a live object — a file handle, a lambda, an arbitrary class instance — is rejected by the canonical freeze (Rule 5.3): the value has no canonical encoding, so it may not enter a board.
- *Type-punned parameter.* A `fuse` authored with `action=True` fails the string-typed choice rule; authored with `action="permanent_block"` it passes; authored with `action="Permanent_Block"` it fails the closed choice set — case is content.
- *Mutation attempt.* Given a constructed net `n`, the assignment `n.safety_critical = True` raises: frozen records have no assignment path (Rule 5.1). The only way to "edit" a board is to construct a new one — which, by Proposition 5.15, has a new identity.

12.4. Illegal boards at the check layer.

EXAMPLE 5.24 (Type-legal, check-illegal). The record layer accepts each of the following boards — every record is locally well-formed — and the DRC then fails each with a distinct diagnostic, illustrating the division of labor of Section 3:

- Two components both named `u1`: duplicate-reference, a model-class error.
- A component of kind `transistor` (not in the closed catalog): unknown kind, a semantics-class error — not an attempt at nearest-match resolution.
- A net whose endpoint names port `inptu` on `u1`: unknown port, a net-class error; typos are errors, not warnings.
- A net driven by two outputs: invalid driver count. A net driven by none, or loading no input: likewise distinct net-class errors.
- A decision output wired to an activation input: signal-kind mismatch under the closed admissibility table; no coercion from boolean to $[0, 1]$ exists anywhere in the candidate.
- A safety-critical net with no marked test point, or a test point whose calibration reference names a nonexistent calibration: distinct test-point-class and calibration-class errors.
- A cycle of components with no feedback declaration: undeclared feedback. With a declaration but no delay component on the cycle: feedback without delay. With a declared gain of `Decimal("1.3")`: unsafe feedback gain.

In every case the board *exists* and has a digest — one can refer to it, archive it, and regression-test against it — but it is failed, and a failed report is bound to that digest permanently. The candidate's known-fault fixtures (Chapter 11) rely on exactly this: a board that must fail, kept, with its failure guarded by tests.

13. Invariants and boundary

Invariant 5.25 (Board-model invariants). For every board accepted by the model: (i) the board and every contained record are frozen, exact-typed, and revalidatable, and all open-ended metadata is recursively frozen and detached; (ii) every identifier satisfies the safe-reference profile, with `$board` admitted only as an endpoint owner; (iii) every numeric field that enters semantics is a finite `Decimal`, and no non-finite value is representable in any parameter, gain, uncertainty, or weight; (iv) the canonical payload embeds the complete structure together with source, engine, catalog, and decimal-context identities, and the board digest is a deterministic function of exactly that payload; (v) authoring order never affects identity; structural content always does; (vi) calibration records constructed locally are never packet-eligible, and no board-resident flag can grant evidentiary authority; and (vii) every public operation rederives and revalidates the fixed research boundary rather than trusting any stored or supplied authority object.

Threat 5.26 (Residual limits). The invariants above are enforced by tested, not proved, code; the bounded-evidence posture of Chapter 12 applies. Identity rests on the disclosed SHA-256 collision-resistance assumption. Revalidation defends against corruption it can observe at the record surface; it cannot detect a hostile interpreter or a modified source bundle except insofar as the source digest exposes them. And nothing in the model prevents an author from writing a board that is well-formed, checkable, and meaningless — well-formedness is the floor of the discipline, not its ceiling.

Claim Boundary 5.27. This chapter specifies the board model of the CAS PCBA 8 v0.1 research candidate, at Ring 4, NOT LICENSED. A board is a typed model description; its digests identify software structures only. No board, binding, flag, calibration reference, feedback declaration, or worked example in this chapter constitutes evidence about any cognitive system, deployed model, or physical circuit; no safety, causal, physical, clinical, regulatory, or

product claim is made or licensed; and all example fragments are illustrative, non-normative, and assert nothing beyond the acceptance behavior of the declared software contracts.

CHAPTER 6

Geometry Profiles

1. Why geometry, and under what discipline

Chapters 4 and 5 built a world of exact identities: tagged values, canonical payloads, digests that separate everything distinct. This chapter introduces the candidate's two *geometric* vocabularies — the mixed-simplex Fisher–Rao baseline and the optional projector research profile — and, with equal care, the disciplines that keep geometric numbers from being mistaken for exact identities or for empirical findings.

The motivating need is real. A simplex address (Definition 4.15) is a strictly positive normalized profile over the forty canonical ND-cells; analyses over such profiles want a notion of *closeness*: how far did a profile drift between two ticks, how dissimilar are two declared design profiles, is this fixture's output profile nearer to one reference than another. A digest cannot answer these questions — it separates everything and grades nothing. A metric can. Information geometry supplies the canonical candidate metric on the open simplex, with a mathematical pedigree [**42**, **43**, **25**, **26**] that Section 2 reviews.

But the candidate's evidence discipline requires that every geometric capability arrive with its limits attached, and this chapter is structured around three of them, stated here in advance:

(1) The implemented Fisher–Rao distance is a *binary64 numerical pseudometric at its resolution*. It is disclosed as such, profiled as such, and it is *not* a proof that all distinct profiles are separated (Section 3).

(2) The projector profile is a *separate, optional research profile*, never a canonical PCBA address; its Gram-kernel overlap result requires common rank $d > 0$ and establishes an algebraic relation between declared artifacts, *not* latent cognitive capacity (Sections 4–5).

(3) The CAS capacity transfer retains a *proof gap* governed solely by the formal-mathematics Paper 3 record; every engineering-side statement about it, including everything in this chapter, is a read-only dependency summary with no authority to narrow, close, or reinterpret the gap (Section 6).

And over all of it stands the boundary restated in Section 10: geometry profiles do not identify learned model states with CAS states.

2. Primer: information geometry on the open simplex

2.1. The open simplex as a statistical manifold. Identify a simplex address $p \in \Delta^\circ_{39}$ with the categorical probability distribution it defines over the forty canonical cells: cell k receives probability $p_k > 0$. The open simplex is then the interior of the parameter space of the categorical family — a smooth manifold of dimension 39, with global coordinates given by any thirty-nine of the forty weights. Its tangent space at p is the set of zero-sum vectors,

$$T_p\Delta^\circ_{39} = \Big\{\, u \in \mathbb{R}^{40} : \textstyle\sum_k u_k = 0 \,\Big\},$$

reflecting that perturbations must preserve normalization.

Statistical manifolds carry a distinguished Riemannian metric, obtained by Fisher's information quantity [42] read as a quadratic form on parameter perturbations; Rao's foundational observation [43] was that this form defines a Riemannian geometry on the parameter space, whose geodesic distance measures the intrinsic distinguishability of nearby distributions. On the categorical family the construction is completely explicit, and it is worth carrying it out once, because the result is exactly the quadratic form the candidate uses.

Let X be a categorical random variable over the forty cells with distribution p, and perturb p along a zero-sum direction u: $p(t) = p + tu$, which stays in the open simplex for small t. The log-likelihood of outcome k is $\log p_k(t)$, its t-derivative at $t = 0$ is u_k/p_k, and the Fisher information of the one-parameter family at $t = 0$ is the variance of that score:

$$I(0) = \sum_{k=1}^{40} p_k \Big(\frac{u_k}{p_k}\Big)^2 - \Big(\sum_{k=1}^{40} p_k \frac{u_k}{p_k}\Big)^2 = \sum_{k=1}^{40} \frac{u_k^2}{p_k},$$

the cross term vanishing because $\sum_k u_k = 0$. Polarizing the quadratic form gives the metric.

Definition 6.1 (Fisher–Rao metric on the open simplex). For $p \in \Delta^\circ_{39}$ and $u, v \in T_p\Delta^\circ_{39}$,

$$g^{\mathrm{FR}}_p(u, v) = \sum_{k=1}^{40} \frac{u_k\, v_k}{p_k}.$$

The reasons information geometry — rather than an arbitrary embedding metric — is the right setting are threefold, and each matters to how the candidate is allowed to talk about its numbers.

Invariance. The Fisher–Rao metric is, up to an overall scale, the unique Riemannian metric on families of distributions invariant under the natural statistical maps (sufficient statistics, refinement and coarsening of outcomes); this characterization and its history are developed in the standard treatments [25, 26]. Invariance means the geometry measures properties of the *distributions*, not of an arbitrary coordinate choice — exactly the property one wants if profile comparisons are ever to be more than numerology.

Degeneracy at the boundary. The metric coefficients $1/p_k$ diverge as $p_k \to 0$: the closed simplex is not a Riemannian manifold under g^{FR}. This is the geometric face of the strict positivity rule of Chapter 4, Section 5.1: the type system admits exactly the points where the geometry is defined.

Distinguishability semantics. Locally, the squared Fisher–Rao length of a perturbation measures how statistically distinguishable the perturbed distribution is from the original — large distance means easy to tell apart from samples, small distance means hard. This gives distance values an operational reading *as statements about distributions*. It gives them no reading whatsoever as statements about cognition; the distributions here are declared software profiles.

2.2. The sphere embedding and the geodesic distance. The categorical Fisher–Rao geometry admits a classical closed form.

Proposition 6.2 (Sphere embedding). *The map $\varphi(p) = 2\sqrt{p} = (2\sqrt{p_1}, \dots, 2\sqrt{p_{40}})$ is a diffeomorphism from Δ°_{39} onto the open positive orthant of the sphere of radius 2 in $\mathbb{R}^{40}$, and it is an isometry between g^{FR} and the metric induced on that sphere by the Euclidean ambient metric.*

PROOF. If $x = \varphi(p)$, then $\sum_k x_k^2 = 4\sum_k p_k = 4$, so the image lies on the radius-2 sphere, with all coordinates positive; the inverse $p_k = x_k^2/4$ is smooth on the open orthant. For the isometry, differentiate: $dx_k = dp_k/\sqrt{p_k}$, hence $\sum_k dx_k^2 = \sum_k dp_k^2/p_k = g_p^{\mathrm{FR}}(dp, dp)$. □

COROLLARY 6.3 (Geodesic distance). *For $p, q \in \Delta_{39}^\circ$,*

$$d_{\mathrm{FR}}(p, q) = 2\arccos\Big(\sum_{k=1}^{40} \sqrt{p_k\, q_k}\Big),$$

the great-circle distance between $\varphi(p)$ and $\varphi(q)$ on the radius-2 sphere. The argument of the arccosine — the Bhattacharyya coefficient of p and q — lies in $(0, 1]$ on the open simplex, so $d_{\mathrm{FR}}(p, q) \in [0, \pi)$, with 0 attained exactly at $p = q$.

PROOF. The image of the open simplex is a geodesically convex subset of the open positive orthant of the sphere (any two points of the open orthant are joined by a unique minor great-circle arc lying in the orthant), so the induced intrinsic distance is the ambient great-circle distance: radius times the angle, and the angle between unit directions $\varphi(p)/2$ and $\varphi(q)/2$ has cosine $\sum_k \sqrt{p_k}\sqrt{q_k}$. By Cauchy–Schwarz this coefficient is at most 1 with equality iff $\sqrt{p}$ and $\sqrt{q}$ are proportional, which under normalization forces $p = q$; positivity of all coordinates makes the coefficient strictly positive, bounding the distance strictly below π. □

REMARK 6.4 (Geodesic paths). The embedding also yields the geodesic itself, not only its length. For $p \neq q$ with angle $\alpha = \arccos\big(\sum_k \sqrt{p_k q_k}\big)$, the unit-speed great-circle arc between $\varphi(p)$ and $\varphi(q)$ pulls back to

$$\gamma(t)_k = \Big(\frac{\sin\big((1-t)\,\alpha\big)}{\sin\alpha}\sqrt{p_k} + \frac{\sin(t\,\alpha)}{\sin\alpha}\sqrt{q_k}\Big)^2, \qquad t \in [0, 1],$$

which remains in the open simplex throughout (both coefficients are positive for $t \in (0, 1)$, and squares of positive combinations of positive coordinates are positive; normalization follows from the arc lying on the sphere). Interpolation between profiles, where an analysis wants it, therefore has a canonical form — and it is *not* the straight-line mixture $(1-t)p+tq$, which is a geodesic of a different (flat) geometry. The candidate registers no interpolation operation in v0.1; the formula is recorded so that any future one has its exact target stated in advance.

REMARK 6.5 (Neighboring distances and local behavior). Writing $H^2(p, q) = 1-\sum_k \sqrt{p_k q_k}$ for the squared Hellinger distance, the exact relation $\cos\big(d_{\mathrm{FR}}(p, q)/2\big) = 1 - H^2(p, q)$ holds, and for close profiles the expansion $\arccos(1 - x) = \sqrt{2x}\,(1 + O(x))$ gives

$$d_{\mathrm{FR}}(p, q) = 2\sqrt{2}\,H(p, q)\,\big(1 + O(H^2)\big), \qquad d_{\mathrm{FR}}(p, p+u)^2 = \sum_k \frac{u_k^2}{p_k} + O(\|u\|^3),$$

the latter being the infinitesimal form one expects from Definition 6.1: to second order, squared geodesic distance is the chi-square-like quadratic form of the perturbation. These classical relations [**25, 26**] matter operationally in Section 3.3, where the square-root behavior of arccos near 1 is exactly what amplifies floating-point error for nearby profiles.

EXAMPLE 6.6 (A worked exact distance). Illustrative computation, exact mathematics only. Let p be the uniform address ($p_k = 1/40$ for all k) and let q move mass 0.005 from cell `P0` to cell `P1`: $q_1 = 0.02$, $q_2 = 0.03$, $q_k = 0.025$ otherwise. Then

$$\sum_k \sqrt{p_k q_k} = 38\cdot 0.025 + \sqrt{0.025\cdot 0.02} + \sqrt{0.025\cdot 0.03} \approx 0.95 + 0.022361 + 0.027386 = 0.999747,$$

so $d_{\mathrm{FR}}(p,q) = 2\arccos(0.999747) \approx 0.0450$, while the Hellinger approximation of Remark 6.5 gives $2\sqrt{2}\,H \approx 2\sqrt{2}\cdot 0.015912 \approx 0.0450$ — agreement to the displayed precision, as expected at this scale. The computed magnitude has, as always, no cognitive reading; it is a distance between two declared profiles.

REMARK 6.7 (Exact-mathematics status). Definition 6.1 through Corollary 6.3 are classical mathematics [43, 25, 26], stated over the reals, and at that level d_{FR} is a genuine metric on Δ°_{39}: symmetric, positive on distinct pairs, and satisfying the triangle inequality (as a geodesic distance). *None of this transfers automatically to the implementation.* The candidate's discipline is to keep three layers separate: the exact mathematics above; the exact Decimal data of Chapter 4; and the binary64 numerical procedure of the next section, which is where the metric properties are partially lost and where the disclosure lives.

3. The implemented baseline: a disclosed numerical pseudometric

3.1. What the implementation does. The implemented baseline, under the versioned profile name `cas-pcba-v8.binary64-fisher-rao-pseudometric.v1`, accepts two exact mixed-simplex addresses and proceeds as follows: both operands are revalidated; each exact Decimal weight is converted to binary64; the Bhattacharyya coefficient is accumulated by compensated floating summation of the terms $\sqrt{a_k b_k}$; the accumulated coefficient is clamped into $[0,1]$; the distance is computed as $2\arccos(\cdot)$ in binary64; results of magnitude below 10^{-15} are flushed to exactly zero; and the result is returned as a Decimal constructed from the binary64 value's decimal representation. The function object itself carries two disclosure attributes: its profile name, and the flag `is_exact_metric = False`.

Every step of that pipeline is deliberate. Revalidation keeps the garbage-in case out. The clamp exists because accumulated rounding can push the coefficient marginally above 1, where the arccosine is undefined; clamping resolves the exception in the conservative direction (distance 0 rather than a domain error). The flush-to-zero acknowledges that distances at the 10^{-15} scale are numerical noise at binary64 resolution and refuses to report them as structure. And the Decimal return type does *not* launder the number into exactness: it is an exact representation *of the binary64 result*, tagged by the profile that produced it.

3.2. The disclosure, stated precisely.

DEFINITION 6.8 (Numerical pseudometric at resolution). Call the implemented map $\widehat{d}\colon \Delta \times \Delta \to \mathbb{D}$ (where Δ is the set of valid mixed-simplex addresses and $\mathbb{D}$ the finite decimals) a *numerical pseudometric at binary64 resolution*, meaning: $\widehat{d}$ is symmetric and nonnegative by construction; $\widehat{d}(p,p) = 0$ for every p; but $\widehat{d}(p,q) = 0$ does *not* imply $p = q$, and the triangle inequality is inherited from the exact mathematics only up to the accumulated floating-point and flush errors of the pipeline, for which no certified bound is currently registered.

ADVERSE FINDING 6.9 (Separation fails below resolution). There exist pairs of *distinct* valid addresses — distinct as exact Decimal profiles, with distinct canonical identity digests — whose implemented distance is exactly 0. Two mechanisms suffice: (i) two distinct sixty-digit Decimal weight profiles can round to the same forty binary64 values, collapsing the computation entirely; and (ii) profiles distinct even after rounding can yield distances below the 10^{-15} flush threshold, which the implementation reports as zero by design. The implementation's own documentation states the consequence: distinct profiles below binary64 resolution can have distance zero, and the function "is not an exact metric, proof of identity, or projector bridge."

It must be stressed that Finding 6.9 is a *disclosure*, not a defect report: the behavior is documented, versioned into the profile name itself (`binary64`, `pseudometric`), and guarded by the claim discipline below. The alternative — exact Decimal transcendental evaluation with certified rounding — is a legitimate open engineering item, and until it exists the candidate prefers a disclosed pseudometric to an undisclosed pretension of exactness. This is the same posture the seed specification takes: the binary64 implementation "is a numerical pseudometric at its resolution — not a proof that all distinct profiles are separated."

3.3. Resolution arithmetic. How large are the effects the disclosure discloses? No certified error bound is registered, and this subsection does not create one; what it offers is a first-order *heuristic* sizing, clearly labeled as such, so that readers of reported distances can calibrate their skepticism quantitatively rather than vaguely.

Binary64 arithmetic resolves relative differences of order $\varepsilon \approx 2.2 \times 10^{-16}$ (the unit roundoff). Three stages of the pipeline interact with this floor differently.

Conversion. Each sixty-digit Decimal weight is rounded to binary64, a relative perturbation of order ε per weight. Profiles differing only beyond the seventeenth significant digit of every weight become *identical* after conversion — the first mechanism of Finding 6.9.

Accumulation. The Bhattacharyya coefficient is a sum of forty terms of typical magnitude $1/40$; compensated summation keeps the accumulated error near a few units of ε, so the computed coefficient carries an absolute uncertainty on the order of 10^{-16} to 10^{-15}.

Amplification by the arccosine. Near coincidence the coefficient approaches 1, where arccos has unbounded derivative: by Remark 6.5, a coefficient error of magnitude e near 1 moves the distance by roughly $2\sqrt{2e}$. Taking $e \approx 10^{-16}$ gives a distance uncertainty of order $2\sqrt{2 \times 10^{-16}} \approx 3 \times 10^{-8}$.

The heuristic conclusion — and it is a heuristic, pending the independent numerical audit of Threat 6.15 — is that computed distances for *nearly coincident* profiles are noise-dominated below roughly the 10^{-8} scale, even though the implementation only flushes below 10^{-15}: the flush targets exact-coincidence roundoff residue, not the amplified noise floor near coincidence. A reported distance of, say, 3×10^{-9} under profile v1 should therefore be read as "indistinguishable from zero at resolution," and Rule 6.11's discipline applies to it in spirit even though its printed value is positive. Far from coincidence the arccosine's derivative is moderate and the same heuristic assigns reported distances an absolute uncertainty many orders of magnitude below their values; the discipline of Rule 6.12 — report the number, its profile, and no exactness pretension — covers both regimes uniformly.

Example 6.10 (A binary64 collision pair). Illustrative construction of the first mechanism of Finding 6.9. Let p be the uniform address with every weight `Decimal("0.025")`. Build q from p by transferring mass 10^{-30} from cell `P0` to cell `P1` at the exact-Decimal layer, keeping the exact sum at `Decimal("1")`. Then p and q are distinct values with distinct canonical identity digests — the digest layer separates them — but each perturbed weight rounds to the *same* binary64 number as 0.025, since the relative perturbation, 4×10^{-29}, is far below the unit roundoff. The implemented pipeline computes from identical forty-tuples of doubles, obtains a coefficient that clamps to 1, and reports distance exactly 0. Both layers behaved exactly as specified; the example is why the specification insists they never be conflated.

3.4. What this means for evidence claims. The pseudometric disclosure induces a small calculus of which claims a Fisher–Rao number can and cannot support. The candidate states it as rules, because the failure mode — quietly upgrading a numerical zero into an identity claim — is exactly the kind of drift the audit discipline exists to catch.

RULE 6.11 (Zero distance licenses nothing about identity). A computed $\widehat{d}(p, q) = 0$ supports only: *the implemented pipeline could not distinguish p and q at binary64 resolution under profile v1.* Identity of addresses is claimed, if at all, by canonical digest equality (Chapter 4, Rule 4.3) — the exact layer, where separation holds by construction. Any artifact that records a zero distance and needs an identity conclusion must record the digest comparison alongside it.

RULE 6.12 (Positive distance carries its resolution). A computed $\widehat{d}(p, q) = \delta > 0$ supports: *the profiles are distinguishable by the implemented pipeline, with reported magnitude δ under profile v1.* Reported magnitudes are comparable with one another under the same profile; they are not certified approximations of the exact d_{FR} to any stated error bound, because no such bound is currently registered. Downstream text must therefore not present δ as the exact Fisher–Rao distance, and must not chain triangle-inequality arguments through $\widehat{d}$ as if certified.

RULE 6.13 (Profile identity travels with the number). Every reported distance carries the profile name `cas-pcba-v8.binary64-fisher-rao-pseudometric.v1`; the operand identities (their profile digests, which embed the decimal-context digest); and, where the number enters a trace, the standard trace bindings of Chapter 8. A distance whose provenance cannot be replayed is not evidence of anything.

PROTOCOL 6.14 (Reporting template for baseline distances). Any artifact of this research program that reports a Fisher–Rao baseline value records, together: (1) the value as returned (a Decimal transcription of the binary64 result); (2) the profile name `cas-pcba-v8.binary64-fisher-rao-pseu` and its `is_exact_metric = false` disclosure; (3) the operand profile digests; (4) the decimal-context digest (already embedded in the operand identities); (5) where applicable, the trace and result identities binding the computation to a run (Chapter 5, Protocol 5.18); and (6) for values below the heuristic noise scale of Section 3.3, an explicit "indistinguishable at resolution" qualifier in any prose summary. Conclusions of profile *identity* additionally record the digest comparison of Rule 6.11; conclusions of profile *difference* rest on digest inequality, with the distance supplying magnitude only.

THREAT 6.15 (Numerical-validity residual). Independent numerical audit of the baseline — certified error bounds for the conversion, summation, clamp, arccosine, and flush stages; tolerance analysis; and adversarial search for worst-case separation failures beyond the constructive mechanisms of Finding 6.9 — remains open, as the threats chapter of the seed paper records. Until closed, the pseudometric framing is not a conservative courtesy; it is the strongest claim the evidence admits.

4. Projector geometry: a separate optional research profile

The second geometric vocabulary is deliberately quarantined from the first. A *projector profile* is an optional research artifact for frontier analyses that want to reason about linear subspaces — for example, comparing declared feature subspaces across declared bases. It is not part of the board baseline, no component port carries one except under the explicit PROJECTOR_PROFILE signal kind, and the type itself enforces that it can never be mistaken for an address: the discriminator field `canonical_pcba_address` is pinned to false and construction rejects any other value.

DEFINITION 6.16 (Projector profile). A `ProjectorProfile` binds: a profile identifier, basis identifier, and source identifier (all safe references); a nonempty square matrix of finite

Decimals; a declared integer rank r with $1 \le r \le n$ (where n is the dimension); a tolerance — a finite nonnegative Decimal capped by the versioned bound 10^{-12}; the fixed validation-profile name `cas-pcba-v8.projector-validation.v1` and profile-kind discriminator; the fixed research boundary; three *recorded validation residuals* computed at construction (symmetry residual $\max_{i,j}|M_{ij} - M_{ji}|$, idempotence residual $\max_{i,j}|(M^2 - M)_{ij}|$ with the square computed by exact-sum Decimal arithmetic, and trace residual $|\mathrm{tr}M - r|$); and two digests — a matrix digest over the canonical matrix payload, and a profile digest over the complete identity payload.

Rule 6.17 (Capped, versioned tolerance; recorded residuals). Construction fails unless all three residuals are within the declared tolerance, and the tolerance itself cannot exceed the versioned cap. The residuals are not transient checks: they are *recorded into the identity payload*, so the profile digest commits to exactly how close to an orthogonal projector of the declared rank the matrix was found to be, under which tolerance, at validation time. Revalidation recomputes the residuals and fails on any discrepancy with the recorded values — staleness and tampering are both detected — and re-checks the tolerance cap, so a profile cannot quietly loosen its own acceptance criterion after the fact.

Rule 6.18 (One digest binds the whole geometry). The profile digest is computed over a payload containing, together and inseparably: the canonical matrix digest, the basis identifier and source identifier, the dimension, the declared rank, the tolerance, the three recorded residuals, the decimal-context digest, and the research boundary. Consequently no constituent can be swapped independently — a re-based matrix, a re-declared rank, a retuned tolerance, or a re-attributed source each produce a different profile identity. "The projector" in any downstream artifact is always *this matrix, in this basis, at this dimension and rank, validated to this tolerance, under this arithmetic* — one digest, one object.

The design mirrors, in miniature, the board-identity argument of Chapter 5 (Proposition 5.15), and for the same evidentiary reason: subspace claims are only as auditable as the binding between the numbers and their declared interpretation frame. The basis identifier is the frame; the digest makes frame substitution detectable.

Example 6.19 (A minimal validated projector, and a rejected one). Illustrative. The 2×2 matrix diag(`Decimal("1")`,`Decimal("0")`) with declared rank 1 validates exactly: all three residuals are the Decimal zero, within any admissible tolerance. The matrix diag(`Decimal("0.5")`,`Decimal("0.` with declared rank 1 has trace residual zero but idempotence residual 0.25 — twelve orders of magnitude above the tolerance cap — and is rejected: an "average of two projectors" is not a projector, and the type refuses to launder one into subspace vocabulary. A declared rank of 0 or 3 is rejected before any residual is computed; a tolerance of 10^{-6} is rejected by the versioned cap, so no author can admit a sloppy matrix by relaxing the acceptance criterion. For validated $P = \mathrm{diag}(1, 0)$ and $Q = P$ under a shared basis identifier, $\omega(P, Q) = 1$; for $Q' = \mathrm{diag}(0, 1)$, $\omega(P, Q') = 0$ — the two endpoints of Lemma 6.21, realized at the smallest possible dimension.

5. The Gram-kernel overlap result and its limits

The one registered computation over projector profiles is the normalized overlap.

Definition 6.20 (Projector overlap). For validated profiles P and Q with common dimension n, common declared rank r, and *identical declared basis identifier*, the overlap is

$$\omega(P, Q) = \frac{\mathrm{tr}(PQ)}{r},$$

computed in exact Decimal arithmetic (exact-sum matrix products) after revalidating both operands. The operation raises — it does not degrade — on dimension mismatch, rank mismatch, or basis mismatch.

The common-rank requirement is not bureaucratic: the normalization divides by r, and the type system already excludes $r = 0$ (rank is constrained to $[1, n]$ at construction), so the registered result is available exactly when common rank $d > 0$ holds — the condition the seed specification states. What the number means, at the level of exact linear algebra, is bounded by the following.

LEMMA 6.21 (Overlap bounds). *Let P and Q be exact orthogonal projectors on $\mathbb{R}^n$ of common rank $d > 0$. Then $0 \le \mathrm{tr}(PQ) \le d$, hence $\omega(P, Q) \in [0, 1]$; moreover $\omega(P, Q) = 1$ if and only if $\mathrm{im}P = \mathrm{im}Q$, and $\omega(P, Q) = 0$ if and only if $\mathrm{im}P \perp \mathrm{im}Q$.*

PROOF. Using symmetry and idempotence, $\mathrm{tr}(PQ) = \mathrm{tr}(PQQP) = \mathrm{tr}\big((QP)^\top (QP)\big) = \|QP\|_F^2 \ge 0$, with equality iff $QP = 0$, i.e. $\mathrm{im}P \subseteq \ker Q$, i.e. the ranges are orthogonal. For the upper bound, write $\mathrm{tr}(PQ) = \mathrm{tr}(PQP)$ and note $0 \preceq PQP \preceq P$ in the Loewner order, so $\mathrm{tr}(PQP) \le \mathrm{tr}(P) = d$; equality forces $PQP = P$, hence for every unit $x \in \mathrm{im}P$, $x^\top Q x = 1$, which for a projector means $x \in \mathrm{im}Q$; thus $\mathrm{im}P \subseteq \mathrm{im}Q$, and equal finite rank upgrades inclusion to equality. □

RULE 6.22 (What an overlap value may claim). An overlap $\omega(P, Q)$ computed by the candidate supports exactly: *an exact-arithmetic algebraic relation between two validated matrices that were declared to share a basis frame, each within its recorded residuals of an orthogonal projector of the declared rank.* Lemma 6.21 then gives the value its geometric reading — subspace alignment — *up to* the residual tolerances, whose propagation into an error bar on ω is an open numerical item. What the value does *not* support, under any magnitude: that either subspace corresponds to a capability, feature, concept, or state of any learned model or cognitive system. The overlap is a statement about declared artifacts, and the registered result is explicitly recorded as *not* establishing latent cognitive capacity.

The last sentence deserves its own paragraph, because it is where a plausible-sounding inference chain must be cut. The tempting chain runs: a projector was derived from model activations (elsewhere, under some analysis); two such projectors overlap strongly; therefore the model has a shared latent structure; therefore a cognitive capacity is present. Every link after the first imports assumptions the candidate does not admit: that the derivation basis is meaningful, that overlap of declared artifacts measures overlap of whatever produced them, and that subspace structure is capacity. The projector profile's fields are designed to keep the first link honest (source and basis identifiers, recorded residuals) precisely so that the later links can be seen to be *missing* rather than implied. The Gram-kernel result is mathematics about matrices; capacity claims are empirical claims about systems; the candidate registers the former and quarantines the latter.

6. The capacity-transfer proof gap

Beyond the overlap result lies a stronger statement that the research program cares about: a transfer principle by which structure validated in one representation would carry CAS-semantic weight in another. This monograph's obligation is to record its status exactly.

REGISTERED HYPOTHESIS 6.23 (CAS capacity transfer — status: open, not licensed). The proposed transfer from projector-level structure to CAS capacity claims is a hypothesis. It retains a *proof gap*: no admitted derivation closes the distance between algebraic

subspace relations (Lemma 6.21 and its registered computational form) and any statement about cognitive or model capacity.

RULE 6.24 (Governance of the gap). The proof gap is governed *solely* by the formal-mathematics Paper 3 record, at that record's own commit. Engineering records — the implementation, its tests, the seed research paper, and this monograph — are *read-only dependency summaries* with respect to that record: they may restate its status, as this section does, and they must track it; they have no authority to close, narrow, extend, or reinterpret it. In particular, no accumulation of green software tests, no overlap computation of any magnitude, and no prose in an engineering artifact changes the status of Hypothesis 6.23. If the Paper 3 record changes, engineering summaries update to match — never the reverse.

This governance rule is an instance of the candidate's general authority discipline (the same one that keeps locally constructed calibration records out of Evidence Packets and self-hashed receipts out of licensing): the artifact that *could* benefit from a claim is never the artifact that adjudicates it. A geometry module that could declare its own mathematics sufficient would be exactly the kind of self-certifying loop the architecture exists to prevent.

7. Two arithmetic regimes, and why they differ

A reader tracking the candidate's arithmetic discipline (Chapter 4, Section 13) will have noticed an asymmetry in this chapter that deserves to be made explicit, because it looks like an inconsistency and is in fact a design rule.

The *projector* path is exact end to end. Residual computation multiplies matrices with exact aligned-integer Decimal summation; overlap traces are exact sums of exact products; the tolerance comparison is exact Decimal comparison. Nothing in the projector path introduces a rounding step beyond the fixed sixty-digit context's governance of individual multiplications, and the recorded residuals are reproducible to the digit on any conforming interpreter.

The *simplex distance* path is not. It leaves the exact world at the conversion step and returns only a transcription of a binary64 result, with the pseudometric disclosure attached.

The difference is not carelessness on one side; it is the difference between polynomial and transcendental computation. Residuals, traces, and overlaps are polynomial in the matrix entries: finitely many exact multiplications and exact additions, closed under Decimal arithmetic. The Fisher–Rao distance requires square roots and an arccosine — transcendental functions whose exact values are not decimals at all, so *some* approximation contract is mandatory. The candidate's v0.1 choice is the weakest honest one: use the platform's binary64 transcendentals, and disclose the resulting object as a pseudometric at that resolution rather than dress it in exactness it does not have. A future profile could tighten the contract — correctly rounded sixty-digit square roots exist in the Decimal context, and arbitrary-precision arccosine with certified error is implementable — but that would be a *new versioned profile* with its own name and its own disclosure, not a quiet upgrade of v1 (Rule 6.13 makes the version part of every reported number precisely so that such an upgrade would be visible in the evidence).

The same asymmetry explains an operational rule that might otherwise puzzle: digest-bearing identity payloads embed *weights*, never *distances*. A simplex address's identity contains its forty exact Decimals; a projector's identity contains its exact matrix and exact residuals; no identity anywhere contains a Fisher–Rao value. Exact objects are identity material; approximate objects are reportable results. Keeping the two segregated is what lets the candidate promise that identities are deterministic across conforming platforms while transcendental libraries vary.

8. Geometry values in the wiring and the trace

Both geometry types are first-class citizens of the board vocabulary, and their integration points are worth enumerating because each is also a containment point.

Signal kinds and units. A net can carry a simplex address only under the MIXED_SIMPLEX_ADDRESS signal kind with unit PROBABILITY, and a projector profile only under PROJECTOR_PROFILE with unit NONE (Chapter 4, Table 2). The closed admissibility table means geometry values cannot ride generic scalar wiring, and scalars cannot pose as geometry.

Port validation on canonical bytes. When a geometry value crosses a port, validation happens against its serialized canonical identity: a mixed-simplex port requires a payload of mixed-simplex type whose embedded identity revalidates (canonical cell labels, forty tagged positive weights, exact sum one, matching decimal-context digest); a projector port likewise requires a full projector identity whose recorded residuals, tolerance cap, rank bounds, and boundary block all re-check, and whose matrix-digest linkage is intact. A tampered geometry payload does not reach a component; it fails the port.

Measurements. A geometry value observed at a test point enters the trace as a measurement record whose value text is the canonical identity payload — so the full forty weights, or the full projector identity, are in the evidence verbatim, under the value digest. Any distance later computed *from* traced addresses is a derived result governed by Protocol 6.14; the trace holds the exact operands, so the derivation is replayable by any auditor with a conforming implementation.

The adapter boundary. Neither geometry type is finite-adapter-eligible, and the TKS adapter rejects both by type: continuous profiles are exactly the objects the finite-arrow boundary exists to keep out (Chapter 4, Rule 4.17). The geometry chapter therefore ends where the adapter chapter begins: with two vocabularies that touch at zero points, by construction.

9. Separations between the two profiles

It is worth closing the technical development with the separations the two geometry profiles maintain, since their coexistence in one chapter could otherwise suggest a unity the candidate does not claim.

INVARIANT 6.25 (Profile separations). (i) A mixed-simplex address is never a projector profile, and neither is ever coerced to the other; they occupy distinct signal kinds (MIXED_SIMPLEX_ADDRESS versus PROJECTOR_PROFILE) with distinct admissible units, so no net can deliver one where the other is expected. (ii) The Fisher–Rao baseline consumes addresses only; the overlap consumes projector profiles only; each raises on the other's type. The baseline's own documentation disclaims being a "projector bridge." (iii) The baseline is part of the candidate's named geometry (`cas-pcba-v8.mixed-simplex.v1` addresses; `cas-pcba-v8.binary64-fisher-rao-pseudometric.v1` distance); the projector profile is optional research apparatus (`cas-pcba-v8.projector-validation.v1`) that no baseline artifact depends on. (iv) Both profiles bind the decimal-context digest and the fixed research boundary into their identities; neither can be constructed in an "accepted" or "licensed" state.

EXERCISE 6.26 (Boundary behavior of the distance). Show that on the *open* simplex the supremum π of d_{FR} is not attained: for any two valid addresses the Bhattacharyya coefficient is strictly positive, while sequences of addresses concentrating on disjoint cells drive it to 0.

Conclude that "maximally distant profiles" is not a well-posed superlative in the candidate's vocabulary, and relate this to the exclusion of one-hot addresses (Chapter 4, Rule 4.16).

EXERCISE 6.27 (Geodesic midpoint). Using Remark 6.4, derive the closed form of the geodesic midpoint of two addresses p, q and verify it lies in the open simplex; then explain why the arithmetic mixture $(p+q)/2$ generally differs from it, and compute both for the pair of Example 6.6.

REMARK 6.28 (Why the quarantine is architecture, not caution). The two vocabularies answer different questions with different mathematics: distances between normalized profiles (a Riemannian question on a statistical manifold) versus alignment between declared subspaces (a linear-algebra question in a declared frame). Merging them — for example, defining an ad hoc "distance" between an address and a projector — would require exactly the identification of profile space with representation space that the boundary below prohibits. The type-level quarantine makes the prohibited identification not merely unclaimed but unrepresentable in the candidate's vocabulary, which is the strongest form of claim discipline software can provide.

10. Claim boundary

CLAIM BOUNDARY 6.29. This chapter presents mathematics and software contracts of the CAS PCBA 8 v0.1 research candidate, at Ring 4, NOT LICENSED. The exact Fisher–Rao results (Definition 6.1–Corollary 6.3) are classical statements about probability distributions [**42**, **43**, **25**, **26**]; their subjects here are declared software profiles, not measured states of any system. The implemented distance is a disclosed binary64 numerical pseudometric at its resolution and is not a proof that all distinct profiles are separated (Finding 6.9); every claim built on its values is bounded by Rules 6.11–6.13. The projector Gram-kernel result requires common rank $d > 0$ and establishes algebraic relations between declared, validated artifacts only; it does not establish latent cognitive capacity (Rule 6.22). The CAS capacity transfer retains a proof gap governed solely by the formal-mathematics Paper 3 record, with respect to which all engineering records, including this chapter, are read-only dependency summaries (Hypothesis 6.23, Rule 6.24). Geometry profiles do not identify learned model states with CAS states; no physical, causal, safety, clinical, regulatory, or product claim is made or licensed by anything in this chapter.

CHAPTER 7

The Design-Rule Checker

The design-rule checker (DRC) is the gatekeeper of the entire CAS PCBA 8 candidate. Nothing downstream of it—not the reference simulator, not the measurement and trace machinery, not the TKS v8 adapter—will accept a board that the DRC has not passed, and nothing upstream of it can negotiate a pass. The checker's design goal is deliberately modest and deliberately rigid: given one typed board object, compute one deterministic diagnostic report under one closed catalog of diagnostic codes, and make the report itself an identity-bearing artifact that can be re-verified byte for byte. This chapter specifies that contract in full. We first present the architectural commitments (Section 1), then walk through every one of the thirty-five diagnostic codes in the implementation's catalog, one section per code family and one subsection per code, giving for each its purpose, its exact trigger condition, an illustrative scenario, the rationale for its severity, and its fail-closed behavior (Sections 2–12). We close by regrouping the catalog into the five check classes that organize the checker's execution (Section 13) and by restating the claim boundary that governs every interpretation of a DRC result (Section 14).

Throughout this chapter, every diagnostic code is reproduced exactly as it appears in the implementation's catalog. The codes are stable strings of the form `PCBA8-CLASS-NNN`; they are part of the candidate's public contract, and any change to the catalog is by construction a change to the engine identity that enters every report digest. All scenarios below are *illustrative*: they are constructed examples that show how a code fires, and they assert nothing about any published board, any cognitive system, or any physical circuit.

1. Architecture: one deterministic report

1.1. The report as an artifact. The DRC entry point takes exactly one validated `Board` value and produces exactly one report object. The report is not a stream of console messages, not a mutable accumulator, and not a boolean; it is a frozen record that embeds the exact board it judged, the complete ordered tuple of diagnostics, and a family of SHA-256 identities binding the verdict to everything that could have influenced it: the canonical board payload, the package source digest, the closed component-catalog digest, the engine digest, the immutable decimal-context digest, a DRC-profile digest derived from all of the preceding, and finally a report digest computed over the whole canonical payload.

Definition 7.1 (Diagnostic). A *diagnostic* is a quadruple (c, s, u, m) where c is a code drawn from the closed catalog of Table 1, s is a severity drawn from the closed set $\{\text{ERROR}, \text{WARNING}, \text{INFO}\}$, u is a nonempty printable canonical-ASCII subject string of at most 512 characters naming the board element under judgment, and m is a nonempty message string free of control characters. Diagnostics are exact values: subclassing is rejected, the code and severity must use the exact enumeration types, and every accessor revalidates the value before use.

Definition 7.2 (DRC report). A *DRC report* is a frozen record containing: the exact embedded board; the board reference and board digest; the source, catalog, engine, decimal-context, and DRC-profile digests; the tuple of diagnostics in canonical sort order; the report digest; and the fixed Ring-4 research boundary. A report whose linkage is stale—whose embedded board no longer hashes to the recorded board digest, whose diagnostics differ from a fresh recomputation over the embedded board, or whose recorded digests differ from the current package identities—fails its own validity check and cannot be used.

The last clause of Definition 7.2 deserves emphasis because it is unusual among diagnostic tools. The report's `verify_identity` operation does not merely recheck a hash; it *re-runs the checker* over the embedded board and compares the resulting diagnostics field by field against the stored tuple. A report is therefore not a cached opinion that might drift away from the code that produced it. It is a reproducible claim: "this exact engine, at this exact source digest, over this exact board, produces exactly these diagnostics." If any of those identities has changed—a source file edited, the component catalog revised, the decimal context replaced—the old report refuses to answer questions rather than answering them under a silently different regime.

Invariant 7.3 (Deterministic ordering). The diagnostics tuple in every report is sorted by the total order (severity rank, code string, subject string, message string), with severity ranked ERROR before WARNING before INFO. A report whose diagnostics are not in this order is invalid at construction. Consequently two runs of the checker over equal boards produce byte-identical reports, and a textual diff of two reports is a semantically meaningful diff of their verdicts.

Determinism here is not an optimization or a convenience; it is a load-bearing evidential property. The candidate's evidence discipline (Chapter 14 restates the local form) treats a DRC report as a citable object. That is only coherent if the object is a pure function of its declared inputs. The checker therefore iterates all board collections in sorted order, uses no ambient randomness, no wall-clock values, no environment-dependent state, and performs all numeric comparisons under the one immutable decimal context whose identity enters the report.

1.2. The closed catalog.

Rule 7.4 (Closed catalog). The set of diagnostic codes is closed: it consists of exactly the thirty-five codes of Table 1 and nothing else. The checker cannot mint a new code at runtime, a caller cannot register additional checks, and the component-contract catalog against which components are judged cannot be overridden by a caller-supplied substitute—an attempt to pass a different catalog is rejected before any checking begins.

Closure serves two audiences. For the implementer, it makes the checker's coverage auditable: the complete list of things the checker can say is finite, versioned, and hashed. For the reader of a report, it makes silence meaningful in a bounded way: if a board passes, then none of the thirty-five failure conditions was detected—no more, and importantly, no less. The checker's silence about conditions outside its catalog (for example, semantic adequacy of a state binding, or physical realizability of anything) is not a verdict, because those conditions were never in the catalog to begin with. Rule 7.4 is thus the structural companion of the claim boundary: a closed catalog is what allows "DRC-clean" to be a precise predicate rather than an open-ended endorsement.

RULE 7.5 (Errors fail the board; warnings never become implicit clean). A report is *clean* if and only if it contains no diagnostic of severity ERROR. Every error fails the board for all downstream purposes: simulation refuses the board, the adapter refuses the board, and no override path exists. Severities WARNING and INFO exist in the severity vocabulary and are preserved verbatim in the report and its digest; they never disappear, are never aggregated away, and never convert a flagged condition into an implicit pass. In the current engine every catalog check emits at severity ERROR; the weaker severities are reserved vocabulary, and any future use of them must preserve this rule.

The second half of Rule 7.5 addresses a familiar failure mode of industrial linters, where warnings accumulate, are suppressed, and eventually rot into a de facto extension of the pass predicate. The candidate forecloses that path structurally: the clean predicate is defined only over errors, but the report digest covers all diagnostics of every severity, so a "pass with warnings" is a different artifact than a pass without them, and any consumer that wants to treat them identically must do so explicitly and visibly, against a report whose warnings are permanently on the record.

1.3. The catalog at a glance. Table 1 lists the complete closed catalog. Each code is specified in detail in the sections that follow.

Table 1: The closed `PCBA8-*` diagnostic catalog (35 codes).

Code	Name	Class
`PCBA8-MDL-001`	invalid board metadata	Model
`PCBA8-MDL-002`	duplicate reference	Model
`PCBA8-SEM-001`	unknown component kind	Semantics
`PCBA8-SEM-002`	unresolved semantics	Semantics
`PCBA8-PAR-001`	unknown parameter	Parameters
`PCBA8-PAR-002`	missing parameter	Parameters
`PCBA8-PAR-003`	invalid parameter	Parameters
`PCBA8-STATE-001`	unknown state binding	States
`PCBA8-STATE-002`	missing state binding	States
`PCBA8-STATE-003`	invalid state binding	States
`PCBA8-BIAS-001`	bias incoherent	Bias
`PCBA8-NET-001`	unknown endpoint owner	Nets
`PCBA8-NET-002`	unknown port	Nets
`PCBA8-NET-003`	invalid driver count	Nets
`PCBA8-NET-004`	missing net load	Nets
`PCBA8-NET-005`	signal kind mismatch	Nets
`PCBA8-NET-006`	unit mismatch	Nets
`PCBA8-NET-007`	port multiply connected	Nets
`PCBA8-NET-008`	required port unconnected	Nets
`PCBA8-NET-009`	invalid signal/unit pair	Nets
`PCBA8-TP-001`	unknown test-point net	Test points
`PCBA8-TP-002`	missing safety test point	Test points
`PCBA8-TP-003`	safety test point not marked	Test points
`PCBA8-CAL-001`	unknown calibration	Calibration
`PCBA8-CAL-002`	invalid calibration	Calibration
`PCBA8-CAL-003`	calibration unit mismatch	Calibration
`PCBA8-CAL-004`	missing test-point calibration	Calibration

Code	Name	Class
PCBA8-FBK-001	undeclared feedback	Feedback
PCBA8-FBK-002	feedback without delay	Feedback
PCBA8-FBK-003	unsafe feedback gain	Feedback
PCBA8-FBK-004	invalid feedback declaration	Feedback
PCBA8-SAF-001	unreachable safety threshold	Safety
PCBA8-SAF-002	unsafe fuse action	Safety
PCBA8-SAF-003	unresolved safety analysis	Safety
PCBA8-TKS-001	invalid finite transition	TKS adapter

2. Model integrity: the MDL family

The model family judges the board record itself, before any component semantics are consulted. Its two codes enforce the minimal preconditions for every later check: that the board is nameable and that its element references form usable keys.

2.1. PCBA8-MDL-001: invalid board metadata.

Purpose. Every downstream artifact—the DRC report, the simulation result, the adapter certificate—identifies its board by reference and version and hashes the canonical board payload. A board whose own name or version is blank, or whose element collections contain blank or non-string references, cannot participate in that identity discipline; PCBA8-MDL-001 exists to stop such a board at the door.

Trigger condition. The code fires when the board reference or the board version is empty after stripping whitespace, and it fires again, once per offending entry, when any element of the six reference groups—board ports, components, nets, test points, calibrations, feedback declarations—has a reference that is not a string or is blank after stripping. The subject string names either `board` itself or the offending group and index, so a report reader can locate the defect without parsing the message.

Example 7.6 (Illustrative scenario). Illustrative only. A board is constructed programmatically from a spreadsheet export in which the version column of the header row was left empty. The board object carries `ref "demo-board"` and `version " "` (a single space). The checker strips the version, finds it blank, and emits PCBA8-MDL-001 with subject `board` and the message that board reference and version must be non-blank. No further meaning is assigned: the diagnostic asserts a defect of the record, not of any modeled system.

Severity rationale. ERROR. Identity is the precondition for every other guarantee the candidate offers; a board that cannot be exactly named cannot be exactly hashed, cited, or re-verified. Downgrading this condition would poison every digest downstream.

Fail-closed behavior. The board fails. Because the checker continues collecting diagnostics after this code fires, the report may contain further findings, but no consumer will act on them: the clean predicate is already false, simulation refuses the board, and the adapter refuses the board. There is no repair path inside the checker; the caller must construct a corrected board, which by construction has a different digest.

2.2. PCBA8-MDL-002: duplicate reference.

Purpose. References are the keys of the whole board namespace: nets name their endpoints by component reference, test points name nets, feedback declarations name components and calibrations. If two elements of the same group share a reference, every such link becomes ambiguous, and any checker or simulator that "resolves" the link would be silently choosing one of two meanings. PCBA8-MDL-002 makes the ambiguity a hard defect instead.

Trigger condition. For each of the six reference groups independently—board ports, components, nets, test points, calibrations, feedback declarations—the checker counts occurrences of each reference and emits one diagnostic per value that occurs more than once, with subject *group*:*value*. Duplication is judged within a group; the same string appearing once as a net reference and once as a calibration reference is not a duplicate, because the groups are disjoint namespaces.

EXAMPLE 7.7 (Illustrative scenario). Illustrative only. During a refactor, a component list is concatenated from two fragments and both fragments contain a component with reference cjt_main. Nets in the merged board attach endpoints to cjt_main. The checker emits PCBA8-MDL-002 with subject component:cjt_main. Note what the checker does *not* do: it does not pick the first, the last, or the "most compatible" of the two components and continue; each duplicate is a standing error until the board is rebuilt.

Severity rationale. ERROR. Silent disambiguation is precisely the class of hidden repair the candidate's evidence discipline forbids. A duplicate key does not have a best reading; it has two readings, and any choice between them is new design intent that must come from the board author, not from the checker.

Fail-closed behavior. The board fails. Downstream resolution maps in the same run are built with first-occurrence semantics purely so that diagnostic collection can continue and report additional independent defects; those maps are never used to produce a clean verdict, because the error already blocks it.

3. Semantic resolution: the SEM family

The semantics family connects each component instance to its contract in the closed component catalog. A contract fixes the component's port list (directions, signal kinds, units), its exact parameter schema, its required state bindings, and its semantic-profile string. The two SEM codes police the existence and the exactness of that connection.

3.1. PCBA8-SEM-001: unknown component kind.

Purpose. The candidate's component vocabulary is closed: the catalog declares the kinds pass, scale, cjt, decision_threshold, mux, fuse, delay, and finite_transition, each with a versioned semantic profile. A component whose kind is not in the catalog has no contract, hence no ports, no parameter schema, and no executable meaning. PCBA8-SEM-001 rejects such components rather than guessing.

Trigger condition. For each component, sorted by reference and kind, the checker looks up the kind in the closed catalog. If the lookup fails, it emits PCBA8-SEM-001 with subject component:*ref* and a message quoting the unknown kind, and it *skips all further component-level checks for that component*: no parameter, state-binding, or profile checks are attempted, because there is no contract to check against.

EXAMPLE 7.8 (Illustrative scenario). Illustrative only. A board author, anticipating a future capacitor-like element, declares a component of kind capacitor. The kind is documented in prose in the source book but is not in the candidate's closed catalog (Chapter 14; the stateful-device gap is an explicitly open obligation). The checker emits PCBA8-SEM-001

for the component. Every net endpoint that names one of the component's would-be ports will additionally fail in the net family, since no contract port can be resolved.

Severity rationale. ERROR. An unknown kind is not a stylistic issue; it is a component with no semantics at all. Accepting it would make the board's meaning depend on whatever a future catalog might say, which inverts the direction of authority: boards must be judged against the catalog that exists, not against one that might.

Fail-closed behavior. The board fails, and the skip rule ensures the checker never fabricates a partial judgment of an unknown component. Absence from the catalog is treated as absence of meaning, never as permission.

3.2. PCBA8-SEM-002: unresolved semantics.

Purpose. Even when a kind is known, the component instance must claim *exactly* the semantic profile its contract declares—for example `pcba8.cjt.reference.v0` for a `cjt`—so that every trace event and certificate downstream carries an unambiguous versioned statement of which semantics were executed. `PCBA8-SEM-002` fires when that claim is absent, wrong, or contradicted by extra semantic baggage.

Trigger condition. The code fires in two situations. First, when a component's semantic-profile string differs from the exact profile string in its contract; the message quotes both. Second, when a component that is *not* of kind `finite_transition` nevertheless carries an attached finite-transition table; a transition table on a non-finite component is semantic content the component's contract does not license.

EXAMPLE 7.9 (Illustrative scenario). Illustrative only. A migration script upgrades boards to a hypothetical future profile and rewrites every profile string to `pcba8.cjt.reference.v1`. The engine's catalog still declares `pcba8.cjt.reference.v0`. Every `cjt` on the migrated board receives `PCBA8-SEM-002`. The defect is real even though the component parameters are untouched: a profile string is a versioned semantic commitment, and the candidate refuses to execute v0 semantics while labeling the results v1.

Severity rationale. ERROR. Profile strings enter trace events, measurement scopes, and adapter certificates; a mismatch here would let an artifact claim semantics that were never executed, which is a provenance corruption rather than a cosmetic defect.

Fail-closed behavior. The board fails. There is no "closest profile" resolution and no default: the profile either matches exactly or the component's semantics are unresolved.

4. Typed parameters: the PAR family

Each contract declares an exact parameter schema: a set of keys, each with a type (finite decimal or string), optional bounds or choice sets, and a required flag. The three `PAR` codes cover the three ways an instance can disagree with its schema: a key the schema does not know, a required key the instance lacks, and a value the schema rejects.

4.1. PCBA8-PAR-001: unknown parameter.

Purpose. A parameter key outside the contract is unexecutable configuration: no transfer rule reads it, no bound propagation uses it, and no simulator honors it. Accepting it silently would let boards accumulate dead knobs that readers reasonably but wrongly believe have effect. `PCBA8-PAR-001` makes every such knob a defect.

Trigger condition. For each component with a resolved contract, the checker computes the set difference between the instance's parameter keys and the contract's declared keys, and emits one diagnostic per unknown key, in sorted order, with subject `component:`*`ref`*`.parameter:`*`key`*.

EXAMPLE 7.10 (Illustrative scenario). Illustrative only. A `scale` component is given parameters `factor`, `ceiling`, and `gain`. The contract declares only `factor` and `ceiling`; the author intended `gain` as a synonym for `factor`. The checker emits `PCBA8-PAR-001` for `gain`. It does not alias, coalesce, or warn-and-ignore: the author's intent is recoverable only by the author.

Severity rationale. ERROR. The alternative—ignoring unknown keys—is exactly how configuration drift becomes invisible. A key that does nothing must not survive on a board whose purpose is to be an exact, replayable description.

Fail-closed behavior. The board fails. Unknown keys are never forwarded to simulation, so there is no state in which an unrecognized parameter influences or fails to influence an execution ambiguously.

4.2. `PCBA8-PAR-002`: missing parameter.

Purpose. Required parameters are the ones the reference semantics read unconditionally: a `cjt` without `beta`, `saturation`, and `cutoff` has no defined transfer behavior, and a `fuse` without `threshold` and `action` has no defined trip rule. `PCBA8-PAR-002` guarantees that a clean board never reaches the simulator with a hole where a required value should be.

Trigger condition. For each contract-declared parameter with the required flag, in sorted key order, the checker emits the code if the key is absent from the instance's parameters. Optional keys are exempt; supplying them subjects them to `PCBA8-PAR-003` validation, and omitting them is legal.

EXAMPLE 7.11 (Illustrative scenario). Illustrative only. A board template instantiates a `delay` component but the templating layer fails to fill the `initial` parameter, which the contract requires because the unit delay must emit a defined value at tick zero. The checker emits `PCBA8-PAR-002` with subject `component:`*`ref`*`.parameter:initial`. Without this check, the simulator would face an undefined tick-zero output; with it, the board never reaches the simulator.

Severity rationale. ERROR. There are no engine-side defaults for required parameters, by design: a default is a design decision, and design decisions live on the board where they are hashed, not in the engine where they would be invisible.

Fail-closed behavior. The board fails; the missing value is never synthesized. This is the DRC-level face of the general rule that the candidate does not repair inputs.

4.3. `PCBA8-PAR-003`: invalid parameter.

Purpose. A present parameter must satisfy its rule exactly. The candidate's arithmetic discipline is strict—finite decimals only, under one immutable decimal context—and the contract may impose bounds (a minimum, a maximum, possibly exclusive) or a closed choice set for strings. `PCBA8-PAR-003` is the single code for every value-level violation.

Trigger condition. For a decimal-typed rule, the code fires when the value is not a finite `Decimal`—binary floats are rejected outright, as are infinities and NaN—or when it lies below the declared minimum, or at or above an exclusive maximum, or above an inclusive maximum. For a string-typed rule, it fires when the value is not a nonempty string, or is outside the declared choice set. Additionally, on a `cjt` whose `saturation` and `cutoff` are both finite

decimals, the code fires when `cutoff` is not strictly below `saturation`; this cross-parameter check guards the coherence of the piecewise transfer regions.

Example 7.12 (Illustrative scenario). Illustrative only. An author encodes an idealized amplification stage by setting a `cjt` gain `beta` to positive infinity, mirroring an idealization that the audited source book itself uses and then disclaims. The value is not a finite decimal, so the checker emits `PCBA8-PAR-003` with the message that numeric parameters require a finite `Decimal` and that binary floats and non-finite values are rejected. The known-fault Cognitive Safety Board fixture of Chapter 11 exercises exactly this rejection; here it suffices to note that the rejection is unconditional.

Severity rationale. ERROR. Non-finite and float-typed values would silently change meaning under decimal arithmetic, and out-of-range values would make the conservative bound propagation of Section 13 unsound. Both are integrity failures, not stylistic ones.

Fail-closed behavior. The board fails. No clamping, rounding, or coercion is performed; the recorded value is judged exactly as written.

5. Typed state bindings: the STATE family

Components that carry CAS-semantic annotations do so through *state bindings*: named slots (for a `cjt`, the slots `cause`, `bias`, and `effect`) that must be filled with exact typed `CasState` values drawn from the candidate's type system (Chapter 4). The `STATE` codes are the exact analogue of the `PAR` codes, transposed from configuration values to semantic annotations, and their existence enforces a central design commitment: semantic labels are typed data with a schema, not free-form tags.

5.1. PCBA8-STATE-001: unknown state binding.

Purpose. A binding key the contract does not declare is a semantic annotation with no reader. Permitting it would create a shadow annotation layer—labels that look load-bearing but bind to nothing.

Trigger condition. For each component with a resolved contract, the checker computes the set difference between the instance's state-binding keys and the contract's required binding keys and emits one diagnostic per unknown key, sorted, with subject `component:`*`ref`*`.state:`*`key`*.

Example 7.13 (Illustrative scenario). Illustrative only. An author adds a binding `mood` to a `pass` component to record an informal impression of the signal's affective tone. The `pass` contract declares no state bindings. The checker emits `PCBA8-STATE-001`. If the annotation matters, it belongs in a typed extension of the contract vocabulary, where it would be versioned and hashed; it does not belong in an unread side channel.

Severity rationale. ERROR. Untyped semantic residue is precisely the drift vector the candidate's typed model exists to eliminate.

Fail-closed behavior. The board fails; the binding is never interpreted.

5.2. PCBA8-STATE-002: missing state binding.

Purpose. Where a contract requires a binding, the binding is part of the component's meaning: a `cjt` without a declared `cause` state is not a semantically annotated junction but a bare arithmetic gadget, and the candidate refuses to blur that distinction.

Trigger condition. For each key in the contract's required binding set, sorted, the code fires when the instance's bindings lack the key.

EXAMPLE 7.14 (Illustrative scenario). Illustrative only. A board is assembled by cloning a `cjt` from an example and deleting its bindings with the intent to fill them in later. The intent is forgotten. The checker emits `PCBA8-STATE-002` once per missing required key. The board cannot pass in the half-annotated state, so "later" cannot silently become "never" on a clean board.

Severity rationale. ERROR. Missing required semantics is a completeness failure of the board as a semantic document, independent of whether any simulation would numerically succeed.

Fail-closed behavior. The board fails; no default state is assumed, since a default semantic annotation would be an engine-authored semantic claim.

5.3. `PCBA8-STATE-003`: invalid state binding.

Purpose. Bindings must be *exact* typed values. The type system's tagged-value discipline (integer 1, Boolean true, and string `"true"` are everywhere distinct) extends to states: a string that happens to spell a state's name is not a state.

Trigger condition. For each required binding key that is present, the code fires when the bound value is not of the exact `CasState` type—subclasses included; the check is on exact type identity, not on interface compatibility.

EXAMPLE 7.15 (Illustrative scenario). Illustrative only. A JSON import path deserializes state bindings as plain strings, so a `cjt`'s `cause` slot holds the string `"C8"` rather than a typed state value. The checker emits `PCBA8-STATE-003` with the message that the binding must be a typed `CasState`. The import path, not the checker, must be fixed: the deserializer has to reconstruct exact values, because nothing downstream will.

Severity rationale. ERROR. Accepting look-alike values would reintroduce the label/identity conflation that the semantic-identity discipline of Chapter 3 exists to prevent.

Fail-closed behavior. The board fails; there is no coercion from strings or tuples into states.

6. Bias coherence: `PCBA8-BIAS-001`

6.1. `PCBA8-BIAS-001`: bias incoherent.

Purpose. The candidate's domain system carries a structural precedence order over processing domains, and the `cjt` component's semantics distinguish a *cause* input from a *bias* input: the bias is the standing condition against which the cause fires. The modeling convention enforced here is that a bias must not come from a strictly lower-precedence domain than its cause. `PCBA8-BIAS-001` makes a violation of that convention a board defect rather than a silent oddity. The code checks a structural coherence property of the annotation pair; it does not assert that any cognitive system exhibits domain precedence.

Trigger condition. On each `cjt` whose `cause` and `bias` bindings are both present and exactly typed, the checker compares domain precedence ranks and fires when the bias domain's rank is strictly greater (lower precedence) than the cause domain's rank. The message quotes both domain values. The check is deliberately conditional on well-typed bindings: if either binding already failed the `STATE` family, no incoherence claim is stacked on top of it.

EXAMPLE 7.16 (Illustrative scenario). Illustrative only. An author annotates a `cjt` with a cause in a Principled-tier domain and a bias in an Operational-tier domain, intending to

model an operational habit modulating a principled trigger. Under the domain order this places the bias strictly below the cause, and the checker emits PCBA8-BIAS-001. Whether the author's modeling intuition is right is a research question outside the checker; what the checker enforces is that the board either respects the declared convention or is revised together with an explicit change to the convention—not quietly at odds with it.

Severity rationale. ERROR. The precedence convention is part of the semantic contract the board claims by using cjt components with typed bindings; an incoherent pair is a self-contradiction inside the board's own declared vocabulary.

Fail-closed behavior. The board fails. The checker does not swap the bindings, reinterpret the domains, or downgrade the annotation; only the author can decide which of the two annotations was wrong.

7. Typed connectivity: the NET family

Nets are the board's wiring: each net is a set of endpoints, each endpoint naming an owner (a component reference or the distinguished board endpoint) and a port. The nine NET codes jointly enforce a single discipline: every net resolves to exactly one typed driver and at least one load, every endpoint is real and uniquely placed, and the types agree end to end. Driver-hood is directional: a component's *output* port drives, a component's *input* port loads, and—dually—a *board input* port drives (it injects an external sample into the board) while a *board output* port loads.

7.1. PCBA8-NET-001: unknown endpoint owner.

Purpose. An endpoint that names a nonexistent component is a wire into nothing. The code exists so that dangling references are located precisely, at the net and endpoint where they occur.

Trigger condition. While resolving each endpoint of each net (nets sorted by reference), the checker fires this code when the endpoint's owner reference is neither the distinguished board endpoint nor a declared component, with subject net:*ref*.endpoint:*owner.port*.

EXAMPLE 7.17 (Illustrative scenario). Illustrative only. A component filter_a is deleted during cleanup, but one net still lists endpoint filter_a.output. The checker emits PCBA8-NET-001 at that endpoint. Because the endpoint cannot be resolved, it also cannot be counted as a driver; if it was the net's only driver, PCBA8-NET-003 will additionally record the driverless state of the net.

Severity rationale. ERROR. A dangling wire is a structural contradiction between the net list and the component list; there is no conservative reading of it.

Fail-closed behavior. The board fails; the unresolved endpoint contributes nothing to driver or load counting, so no phantom connectivity is created.

7.2. PCBA8-NET-002: unknown port.

Purpose. Even with a real owner, the named port must exist: on the board side it must be a declared board port, and on the component side it must be a port of the component's contract. Port lists are part of the closed catalog, so this code is where wiring meets the catalog's authority.

Trigger condition. The code fires when an endpoint names the board endpoint with a port reference that is not a declared board port, or names an existing component whose contract has no port of that name; the message distinguishes the two cases and quotes the component kind in the latter.

EXAMPLE 7.18 (Illustrative scenario). Illustrative only. An author wires a net to `mux_1.sel`, abbreviating the port name; the `mux` contract declares the port as `select`. The checker emits `PCBA8-NET-002` with the message that the component kind `mux` has no such port. No fuzzy matching is attempted—`sel` is not "close enough," it is absent.

Severity rationale. ERROR. Port identity is the unit of typed connectivity; guessing a port would amount to the checker rewiring the board.

Fail-closed behavior. The board fails; the endpoint is excluded from resolution just as in `PCBA8-NET-001`, with the same secondary-diagnostic consequences.

7.3. `PCBA8-NET-003`: invalid driver count.

Purpose. The candidate's execution semantics assume single-driver nets: each net's value at each tick is the value of its one driver. Zero drivers means the net has no source of value; two or more means contention with no resolution rule. Both are excluded at the DRC rather than "resolved" at runtime.

Trigger condition. After resolving a net's endpoints, the checker partitions them into drivers and loads by port direction (component outputs and board inputs drive). The code fires when the number of resolved drivers differs from exactly one; the message reports the count found.

EXAMPLE 7.19 (Illustrative scenario). Illustrative only. Two `scale` components' outputs are wired onto one net in an attempt to model additive superposition of two influences. The checker emits `PCBA8-NET-003` with a driver count of 2. The candidate has no implicit summing junction; if superposition is intended, it must appear as an explicit component with declared semantics, not as a wiring pun.

Severity rationale. ERROR. Any runtime resolution of contention (last-writer-wins, summation, nondeterministic choice) would be an undeclared semantic rule—exactly the category of hidden behavior the candidate exists to eliminate.

Fail-closed behavior. The board fails. When the count is not one, the checker also skips the driver-relative type comparisons (`NET-005`, `NET-006`) for that net, since there is no unique driver type to compare against; the driver-count error stands on its own.

7.4. `PCBA8-NET-004`: missing net load.

Purpose. A net with a driver but no load is a value computed and discarded—dead wiring. On an artifact whose purpose is exact accountability, dead wiring is not harmless decoration; it is either a mistake (a forgotten connection) or unexecutable intent, and both deserve a defect.

Trigger condition. The code fires when a net's resolved endpoints include no load—no component input port and no board output port.

EXAMPLE 7.20 (Illustrative scenario). Illustrative only. An author adds a probe-like net attached only to a `cjt`'s `effect` output, planning to attach a test point later, and forgets. The net has one driver and zero loads. The checker emits `PCBA8-NET-004` with the message that the net has no input load or board output. Notably, the correct repair here is often exactly what the author intended: attach a test point's net properly or route the value to a board output—either of which makes the value observable and accountable.

Severity rationale. ERROR. Tolerating unconsumed values would create a class of board content invisible to simulation coverage: values that exist in no trace and no measurement.

Fail-closed behavior. The board fails; the dangling computation never enters a trace where its absence of consumers would be undetectable.

7.5. PCBA8-NET-005: signal kind mismatch.

Purpose. Ports carry a signal kind (for example activation, decision, or finite-state), and the type system treats these as disjoint carriers: a Boolean decision is not a small activation, and a finite state is not a number. PCBA8-NET-005 enforces kind agreement between a net's driver and each of its loads.

Trigger condition. For each net with exactly one resolved driver, and for each resolved load on that net, the code fires when the load port's signal kind is not identical to the driver port's signal kind; the message names both kinds and the offending endpoint.

Example 7.21 (Illustrative scenario). Illustrative only. An author wires a decision_threshold component's decision output (signal kind decision, Boolean-valued) directly into a scale component's input port (signal kind activation, decimal-valued), intending "true acts like one." The checker emits PCBA8-NET-005. The intended coercion is a semantic rule, and if wanted it must be a declared component (for example a mux selecting between explicit constants), not an implicit cast buried in a wire.

Severity rationale. ERROR. Implicit cross-kind coercion is a classic source of silent meaning change; the tagged-value discipline of the type system would be vacuous if wiring could launder kinds.

Fail-closed behavior. The board fails; no runtime conversion exists to reach anyway, since the simulator validates value-to-port kind agreement independently.

7.6. PCBA8-NET-006: unit mismatch.

Purpose. Orthogonally to kind, ports carry units, and unit agreement is checked separately so that a mismatch is reported for what it is. A dimensionless activation and a unit-bearing activation must not be conflated even though both are decimal-valued.

Trigger condition. For each net with exactly one resolved driver, and each load, the code fires when the load port's unit is not identical to the driver's unit; driver and load units are both quoted.

Example 7.22 (Illustrative scenario). Illustrative only. A board mixes two conventions: one subsystem expresses activations on a normalized dimensionless scale, another was drafted with a unit-bearing activation port. Wiring them directly produces PCBA8-NET-006 at each crossing net. The defect is a genuine modeling decision surfaced early: either one subsystem is re-expressed, or an explicit conversion stage with declared semantics is inserted. The checker performs no unit conversion itself—conversion factors are design content, not checker content.

Severity rationale. ERROR. Silent unit mixing is among the most empirically damaging classes of engineering defect precisely because each side remains locally plausible; the whole point of carrying units in the type system is to make the crossing loud.

Fail-closed behavior. The board fails; values never flow across a unit-mismatched net in any execution because no clean report can be produced for the board.

7.7. PCBA8-NET-007: port multiply connected.

Purpose. Each endpoint—each (owner, port) pair—may appear on at most one net. A port on two nets would make the port's value ambiguous (for an input) or create hidden fan-out identity (for an output) in a way the board text does not declare. Fan-out is expressed within one net (one driver, many loads), never by enrolling a port in several nets.

Trigger condition. The checker records, for every endpoint, the list of nets on which it appears. The code fires once per endpoint whose list has more than one entry, sorted by owner and port, quoting the net references involved.

EXAMPLE 7.23 (Illustrative scenario). Illustrative only. During a merge of two board fragments, the shared component `gate_1` ends up with its `input` port listed on a net from each fragment. The checker emits `PCBA8-NET-007` with both net references. The two fragments each had a coherent local story about what feeds `gate_1`; the merged board has two stories, and the checker declines to pick one.

Severity rationale. ERROR. This is the wiring-level analogue of `PCBA8-MDL-002`: a key (the endpoint) bound twice, with no principled tie-break.

Fail-closed behavior. The board fails; endpoint-to-net resolution retains only a first occurrence for continued diagnostic collection, and that resolution never supports a clean verdict.

7.8. `PCBA8-NET-008`: required port unconnected.

Purpose. Contracts and board declarations mark ports as required when the semantics cannot execute without them: a `cjt` with an unconnected `cause` has no defined effect, and a required board input with no net feeds nothing. `PCBA8-NET-008` guarantees that a clean board's execution never encounters a port with no value and no declared default.

Trigger condition. The code fires for each required board port whose endpoint appears on no net, and for each required component port (per the contract) whose endpoint appears on no net; subjects distinguish `board-port:`*`ref`* from `component:`*`ref`*`.port:`*`ref`*.

EXAMPLE 7.24 (Illustrative scenario). Illustrative only. An author instantiates a `mux` and wires `when_true` and `when_false` but forgets the `select` port. The contract marks `select` required. The checker emits `PCBA8-NET-008` for the port. Had the code not fired, simulation would reach the `mux` with no selector value; instead the defect is caught while it is still a wiring omission rather than a runtime mystery.

Severity rationale. ERROR. An unconnected required input is an execution hole. The candidate's rule that required inputs must connect *or* have an executable default in the selected simulator profile is enforced conservatively: the current profile declares no such defaults, so connection is the only satisfying condition.

Fail-closed behavior. The board fails; the simulator additionally cross-checks at admission that its input-sample set equals the connected board inputs, so this condition is doubly guarded.

7.9. `PCBA8-NET-009`: invalid signal/unit pair.

Purpose. Not every (signal kind, unit) combination is meaningful: the candidate maintains a closed admissibility table of canonical pairs (for example, decision signals are dimensionless; finite-state signals carry no numeric unit). `PCBA8-NET-009` rejects port declarations outside the table—including, defensively, any noncanonical pair that would ever appear in the component catalog itself.

Trigger condition. The code fires for each declared board port whose (signal kind, unit) pair is not in the closed admissibility table, and for each component-contract port, as instantiated on the board, whose pair is not in the table; the latter case is a self-audit clause whose message states that the closed catalog contains a noncanonical pair.

EXAMPLE 7.25 (Illustrative scenario). Illustrative only. A board declares an input port with signal kind decision and a non-dimensionless unit, the author having copied a unit field from an adjacent activation port. The pair is outside the admissibility table, and the checker emits `PCBA8-NET-009` for the board port. The defensive clause matters for a different audience: if a future catalog revision ever introduced an incoherent pair, every board using

the affected component kind would fail loudly rather than executing under an incoherent type.

Severity rationale. ERROR. The admissibility table is part of the type system's definition of what a port *is*; a pair outside it is not a port with an unusual taste but a non-value of the port type.

Fail-closed behavior. The board fails. The self-audit clause means even a defect in the engine's own catalog surfaces as a board-failing diagnostic rather than as silently accepted typing.

8. Probe integrity: the TP family

Test points are the board's declared observation sites: each names a net and (for executable use) a calibration reference, and each may be marked safety-critical. The three TP codes enforce that observation sites are real, that safety-relevant paths have them, and that safety marking is explicit. The recurring theme is that *observability is declared, never assumed*: a probe's existence is a board fact with obligations, and a safety label is an explicit commitment, not an inference.

8.1. PCBA8-TP-001: unknown test-point net.

Purpose. A test point observing a nonexistent net observes nothing; worse, it *appears* to add observability while adding none. The code keeps the board's observation claims honest.

Trigger condition. For each test point, sorted by reference, the code fires when the named net reference is not among the board's nets, with subject `test-point:`*`ref`*. When it fires, the remaining calibration checks for that point are skipped, since they would be judgments about an observation site that does not exist.

EXAMPLE 7.26 (Illustrative scenario). Illustrative only. A net is renamed from `net_effect` to `net_effect_main` during a cleanup, and one test point retains the old name. The checker emits PCBA8-TP-001 quoting the stale reference. If that test point was also the safety probe for a safety-critical path, the path is now uncovered, and PCBA8-TP-002 will additionally fire on the affected net—the two diagnostics jointly describing both the broken pointer and its safety consequence.

Severity rationale. ERROR. Phantom observability is strictly worse than declared absence of observability, because it defeats the reader's audit.

Fail-closed behavior. The board fails; the phantom point contributes nothing to safety-coverage accounting.

8.2. PCBA8-TP-002: missing safety test point.

Purpose. Safety-critical paths must be observable. The candidate's rule is structural: every net that is declared safety-critical, and every net driven by an output port of a safety-critical component, requires at least one test point. A safety mechanism whose activity cannot be observed at a declared, calibrated site would be unauditable by construction.

Trigger condition. The checker computes the required-net set as the union of (i) nets marked safety-critical and (ii) nets attached to output ports of components marked safety-critical (resolved through the contract's port list). For each required net, sorted, the code fires when no test point names that net.

EXAMPLE 7.27 (Illustrative scenario). Illustrative only. A board contains a safety-critical `fuse`; its `output` port drives a net that carries the possibly-blocked signal onward, but the author placed a probe only upstream of the fuse. The output net is in the required set and has

no test point, so the checker emits PCBA8-TP-002 for it. The placement matters: observing only the fuse's input would show what the fuse receives, never whether it blocked—precisely the observation the safety claim would need.

Severity rationale. ERROR. This code encodes a governance stance, not merely hygiene: the candidate refuses to let a board describe a safety mechanism while omitting the instrumentation that could ever ground a claim about it. A probe's presence still establishes nothing about safety (see the boundary in Section 14); its *absence*, however, forecloses even the possibility of evidence.

Fail-closed behavior. The board fails; there is no waiver mechanism, because an unobservable safety path is exactly the situation in which a waiver would be least defensible.

8.3. PCBA8-TP-003: safety test point not marked.

Purpose. Coverage of a safety-critical net must itself be explicit: at least one test point on such a net must carry the safety-critical mark. The mark is not decorative; it declares that this probe is load-bearing for the safety path, which future tooling and reviewers can rely on and which cannot be quietly withdrawn by deleting an "ordinary" probe.

Trigger condition. For each net in the required set of Section 8.2 that *does* have test points, the code fires when none of those points carries an explicit safety-critical mark of Boolean true; the message states that a safety-critical probe must explicitly carry the mark.

EXAMPLE 7.28 (Illustrative scenario). Illustrative only. A safety-critical net has two probes, both created by a generic instrumentation script that leaves the safety flag at its default false. Coverage exists numerically, but no probe has accepted the safety role. The checker emits PCBA8-TP-003 for the net. The repair is one explicit flag—small in the file, significant in the record: it converts incidental coverage into declared coverage.

Severity rationale. ERROR. Implicit safety coverage is fragile under maintenance; any refactor that removes a probe nobody marked as critical would silently strip a safety path of observation. The explicit mark moves that risk from silent to impossible.

Fail-closed behavior. The board fails until some probe on the net is explicitly marked. The checker never promotes a probe on its own initiative.

9. Calibration integrity: the CAL family

Calibration records are the board's declared measurement metadata: versioned records carrying a profile digest, instrument and protocol identities, an uncertainty, a measurement kind, and a unit. The four CAL codes enforce their existence, well-formedness, unit compatibility, and attachment. One rule towers over the family and is restated here before the codes, because every code below is partly in its service:

RULE 7.29 (Locally asserted calibration is structural). A calibration record constructed on a board is structural, test-only metadata. It can make a board *well-formed*; it cannot make any measurement *authenticated*. In particular, no locally asserted calibration record can confer Evidence-Packet eligibility on anything, and a record that claims packet eligibility for itself is by that very claim invalid.

9.1. PCBA8-CAL-001: unknown calibration.

Purpose. References to calibration records must resolve. This code covers both consumers of calibrations on a board: test points (which name a calibration for their observations) and feedback declarations (which must cite an existing *dimensionless* calibration for their gain bound—a gain is a ratio, and a unit-bearing gain calibration is a category mistake).

Trigger condition. The code fires when a test point's named calibration reference does not resolve to any calibration on the board; and it fires on a feedback declaration when the declaration's calibration reference either does not resolve or resolves to a record whose unit is not the dimensionless unit, with the message that feedback gain requires an existing dimensionless calibration.

EXAMPLE 7.30 (Illustrative scenario). Illustrative only. A calibration record `cal_probe_v2` is renamed `cal_probe_v3` while a test point still cites the old name. The checker emits `PCBA8-CAL-001` at the test point. Separately, a feedback declaration cites a calibration whose unit was copied from a signal-bearing probe; even though the record exists, the unit disqualifies it for gain duty and the same code fires with the dimensionless-calibration message.

Severity rationale. ERROR. A dangling calibration reference converts a declared-instrumentation claim into an empty pointer; the feedback clause additionally protects the gain analysis of Section 10 from resting on typed nonsense.

Fail-closed behavior. The board fails; no default or substitute calibration is ever selected.

9.2. `PCBA8-CAL-002`: invalid calibration.

Purpose. A calibration record that exists must be internally complete and well-typed. This code aggregates all record-level defects into one diagnostic per record, with a semicolon-joined message enumerating every problem found, so a defective record is reported once, fully.

Trigger condition. For each calibration record, the checker accumulates problems: blank reference or version; a profile digest that is not exactly 64 uppercase hexadecimal digits; an untyped measurement kind; an untyped unit; blank instrument or protocol identity; an uncertainty that is not a finite non-negative decimal; and—per Rule 7.29—a packet-eligibility flag set true, with the message that self-attested calibration metadata is structural/test-only and cannot confer packet eligibility. If the problem list is nonempty, the code fires once with the joined list.

EXAMPLE 7.31 (Illustrative scenario). Illustrative only. An author drafts a calibration record with instrument identity left as an empty string ("to fill in after the bench session") and, misunderstanding the flag, sets packet eligibility true so that "the data will count." The checker emits `PCBA8-CAL-002` listing both problems. The second is the important one: eligibility is precisely what a local record cannot self-assert. Authenticated eligibility, if it ever exists, is conferred by an external evidence process, not by a field on the board.

Severity rationale. ERROR. Each enumerated condition either breaks identity discipline (digest form, blank identities), breaks arithmetic discipline (uncertainty), or attempts privilege escalation (packet eligibility). None admits a lenient reading.

Fail-closed behavior. The board fails. The eligibility clause is the fail-closed form of Rule 7.29: the flag is not ignored or reset; its very assertion is the defect.

9.3. `PCBA8-CAL-003`: calibration unit mismatch.

Purpose. A test point's calibration must speak the unit of the net it observes. A probe calibrated in one unit attached to a net carrying another would produce observations whose declared metadata contradicts their provenance.

Trigger condition. For each test point whose net exists, whose calibration reference resolves, and whose net has a resolved driver type, the code fires when the calibration record's unit is not identical to the net's unit (the unit of its driving port); both units are quoted.

EXAMPLE 7.32 (Illustrative scenario). Illustrative only. A probe with a dimensionless calibration is moved from a normalized activation net to a unit-bearing net during a board revision. Nothing else changes; the calibration record is internally valid and the net is well-formed. Only the pairing is wrong, and PCBA8-CAL-003 reports exactly that pairing. This separation of concerns is deliberate: record-level validity (CAL-002) and attachment-level compatibility (CAL-003) fail independently and are repaired independently.

Severity rationale. ERROR. Observations under mismatched units would be born self-contradictory; the simulator's measurement validation would also refuse them, but the defect belongs at design time, on the board, where the wrong attachment was made.

Fail-closed behavior. The board fails; no unit conversion is interposed (conversion is design content, as in PCBA8-NET-006).

9.4. PCBA8-CAL-004: missing test-point calibration.

Purpose. Every executable test point requires a versioned calibration reference. An uncalibrated probe would yield observations with no declared measurement metadata at all—no instrument identity, no protocol, no uncertainty—which the candidate's measurement schema cannot represent honestly.

Trigger condition. For each test point whose net exists, the code fires when the point's calibration reference is absent, with the message that every executable test point requires a versioned calibration reference. When it fires, the resolution and unit checks (CAL-001, CAL-003) are necessarily skipped for that point.

EXAMPLE 7.33 (Illustrative scenario). Illustrative only. A board author sketches observation sites early, before deciding on calibration records, and leaves the calibration field empty on three points. All three receive PCBA8-CAL-004. The design pressure this creates is intentional: declaring *where* to observe is inseparable, on a clean board, from declaring *under what metadata* the observation would be made—even though, per Rule 7.29, that metadata remains structural until authenticated evidence exists.

Severity rationale. ERROR. Metadata-free observation is the seed of untraceable data; the trace schema of Chapter 14 (developed fully in the next chapter) has no slot for an observation of unstated provenance, and the DRC keeps it that way.

Fail-closed behavior. The board fails; simulation never runs, so no uncalibrated observation is ever emitted.

10. Feedback analysis: the FBK family

Feedback is where a wiring formalism either becomes disciplined or becomes dangerous. The candidate's stance is that *every* cycle in the component graph must be declared, delayed, calibrated, and bounded—no exceptions, no inferred loops. The checker builds a directed graph over components (an edge from the driver of each net to every distinct component loading that net, with self-loops when a component drives itself), computes its strongly connected components with the classical linear-time algorithm [**20**], and confronts the set of *actual* cycles with the set of *declared* feedback loops. Four codes partition the ways this confrontation can fail.

10.1. PCBA8-FBK-001: undeclared feedback.

Purpose. A cycle that exists in the wiring but nowhere in the declarations is feedback the board author has not acknowledged—and therefore feedback with no declared delay, no gain bound, and no calibration. This is the single most important structural check in the family: it converts "the author forgot about this loop" from a latent runtime property into a named defect.

Trigger condition. After computing the cyclic strongly connected components (those with more than one member, or one member with a self-edge), the checker matches each against the component sets of the board's feedback declarations. The code fires for each cyclic component matched by no declaration, with a subject listing the cycle's sorted member references and the message that every graph cycle must have an exact, calibrated, sub-unity declaration.

EXAMPLE 7.34 (Illustrative scenario). Illustrative only. An author adds a "context reinforcement" path: the output of a downstream `scale` is routed back as the `bias` input of an upstream `cjt`. The wiring is locally reasonable; globally it closes a loop the author never thought of as feedback. The SCC computation finds the cycle {`cjt_a`, `scale_b`}; no declaration matches; `PCBA8-FBK-001` fires with subject `feedback-cycle:cjt_a,scale_b`. The board cannot pass until the author either removes the loop or declares it—with a delay element, a gain bound, and a calibration, per the remaining codes of this family.

Severity rationale. ERROR. Undeclared feedback is undeclared dynamics. Every guarantee the simulator offers (topological evaluation, unit-delay breaking) is conditioned on the loop inventory being complete; a single unacknowledged cycle voids the condition.

Fail-closed behavior. The board fails. The match required is *exact* set equality between the declared component set and the SCC's member set—a declaration covering a subset or superset does not excuse the cycle (it instead fails its own checks below).

10.2. `PCBA8-FBK-002`: feedback without delay.

Purpose. The reference simulator evaluates the non-delay portion of the graph in topological order and breaks cycles *only* at declared unit-delay components. A declared loop must therefore contain exactly one delay component, and the declaration must name it: the delay is the point where the loop's present is severed from its past.

Trigger condition. For each feedback declaration, the code fires unless all of the following hold: the named delay component exists on the board; it is a member of the declared component set; its kind is `delay`; and it is the *only* component of kind `delay` in the declared set. The message states that feedback requires exactly one declared unit-delay component inside the cycle.

EXAMPLE 7.35 (Illustrative scenario). Illustrative only. An author declares a loop over {`cjt_a`, `scale_b`} with a gain bound and a calibration, but the loop's wiring is purely combinational—no delay element exists in it. The declaration names `scale_b` as its delay component; `scale_b` is not of kind `delay`, so `PCBA8-FBK-002` fires. Structurally, this scenario is a combinational cycle wearing a declaration as a costume; the code strips the costume. Conversely, a loop containing two delay elements also fails: the breaking point must be unique, or the loop's time semantics are ambiguous.

Severity rationale. ERROR. A cycle without a unique declared delay has no well-defined synchronous semantics in the candidate's execution model; there is nothing conservative the simulator could do with it.

Fail-closed behavior. The board fails; the simulator independently treats a surviving combinational cycle as a hard error, but on a clean board that state is unreachable.

10.3. PCBA8-FBK-003: unsafe feedback gain.

Purpose. Declared feedback must be contractive: the declared round-trip gain bound must be a finite decimal in $[0, 1)$, and the *derived* gain of every directed simple cycle through the declared component set—computed as the product of conservative per-component gain bounds—must both stay below one and stay within the declared bound. This is a conservative worst-case discipline, not a stability proof: it bounds what the declared parameters permit, and asserts nothing about any dynamical system beyond the software semantics.

Trigger condition. The code fires in two ways. First, at the declaration level, when the declared maximum round-trip gain is not a finite decimal, is negative, or is greater than or equal to one. Second, at the cycle level: for each directed simple cycle enumerated within the declared set, the checker multiplies per-component bounds—1 for `pass`, `delay`, `fuse`, and `mux`; the `factor` parameter for `scale`; the `beta` parameter for `cjt`—and fires when the product is at least one or exceeds the declared bound, quoting the cycle path and both numbers.

Example 7.36 (Illustrative scenario). Illustrative only. A declared loop contains a `cjt` with `beta` of `1.6` and a `scale` with `factor` `0.5`, plus the delay; the author declares a bound of `0.9`. The derived product is $1.6 \times 0.5 \times 1 = 0.8$, within bounds—no diagnostic. A revision raises `beta` to `2.5`: the product becomes $1.25 \geq 1$ and `PCBA8-FBK-003` fires with the cycle path (for example `cjt_a->scale_b->delay_c->cjt_a`) and the numbers. Note the conservatism: the bound multiplies worst-case factors and ignores attenuating regions such as cutoff and saturation; a loop that would "probably" settle in practice still fails if its declared parameters permit expansion.

Severity rationale. ERROR. A non-contractive declared loop makes every conservative bound downstream (notably threshold reachability) unsound; and a declaration claiming a bound its own parameters exceed is self-contradictory board text.

Fail-closed behavior. The board fails. Where a cycle's gain *cannot be derived*—some member has no defined gain bound—the checker does not assume contractiveness; that case escalates to `PCBA8-SAF-003` (Section 11.3), never to a pass.

10.4. PCBA8-FBK-004: invalid feedback declaration.

Purpose. The declaration itself must be exact: a nonempty duplicate-free component list, every member existing on the board, the declared set actually forming a graph cycle, and at most one declaration per cycle. This code collects the ways a declaration can be malformed or unmoored from the wiring it claims to describe.

Trigger condition. The code fires when a declaration's component list is empty or contains duplicates; when it names components that do not exist on the board; when the declaration is otherwise valid but its component set is *not* an actual cyclic strongly connected component of the graph; and, dually, when a single graph cycle is matched by more than one declaration, in which case the diagnostic is issued against the cycle with the list of competing declaration references.

Example 7.37 (Illustrative scenario). Illustrative only. After a rewiring that removed a feedback path, the author leaves the old declaration in place "for documentation." The declared set no longer forms a cycle in the graph. The checker emits `PCBA8-FBK-004` with the message that the declared component set is not an actual graph cycle. The stance generalizes: declarations are not documentation, they are claims checked against the wiring, and a stale claim is a defect exactly like a missing one. In the dual scenario, two authors independently declare the same loop under different names; the cycle receives the code with both references

quoted, because two declarations create two potentially divergent bounds for one dynamical structure.

Severity rationale. ERROR. The feedback inventory must be a bijection between declared loops and actual cycles; every deviation from bijectivity—missing (FBK-001), malformed, unmoored, or duplicated (all FBK-004)—breaks the accounting on which the family's guarantees rest.

Fail-closed behavior. The board fails. A declaration that failed earlier structural checks is not additionally judged for cycle-existence (the checker avoids stacking a second complaint on a declaration already found malformed), but the earlier error already blocks cleanliness.

11. Safety analysis: the SAF family

The safety family contains the checker's most consequential and most carefully bounded judgments. Its three codes share one epistemic rule, stated once here: *unresolved is not safe.* Whenever the analysis cannot complete—a bound cannot be derived, an enumeration exceeds its resource budget—the outcome is an explicit unresolved-analysis error, never a silent pass. Everything in this family is a statement about the board's declared parameters under the candidate's conservative propagation semantics; nothing in it is a claim that any system is safe.

11.1. PCBA8-SAF-001: unreachable safety threshold.

Purpose. A protective threshold that the upstream signal can never reach is a safety mechanism in name only. This code exists to detect, at design time and from declared parameters alone, fuse thresholds that exceed the maximum value their input could ever attain. It is the checker-level formalization of the audited source's central known fault: a published safety board whose red-stage clamp of 0.1 sits upstream of a fuse threshold of 0.9 (Chapter 11 develops the finding; the seed paper proves the unreachability proposition).

Trigger condition. The checker first computes conservative upper bounds on every activation net (Check Class 7.46 gives the propagation rules: board activation inputs bound at 1; `cjt` bounded by 0 in the cutoff region and otherwise by min(`saturation`, `beta` · cause bound · bias bound); `scale` by min(`ceiling`, input bound · `factor`); `mux` by the maximum of its branch bounds; `pass` and `fuse` transparent; `delay` bounded at 1; iterated to a fixed point). Then, for each `fuse` component with a finite threshold whose input net has a resolved bound, the code fires when the bound is strictly below the threshold, with the message quoting both values and stating that the fuse cannot trip.

EXAMPLE 7.38 (Illustrative scenario). Illustrative only, mirroring the known-fault fixture's structure. A red-stage `cjt` carries `saturation 0.1`; its effect feeds, possibly through `pass` stages, a safety-critical `fuse` with `threshold 0.9`. Propagation bounds the fuse input at 0.1; since $0.1 < 0.9$, `PCBA8-SAF-001` fires with the message that the maximum upstream activation `0.1` is below the fuse threshold `0.9`. The unmodified published parameters are retained in the candidate's test suite as a permanent regression guard: the checker must flag this configuration forever.

Severity rationale. ERROR. Among all defects in the catalog, this one carries the greatest potential for misplaced trust: a reader who sees a fuse on a board naturally credits the design with a protective crossing. The code ensures that credit can only attach to boards where the crossing is at least *reachable* under conservative bounds—which is still, emphatically, not a claim that it is *effective* (see Section 14).

Fail-closed behavior. The board fails. The dual danger—treating an *unresolved* bound as reachable—is separately foreclosed by PCBA8-SAF-003: reachability is only ever concluded from a resolved bound at or above threshold.

11.2. PCBA8-SAF-002: unsafe fuse action.

Purpose. A fuse's post-trip behavior is its `action` parameter. For a safety-critical fuse the candidate admits exactly one action: `permanent_block`, the latching behavior in which a tripped fuse emits zero for the remainder of the run. Any weaker action on a safety-critical fuse—in particular any transient or self-resetting behavior—is rejected, because a protective mechanism that quietly resumes passing signal after the hazardous condition recurs is not the mechanism the safety marking advertises.

Trigger condition. For each component of kind `fuse` that is marked safety-critical, the code fires when the `action` parameter is not exactly the string `permanent_block`.

Example 7.39 (Illustrative scenario). Illustrative only. An author marks a fuse safety-critical but selects a pass-through action so that downstream analysis can continue observing the signal after a trip during exploratory runs. The checker emits PCBA8-SAF-002 with the message that a safety-critical fuse must use `action='permanent_block'`. The exploratory intent has a legitimate expression: an ordinary, non-safety-critical fuse. What the code forbids is the combination—the safety marking with the non-latching behavior—because it is the marking that downstream tooling and reviewers rely on.

Severity rationale. ERROR. This code addresses the plateau-versus-latch confusion documented in the source audit: saturation clamps are plateaus, not latches, and the audited book supplies no fail-closed latch state machine for its safety board. On the candidate's boards, the latch requirement is not optional prose; it is a checked parameter constraint.

Fail-closed behavior. The board fails. The simulator's fuse semantics implement the latch (a tripped safety fuse emits zero for the rest of the run, with run-local state that resets only at a new `simulate` call), so on clean boards the checked contract and the executed behavior coincide.

11.3. PCBA8-SAF-003: unresolved safety analysis.

Purpose. This is the family's epistemic keystone: the code that fires when the checker *cannot decide*. Conservative analysis has boundaries—bounds that do not resolve, enumerations that exceed budgets—and the candidate's rule is that reaching such a boundary is itself an error on a safety-relevant path. The alternative, treating analysis failure as absence of findings, is the classic soundness hole of lint-style tools; the candidate closes it by construction.

Trigger condition. The code fires in three situations. First, when the directed-simple-cycle enumeration for a declared feedback set exceeds its conservative resource bound (the enumerator caps the cycle count and reports failure past the cap), with the message that enumeration exceeded the bound. Second, when a directed cycle's gain cannot be derived because some member component has no defined gain bound, quoting the cycle path. Third, when a safety-critical fuse's input-net bound cannot be resolved by the propagation of Section 11.1, with the message that the safety-critical fuse input bound cannot be resolved. (For a non-safety-critical fuse, an unresolved bound produces no reachability verdict at all—neither pass nor fail—because no safety reliance was declared.)

Example 7.40 (Illustrative scenario). Illustrative only. A safety-critical fuse's input is driven, through a chain, by a `finite_transition` component's output. Finite transitions

carry no activation bound in the propagation rules, so the fuse input's bound never resolves. The checker emits PCBA8-SAF-003 for the fuse. The author's remedies are all explicit: restructure the path so the bound derives, or remove the safety marking if no safety reliance is actually intended. What the author cannot do is leave the question open on a clean board.

Severity rationale. ERROR. "The analysis did not finish" and "the analysis found nothing" are different epistemic states, and only a tool that distinguishes them can be cited honestly. The code is the distinction, made mandatory.

Fail-closed behavior. The board fails. This is fail-closed in its purest form: the failure being closed against is the checker's own incompleteness.

12. The adapter boundary: PCBA8-TKS-001

12.1. PCBA8-TKS-001: invalid finite transition.

Purpose. The finite_transition component kind is the only bridge between PCBA boards and the TKS v8 finite kernel (Chapter 14 defers the full adapter treatment to the next chapters). The bridge is available only to transition tables that are certified total, deterministic, stateless, and non-stochastic over exact finite carriers, with a self-consistent certificate digest. PCBA8-TKS-001 is the DRC-side enforcement of those certification conditions: it validates the table *as board content*, before any adapter is invoked.

Trigger condition. For each component of kind finite_transition, the checker accumulates problems and fires the code once with the joined list if any are found: the transition table is absent entirely (the message states that such components require an exact self-hashed transition table); the certificate's profile differs from the component's semantic profile; the source or target object name is blank; the source or target element lists fail exact finite-value identity or are not pairwise distinct; the mapping's length differs from the source carrier's (non-totality); a mapping entry is not an integer or indexes outside the target carrier; the certificate flags do not assert total and deterministic, or do assert stateful or stochastic; or the certificate digest is malformed (not 64 uppercase hexadecimal digits) or stale (differs from a fresh recomputation over the canonical payload).

EXAMPLE 7.41 (Illustrative scenario). Illustrative only. An author edits one entry of a transition table by hand to fix a mapping and forgets to recompute the certificate digest. The stored digest no longer matches the recomputed one, and the checker emits PCBA8-TKS-001 with the problem that the transition-table certificate digest is missing, malformed, or stale. In a second scenario, an author marks a table stochastic to represent a probabilistic transition; the code fires with the problem that stateful or stochastic transitions are outside the finite adapter. The second scenario is not a missing feature but a boundary: probabilistic kernels require their own types and are deliberately outside the finite arrow (Chapter 9).

Severity rationale. ERROR. The adapter downstream re-derives and re-checks all of these conditions and refuses non-clean boards outright; the DRC code exists so that the failure surfaces at design time, attached to the offending component, in the same report as every other defect—rather than later, as an adapter exception detached from the board's diagnostic record.

Fail-closed behavior. The board fails, and consequently no certificate can be minted for any of its components: the adapter's first act is to require a clean DRC report. The certification conditions are thus enforced twice on independent code paths, and the weaker path (the checker) is never the last line.

13. The five check classes

The thirty-five codes are executed by a pipeline of check groups, and it is worth seeing the catalog the way the checker itself is organized: five classes, each with a characteristic algorithmic core, together covering the board from record integrity to safety reachability. The classes also serve as the reviewer's map: to audit the checker's coverage, one audits five bounded analyses rather than thirty-five scattered rules.

Check Class 7.42 (Structural integrity). Unique nonblank identities for components, ports, nets, test points, calibrations, and feedback declarations; component kinds and exact parameter keys from a closed catalog; typed state bindings; bias-domain coherence; missing or unknown semantics fail closed. Codes: the `MDL`, `SEM`, `PAR`, `STATE`, and `BIAS` families, and the component-level clauses of `SAF-002` and `TKS-001`. Algorithmic core: sorted iteration with set-difference and exact-type judgments; no graph reasoning.

Check Class 7.43 (Typed connectivity). Every endpoint resolves to a declared owner and port; every net has exactly one typed driver and at least one load; driver and load signal kinds and units agree; endpoints are uniquely placed; required ports connect (the current simulator profile declares no executable input defaults, so connection is the only satisfying condition); and every port's signal/unit pair lies in the closed admissibility table. Codes: `NET-001` through `NET-009`. Algorithmic core: endpoint resolution and per-net driver/load partition, followed by pairwise type agreement against the unique driver.

Check Class 7.44 (Calibration and probe integrity). Executable test points require versioned calibration references with digest-shaped profiles, instrument and protocol identities, uncertainty, measurement kind, and net-unit compatibility; safety-relevant paths require explicitly marked probes; and locally constructed calibration records cannot make themselves Evidence-Packet eligible. A probe's presence never establishes safety. Codes: the `TP` and `CAL` families. Algorithmic core: reference resolution across the test-point/calibration/net tripartite structure, plus the safety-coverage set computation over safety-critical nets and safety-critical components' output nets.

Check Class 7.45 (Feedback analysis). The checker builds the directed component graph and computes its strongly connected components with the classical depth-first algorithm [20]; each realizable cycle requires exactly one exact declaration carrying a unique in-cycle unit delay, an existing dimensionless calibration, and a declared sub-unity round-trip gain bound corroborated by conservative per-cycle gain products; enumeration is resource-bounded, and stateful or stochastic loop semantics are unsupported. Unsupported or resource-bounded analysis yields `SAF-003`, never a pass. Codes: `FBK-001` through `FBK-004`, with `CAL-001` and `SAF-003` clauses in support. Algorithmic core: Tarjan SCC computation, bounded directed-simple-cycle enumeration, and decimal gain-product bounding.

Check Class 7.46 (Threshold reachability). For supported activation paths, conservative upper bounds propagate to a fixed point through causes, bias and gain stages, clipping, and cutoffs, under the rules itemized in Section 11.1. A safety-critical fuse whose finite threshold exceeds its resolved upstream bound receives `SAF-001`; an unresolved bound on a safety-critical fuse receives `SAF-003` and is never treated as reachable. Codes: `SAF-001` and the reachability clause of `SAF-003`. Algorithmic core: monotone interval-style bound propagation over the net graph, iterated with a component-count-bounded fixed-point loop under the immutable decimal context.

Remark 7.47 (Conservatism is directional). The bound propagation of Check Class 7.46 over-approximates attainable values (it takes worst cases through every stage), which makes the *unreachability* verdict sound within the software semantics: if even the over-approximation cannot reach the threshold, no admitted execution can. The same propagation says nothing in the other direction—a bound at or above threshold does not show the threshold is attained by any actual input, only that conservative analysis cannot exclude it. The checker uses the propagation exclusively in the sound direction.

14. Claim boundary

Claim Boundary 7.48. A clean DRC report means exactly this: the exact engine at the recorded source, catalog, and decimal-context identities, executed over the exact recorded board, detected none of the thirty-five failure conditions of the closed catalog. It does not mean the board is a correct model of anything; it does not mean any simulated behavior corresponds to a cognitive or physical mechanism; it does not mean any safety mechanism on the board is effective, certified, or fit for reliance; and it confers no Evidence-Packet eligibility, disclosure authority, or license status on any artifact. All artifacts of this chapter are Ring 4 research materials, NOT LICENSED, and every safety-flavored diagnostic in the catalog is a statement about declared parameters under conservative software semantics—never about the world.

The checker's authority is exactly coextensive with its catalog, and the catalog is closed on purpose. What Chapter 8 adds on top of a clean report is narrow deterministic execution with full trace accountability; what Chapter 9 adds is a certified bridge into the TKS v8 finite kernel. Both begin where this chapter ends: with a board that has survived thirty-five ways of failing, and a report that will re-derive that survival, byte for byte, for as long as the identities hold.

CHAPTER 8

Deterministic Reference Simulation

The reference simulator is the candidate's only executable semantics, and it is deliberately narrow. It runs exactly one class of board—DRC-clean boards built entirely from the closed component catalog—under exactly one declared execution discipline: synchronous ticks, topological evaluation of the non-delay graph, and feedback broken only at declared unit delays. Its output is not a number or a plot but a single frozen *result* artifact that binds the run to every identity that could have influenced it and that carries a complete, internally cross-validated record of everything that happened: every input sample, every component event, every test-point observation, every derived measurement with its full parentage. This chapter specifies the admission rules (Section 1), the execution order and feedback discipline (Section 2), the canonical trace schema field by field (Section 4), the validation obligations that every result must satisfy to exist at all (Section 5), the status of locally asserted calibration (Section 8), the fail-closed treatment of unsupported behavior (Section 9), the relationship of the record discipline to W3C PROV (Section 10), and the interpretation rules that bound what any of this can ever mean (Sections 11 and 12).

One sentence from the implementation's own preamble governs everything below: the equations are versioned design semantics, not measured model behavior. The simulator executes what the board declares; it observes nothing about any cognitive or physical system.

1. Admission rules

Simulation is not a service the candidate offers to arbitrary inputs. A run is *admitted* only when a conjunction of exact conditions holds, and every condition is checked before the first tick executes.

Rule 8.1 (Admission). A simulation call is admitted if and only if all of the following hold:

(1) The board is an exact, revalidated `Board` value, and a fresh DRC run over it is clean. The simulator does not accept a caller-supplied report as proof of cleanliness; it recomputes the verdict itself, and on failure it raises an error quoting the complete list of diagnostic codes found.
(2) The tick count is a positive integer of exact type.
(3) The input mapping's key set equals, exactly, the set of *connected* board input ports—board ports of input direction whose endpoints appear on some net. Missing inputs are rejected; surplus inputs are rejected; the error message quotes both the expected and the received sets.
(4) Every input value is a typed `InputSample` carrying a value, a measurement kind, and an optional calibration reference. A sample claiming `MODEL_DERIVED` provenance is rejected at construction: board inputs come from outside the run and cannot claim to have been derived by it.

(5) Every component on the board is of a supported kind—which is guaranteed by DRC cleanliness, since unknown kinds fail `PCBA8-SEM-001`—and therefore has a contract declaring its ports (directions, signal kinds, units), its transfer rule, its clipping behavior, its statefulness, and its time semantics.

The fifth clause deserves expansion, because "supported component profile" is the load-bearing phrase of the whole chapter. Each of the eight catalog kinds carries a versioned semantic profile string (for example `pcba8.cjt.reference.v0`, `pcba8.unit-delay.v0`) and a fully declared behavior:

- `pass`: identity transfer from `input` to `output`; stateless; combinational.
- `scale`: output is $\min(\texttt{ceiling}, \max(0, \texttt{input} \cdot \texttt{factor}))$; the clipping to the declared ceiling and the floor at zero are part of the declared profile, not incidental; stateless; combinational.
- `cjt`: the piecewise junction rule. If the bias input is at or below `cutoff`, the effect is exactly zero (cutoff region); otherwise the effect is $\min(\texttt{saturation}, \max(0, \texttt{beta} \cdot \texttt{cause} \cdot \texttt{bias}))$, and the executed region—cutoff, active, or saturation—is recorded verbatim in the trace event's details; stateless; combinational.
- `decision_threshold`: Boolean decision, true exactly when the input is at or above `threshold`; stateless; combinational.
- `mux`: selects `when_true` or `when_false` by the Boolean `select` input, recording which branch was taken; stateless; combinational.
- `fuse`: passes its input until the input first reaches `threshold`; from that tick on, if the action is `permanent_block`, it emits exactly zero for the remainder of the run. The fuse is the one deliberately stateful non-delay element, and its state is *run-local*: initialized at simulation entry, persistent only across the ticks of that one result, reset on every new call. This scope is itself a named, versioned string that enters the result identity.
- `finite_transition`: applies its certified table to an exact finite input value; an input outside the certified source carrier is a hard error, not a default; stateless; combinational.
- `delay`: the unit delay. It is *not* evaluated by the combinational rule at all—an attempt to evaluate it combinationally is an internal error—but by the synchronous state discipline of Section 2: at each tick it emits the value its input net carried at the previous tick, and at tick zero it emits its declared `initial` parameter.

This is the entire executable vocabulary. Statefulness is confined to exactly two places (the fuse latch and the delay register), both with declared, versioned time semantics; everything else is combinational and clipped or guarded exactly as written. There is no partial support, no best-effort mode, and no extension point: behavior outside this list does not degrade, it fails (Section 9).

1.1. Typed input samples and canonical value text. The input side of admission deserves its own treatment, because it is where the outside world touches the run and where the provenance vocabulary is fixed for everything downstream.

Definition 8.2 (Input sample). An *input sample* is an exact typed value carrying: the value to inject; a *measurement kind*, which for board inputs may be a design value (the default: a number the author chose) or an instrumented value (a number the author asserts came from an instrument, in which case the sample should name a calibration reference and the resulting root measurement will be validated against it); and an optional *calibration reference* into the

board's calibration records. The kind `MODEL_DERIVED` is structurally excluded at sample construction: it is reserved for values the run itself computes, and a board input claiming it would be asserting an impossible provenance.

Two consequences follow. First, the provenance partition of the whole artifact is fixed at the boundary: everything that enters is a root (design value or instrumented), everything computed inside is `MODEL_DERIVED`, and the two vocabularies cannot be confused even by a hostile caller, because the types refuse the confusion. Second, the instrumented kind buys nothing epistemically at this layer—per Section 8, the resulting record is still structural and never packet-eligible—but it is preserved exactly, so that a future authentication process examining the artifact can distinguish which roots *claimed* instrument provenance and which never did. The candidate records the claim without endorsing it.

Values themselves travel as *canonical value text*: a deterministic, type-tagged textual rendering of the typed value (a decimal renders with its exact digits under the immutable context, a Boolean renders distinctly from any string, a finite carrier element renders with its type tag). Canonicalization serves three masters at once. It makes every artifact byte-stable, so digests are meaningful; it makes every recorded value re-parsable without ambiguity, so the artifact is self-contained; and it extends the tagged-value discipline of the type system into the serialized world, so that integer 1, Boolean true, and string `"true"` remain distinct *as text*. A value that cannot be canonically rendered—a binary float, a NaN, an untagged composite—cannot enter a measurement record at all; the rendering failure is a hard error at the point of production (Section 9).

At each tick, every connected board input's sample value is validated against its port before use: the value must be well-typed for the port's signal kind (a decimal for activation, an exact Boolean for decision, a certified carrier element for finite-state signals) and the port's (kind, unit) pair must lie in the closed admissibility table. Validation happens per tick even though the sample is constant across ticks, because the invariant being protected is per-record: every measurement, including every root, must individually withstand the unit and value discipline of Section 5.

Remark 8.3 (Why re-run the DRC at admission). Requiring a fresh DRC pass rather than trusting a presented report removes a time-of-check/time-of-use gap: a board could in principle be validated, mutated, and then simulated under a stale report. Because boards are immutable values and reports embed their board, the gap is already narrow; recomputation closes it entirely and costs only a deterministic recheck. The recomputed report is then embedded in the result, so the result's DRC linkage is to the verdict actually used, not to any earlier one.

2. Execution order and feedback breaking

2.1. Topological ordering of the non-delay graph. The simulator evaluates each tick in a fixed component order computed once, before the first tick. The order is a topological sort of the *non-delay* graph: the directed graph whose nodes are all components other than delays, with an edge from the driver of each net to every distinct non-delay component that loads the net, where edges originating from board inputs or from delay outputs are omitted. Omitting delay-driven edges is what makes the graph acyclic on a clean board: every cycle, by the feedback discipline of Chapter 7, contains exactly one delay, so cutting the delay's outgoing dependency cuts every cycle exactly once.

CONSTRUCTION 8.4 (Deterministic Kahn order). The simulator computes indegrees over the non-delay graph and repeatedly removes the lexicographically least zero-indegree node, appending it to the order and decrementing successors, keeping the ready set sorted at every step. The result is the unique lexicographically-least topological order of the graph. Determinism here is essential rather than cosmetic: the evaluation order is observable through trace sequence numbers, and two runs over equal boards must produce byte-identical results.

RULE 8.5 (Residual cycles are fatal). If the Kahn process terminates with fewer nodes ordered than exist—that is, if a combinational cycle survives delay removal—the simulator raises a hard error stating that a clean DRC still contains a combinational cycle. On a clean board this state should be unreachable, because the DRC's feedback family requires a unit delay in every cycle; the check exists anyway, as an independent guard on an invariant the simulator's soundness depends on. Defense in depth here is cheap and the alternative—evaluating a combinational cycle by some fixpoint or iteration scheme—would be new, undeclared semantics.

2.2. Declared unit-delay feedback breaking. Each tick then executes in three phases, and the phase structure is the entire time semantics of the candidate:

(1) *Delay emission.* Every delay component, in sorted reference order, emits its held value: at tick 0, the declared `initial` parameter; at tick $t > 0$, the value its input net's driver produced at tick $t - 1$. Each emission is recorded as a trace event whose details note that the output is the unit-delay output from the prior tick, and as a measurement whose parentage points at the prior-tick driver measurement (empty parentage at tick 0, where the value originates from the declared parameter rather than from any in-run producer).

(2) *Combinational sweep.* The non-delay components execute in the fixed topological order. Each reads the current-tick values of its input nets—which are guaranteed present, because every net's driver is either a board input (sampled at tick start), a delay (phase 1), or an earlier component in the order—applies its declared transfer rule, and emits outputs, trace events, and measurements. If any input value has not been produced when a component needs it, the simulator raises an error stating that no value reached the port and that ordering is unresolved; like Rule 8.5, this is an internal-invariant guard that must be unreachable on admitted runs.

(3) *State update.* After the sweep, every delay's register is loaded with the current-tick value of its input net (together with the identity of the measurement that produced it, for next-tick parentage), and the tick's board outputs and test-point observations are snapshotted. Fuse latch state carries forward implicitly in the run-local fuse map.

The discipline is the classical synchronous one: feedback influences the future, never the present. A declared loop's value at tick t depends on the loop's state at tick $t - 1$ through exactly one register, and the DRC has already certified that the loop's declared round-trip gain is sub-unity under conservative bounds. The simulator adds no analysis of its own on top of that; it simply executes the declared semantics and records everything.

ALGORITHM 8.6 (The tick loop). Given an admitted board, its fresh clean DRC report, the fixed topological order O over non-delay components, the sorted delay list D, the run-local fuse map F (empty at entry), and the delay register map R (initialized from each delay's `initial` parameter with empty provenance):

For each tick $t = 0, \ldots, T - 1$:

(1) For each connected board input port, in sorted order: validate the sample value against the port; mint the root measurement (kind and calibration from the sample, empty children); publish the value onto the input's net.
(2) For each delay reference $d \in D$, in order: emit the registered value of $R[d]$ onto d's output net; mint the `MODEL_DERIVED` measurement whose children are the registered provenance (empty at $t = 0$, the prior-tick driver measurement otherwise); append the trace event with the prior-tick provenance detail.
(3) For each component $c \in O$, in order: gather current-tick values of c's input nets (error if any is unpublished); apply c's declared transfer rule, updating F if c is a fuse; for each output port, mint the `MODEL_DERIVED` measurement whose children are the same-tick driver measurements of c's input nets; append the trace event with the rule's declared details.
(4) For each test point, in sorted order: read the observed net's current value; mint the observation measurement (child: the net's driver measurement this tick; calibration: the point's declared reference; unit: the calibration's unit); append the observation trace event.
(5) For each delay reference $d \in D$: load $R[d]$ with the current-tick value and driver-measurement identity of d's input net.
(6) Snapshot the connected board outputs as (reference, canonical value text) pairs in sorted order.

After the last tick, assemble the result payload (Section 4), compute the result digest, and construct the `SimulationResult`—whose constructor immediately runs the full validation of Section 5 against the artifact just built. The simulator does not exempt itself from its own audit.

The final sentence of Algorithm 8.6 states a structural humility worth underlining. The validation invariants were designed as external obligations—what any presented result must satisfy—and the simulator's own output passes through them unprivileged. An implementation defect that produced an inconsistent artifact (a measurement missing, a linkage wrong, a digest stale) would therefore not ship a subtly corrupt result; it would fail to construct a result at all, loudly, at the source.

3. A worked illustrative run

Example 8.7 (Illustrative two-tick run). Illustrative only; the board, values, and identities are constructed for exposition and correspond to no published artifact. Consider a minimal board `demo-jct` at version `0.1` with: board input ports `stim` and `ctx` (activation, dimensionless); one `cjt` component `j1` with `beta 2`, `saturation 0.8`, `cutoff 0.05`, cause wired to `stim`, bias wired to `ctx`; one safety-critical `fuse f1` with `threshold 0.75`, action `permanent_block`, input wired to `j1`'s effect; a board output `out` on `f1`'s output net; a test point `tp1` on the effect net with a dimensionless calibration `cal1`; and the safety probe marking required by the `TP` family. Assume the DRC report is clean (in particular, the propagated bound on `f1`'s input is $\min(0.8, 2 \cdot 1 \cdot 1) = 0.8 \geq 0.75$, so `PCBA8-SAF-001` does not fire). The caller supplies design-value samples `stim` $= 0.5$, `ctx` $= 0.9$, and requests two ticks.

Tick 0. Phase 1 mints two root measurements: $(0, \texttt{board}, \texttt{stim})$ with value text for `0.5` and $(0, \texttt{board}, \texttt{ctx})$ with `0.9`, both design-value kind, both with empty children. There are no delays, so phase 2 is empty. Phase 3 evaluates `j1` first (it precedes `f1` in the unique topological order): bias `0.9` exceeds cutoff `0.05`, so the effect is $\min(0.8, \max(0, 2 \cdot 0.5 \cdot 0.9)) = \min(0.8, 0.9) = 0.8$—the saturation clamp binds, and the trace event for `j1` records

`region=saturation` in its details, inputs `cause` = 0.5 and `bias` = 0.9, output `effect` = 0.8, and one linked measurement whose children are the two root measurement identifiers. Then `f1`: input `0.8` reaches threshold `0.75`, the run-local latch trips, and because the action is `permanent_block` the output is `0`; the event records `tripped=true`, and its measurement's child is `j1`'s effect measurement. The test point `tp1` observes the effect net: its event carries the observation profile, input `net` = 0.8, output on the net's reference, and its measurement's child is `j1`'s effect measurement with calibration reference `cal1`. The tick-0 output snapshot records `out` = 0.

Tick 1. Fresh roots are minted for the same sample values (new measurement identifiers: the tick enters the payload, so tick-1 roots are distinct records). `j1` computes `0.8` again, with tick-1 parentage. `f1` now demonstrates the latch: its input is again `0.8`, but the run-local state already records the trip, so the output is `0` with `tripped=true`—and had the inputs instead dropped below threshold, the output would *still* be `0`, which is precisely the plateau-versus-latch distinction the safety family enforces (a saturation clamp would have resumed passing; the latch does not). The observation and snapshot repeat with tick-1 identities.

The resulting artifact contains: $2 \times 2 = 4$ root measurements, $2 \times 2 = 4$ component-output measurements, 2 observation measurements, $2 \times 3 = 6$ trace events (two components and one test point per tick) numbered 0 through 5 in execution order, two output snapshots, the embedded clean report, and the full identity block. Calling `ancestry` on tick-1's `f1` output measurement returns exactly the chain: `j1`'s tick-1 effect measurement, then the two tick-1 roots—the complete and only provenance of the value `0` at `out`.

What this example licenses is exactly Boundary 8.20: the software executed the declared clipped-product, latching semantics on these inputs and recorded these values. It does not show that any junction saturates, that any protective mechanism works, or that `0.8` measures anything.

4. The canonical trace schema

Every admitted run produces exactly one `SimulationResult`. The result is a frozen record whose fields we now specify one by one; every field participates in the canonical result payload whose SHA-256 digest is the result's identity, so nothing listed here is optional annotation—each is load-bearing for the artifact's meaning and its verifiability.

4.1. Board and component identities.

`board_ref`, `board_version`: The board's declared reference and version strings, revalidated as safe references. These are the human-facing coordinates of the run.

`board_sha256`: The digest of the canonical board payload. Together with the embedded DRC report (next subsection), this pins the run to one exact board value: any structural edit to the board—one parameter, one net, one annotation—produces a different digest and therefore a different, non-interchangeable result lineage.

Component identities enter the schema indirectly but completely: the embedded DRC report embeds the exact board, which carries every component with its kind, semantic profile, parameters, and bindings; and every trace event names its subject component by reference with its exact semantic profile string. There is no summarized or abbreviated component view anywhere in the artifact.

4.2. DRC report identity.

`drc_report_sha256`: The report digest of the DRC report computed at admission.

`drc_report`: The exact embedded `DrcReport` object itself—not a pointer, the report. Validation (Section 5) requires it to be clean, to hash to the recorded digest, and to agree with the result on every shared identity: board reference, board digest, source, catalog, engine, and decimal-context digests, and board version.

`drc_sha256`: The DRC profile digest—the digest over the checker's profile constant and the source, engine, catalog, and decimal-context digests—identifying *which checker* produced the verdict, independently of which board it judged.

Embedding the full report is a deliberate redundancy. A result is meant to be citable in isolation; a reader holding only the result can inspect the exact diagnostics (none, since it is clean), the exact board, and the exact checker identity without resolving any external reference.

4.3. Catalog and decimal-context identities.

`source_sha256`: The package source digest: a digest over the implementation's own source files. The result records not just what was run but the exact code that ran it.

`catalog_sha256`: The digest of the closed component catalog—the port lists, parameter schemas, and semantic profiles of Section 1 in canonical form.

`engine_sha256`: The engine digest derived from the source and catalog identities.

`decimal_context_sha256`: The digest of the one immutable decimal context under which every arithmetic operation of the run executed. Two results with different context digests are results of different arithmetic regimes and are never comparable as if they were replicates.

4.4. Simulator profile and source baseline.

`simulator_sha256`: The simulator profile digest, computed over the simulator's profile constant, the source, engine, and catalog digests, the decimal-context digest, the DRC profile digest, and the fuse-state scope string. This is the identity of the executable semantics as a whole.

`fuse_state_scope`: The versioned scope string declaring the run-local fuse discipline (initialized at entry, persistent across ticks within the result, reset on every call). Recording the scope makes the statefulness contract itself part of the artifact: a future revision of fuse state semantics would change the string, hence the digest, hence the lineage.

`measurement_scope_sha256`: The measurement scope digest, computed over the board digest, the source, engine, and catalog digests, the decimal-context digest, the DRC report digest, and the simulator digest. Every measurement in the result carries this scope; a measurement therefore cannot be detached from its run and grafted onto another without its scope betraying the move.

4.5. Typed inputs and outputs.

`ticks`: The positive tick count.

`outputs`: A tuple with exactly one entry per tick, in tick order, each entry pairing the tick index with a tuple of (port reference, canonical value text) pairs covering exactly the connected board output ports, sorted by reference, with no duplicates. Values appear in canonical text form—the exact, type-tagged textual representation of the typed value—so that the artifact is self-contained and byte-stable without requiring the reader to re-execute anything.

Inputs appear in the schema as measurements rather than as a separate field: each connected board input, at each tick, yields a root measurement (next subsection) recording the sample's canonical value, unit, measurement kind, and optional calibration reference. The admission rule that the sample set equals the connected input set guarantees the input record is complete.

4.6. Component events and tick linkage.

`trace:` The tuple of trace events. Each `TraceEvent` carries: a *sequence* number—the events are numbered $0, 1, 2, \ldots$ contiguously across the whole run, so the artifact records not only what happened at each tick but the exact total order in which the simulator did it; a *tick* index; a *subject reference*—a component reference, or a test-point subject of the form `test-point:`*`ref`*; the subject's exact *semantic profile* string; *inputs*: (reference, canonical value text) pairs for each input port read, in contract order, with unique references (empty for delays, whose inputs at the emitting tick are definitionally the prior tick's values); *outputs*: (reference, canonical value text) pairs for each output port written, in contract order; *output measurement identifiers*: exactly one measurement identifier per output pair, positionally aligned, linking each output value to its measurement record; and *details*: an ordered tuple of strings recording declared execution facts—the `cjt` region taken (`region=cutoff`, `region=active`, or `region=saturation`), the `mux` branch selected, the fuse trip status (`tripped=true` or `tripped=false`), the delay's prior-tick provenance note.

Tick linkage is structural, not conventional: validation requires that every measurement referenced by an event was produced at the event's own tick by the event's own subject, so an event cannot cite a value from elsewhere in time or from another producer.

4.7. Test-point observations. Test points are executed as first-class producers. At every tick, each declared test point yields one trace event with subject `test-point:`*`ref`* under the fixed semantic profile `pcba8.test-point-observation.v1`, whose single input is the observed net's current value and whose single output re-emits that value on the net's name, plus one measurement whose parent is the net's driver measurement at that tick and whose calibration reference is exactly the test point's declared calibration. An observation is thus never a copy of a number; it is a recorded derivation from an identified producer under an identified calibration record.

4.8. Measurements: identifiers, derivations, calibration references.

`measurements:` The tuple of measurement records. Each `MeasurementRecord` carries: a *measurement identifier*, which is the SHA-256 digest of the record's own canonical payload—identity is content-derived, so equal records are the same record and any tampering changes the identifier; the *measurement scope digest* tying the record to this run (see above); the *tick*; the *producer reference*: the distinguished board endpoint for input roots, a component reference for component outputs, or a `test-point:` subject for observations; the *port reference* (for test points, the observed net's reference); the *canonical value text* and its *value digest*; the typed *unit*; the *measurement kind*: board-input roots carry the sample's declared kind (design value or instrumented, never model-derived), while every simulated producer's record carries `MODEL_DERIVED`—the kinds partition provenance into what entered the run and what the run computed; the optional *calibration reference*; the *packet-eligibility flag*, which is always false (Section 8); and the *child measurement identifiers*: the exact tuple of parent-input measurements from which this value was

derived—for a component output, the driver measurements of its input nets at the same tick; for a delay output at tick $t > 0$, the driver measurement of its input net at tick $t - 1$; for board-input roots, empty, because roots have no in-run ancestry.

The result exposes an `ancestry` operation that walks child links transitively, so any output value can be traced, measurement by measurement, back to the exact set of input roots that fed it. Derivation direction follows the convention that a record lists the records it was derived *from*; the graph is acyclic by validation (Section 5).

4.9. The fixed Ring-4 boundary.

`boundary`: Every result, every measurement, every trace event, and every embedded report carries the fixed research boundary object—the Ring-4, not-licensed research classification—and every public operation re-derives and rechecks it rather than trusting a stored copy. The boundary is inside the digest: an artifact stripped of its research classification is not a cleaned-up version of the artifact; it is a different byte string that fails identity verification.

`result_sha256`: Finally, the result digest over the entire canonical payload: every field above, the full diagnostics-bearing report, every event, every measurement, the boundary. This is the identity by which the result is cited.

5. Measurement and result validation

A `SimulationResult` cannot exist in an invalid state: construction runs the full validation, and every subsequent public operation re-runs it. The validation is worth specifying in detail because it is the difference between a log (which records claims) and this artifact (which refuses to exist unless its claims are mutually consistent).

Invariant 8.8 (Digest form and identity). Every digest-valued field—board, source, catalog, engine, decimal-context, DRC report, DRC profile, simulator, measurement scope, and result digests, every measurement identifier, every value digest—must be exactly 64 uppercase hexadecimal characters, and every content-derived digest must equal a fresh recomputation over the canonical payload it names. The result additionally requires that the recorded source, catalog, engine, decimal-context, and simulator identities equal the *current* package identities: a result does not merely remember which engine produced it, it refuses to operate under a different one.

Invariant 8.9 (Measurement uniqueness and coverage). Measurement identifiers are unique. Producer coordinates (tick, producer, port) are unique. And coverage is *exact*: the coordinate set must equal, precisely, the set consisting of every connected board input at every tick, every component output port at every tick, and every test point at every tick—nothing missing, nothing extra. A result cannot quietly omit an inconvenient observation, and it cannot smuggle in a record no declared producer accounts for.

Invariant 8.10 (Child existence, acyclicity, and tick monotonicity). Every child identifier resolves to a measurement in the same result; no measurement is its own child; children are unique per record; a depth-first check certifies the derivation graph contains no cycle; and no record depends on a future tick—every child's tick is at most its parent's. Moreover, parentage is not free-form: for each record, the validator recomputes the *expected* children from the embedded board's wiring (same-tick input-net drivers for combinational components, the prior-tick driver for delays, the same-tick net driver for test points, empty for roots) and requires exact equality. Provenance must match the board, edge for edge.

INVARIANT 8.11 (Unit and value discipline). Every record's unit must equal the unit of the port or calibration it reports (board-input records carry the port's unit; test-point records carry their calibration's unit; component records carry the output port's unit), and every canonical value text must validate against the receiving port's signal kind and unit under the closed admissibility table. A Boolean cannot inhabit an activation record, and a unit-bearing value cannot inhabit a dimensionless one, anywhere in the artifact.

INVARIANT 8.12 (Measurement-kind eligibility). Board-input records must not claim `MODEL_DERIVED` and must have empty parentage—roots are roots. Every simulated producer's record must claim exactly `MODEL_DERIVED`. A board-input record claiming instrumented provenance must name a calibration record, and any named calibration must exist on the board and agree with the record in unit and measurement kind. Test-point records must carry `MODEL_DERIVED` with exactly their test point's declared calibration reference. Component-output records must carry no calibration reference at all: a simulated value is not an instrument reading, and the schema will not let it dress as one.

INVARIANT 8.13 (Trace linkage). Trace sequence numbers are exactly $0, 1, \ldots, n-1$. Every event's tick lies inside the run. Subjects are unique within each tick, and the subject set at every tick equals exactly the components plus test points of the embedded board—full producer coverage, every tick. Every event links exactly one measurement per output, positionally; linked measurements must match the event in tick, producer, port, and value text. Each measurement is linked by at most one event, and the set of trace-linked measurements equals exactly the set of `MODEL_DERIVED` measurements: every computed value is explained by exactly one event, and no event explains a value that does not exist. Event input values must equal the value texts of the output record's children (for test points, of the observed driver), so the narrative told by events and the graph told by measurements are forced to agree. Finally, the per-tick output snapshots must list exactly the connected board outputs with values equal to their nets' driver records.

The cumulative effect of Invariants 8.8 through 8.13 is that the result artifact is closed under cross-examination: the identities agree with the payloads, the events agree with the measurements, the measurements agree with the wiring, the wiring agrees with the board, and the board agrees with the DRC report embedded beside it. There is no consistent way to alter one stratum of the artifact without the others convicting it.

6. Reading the artifact: three audit walkthroughs

The schema and its invariants were specified from the producer's side; it is equally important that they support the *reader's* side—the auditor who holds a result and wants to interrogate it. Three canonical walkthroughs show how the fields compose into audit procedures, and each is a pure read of the artifact: no re-execution, no external state.

Walkthrough 1: from an output value to its causes. Start from a tick's output snapshot entry, say `out` at tick t. Resolve the board output's net through the embedded board's wiring, take the net's driver coordinate $(t, \text{driver}, \text{port})$, and fetch that measurement. Its child identifiers give the same-tick (or, through a delay, prior-tick) parent measurements; iterating—or calling the result's `ancestry` operation, which performs exactly this walk—terminates at root measurements, whose kinds and calibration references state exactly what entered the run and under what claimed provenance. Every step is validated in advance by Invariant 8.10: the walk cannot cycle, cannot dangle, and cannot cross ticks in the wrong

direction. The auditor ends with a finite, exact causal set—in the software sense of causation: dataflow under the declared semantics—for the output value.

Walkthrough 2: from a claim about behavior to the event that grounds it. Suppose a report elsewhere asserts "the junction saturated at tick 3." The artifact either grounds this or refutes it in two lookups: the trace event at tick 3 with the junction's subject reference (unique, by Invariant 8.13) either contains `region=saturation` in its details or does not. Because details are inside the digested payload, the grounding is as tamper-evident as the result itself. The same pattern serves fuse trips (`tripped=true`), mux routing, and delay provenance: every discretionary branch the semantics took is a recorded, digested fact, not an inference the auditor must reconstruct from values.

Walkthrough 3: from two results to a defensible comparison. Given results R_1 and R_2, the identity block decides what comparison is even meaningful. Equal board, source, catalog, decimal-context, and simulator digests with different input roots: the comparison is between input conditions under one semantics, and per-tick outputs can be diffed value by value. Different board digests: the comparison is between designs, and the embedded boards themselves state the design delta. Different source or context digests: the results belong to different engines, and any numerical comparison must be labeled cross-engine, whatever the numbers look like. The artifact does not prevent an auditor from making a careless comparison, but it makes carefulness cheap and its absence visible: every published comparison can be checked against the identity blocks of its operands.

These walkthroughs are the practical justification for the schema's apparent redundancy. Each field earns its place by shortening some audit from a reconstruction to a lookup—and the invariants of Section 5 are what entitle the auditor to trust the lookup.

7. Determinism and replayability

The chapter has asserted determinism repeatedly; it is worth collecting the argument in one place, both because the property is load-bearing for the artifact's evidential role and because its *scope* needs stating as carefully as its content.

PROPOSITION 8.14 (Replay determinism). *Let two simulation calls be made with equal boards (equal canonical payloads), equal input samples (equal values, kinds, and calibration references per port), and equal tick counts, under the same package source, component catalog, and decimal context. Then the two calls either both fail with the same error class, or both produce results with equal canonical payloads and therefore equal result digests.*

PROOF SKETCH. Every source of nondeterminism is either excluded or canonicalized. The DRC verdict is a pure function of the board under the fixed catalog and decimal context (Chapter 7, deterministic ordering invariant). The evaluation order is the unique lexicographically-least topological order (Construction 8.4); delay and test-point iteration is over sorted references; input publication is over sorted ports. Arithmetic runs under the one immutable decimal context, so every operation's rounding and exponent behavior is fixed. All values are rendered by canonical value text, which is a function of the typed value alone. Measurement identifiers and the result digest are content-derived SHA-256 over canonical JSON payloads with sorted keys and fixed separators, so equal content yields equal identity. No wall-clock time, randomness, environment variable, iteration-order-sensitive container, or caller-visible mutable state enters any payload. The fuse and delay states are initialized at entry from declared parameters and evolve only by the declared rules over already-determined values. Hence every intermediate and final datum is a function of the declared inputs, and the conclusion follows by induction over the tick loop's phases. □

Remark 8.15 (Scope of the claim). Proposition 8.14 is a design-level argument about the declared semantics, supported by the bounded regression suites—not a machine-checked theorem about the implementation, and not a statement that survives a change to any identity in its hypothesis. Two runs under different source digests are different experiments regardless of how similar their outputs look; the identity block exists precisely so that "same result" can never be judged by output resemblance alone. Formal verification of the determinism property, like the property-based campaign of Remark 8.18, is open.

Replayability is the practical payoff. Because a result names its complete identity set and its digest recomputes from content, an independent party holding the same sources can re-execute the fixture and compare digests—one string—instead of comparing artifacts field by field. Divergence localizes fast: a digest mismatch with equal board and input identities indicts the engine identities; equal engine identities indict the inputs; and the embedded report and measurement graph give the diff a structure to walk. This is the modest, checkable sense in which the candidate's results are reproducible, and the only sense claimed.

8. Locally asserted calibration is structural

Rule 8.16 (No self-attested eligibility). Every measurement record in every simulation result carries a packet-eligibility flag that is *always false*, and the record type rejects construction of any record claiming otherwise: self-attested PCBA measurements are never packet-eligible. Calibration references inside a result are structural, test-only metadata—they record which declared calibration profile a test point or input sample *named*, and nothing more. They confer no authenticated measurement status, no uncertainty validation, no instrument-chain evidence, and no Evidence-Packet eligibility on anything.

This rule is the runtime continuation of the DRC-level Rule on locally asserted calibration (Chapter 7): the DRC rejects calibration records that claim eligibility for themselves, and the simulator refuses to mint measurement records that claim it either. The two together close the loop against a specific failure mode the candidate's source audit documents in another guise—artifacts that assert their own authority. A calibration profile whose digest, instrument identity, and protocol identity are all locally authored is a well-formed *declaration of intent to measure*; turning it into evidence requires an authentication path that lies entirely outside this software and is, at the bound sources, an open obligation.

9. Unsupported behavior fails closed

Rule 8.17 (Fail closed). Every condition the simulator does not affirmatively support is a hard error: a component kind without reference semantics; a delay reached by the combinational evaluator; a finite-transition input outside the certified carrier; a value that cannot be rendered in canonical text; a port/value pairing outside the admissibility table; a non-positive or non-integer tick count; an input set differing from the connected inputs; a board that fails DRC; a residual combinational cycle; an unresolved evaluation order. None of these produces a partial result, a default value, a skipped component, or a warning-bearing artifact. The run raises, and no result exists.

The rationale is evidential rather than defensive. A partial result would be an artifact whose completeness invariants (Invariant 8.9, Invariant 8.13) are false, and every consumer would need to reason about which guarantees survived the degradation. By making nonexistence the only failure mode, the candidate keeps the result class pure: *every* result that exists satisfies *every* invariant of Section 5. There are no second-class results to audit around.

Remark 8.18 (Open testing obligations). The invariants above are enforced by construction and exercised by the bounded regression suites recorded in the Gate 4 receipt; they are not proofs over all inputs. A property-based and random-generation campaign over the admission, ordering, and validation invariants—in the style of QuickCheck [24]—remains an explicitly open obligation of the research program, as does independent reimplementation. Nothing in this chapter should be read as claiming the absence of defects; it claims the declared contract and its fail-closed enforcement discipline.

10. Relation to PROV, without a conformance claim

Readers familiar with the W3C provenance model will recognize the shape of the record discipline: trace events resemble PROV activities, measurement records resemble entities, child-measurement links resemble derivation relations, and producers resemble agents associated with activities [22]. The resemblance is acknowledged and deliberate at the level of *method*—the separation of the thing produced, the happening that produced it, and the lineage between things is the load-bearing idea, and the candidate independently converged on it because the idea is correct.

Claim Boundary 8.19. The candidate makes no PROV-O conformance claim. Its record types are not serialized in any PROV vocabulary, have not been checked against PROV constraints, and add obligations PROV does not impose (content-derived identifiers, exact-coverage requirements, digest-bound scopes, a mandatory research boundary) while omitting structure PROV provides (plans, delegation, alternate accounts). The comparison in this section is an orientation aid for the reader, not an interoperability statement, and no artifact of this chapter should be represented as PROV-conformant in any downstream use.

The distinction matters for the same reason the rest of the candidate's claim discipline matters: "resembles a standard" and "conforms to a standard" license very different reliance, and only the second is checkable. If PROV interoperability is ever wanted, it is a separately specified, separately tested export surface with its own construct-coverage matrix—the same rule the candidate applies to the CCDL grammar.

The instructive divergences run in both directions. Where PROV is permissive, the candidate is strict: PROV allows an entity to exist without recorded derivation, whereas the coverage invariant forces every producible coordinate to be produced and every derived record to carry its exact wiring-derived parentage; PROV allows multiple accounts of the same happening, whereas the trace-link bijection forces exactly one event per derived measurement. Where PROV is expressive, the candidate is silent: it has no notion of a plan (the board plays that role, but as a hashed input, not a provenance node), no delegation between agents (there is one producer per record, period), and no revision or invalidation relations (artifacts are immutable; a corrected run is a new result with a new digest, not a revision edge). These are not defects of either side; they are the difference between a general interchange vocabulary and a closed evidential contract. The comparison earns its place in this chapter because a reviewer fluent in PROV can use it to find the candidate's load-bearing walls quickly—and because stating the non-conformance explicitly prevents the resemblance from ever being cited as compliance.

11. The green-fixture interpretation rule

The simulator's test fixtures execute boards and compare complete result artifacts—digests, traces, measurements, and all—against expected values. A passing fixture is colloquially "green." Because green fixtures are the candidate's most frequently produced evidence artifact, their interpretation is fixed by rule rather than left to context:

Claim Boundary 8.20. The correct interpretation of a green fixture is exactly this: *the software performed the declared typed transition and produced the declared trace for the exact fixture inputs under the exact profile identities recorded.* It is never evidence that a cognitive mechanism exists or behaves as modeled; never evidence of a causal relation in any system outside the software; never evidence that a physical circuit law holds or that any CAS quantity is an electrical quantity; never evidence that a safety mechanism is effective, that a refusal path is robust, or that any deployment is safe; and never evidence of a product capability. A thousand green fixtures license exactly the same class of claim as one: tested software behavior on tested inputs.

The rule has a constructive reading as well as a restrictive one. Because a result binds its complete identity set, a green fixture is *strong* evidence within its licensed class: it certifies byte-exact behavior of a named engine on a named board, replayable by anyone holding the same sources. The discipline is not modesty for its own sake; it is the exchange rate at which small, hard claims are preferred to large, soft ones.

Two corollaries prevent the most common misuses. First, green fixtures do not aggregate upward: a suite of fixtures covering many boards licenses the conjunction of the per-fixture claims, not any generalization over boards, inputs, or behaviors not fixtured—coverage of the input space is a property the suite must argue separately (and the candidate's suites, being bounded regression suites, do not argue it; see Remark 8.18). Second, *red* fixtures are first-class evidence of the same kind. The known-fault Cognitive Safety Board fixture of Chapter 11 is deliberately kept failing—the checker must flag its unreachable threshold forever—and its value lies exactly in what a red fixture certifies: that the exact engine, on the exact adverse input, produced the exact refusal. A fixture suite that quietly converted expected failures into skipped tests would be discarding the candidate's most defensible negative evidence, and the suite discipline forbids it for the same reason the DRC forbids warnings decaying into passes.

12. Simulation is not validation

The simulation literature has long distinguished *verification*—does the implementation correctly execute the conceptual model?—from *validation*—does the model adequately represent the system of interest for the intended purpose? [**23**]. The candidate adopts the distinction and locates itself entirely on the verification side. The reference simulator, its fixtures, and its identity-bound results are machinery for verifying that the software faithfully executes the declared board semantics. They contain no mechanism whatsoever for validation: no calibrated correspondence between board quantities and observed quantities of any cognitive or model system, no held-out behavioral data, no experimental design, no error model, no uncertainty propagation from world to artifact.

Rule 8.21 (Validation is out of scope and unclaimed). No output of the reference simulator constitutes operational validation of any board as a model of any system. Any future validation effort is a separate activity with its own preregistered design, calibration and uncertainty discipline, held-out data, and independent review—the dependency-ordered research

program of the seed paper places it behind the grammar, profile, and testing obligations that precede it. Until such an effort exists and survives review, every board is a hypothesis expressed in a typed language, and every simulation is an execution of the hypothesis's declared consequences, not a test of its truth.

The temptation the rule guards against is specific and recurrent: verification artifacts *look like* validation artifacts. A trace of a junction saturating looks like an observation of saturation; a fuse latching in a fixture looks like a demonstration of protection; a curve of outputs over ticks looks like data. The resemblance is not accidental—the schema was designed to record executions with the completeness one would want of experiments—but the referent differs categorically. An experiment's record binds observations to a system in the world through an instrument chain with authenticated calibration and quantified uncertainty; the simulator's record binds computed values to a hypothesis through a digest chain with declared, unauthenticated calibration references and no uncertainty at all. The two artifacts can and should share record discipline; they can never share evidential class, and every field of the schema that carries calibration or measurement-kind metadata exists to keep the classes distinguishable *inside the artifact itself*, not merely in surrounding prose.

That closing formulation is the chapter in one sentence. The simulator makes hypotheses *exact* and their declared consequences *replayable*; exactness and replayability are preconditions for honest validation, not substitutes for it. All artifacts of this chapter remain Ring 4, NOT LICENSED, and the interpretation rules of Sections 11 and 12 are part of the artifact contract, not commentary on it.

CHAPTER 9

The TKS v8 Adapter and the Noetic Topos Boundary

The candidate touches two formal neighbors. Downward, it can hand a single, narrowly qualified class of board component—the certified finite transition—to the TKS v8 finite reference kernel, where the component becomes an exact finite arrow between exact finite objects. Upward, it inherits from the Noetic Topos 2.0 specification an evidence boundary separating *ambient* constructions from *resolved* semantics. Both contacts are boundaries first and bridges second: the adapter is engineered so that crossing it can only ever transport what was already exact, and the topos boundary is inherited so that nothing the candidate produces can drift into semantic authority it has not earned. This chapter specifies the certification conditions under which adaptation is permitted (Section 1), the treatment of composition and iteration (Section 2), the adapter certificate field by field (Section 3), the rejection of ambient substitution (Section 5), the DRC-side face of the same discipline (Section 7), the rule that no semantic theorem is created by adaptation (Section 8), the inventory of everything that stays outside the finite arrow and why each exclusion demands its own future types (Section 9), and finally the ambient/resolved evidence boundary itself (Section 10).

1. Certification conditions for adaptation

The TKS v8 finite kernel is a small, exact universe: finite objects are named tuples of distinct exact elements, and finite arrows are total functions between them given by explicit index mappings, validated at construction. Nothing about a PCBA board is automatically welcome there. The adapter admits exactly one component kind under exactly one qualification regime.

Definition 9.1 (Kernel objects and arrows, as consumed by the adapter). For this chapter's purposes, a *finite object* is a pair (name, elements) where the name is a nonempty string and the elements are a finite tuple of pairwise-distinct exact tokens; a *finite arrow* is a quadruple (name, source object, target object, mapping) where the mapping is a tuple of integers, one per source element, each a valid index into the target elements, so that the arrow denotes the total function sending the i-th source element to the element at the mapping's i-th index. Arrow application is index lookup; nothing else happens at application time. The kernel validates these shape conditions at construction, which is why the adapter can treat a successfully constructed arrow as structurally sound—though it still re-verifies, on its own side, that the constructed arrow's names, carriers, and mapping equal the certified table's (Invariant 9.7).

Rule 9.2 (Certified transitions only). Only a certified *total, finite, deterministic, stateless, non-stochastic* component transition may be adapted to a TKS v8 finite arrow. Concretely, the adapter admits a component if and only if all of the following hold:

(1) The board and component are exact typed values, both revalidated at entry.
(2) A fresh DRC run over the board is clean; the adapter refuses any board with errors, and it recomputes the verdict rather than accepting one.

(3) The component is present on the board exactly once under its reference, and the board's copy hashes to the same component digest as the value presented—the caller cannot adapt a private variant of a board component.
(4) The component's kind is exactly `finite_transition` and its semantic profile is exactly `pcba8.finite-transition.v0`.
(5) The component carries no parameters and no state bindings: a finite transition's entire meaning is its table, and undeclared side metadata is refused rather than ignored.
(6) The attached transition semantics exist, carry the same profile, and are flagged certified total and deterministic, not stateful, and not stochastic.
(7) The transition's certificate digest is fresh: the stored digest equals a recomputation over the canonical table payload, and a payload that cannot be canonically rendered is rejected with the failure quoted.

Each condition failing raises a typed adapter error; there is no partial admission.

The five adjectives in Rule 9.2 are the entire theory of the boundary, and each earns its place by what the kernel's arrow type can and cannot represent. *Total*: a finite arrow is defined on every element of its source object; a partial table would require either an error element or an implicit default, both of which are semantic inventions. *Finite*: the source and target carriers must be explicitly enumerated tuples of distinct exact values; nothing with an unbounded or dense carrier can be a kernel object. *Deterministic*: the mapping assigns one target index per source element; a relation or a distribution is not a function. *Stateless*: the arrow's output depends on its input alone; any dependence on history would make the "function" a family of functions indexed by hidden state. *Non-stochastic*: the kernel has no randomness anywhere, and a probabilistic transition is a different mathematical object (Section 9). The adapter checks all five as recorded certificate flags *and* re-derives the checkable ones (totality and index-range validity are re-verified against the actual carriers), so a mislabeled table fails even if its flags lie.

Remark 9.3 (Why the DRC is re-run at the adapter). The DRC already validates finite transitions under `PCBA8-TKS-001` (Section 7), so the adapter's own re-checks are partially redundant by design. The redundancy is the point: the adapter is the last gate before another formal system receives the data, and it treats every upstream assurance as advisory. If the checker and the adapter ever disagree—which should be impossible—the disagreement surfaces as an adapter refusal, never as a kernel object of doubtful pedigree.

2. Chains and powers: paths and endomaps

Adaptation of single components raises the immediate question of composition: boards chain components, so what does a chain become on the kernel side?

Rule 9.4 (Chains become semigroupoid paths). A typed sequence of certified finite transitions, where each transition's target object matches the next transition's source object exactly, is represented on the kernel side as a *semigroupoid path*: a nonempty, composition-compatible sequence of finite arrows. The path is the honest object: it records the chain as a chain, composable stepwise, without asserting the existence of identities or of any ambient categorical closure beyond what the kernel's finite structure provides.

Rule 9.5 (Powers require a certified endomap). Iteration—applying a transition repeatedly, as declared feedback through a delay would induce tick after tick—is admitted on the

kernel side only for a certified *endomap*: a finite arrow whose source and target are the same exact object, element for element and tag for tag. Only an endomap has powers. A transition whose source and target carriers are merely isomorphic, or equal up to renaming, or equal up to retagging of values, is not an endomap and has no powers until the identification is itself made explicit as a certified arrow and composed in.

Rule 9.5 is stricter than casual intuition expects, and deliberately so. "The same set, basically" is precisely the reasoning the tagged-value discipline exists to forbid: integer 1 and string `"1"` are different elements, an object named `Stage` and an object named `Stage'` with equal carriers are different objects, and squaring an arrow between them is a type error, not a formality. The practical consequence for board modeling is clean: a feedback loop's per-tick finite dynamics can be studied as kernel powers exactly when the loop's transition is certified on one fixed carrier—which is also the only situation in which "the state space of the loop" is a well-defined finite object rather than a manner of speaking.

3. The adapter certificate

A successful adaptation returns neither a bare kernel arrow nor a boolean, but an immutable *adapter certificate*: a frozen record binding the arrow to everything that licensed its creation. The certificate is the unit of citation for cross-system claims—any statement of the form "this board component is represented in the TKS kernel" is, in the candidate's discipline, a statement that a certificate with a given digest exists.

Definition 9.6 (Adapter certificate). The certificate carries the following fields, all covered by its own content-derived digest:

Kernel identity—exact bytes and profile.: The versioned kernel profile string and the SHA-256 digest of the kernel's exact source bytes, computed file by file over the kernel package's module files. This is not a version number; it is the hash of the code that defines the arrow type the certificate holds, pinned to an approved constant. A kernel whose bytes differ—by one character—is not the approved kernel, and adaptation refuses to begin.

Board, component, and transition identities.: The full canonical board payload and the board digest; the full canonical component payload and the component digest; the transition-table certificate digest, recorded twice by design—once as the transition digest and once as the PCBA-side certificate digest—with validation requiring the two to be equal and fresh. The certificate does not point at its board; it contains it.

Tagged carrier.: The exact source and target element tuples, serialized as tagged type-plus-value payloads, together with the exact integer mapping. The carrier travels with the certificate so that the certified function is fully inspectable without dereferencing anything.

Semantic profile and names.: The component's semantic profile string, the component reference (which names the kernel arrow), and the source and target object names.

Fixed boundary.: The Ring-4 research boundary, re-derived and re-checked at every validation, inside the digest as always.

Adapter digest.: The SHA-256 of the certificate's own canonical payload—the identity under which the adaptation is cited.

INVARIANT 9.7 (Certificate self-verification). Every public use of a certificate re-runs its full validation, which: revalidates board and component; recomputes both digests and the unique-membership condition of Rule 9.2; re-runs the DRC and requires the embedded board to still be clean; reloads the kernel with full provenance checks (Section 5) and requires the recorded kernel identity to match the currently loaded one byte for byte; requires the held arrow, its source and target objects, and their element tokens to be exact instances from that kernel, with names, tagged carriers, and mapping all equal to the recorded ones; re-certifies the transition table from the recorded fields and requires the recomputed digest to equal both recorded copies; and finally recomputes the certificate digest over the canonical payload and requires equality. A certificate that fails any clause is not degraded or flagged—it is unusable, and every operation on it raises.

Application is likewise disciplined. Calling the certificate on a value performs an exact tagged lookup in the recorded source carrier (a merely equal-looking value of the wrong exact type is outside the carrier and is refused), converts to the kernel's token type, applies the kernel arrow, verifies that the returned token is an exact kernel token belonging to the recorded target carrier, and returns the corresponding recorded target value. The round trip through the kernel is thus checked on both shores: nothing untagged goes in, and nothing unrecognized comes out.

3.1. Tagged carriers and exact identity. The phrase "tagged carrier" compresses a discipline that deserves unpacking, because it is where the candidate's type-plus-value identity rule crosses the system boundary intact. A finite carrier element is never a bare value; it is a value under a type tag, and equality of elements is equality of (tag, canonical value) pairs. The consequences at the adapter are concrete:

- The integer 1, the Boolean true, and the string `"true"` are three distinct carrier elements. A transition table over {true, false} is not a table over $\{1, 0\}$, and the adapter will neither confuse them at admission (the distinctness check of Rule 9.2 operates on tagged keys) nor at application (the lookup of an integer in a Boolean carrier simply fails).
- Carrier serialization in the certificate is by tagged payloads, so a certificate read back from its canonical form reconstructs exactly the elements that were certified—no locale, encoding, or numeric-widening ambiguity can produce a look-alike carrier.
- On the kernel side, elements travel as the kernel's own exact token type carrying the same (tag, canonical value) content; the adapter converts on entry, and on exit verifies both the token's exact type and its membership in the recorded target carrier before translating back. A kernel that returned a well-formed token *outside* the certified target carrier—impossible for the approved kernel, but the adapter does not assume approval is competence—would be caught at this membership check.

The strictness has one practical corollary for board authors worth stating plainly: two transition components intended to "share a state space" share it only if their carriers are element-for-element identical under tags, in the same order, under the same object name. Order matters because the mapping is positional; names matter because kernel objects are named; tags matter because identity is tagged. The candidate provides no carrier-unification convenience, and this is a feature: every identification of state spaces is a modeling decision, and modeling decisions are made on boards, in hashed content, where reviewers can see them.

4. A worked illustrative adaptation

Example 9.8 (Illustrative certified transition and its certificate). Illustrative only; every identifier and value is constructed for exposition. A board `demo-fsm` carries a `finite_transition` component `step` whose certified table declares: source object `Stage` with elements (string `"idle"`, string `"primed"`, string `"fired"`); target object `Stage` with the same three tagged elements in the same order; and mapping $(1, 2, 2)$—`idle` goes to `primed`, `primed` to `fired`, `fired` to `fired`. The certificate flags assert total and deterministic, deny stateful and stochastic, and the table's digest is fresh. The component has no parameters and no state bindings, and its profile is exactly the finite-transition profile.

Adaptation proceeds: the board revalidates; the DRC re-runs clean (the table also passed `PCBA8-TKS-001` inside that verdict); `step` is found exactly once on the board with matching digest; the kernel loads under Protocol 9.10; the three tagged strings become kernel tokens; the kernel constructs the source and target objects and the arrow named `step` with mapping $(1, 2, 2)$; and the adapter mints the certificate, whose digest now names this adaptation. Applying the certificate to the string `"primed"` performs the tagged lookup (index 1), applies the arrow (index 2), verifies the returned token, and yields the string `"fired"`.

Three refusals mark the boundaries. Applying the certificate to the string `"Primed"` fails: no tagged key matches, and the error states the value is outside the tagged source carrier—no case folding, no fuzzy match. Editing the board (say, adding an unrelated component) and presenting the old certificate fails validation: the embedded board no longer matches the presented lineage, and—because validation re-runs the DRC and recomputes digests—the certificate reports its linkage stale rather than pretending the adaptation still stands. And declaring a variant `step2` whose flags claim determinism but whose mapping has only two entries fails both the DRC (`PCBA8-TKS-001`: mapping not total on the source carrier) and, were the DRC somehow bypassed, the adapter's own totality re-derivation.

Example 9.9 (Illustrative chain and endomap powers). Illustrative only, continuing Example 9.8. Because `step`'s source and target are the same exact object—same name `Stage`, same tagged elements in the same order—its arrow qualifies as a certified endomap, and kernel powers are defined: $\texttt{step}^2$ has mapping $(2, 2, 2)$, and $\texttt{step}^n = \texttt{step}^2$ for all $n \geq 2$; the finite dynamics reach a fixed point at `fired`. Now modify the scenario: a second component `route` certifies a table from `Stage` to a two-element object `Gate` (elements `"open"`, `"shut"`). The pair (`step`, `route`) is composition-compatible—`step`'s target equals `route`'s source exactly—so it forms a semigroupoid path of length two, denoting the composite function; but `route` has no powers, and no request for them is expressible, because its source and target differ. Finally, suppose an author re-declares `Stage` in a second table as `Stage'` with identical elements: the path (`step`, `step'`-based arrow) is *not* composition-compatible—the objects differ by name—and the identification, if intended, must itself be certified as an explicit arrow. What looks like pedantry is the discipline of Rule 9.5 doing its work: every "obvious" identification is either made explicit and checkable or does not exist.

What the two examples jointly license is narrow and exact, per Rule 9.13: the tables are representable and represented; the powers are properties of the kernel arrow. That the fixed point at `"fired"` models any process reaching any absorbing state is a semantic claim no artifact in this chapter makes.

5. Ambient module substitution is rejected

A hash-bound certificate would be worthless if the kernel it names could be quietly swapped at import time—the standard failure being an ambient module on the import path shadowing the intended one, or a monkey-patched class substituted for the arrow type. The adapter treats kernel loading as a provenance problem and solves it with four independent checks.

Protocol 9.10 (Exact kernel loading). Before any adaptation or validation, the adapter:

(1) resolves the expected kernel package location relative to its own installed source tree and computes the SHA-256 over the kernel's exact module bytes, file by file with name separators, requiring equality with the approved constant—a kernel edited in place fails here;
(2) imports the kernel module and requires the module's reported file path to resolve to exactly the expected location—a same-named module found elsewhere on the path fails here;
(3) retrieves the finite-object and finite-arrow types and requires each to be a genuine class whose reported source file resolves to exactly the expected kernel module—a substituted or wrapped class, even one installed into the genuine module's namespace, fails here;
(4) records the kernel identity (profile, byte digest, and the package-relative source names) into the certificate, so that later validations repeat the entire procedure and additionally require the recorded identity to equal the freshly derived one.

Any failure raises a typed adapter error naming the untrusted step; no fallback kernel, vendored copy, or degraded mode exists.

Threat 9.11 (Substitution scenarios the protocol forecloses). Illustrative only, as engineering rationale. (i) A test harness inserts a permissive stub kernel earlier on the import path to "simplify fixtures"; step 2 rejects the foreign path. (ii) A well-meaning patch adds a convenience subclass of the arrow type and registers it in the kernel's namespace; step 3 rejects the class whose source is not the kernel module, and the exact-type checks of Invariant 9.7 reject instances of any subclass regardless. (iii) A local edit relaxes the kernel's mapping validation; step 1 rejects the changed bytes before the import is even attempted. (iv) A stale certificate from an older kernel is presented after a kernel upgrade; validation's identity-equality clause rejects it, which is the intended behavior—certificates do not survive their kernel, they are re-earned under the new bytes.

The protocol's philosophy generalizes a theme that runs through the whole candidate: an artifact's dependencies are part of the artifact. The certificate names its kernel by content, and the adapter refuses to reason about arrows whose defining code it cannot re-derive byte for byte.

6. Certificate lifecycle and staleness

Because a certificate re-verifies its entire licensing basis at every use (Invariant 9.7), it has a well-defined lifecycle governed by one principle: a certificate is valid exactly as long as every identity it binds still holds, and not one moment longer. It is worth enumerating what does and does not invalidate a certificate, since the answers follow from the validation clauses rather than from convention.

- *Nothing the caller does to their copy of the board matters*, because the certificate does not reference the caller's board—it embeds its own. Boards are immutable values; a caller who "edits the board" has constructed a new board, and the certificate's embedded original is untouched and still valid.
- *A change to the implementation's source invalidates every certificate*, because validation recomputes the current package identities and the DRC verdict under them; a certificate minted under prior sources fails the freshness comparisons. This is deliberate and symmetric with the simulation result's behavior: artifacts do not outlive the code whose behavior they certify.
- *A change to the kernel's bytes invalidates every certificate*, through the identity-equality clause of Protocol 9.10. Certificates are re-earned under a new kernel by re-running adaptation, which re-derives everything; there is no migration path that carries an old certificate forward, because a migration would be precisely an unreviewed assertion that the old and new kernels agree.
- *Time, by itself, changes nothing.* No certificate field carries a timestamp or expiry; the lifecycle is entirely content-driven. Two parties holding the same sources and the same certificate bytes will agree on its validity regardless of when they check.

REMARK 9.12 (Staleness is a feature with a cost, and the cost is accepted). The strict lifecycle means that routine maintenance—reformatting a source file, say—invalidates every outstanding certificate even though "nothing meaningful changed." The candidate accepts this cost knowingly, for the same reason it hashes whole source trees: *meaningful* is exactly the judgment that cannot be delegated to the artifact layer. A digest cannot distinguish a reformatting from a semantics change; a human reviewer can, and the re-adaptation run that refreshes the certificates is cheap, deterministic, and itself an audit event. The alternative—certificates that survive "insignificant" changes under some significance policy—would relocate trust from checkable bytes to an uncheckable policy, which is the trade the whole architecture exists to refuse.

7. `PCBA8-TKS-001`: the DRC face of the boundary

The certification conditions of Rule 9.2 appear twice in the candidate, on independent code paths. Their design-time face is the DRC diagnostic `PCBA8-TKS-001` (specified in full in Chapter 7): for every `finite_transition` component on a board, the checker validates the presence of the table, profile agreement, non-blank object names, pairwise-distinct exact elements in both carriers, totality of the mapping, index-range validity, the certified-flag conjunction (total and deterministic, not stateful, not stochastic), and the freshness and well-formedness of the certificate digest—failing the board on any violation.

The division of labor is deliberate. The DRC catches boundary violations *early*, attached to the offending component, in the same deterministic report as every other board defect, so that an author sees the complete state of the board in one artifact. The adapter catches them *last*, immediately before another formal system is asked to accept the data, trusting nothing upstream. Between the two, the certification conditions are enforced at both ends of the pipeline with no path around either: a board with an invalid table cannot be clean, and a clean board's table is still re-verified at adaptation. Should the two enforcements ever diverge through an implementation defect, the composition fails closed—the adapter's refusal governs.

8. No automatic semantic theorem

What does a successful adaptation *prove*? The candidate's answer is deliberately minimal.

Rule 9.13 (No-automatic-semantic-theorem rule). An adapter certificate proves at most this: the named component's transition table is representable, and has been exactly represented, as a type-correct finite arrow in the exact TKS v8 reference kernel identified by the recorded bytes. It carries no automatic CAS semantic theorem whatsoever. In particular, adaptation does not establish that the transition models any cognitive process; that the source or target carriers correspond to any TKS object, world, foundation, or element beyond the names the board author chose; that composition of adapted arrows mirrors any composition of cognitive operations; or that any property provable of the arrow in the kernel (totality, functionality, iterability of an endomap) transfers to any system outside the software. Semantic claims connecting kernel arrows to CAS or TKS meaning require their own separately stated, separately reviewed arguments, none of which is supplied by the adapter.

The rule guards against a subtle laundering pattern: formal systems confer prestige, and an object that has "made it into the kernel" can appear validated by association. The candidate's semantic-identity discipline (namespaced references, no silent identity across namespaces) already blocks the crudest version—two things sharing a label do not thereby share a referent—and Rule 9.13 blocks the refined version: two things sharing a *formal representation* do not thereby share a semantics. The adapter is a typed transport, not an oracle; what it transports faithfully is structure, and structure is all it certifies.

Three misreadings recur often enough in adjacent literatures to deserve explicit refutation here, each stated as the inference it tempts and the reason the inference fails.

"The kernel accepted it, so the model is coherent." The kernel accepted a total function between finite tagged sets. Coherence of the *modeling*—whether the carrier elements carve any phenomenon at joints, whether the mapping reflects any dynamics—was never examined by anything in the pipeline. A table mapping arbitrary strings arbitrarily adapts exactly as smoothly as a carefully theorized one; the certificate cannot tell them apart, and does not claim to.

"The arrow composes/iterates cleanly, so the modeled process chains/repeats." Composition and powers are closure properties of the representation. The modeled process, if any, has whatever closure properties it has; the board author's claim that stepwise application models repeated exposure, or that composition models sequential processing, is a hypothesis about the world stated in the modeling layer, and the kernel's algebra neither strengthens nor tests it.

"The carrier elements are named after TKS objects, so the adapter connects CAS to TKS semantics." Names are safe references, not imports. The semantic-identity tuple that would bind a carrier element to an authoritative TKS definition (namespace, local identity, version, source identity, definition digest) is not part of the transition table, and no adapter field carries it. A board author who names an element `"B5"` has named a string. The candidate's separation between structural reference and semantic identity is precisely what makes this safe: nothing downstream will ever mistake the string for the object, because nothing downstream resolves strings to objects at all.

The positive statement hiding under the three refutations is worth making once: what the adapter *does* establish—exact structural transport with content-derived accountability—is the necessary substrate for any future semantic argument. A semantic theorem connecting a board's finite dynamics to a TKS-object-level claim would need, at minimum, an exact and

stable representation on both sides to even state its correspondence. The adapter supplies the PCBA side of that substrate and nothing more; the theorem, if one is ever proposed, will be a new artifact with its own review.

9. What stays outside the finite arrow

The finite arrow's admission conditions exclude most of the candidate's own vocabulary, and the exclusions are permanent features of the boundary, not backlog. Each excluded class fails a different admission condition, and each therefore requires a different future type with different comparison maps; coercing any of them into finite arrows would produce objects that type-check and lie.

Claim Boundary 9.14. The following remain outside the finite arrow, each for the stated structural reason:

Continuous simplex profiles.: A strictly positive normalized profile over the forty ND-cells inhabits an uncountable open simplex. No enumeration of its carrier exists, so it can never be a finite object's element list. Its natural home is a metric/geometric type (the mixed-simplex profiles of Chapter 6 with their disclosed numerical limits); relating it to finite data requires explicit discretization maps whose loss is declared, not a coercion that pretends the simplex was finite.

Response curves.: A component transfer rule such as the junction's clipped product is a function on a dense decimal domain. Sampling it at finitely many points yields a finite table *about* the curve, not the curve; the table's fidelity is a claim needing its own error bounds. A future curve type must carry domain, arithmetic context, and approximation contract—three things a finite arrow has no slot for.

Instrumented observations.: A measurement record is not a value; it is a value plus provenance: tick, producer, calibration reference, uncertainty, measurement kind, scope digest. Stripping the provenance to admit the value into a carrier would be exactly the laundering the measurement schema exists to prevent. Observation types belong to the evidence layer, where their calibration metadata is load-bearing (Section 10).

Stateful devices.: The delay and the latching fuse compute functions of history, not of input. The honest formalization of a stateful device is a transition *system*—state carrier, update map, output map—and while each component map might individually be finite, the device is the system, not either map. A future stateful profile must type the state explicitly; the finite arrow's statelessness condition exists precisely to refuse the alternative, an arrow with hidden memory.

Stochastic kernels.: A probabilistic transition assigns each source element a distribution over targets. That is a Markov kernel, a different object with different composition (distributions compose by integration, not by function application) and different equality (almost-sure, not pointwise). Determinism is an admission condition because a finite arrow asserting one output per input would simply misreport a stochastic table.

Memristive history variables.: Device semantics whose parameters drift with accumulated exposure—the memristive elements the source book gestures at—are path-dependent: their behavior is a function of an unbounded history summary. They compound the stateful and continuous exclusions and would additionally require declared time-integration semantics, which the candidate's synchronous tick

discipline does not supply. They are named in the open-obligations register precisely so that their absence is a recorded boundary, not an oversight.

In every case the future path is the same: a distinct type, a declared comparison map to and from existing types, and validation of that map—never coercion into the finite arrow.

9.1. Comparison maps, not coercions. The closing sentence of Boundary 9.14—distinct types with declared comparison maps, never coercion—states the general method for every future extension, and its content is best seen by contrasting the two disciplines on a single case. Take the response curve. A *coercion* discipline would say: sample the junction's clipped product at, say, eleven points, build a finite table, and adapt it; the table type-checks, and downstream consumers cannot tell it from a genuinely finite transition. A *comparison-map* discipline says instead: define a curve type carrying the rule, domain, and arithmetic context; define an explicit sampling map from curves to finite tables carrying its sample grid as data; and let every consumer of the resulting table see, in the types, that it holds a *sampled image of a curve* with a named grid—an object about which the honest questions (What happens between grid points? Was the grid adequate for the use?) remain askable because the provenance was not erased.

The difference is not pedantic. Under coercion, the information that a finite object approximates a continuous one is destroyed at the boundary, and every downstream inference silently inherits an undeclared adequacy assumption. Under comparison maps, the same inference must cite the map, and the map's adequacy is a reviewable claim with an owner. The candidate applies the identical pattern to each excluded class: discretization maps for simplex profiles (with declared loss), sampling maps for curves (with declared grids), summarization maps for histories (with declared state abstractions), support restrictions or derandomizations for stochastic kernels (with declared policies)—each a future, versioned, tested artifact. None exists at the bound sources; the pattern is recorded so that when they are built, they are built as maps and not as leaks.

Remark 9.15 (The boundary as a design asset). It may seem that so restrictive an adapter is barely useful. The restriction is the utility. Because admission is so narrow, the certificate's guarantee is total over its domain: every certified arrow is exactly faithful, with no approximation regime, no sampling caveat, no hidden state. The excluded classes are not banished from the candidate—they live in their own layers with their own contracts. The boundary merely ensures that the one bridge between systems carries only cargo that survives the crossing without loss.

10. The Noetic Topos evidence boundary

The candidate's second formal contact is read-only. From the Noetic Topos 2.0 specification it inherits a single discipline governing how raw records relate to semantic conclusions: the distinction between the *ambient* construction, where records may be associated as external evidence, and *resolved* semantics, which records may enter only through defined and tested restriction and gluing operations.

Rule 9.16 (Ambient association only). Raw or partially resolved records—simulation measurements, test-point observations, locally calibrated traces, adapter certificates—may be treated as external evidence *associated with* the ambient construction. Association is bookkeeping: it records that evidence exists and where it attaches. It confers no semantic status. In particular, a *marked cover proves no measurement true*: marking a family of observation sites as jointly covering a region of interest is a declaration of intended coverage,

and neither the marking, nor the cover's completeness, nor agreement among the covered observations converts any observation into a true, resolved semantic fact.

Rule 9.17 (Resolution requires restriction and gluing under one calibration profile). No record enters resolved semantics until restriction and gluing are *defined and tested* for the record class under *one* calibration profile: a declared operation restricting a record to a sub-context, a declared operation gluing compatible records on overlapping contexts into one record on the union, declared compatibility conditions on overlaps, and tests exercising all three—all relative to a single, versioned calibration profile, so that "compatible" never silently means "measured under different regimes that happen to agree numerically." At the bound sources, no such operations have been defined or tested for any PCBA record class. Consequently the candidate holds *no* resolved semantics: every measurement, trace, and certificate in this monograph is ambient evidence, in full.

Example 9.18 (Illustrative failure of naive gluing). Illustrative only. Suppose two simulation results over the same board region exist: run A observed the effect net through test point `tp1` under calibration profile `cal-v1`, and run B observed the same net under a revised profile `cal-v2` whose declared uncertainty and protocol identity differ. Both artifacts are internally valid; on the overlapping tick range their observed value texts happen to be numerically equal. A naive gluing rule—"records over the same net that agree on overlaps may be merged into one observation series"—would accept the merge, and the merged series would then appear to be a single, longer observation of the net under *a* calibration, when in fact no single calibration covers it and the numerical agreement is uninterpreted coincidence as far as the metadata is concerned. Rule 9.17 refuses the merge twice over: no gluing operation has been defined or tested for observation records at all, and even a future operation would be scoped to one calibration profile, which this pair fails by construction. The honest artifact remains two results, each citable in its own scope—and the example shows why the single-profile condition is not bureaucratic caution but the difference between an evidence series and a concatenation of look-alikes.

The sheaf-theoretic intuition behind the rule is worth making explicit, without claiming any theorem. Gluing is where local observations become a global assertion—it is exactly the step at which evidence is promoted to fact. A formalism that lets records glue by default therefore manufactures global facts from local bookkeeping, and does so most confidently where instrumentation is least consistent. The topos boundary places the promotion step behind explicit definitions and tests, and the single-calibration-profile condition pins the one variable that most easily invalidates a glue: two locally plausible observations under different calibration regimes may satisfy any numerical compatibility check and still describe incomparable quantities. Until the operations exist, are versioned, and are tested, the honest global statement available from any family of PCBA records is the conjunction of its members' individual, licensed claims—tested software behavior, exactly bound—and nothing stronger.

Claim Boundary 9.19. Summary of the chapter's licensed claims. The adapter transports certified total, finite, deterministic, stateless, non-stochastic transitions—and nothing else—into the exact TKS v8 kernel, returning immutable certificates that bind kernel bytes, board, component, transition, tagged carrier, semantic profile, and the fixed Ring-4 boundary, and that re-verify all of it at every use. Chains are semigroupoid paths; powers exist only for certified endomaps; ambient kernel substitution is structurally rejected; `PCBA8-TKS-001` enforces the same conditions at design time. A certificate proves type-correct representation and nothing more: no CAS semantic theorem, no cognitive correspondence, no transfer of

kernel-provable properties to any system outside the software. Continuous, curved, instrumented, stateful, stochastic, and history-dependent semantics remain outside the finite arrow pending their own types and comparison maps. All records are ambient evidence under the Noetic Topos boundary; a marked cover proves no measurement true; and no resolved semantics exist, or are claimed, until restriction and gluing under one calibration profile are defined and tested. All artifacts remain Ring 4, NOT LICENSED, and every obligation named here is open.

CHAPTER 10

The CCDL Grammar Audit

1. Why a grammar audit, and why here

Every chapter of this monograph before this one has described what the CAS PCBA 8 candidate implementation *does*: the types it declares, the boards it models, the diagnostics it emits, the traces it binds, and the adapter certificates it issues. This chapter and the next describe what the candidate deliberately *preserves*: adverse findings about its own source of authority. The present chapter concerns the Cognitive Circuit Description Language (CCDL), the textual board-description language published in the 743-page engineering baseline (source ID SRC-PCBA-ENGINEER-20260726, SHA-256 `294320B39FA521EAC086E99E2D9DB913E2B6CFF0754FF82DB1EF96`). The finding, stated plainly at the outset and developed at full evidential resolution below, is this: *the consolidated formal grammar published in the source does not cover the source's own five-board reference corpus.* Four of the five published reference listings are direct syntactic counterexamples to the consolidated EBNF; the fifth admits no direct counterexample within the bounded surface audit, and its full conformance remains untested.

The finding matters for three reasons, each of which shapes the candidate's engineering posture.

First, it bounds what any implementation can claim. A parser cannot be conformant to a grammar that the source corpus itself violates; a claim of "full CCDL support" would be either a claim about the published EBNF (in which case it fails the published corpus) or a claim about the corpus (in which case it exceeds the published EBNF and rests on an implementer's private reconstruction of the missing productions). The candidate refuses both horns: the v0.1 implementation declares only its programmatic Python object model and does not claim to parse the complete CCDL book language (Rule 10.22).

Second, it is a case study in a failure mode that is endemic to language-bearing engineering documents: the formal appendix and the worked examples evolve on different schedules, and the document itself ships with the divergence unreconciled. The source is partially self-aware of this—its prose expressly instructs the reader to treat a form missing from the appendix as an erratum—but an erratum clause is not a grammar, and Section 4.2 examines why.

Third, it exercises the evidence discipline that the whole monograph is built on. The audit record that this chapter reproduces, `PCBA_CCDL_CONSTRUCT_COVERAGE_v0.1.json` (schema `project_coherence.pcba_ccdl_construct_coverage.v0.1`, as of 2026-08-04, status DRAFT / OPEN / NOT FROZEN, admission Ring 4, NOT LICENSED, disclosure HELD), is a machine-readable artifact with page anchors, per-board observations, and an explicit claim boundary. This chapter is a faithful prose rendering of that record. Where the record says a board is a counterexample, this chapter says so and shows the observed forms; where the record declines to say so, this chapter declines too, and says why the declination is itself informative. Nothing in this chapter extends the record's page anchors, adds observed forms, or upgrades its conclusions.

Claim Boundary 10.1 (Scope of the entire chapter). Everything in this chapter is a claim about the internal syntactic consistency of one identified source document: whether the surface syntax of its published reference listings is derivable from its published consolidated EBNF. Nothing in this chapter is a claim about the semantic correctness of the boards, the quality of the CCDL design, the behavior of any implementation, the validity of the CAS ontology, safety, certification, or product readiness. The record establishes a source-internal syntax-coverage mismatch only.

2. The audit method

The method is deliberately narrow, and the narrowness is load-bearing. The audit record declares its method in three sentences, which we reproduce in substance and then examine one at a time.

(1) Text extraction was used to locate candidate pages.
(2) Every page cited in the record was rendered from the source PDF and visually inspected.
(3) The audit compares only published surface syntax with the consolidated EBNF; it is not a parser implementation and not a semantic-validation result.

2.1. Text extraction as a locator, not as evidence. The source is a 743-page PDF. Locating the grammar appendix, the reserved-word tables, and the five reference listings by hand would be error-prone in the opposite direction from the one usually worried about: the risk is not misreading a located page but *missing* a relevant page altogether. Text extraction over the full document was therefore used as a search instrument—to find candidate pages containing grammar productions, reserved-word enumerations, statement-form tables, and board listings. But extracted text is a lossy artifact of the PDF's internal encoding: ligatures, hyphenation, column ordering, code-listing fonts, and layout reconstruction all introduce artifacts that could manufacture or hide a syntactic discrepancy. For that reason the extraction output was assigned no evidential weight of its own. Its only role in the audit was to nominate pages for the second step.

2.2. Rendered-page visual inspection. Every page cited in the coverage record—the reserved-word page, the two statement-form prose pages, the two consolidated-EBNF pages, and all sixteen reference-listing pages across the five boards—was rendered from the source PDF and visually inspected. The claims in the record are claims about what is visibly printed on those rendered pages: which statement leaders open the lines of a board body, whether a declaration matches the printed component-expression shape, which productions the printed EBNF enumerates. Visual inspection of a rendered page is the closest available analogue to reading the physical book, and it eliminates the extraction-artifact risk of Section 2.1 at the cost of being a human, bounded act. The record accepts that cost and discloses it.

Dual page anchoring follows from the same discipline. The PDF's page numbering and the book's printed folio numbering differ by a fixed front-matter offset; every anchor in the record and in this chapter is therefore given both ways (for example, PDF page 578 is book page 562). A reader with the PDF can verify by PDF index; a reader with a printed copy can verify by folio. The SHA-256 of the source binds every anchor to one exact document, so the anchors cannot silently drift onto a revised build.

2.3. Surface syntax only. The third methodological sentence is the one that most tightly bounds the result, and it deserves expansion because both an overclaim and an underclaim are near at hand.

The audit is a *surface-syntax comparison*: it asks whether the statement forms visibly used in a published listing are among the alternatives that the published consolidated EBNF admits at the corresponding position. It does not implement a parser for the EBNF; it does not tokenize the listings; it does not build derivation trees; it does not check context-sensitive constraints (identifier resolution, type agreement, arity); and it performs no semantic validation of any kind. Consequently:

- A **negative** result at the surface level—a statement leader with no corresponding production alternative—*is* decisive, because no parse of the listing under the published grammar can exist if the grammar has no production for a form the listing uses. This is why the record can assert direct counterexamples without a parser: the counterexample is visible at the coarsest level of analysis, and refining the analysis could not repair it.
- A **positive** result at the surface level—every observed leader fits some published prefix—is *not* decisive, because surface fit does not establish that the full statement bodies, argument lists, expression forms, and nesting derive from the grammar. This is why the record's verdict for the one non-counterexample board is phrased as "no direct counterexample in the bounded surface audit" with full parsing and semantic conformance expressly untested (Section 6).

The asymmetry is the methodological heart of the chapter. Surface-syntax audit is a cheap instrument, but it is cheap in a direction that makes its adverse findings robust and its favorable findings weak; the record exploits the former and refuses to trade on the latter. The distinction between verifying an artifact against its specification and validating the specification against its intended use is a classical one in simulation and model methodology [23]; the audit sits entirely on the verification side, and even there it verifies only one visible layer.

Remark 10.2 (What a parser-level audit would add). A future parser-level audit could, in principle, convert the bounded non-counterexample into either a conformance result or a fifth counterexample, and could enumerate exactly which productions each conforming listing exercises. That instrument does not exist at the bound commit; building it is step one and step two of the research program (Chapter 12), because a grammar with a published coverage matrix and canonical round trips is a prerequisite for every downstream conformance claim. Until then, the surface audit is the strongest honest instrument available, and its results are labeled with its name.

2.4. Threats to the audit itself. An audit that catalogs a source's defects owes the reader an equally frank catalog of its own. Four threats are worth naming, with their mitigations and their residues.

Threat 10.3 (Rendering fidelity). The audit inspects *rendered* pages, and rendering is itself a computation: a defective renderer could, in principle, drop or garble a line of a listing. Mitigation: the claims at issue are coarse—the presence of a statement leader at the start of a printed line, the identity of an outer block keyword—and are far above the failure granularity of mainstream PDF rendering; moreover, the direct counterexamples rest on *presence* of out-of-grammar forms, which a rendering defect could manufacture only by inventing legible text, not merely by omitting it. Residue: the bounded non-counterexample for CCB-2-COB is the

verdict most exposed to an *omission*-type rendering defect, and it is also the verdict asserted most weakly; the exposure and the weakness are aligned.

THREAT 10.4 (Anchor transcription). Page anchors could be mistranscribed between inspection and record. Mitigation: every anchor is carried in two independent numbering systems (PDF index and book folio) whose fixed offset provides a consistency check—a transcription slip in one array that is not mirrored in the other is detectable arithmetically. Residue: a coordinated slip in both arrays remains logically possible and would be caught only by re-inspection, which the record's structure (Section 13) is designed to make cheap.

THREAT 10.5 (Observer effects). A single bounded inspection can miss forms (understating $\sigma(\ell)$) or misread them (overstating it). Mitigation: understatement is conservative for the audit's negative findings—missed forms could only add counterexamples, never remove the recorded ones—and each recorded observation is stated at the coarsest legible level (a leader keyword), minimizing misreading surface. Residue: understatement is *not* conservative for the CCB-2 verdict, which is one reason that verdict is bounded; and the audit is a single-pass human act, not a replicated one. Independent re-inspection is subsumed under the research program's independent-review step.

THREAT 10.6 (Selection of exhibits). The formal-language evidence (Section 4) cites three loci; a critic might ask whether other pages of a 743-page source contain a second, wider grammar that the audit ignored. Mitigation: the exhibit is the source's own *consolidated* EBNF—the construct the document itself designates as the collected formal grammar—and the erratum instruction at PDF pages 591–592 confirms that the source knows of no wider consolidated grammar (else the instruction would be needless). Residue: if a revised build publishes a wider grammar, the audit re-runs against that build's hash; the present findings are bound to the present hash and neither transfer nor need to.

The pattern across all four threats is the one already observed in Section 2.3: the audit's design consistently places its residual uncertainty on the side of its weakest claim. Every threat that survives mitigation attacks the bounded non-counterexample or the completeness of the observation set; none supplies a mechanism by which the four direct counterexamples could be artifacts of the method.

3. A formal statement of the coverage question

Before turning to the evidence, it is worth fixing precisely what mathematical object a "coverage mismatch" is, because the precision explains both why the audit's negative findings are indefeasible and why its positive finding is bounded. Nothing in this section adds content to the audit record; it restates the record's logic in the monograph's own notation so that the per-board sections can appeal to it.

NOTATION 10.7 (Grammar, language, and listings). Let G denote the consolidated structural EBNF as printed at PDF pages 598–599, taken as a context-free grammar over a terminal alphabet of tokens, with distinguished nonterminals including `circuit_item` and `board_item`. Let $L(G)$ denote the set of token sequences derivable from G's start symbol. Let $\ell_1, \ldots, \ell_5$ denote the five published reference listings, regarded as the token sequences visibly printed at their cited pages. The corpus-level conformance claim is the universal statement

$$\forall i \in \{1, \ldots, 5\} : \ \ell_i \in L(G).$$

DEFINITION 10.8 (Surface projection). For a listing ℓ, let $\sigma(\ell)$ denote its *surface projection*: the finite set of statement leaders (initial keywords of top-level and block-level statements) visibly occurring in ℓ, together with the outer block form of ℓ and any expression shapes explicitly recorded by the audit. For a grammar G and a block context B (such as the body of a `board` or `circuit`), let $\mathrm{Adm}_G(B)$ denote the set of statement leaders and expression shapes admissible in context B under G's printed productions.

The surface audit computes $\sigma(\ell_i)$ by rendered-page inspection and compares it with $\mathrm{Adm}_G(B_i)$, where B_i is the outer context of ℓ_i. The logical yield of that comparison is captured by one lemma and one carefully asymmetric proposition.

LEMMA 10.9 (Surface refutation is sound). *If some leader or shape in $\sigma(\ell)$ is absent from $\mathrm{Adm}_G(B)$, where B is the outer context of ℓ, then $\ell \notin L(G)$.*

PROOF. A derivation of ℓ in G must, at the position of the offending statement, apply some production alternative admissible in context B. Every such alternative begins a statement whose leader (or governs an expression whose shape) lies in $\mathrm{Adm}_G(B)$ by the definition of Adm_G. Since the offending leader or shape of ℓ lies outside $\mathrm{Adm}_G(B)$, no alternative matches the printed text at that position, and no derivation of ℓ exists. □

PROPOSITION 10.10 (Surface confirmation is not complete). $\sigma(\ell) \subseteq \mathrm{Adm}_G(B)$ *does not imply* $\ell \in L(G)$.

PROOF. The surface projection forgets everything below the leader: argument lists, operator syntax, nesting, and token-level detail. Two listings with identical surface projections can differ arbitrarily in their statement bodies; a grammar admitting the leaders may still fail to derive the bodies. Hence membership in $L(G)$ is not determined by $\sigma(\ell)$ alone. □

REMARK 10.11 (The audit in this notation). The record's four direct counterexamples are four instances of Lemma 10.9: for $i \in \{1, 3, 4, 5\}$ the audit exhibits elements of $\sigma(\ell_i) \setminus \mathrm{Adm}_G(B_i)$—nine out-of-context block forms for CCB-1, one underivable expression shape for CCB-3, five missing leaders for CCB-4, fourteen missing leaders for CCB-5. The record's verdict for CCB-2 is an instance of Proposition 10.10 read honestly: the audit found $\sigma(\ell_2) \subseteq \mathrm{Adm}_G(B_2)$ under the leader-as-kind reading and therefore asserted nothing about $\ell_2 \in L(G)$. The corpus conclusion is the negation of the universal claim of Notation 10.7, witnessed four times over.

REMARK 10.12 (Robustness under reading disputes). Lemma 10.9 also explains why the finding survives disputes about how to read particular productions. Suppose a reader argues that some printed production should be read more generously—say, that `component_decl` tolerates unfamiliar component kinds. Such generosity enlarges $\mathrm{Adm}_G(B)$ only within the shapes the production actually prints; it cannot admit the `board`-body forms of CCB-1 into a two-alternative `board_item`, cannot supply an arrow-and-bracket expression production for CCB-3, and cannot manufacture alternatives for the nineteen missing leaders of CCB-4 and CCB-5. To defeat the corpus conclusion, a reader would need all four counterexamples to dissolve simultaneously under some uniform reading of the printed pages—and no such reading has been proposed, including by the source itself, whose erratum instruction concedes the gap in advance (Section 4.2).

4. The formal-language evidence

The counterexamples of Sections 5–9 are counterexamples *against* something, and the something must itself be established from the source with the same page-anchored care. Three

bodies of evidence fix what the published CCDL formally is: the reserved-word enumeration, the prose statement-form tables with their erratum instruction, and the consolidated EBNF.

4.1. The reserved Part-IV statement leaders. At PDF page 578 (book page 562), the language reference reserves a substantial vocabulary of Part-IV statement leaders. The record enumerates eighteen of them:

`rail`	`dial`	`spec`	`constraint`	`sequence`	`route`
`derate`	`monitor`	`alert`	`hold`	`use`	`force_state`
`on`	`charge`	`discharge`	`refresh`	`scan`	`select`

Reservation of a word is a strong act in language design. A reserved word cannot be used as an ordinary identifier; the reservation exists precisely so that the word can serve as the leader of a statement form. A language reference that reserves `rail`, `monitor`, and `force_state` is announcing that rail declarations, monitor statements, and forced-state statements are part of the language. The reservation page is therefore the first exhibit: it establishes that the source itself regards these statement families as belonging to CCDL, independently of whether the formal appendix got around to defining them.

Two observations sharpen the exhibit. First, the reserved list includes words—`on`, `use`, `hold`—whose reservation is only intelligible if event-handler, import-or-profile, and hold-condition statement forms exist; these are not plausible identifier names someone would reserve defensively. Second, the list includes an entire memory-discipline vocabulary (`charge`, `discharge`, `refresh`, `scan`, `select`) whose members appear together in exactly one published listing (Section 8), which confirms that the reservations are backed by intended usage rather than being vestigial.

4.2. The prose statement-form tables and the erratum instruction. At PDF pages 591–592 (book pages 575–576), prose tables describe additional statement forms: `select`, `scan`, `interrupt`, and a verification sublanguage. The same passage expressly says that a form missing from the appendix should be treated as an erratum.

The erratum instruction is remarkable, and it cuts in both directions. In the source's favor: the document is not oblivious to the gap between its prose and its appendix; it anticipates that a reader will find statement forms described in Part IV that the consolidated grammar omits, and it tells the reader how to interpret the omission—as a defect of the appendix, not as a prohibition of the form. Against the source: an erratum instruction is an admission wearing the clothes of a repair. It concedes, in the document's own voice, that the consolidated grammar is known to be incomplete relative to the language the document elsewhere describes and uses.

The concession has a precise consequence for audit methodology, and it is worth stating as a rule because a naive audit could be argued out of its own findings by the erratum clause.

Rule 10.13 (The erratum clause does not repair coverage). For the purpose of a syntax-coverage audit, the published consolidated EBNF is taken as the formal grammar of record, and the erratum instruction is taken as evidence that the source anticipated incompleteness—not as a license to imagine the missing productions. A grammar is a generative object: either a production for a form exists on the page or it does not. "Treat the omission as an erratum" names the defect; it does not supply the production, the precedence, the argument structure, or the nesting rules that a production would supply. The audit therefore records forms covered by the printed grammar as covered, and forms not covered as not covered, regardless of the erratum clause.

The rule is not pedantry. Suppose the audit had accepted the erratum clause as constructive—treating every reserved leader as implicitly grammatical. Then no counterexample could ever be recorded, the coverage question would be unfalsifiable, and any implementer's private reconstruction of the missing productions would be as authoritative as any other. Divergent implementations would each be "conformant" to a grammar that exists nowhere. The entire value of a formal grammar—one derivation relation, shared by all implementations—is forfeited the moment omissions are patched by instruction rather than by productions. The audit's refusal to accept the patch is what makes its result a result.

4.3. The consolidated EBNF. At PDF pages 598–599 (book pages 582–583), the source publishes its consolidated structural EBNF. Two productions carry the whole coverage question.

The `circuit_item` production admits exactly seven alternatives:

`io_decl`	`node_decl`	`cjt_decl`
`component_decl`	`feedback_decl`	`probe_decl`
`let_decl`		

That is: input/output declarations, node declarations, cognitive-junction-transistor declarations, generic component declarations, feedback declarations, probe declarations, and let bindings. Nothing else may appear as an item of a circuit body under the printed grammar.

The `board_item` production is even narrower: a board body may contain only `layer_decl` or `via_decl`. Under the printed grammar, a `board` block is a physical-stackup construct—layers and vias—and nothing else.

Setting Section 4.1 beside this section yields the structural tension of the whole chapter before any board is examined: the language reference reserves at least eighteen Part-IV statement leaders, the prose describes further forms and a verification sublanguage, and the consolidated grammar has *seven* circuit-item alternatives and *two* board-item alternatives. None of the eighteen reserved leaders of Section 4.1 corresponds to any of the seven `circuit_item` alternatives. The formal grammar and the reserved vocabulary describe languages of visibly different extent. The reference corpus decides which of the two the source's own boards are written in.

Adverse Finding 10.14 (Consolidated grammar versus reserved vocabulary). The consolidated structural EBNF at PDF pages 598–599 omits the additional reserved statement families that the language reference reserves at PDF page 578 and describes in prose at PDF pages 591–592. This omission is antecedently acknowledged by the source's own erratum instruction and is confirmed against the published corpus by the per-board findings below.

5. Board CCB-1-CSB: the Cognitive Safety Board listing

The first reference listing is the Cognitive Safety Board, CCB-1-CSB, published at PDF pages 606–608 (book pages 590–592). It is the same board whose published parameter values are the subject of Chapter 11; here only its syntax is at issue.

5.1. Outer form and observed body. CCB-1-CSB is the only board of the five whose outer form is a `board` block. Under the consolidated EBNF this is immediately consequential: as established in Section 4.3, a `board` body may contain only `layer_decl` or `via_decl`. The published listing's body, as visually inspected on the rendered pages, contains block forms with the following leaders:

`input`	`output`	`node`
`cjt`	`diff`	`mux`
`filter`	`feedback`	`probe`

None of these is a layer declaration; none is a via declaration. Every single body item of the flagship safety listing falls outside the two alternatives that the printed `board_item` production admits.

Adverse Finding 10.15 (CCB-1-CSB is a direct EBNF counterexample). The published board body of CCB-1-CSB (PDF pages 606–608) contains forms other than `layer_decl` or `via_decl`—namely `input`, `output`, `node`, `cjt`, `diff`, `mux`, `filter`, `feedback`, and `probe`. No derivation of the listing from the consolidated EBNF exists. Verdict of record: DIRECT EBNF COUNTEREXAMPLE.

5.2. Discussion. Two readings of the discrepancy are possible, and the audit is careful to need neither. On one reading, the `board` production was drafted for a physical-stackup construct and the listing authors used `board` as an informal synonym for `circuit`; the intended fix would be to change the listing's outer keyword. On the other reading, `board` was always intended to admit circuit content and the production is truncated; the intended fix would be to widen `board_item`. The source does not say which, and the two fixes produce different languages—under the first, a board is physical and a circuit is logical; under the second, the distinction collapses. An implementer forced to choose would be legislating language design, not implementing it. This is precisely the situation the candidate rule (Section 12) exists to prohibit: the v0.1 implementation models boards programmatically and declines to guess.

It is also worth registering *which* board carries this defect. CCB-1-CSB is the safety exemplar—the listing that Chapter 11 shows to contain an unreachable fuse threshold. The corpus's most safety-salient artifact is thus doubly defective: its syntax is underivable from the published grammar, and its published parameters defeat its published protective function. The two defects are independent (one syntactic, one semantic) and are preserved by the candidate as two separate findings; but their coincidence on the same listing is a fact about the source's review process that the research program's independent-review step (Chapter 12) is designed to address.

6. Board CCB-2-COB: the bounded non-counterexample

The second reference listing, CCB-2-COB, is published at PDF pages 613–614 (book pages 597–598). Its outer form is `circuit`, so the governing production is `circuit_item` with its seven alternatives, not the two-alternative `board_item`.

6.1. Observed forms. Visual inspection of the rendered pages records the following statement leaders in the listing body:

`input`	`output`	`node`	`let`	`inductor`
`capacitor`	`tank`	`pll`	`modulator`	`probe`

Each observed leader can be fitted to a published alternative: `input` and `output` to `io_decl`; `node` to `node_decl`; `let` to `let_decl`; `probe` to `probe_decl`; and `inductor`, `capacitor`, `tank`, `pll`, and `modulator` to `component_decl`, on the reading that these are component declarations whose component kinds are inductors, capacitors, tank stages, phase-locked loops, and modulators.

Adverse Finding 10.16 (CCB-2-COB admits no direct counterexample in the bounded audit). The observed statement leaders of CCB-2-COB (PDF pages 613–614) can fit the published circuit-item and component prefixes. Verdict of record: NO DIRECT COUNTEREXAMPLE IN BOUNDED SURFACE AUDIT. Full parsing and semantic conformance remain untested.

6.2. Why the weak verdict is the honest verdict. It would have been rhetorically convenient for the record to report a clean sweep—five boards, five counterexamples—and the difference between four and five changes no downstream conclusion, since a single counterexample already defeats corpus-level conformance. The record nevertheless declines the fifth counterexample, and the declination is the audit's best credential. A surface audit's favorable finding is weak by construction (Section 2.3): leader fit does not establish that argument lists, expression grammar, and statement bodies derive from the printed productions; the component-kind reading of `inductor` and its siblings assumes a leader-as-kind convention the surface audit cannot itself confirm; and the bodies were not tokenized or parsed. So the record reports exactly what its instrument can see—no visible violation—and marks everything beyond the instrument's reach as untested.

The corpus conclusion (Section 11) is phrased to preserve this asymmetry: four direct counterexamples and one bounded non-counterexample, not five counterexamples and not "one conforming board." A future parser-level audit (Remark 10.2) may move CCB-2-COB in either direction; nothing in this monograph presumes the outcome.

7. Board CCB-3-CDAB: the mirror declaration

The third reference listing, CCB-3-CDAB, is published at PDF pages 618–619 (book pages 602–603). Its outer form is `circuit`. Unlike CCB-1-CSB, where every body item is out of grammar, and unlike CCB-4 and CCB-5, where whole statement families are missing, CCB-3-CDAB is a precision counterexample: a single declaration whose printed shape cannot be produced by the grammar's component-expression production.

7.1. The observed declaration. The listing contains, outside any component expression admitted by the grammar, the declaration:

```
mirror bias_mirror:  replicate(shared_bias) -> [cjt_l, cjt_r]
```

Read as engineering intent, the line is legible: it declares a current-mirror-like construct named `bias_mirror` that replicates a shared bias signal `shared_bias` and fans it out to two cognitive junction transistors `cjt_l` and `cjt_r`. The engineering intent is not in question; the derivability is.

7.2. Why it cannot derive. Under the consolidated EBNF, a component declaration's right-hand side must be a component expression of the shape `type_name<arg_list>`: a type name followed by an angle-bracketed argument list. The mirror declaration matches this shape at no point:

- Its operative construct is a *call-shaped* form, `replicate(shared_bias)`—a function-application syntax with parentheses, which is not the `type_name<arg_list>` shape.
- It employs an arrow `->` mapping the replicated source onto a *bracketed target list* `[cjt_l, cjt_r]`. Neither an arrow operator nor a bracketed fan-out list appears in the component-expression production.
- Its leader, `mirror`, corresponds to no `circuit_item` alternative: it is not an `io_decl`, `node_decl`, `cjt_decl`, `feedback_decl`, `probe_decl`, or `let_decl`, and the only remaining alternative, `component_decl`, requires the `type_name<arg_list>` expression that the line does not use.

Any one of the three mismatches would suffice; jointly they leave no derivation path at all.

Adverse Finding 10.17 (CCB-3-CDAB is a direct EBNF counterexample). The declaration `mirror bias_mirror: replicate(shared_bias) -> [cjt_l, cjt_r]`, observed in the published listing of CCB-3-CDAB (PDF pages 618–619), does not use the required `type_name<arg_list>` component expression. Verdict of record: direct EBNF counterexample.

7.3. Discussion. CCB-3-CDAB demonstrates that the coverage gap is not confined to whole missing statement families (as in CCB-4 and CCB-5) or to a mismatched outer form (as in CCB-1). It reaches down into expression syntax: the corpus uses at least one expression-level idiom—call-shaped replication with arrow fan-out—for which the grammar has no production. This matters for anyone estimating the *size* of the repair. Adding missing statement leaders to `circuit_item` is a bounded, mechanical widening; admitting a new expression grammar (application syntax, an arrow operator, list targets, and their precedence and typing) is language design. A future complete grammar (Chapter 12, program steps 1–2) must therefore publish not merely more alternatives but a genuinely extended expression grammar, with the construct-coverage matrix recording exactly which listings exercise it.

8. Board CCB-4-CMB: the memory-discipline statements

The fourth reference listing, CCB-4-CMB, is published at PDF pages 623–624 (book pages 607–608). Its outer form is `circuit`.

8.1. Observed omissions. The listing employs the memory-discipline statement family whose reservation was noted in Section 4.1. Five statement leaders are observed in the published body for which the consolidated `circuit_item` production has no alternative:

`charge`	`select`	`discharge`	`refresh`	`scan`

Adverse Finding 10.18 (CCB-4-CMB is a direct EBNF counterexample). The published listing of CCB-4-CMB (PDF pages 623–624) uses the statement leaders `charge`, `select`, `discharge`, `refresh`, and `scan`, for which the consolidated `circuit_item` production has no alternative. Verdict of record: direct EBNF counterexample.

8.2. Discussion. CCB-4-CMB closes an evidentiary loop that began at the reservation page. Section 4.1 observed that the reserved-word list includes a coherent memory vocabulary; Section 4.2 observed that the prose tables describe `select` and `scan` among the additional forms; and here the corpus uses all five leaders in a published listing. The three exhibits triangulate: the memory-discipline statements are reserved, described, and used—and absent from the formal grammar. This is the cleanest instance of the general pattern, because no alternative reading is available. One might argue about whether `board` should have admitted circuit content (CCB-1) or whether `mirror` could have been a component kind under a different expression syntax (CCB-3); there is no analogous argument by which `refresh` or `discharge` fits any of the seven printed alternatives. The family is simply missing.

The omission also has a semantic shadow that the audit records only as motivation, not as finding: memory-discipline statements are exactly the statements whose semantics are *stateful* and *temporal*—charging, refreshing, and scanning are operations over persistent state across time. The candidate's reference simulator (Chapter 8) supports only declared deterministic stateless profiles with unit-delay feedback; stateful capacitor, memory, and power semantics

are explicitly open obligations. The grammar gap and the semantics gap coincide on the same constructs, which is unsurprising—the hardest constructs to formalize are the ones the appendix postponed—but the coincidence means a future grammar for these statements cannot be validated by syntax alone: it will need the state/time profiles of program step 3 (Chapter 12) before any listing that uses them can be executed rather than merely parsed.

9. Board CCB-5-CPMB: the power-management vocabulary

The fifth reference listing, CCB-5-CPMB, is the largest, published across five pages at PDF pages 628–632 (book pages 612–616). Its outer form is `circuit`.

9.1. Observed omissions. The listing uses fourteen statement leaders for which the consolidated `circuit_item` production has no alternative:

`rail`	`use`	`spec`	`dial`	`constraint`
`monitor`	`on`	`derate`	`route`	`alert`
`hold`	`sequence`	`gate`	`force_state`	

Adverse Finding 10.19 (CCB-5-CPMB is a direct EBNF counterexample). The published listing of CCB-5-CPMB (PDF pages 628–632) uses the statement leaders `rail`, `use`, `spec`, `dial`, `constraint`, `monitor`, `on`, `derate`, `route`, `alert`, `hold`, `sequence`, `gate`, and `force_state`, for which the consolidated `circuit_item` production has no alternative. Verdict of record: direct ebnf counterexample.

9.2. Discussion. CCB-5-CPMB is the counterexample of scale. Where CCB-3 exhibited one underivable line and CCB-4 one missing family, CCB-5 exhibits an entire register of the language—power rails, usage/import statements, specifications, dials, constraints, monitors, event handlers (`on`), derating, routing, alerts, hold conditions, sequencing, gating, and forced states—that exists in the corpus and the reserved vocabulary but not in the grammar. Thirteen of the fourteen leaders appear on the reservation page examined in Section 4.1; the fourteenth, `gate`, is observed in the listing without appearing in the record's reserved-word enumeration, a detail the record preserves exactly and this chapter neither smooths over nor extends: the audit reports observed forms against printed productions, and `gate` has no production regardless of its reservation status.

The scale has a corpus-level implication. Across the four counterexample boards, the observed out-of-grammar forms number at least twenty-eight distinct leaders and one expression idiom. The consolidated grammar, recall, has seven circuit-item alternatives in total. By raw count, the reference corpus exercises *more* statement vocabulary outside the grammar than inside it. The published EBNF is thus not a grammar with edge-case gaps; it is a core-fragment grammar—an accurate formalization of perhaps the declarative skeleton of the language (I/O, nodes, junctions, generic components, feedback, probes, bindings) that was never extended to the operational, temporal, and management registers in which the corpus's most distinctive content lives. Naming the situation precisely matters because the two diagnoses have different repairs: gap-filling is errata work, while formalizing an unformalized half of a language is a specification project. The research program treats it as the latter.

10. A taxonomy of the counterexamples

The four counterexamples are not four instances of one defect. Classified by *where* in the grammar the derivation fails, they form three kinds, and the kinds have different repair costs, different blast radii for implementers, and different lessons. Making the taxonomy

explicit also guards against a rhetorical compression that would misstate the evidence—"the grammar is missing some keywords"—which is true of only two of the four.

Definition 10.20 (Kinds of surface counterexample). Relative to a grammar G and a listing ℓ with outer context B:

(1) A *context mismatch* occurs when ℓ's body items are drawn from a statement register that G admits, if at all, only in a different block context: the printed forms exist somewhere in G's world, but not under B.
(2) An *expression mismatch* occurs when a statement's leader could be accommodated but its right-hand side uses expression syntax (operators, application forms, aggregate targets) for which G prints no production.
(3) A *register omission* occurs when an entire family of statement leaders used by ℓ has no production alternative anywhere in G.

CCB-1-CSB is a context mismatch: its body forms (`input`, `node`, `cjt`, and the rest) resemble circuit content, but its outer form is `board`, whose printed body admits only `layer_decl` and `via_decl`. CCB-3-CDAB is an expression mismatch: one line, one underivable right-hand side. CCB-4-CMB and CCB-5-CPMB are register omissions: the memory-discipline family (five leaders) and the power-management family (fourteen leaders) respectively.

The repair costs escalate along the taxonomy. A context mismatch can be repaired by a one-line production change (widen `board_item`) or a one-token corpus change (rename the outer block)—but, as Section 5 discussed, choosing between those repairs is a language-design decision the source does not determine. An expression mismatch requires new expression grammar with precedence and typing decisions. A register omission requires specifying entire statement families—syntax, and eventually semantics—essentially from prose. An implementer who reads only the corpus conclusion would budget for keyword additions; an implementer who reads the taxonomy budgets for a specification project. The research program's first two steps (Chapter 12) are sized for the latter.

The taxonomy also refines the audit's robustness argument (Remark 10.12): the three kinds fail derivation at three different grammatical loci (block context, expression production, production inventory), so no single generous reading of any single production can dissolve more than one kind.

11. The corpus conclusion

The record's corpus-level conclusion, reproduced exactly:

Adverse Finding 10.21 (Corpus conclusion of record). FORMAL EBNF DOES NOT COVER PUBLISHED FIVE-BOARD CORPUS. Direct counterexample count: four (CCB-1-CSB, CCB-3-CDAB, CCB-4-CMB, CCB-5-CPMB). Bounded non-counterexample count: one (CCB-2-COB). The corpus-level conformance claim fails without overstating the result for any individual board.

Three features of the conclusion's logical form deserve explicit statement.

Universality fails by existence. "The grammar covers the corpus" is a universal claim over five listings; a single counterexample refutes it. The record has four, of three structurally distinct kinds—an outer-form mismatch (CCB-1), an expression-level mismatch (CCB-3), and two missing-statement-family mismatches (CCB-4, CCB-5). The refutation is therefore robust: it does not hinge on the reading of any single page, and it would survive the reclassification of any single board.

The existential remainder is not converted into a positive. From "CCB-2-COB shows no visible violation" the record does not conclude that CCB-2-COB conforms, because the instrument cannot see conformance (Section 2.3). The count is stated as "four and one bounded," never "four out of five failed" with the implication that one passed.

Per-board resolution is preserved. The record could have reported only the corpus verdict. Instead it carries a per-board block with pages, outer form, observed forms, verdict, and reason—so that any future challenge (a re-rendering of a page, a parser-level re-audit, a revised source build with a different hash) can be adjudicated board by board against the exact printed evidence rather than against a summary.

12. The candidate grammar rule

The audit's findings are about the source; the candidate's obligation is about itself. The v0.1 implementation responds to the coverage gap with a deliberately austere posture, recorded as a rule.

Rule 10.22 (Programmatic model only; no CCDL parsing claim). The CAS PCBA 8 v0.1 implementation declares only its programmatic Python object model (Chapters 4–5). It does not claim to parse the complete CCDL book language, and no part of its test evidence may be represented as CCDL conformance evidence. Any later grammar must: (i) publish an exact construct-coverage matrix stating, for every construct of the published language and every listing of the golden corpus, whether the construct is supported and where it is exercised; (ii) reject unsupported forms fail-closed rather than skipping, warning, or best-effort parsing them; and (iii) round-trip every admitted board canonically, so that parse followed by serialization is the identity on a canonical form and the canonical form's digest is stable.

Each clause answers a specific failure mode that the audit made visible.

Why a coverage matrix. The source's own defect is precisely a missing coverage statement: a grammar published without any accounting of which corpus constructs it covers, discovered years-equivalent later (in document time) to cover a minority register of its own corpus. A published matrix makes the coverage claim itself falsifiable and mechanically checkable—the audit of this chapter becomes a regression test rather than an archaeology project.

Why fail-closed rejection. A best-effort parser that skips unknown statements would silently discard exactly the content—monitors, constraints, forced states, memory discipline—where the operational meaning of a board lives, and downstream tools would draw conclusions from a board that is not the board on the page. Fail-closed rejection converts the coverage boundary into an error surface: every unsupported construct is a loud, coded event (the DRC analogue is the closed `PCBA8-*` catalog of Chapter 7 and Appendix A), never a quiet subtraction.

Why canonical round trips. Round-tripping is the only cheap, total test that a parser and serializer agree about the language they implement; it protects the hash-binding discipline (a board's identity is the digest of its canonical structure, Chapter 5) from divergence between textual and programmatic forms; and property-based generation of boards through the round trip [24] is the planned instrument for exercising the grammar far beyond the five published listings.

Example 10.23 (What a coverage-matrix row looks like). This example is *illustrative only*; no such matrix exists at the bound commit, and the row below describes a hypothetical

future grammar, not the published one. A construct-coverage matrix under Rule 10.22 would contain, for each construct, a row of the shape:

Construct	Grammar production	Status	Exercised by	Round-trip test
`monitor` stmt	`monitor_decl` v1.0	supported	CCB-5 listing	`rt_monitor_001`
`interrupt` stmt	—	rejected (fail-closed)	none admitted	`rej_interrupt_001`

Every construct of the published language appears with exactly one status; "absent from the matrix" is not a state. A listing is admitted only if every one of its constructs has status *supported*, and each supported row names the corpus listings that exercise it and the round-trip test that guards it. Under this discipline, the audit of this chapter reduces to checking that the matrix's "rejected" rows are empty for the golden corpus—a regression assertion, not a research finding.

Remark 10.24 (Relation to the DRC). The candidate's DRC (Chapter 7) operates on the programmatic model directly and is therefore unaffected by the grammar gap in its own domain of claims: a `DrcReport` speaks about a validated in-memory `Board` object with a canonical digest, not about a CCDL text. The gap constrains only the bridge from published listings to programmatic boards—which is exactly the bridge the candidate declines to claim until the Rule 10.22 obligations can be met.

13. The coverage record as a machine-readable artifact

This chapter has repeatedly deferred to "the record." The record deserves a section of its own, because its form—a versioned, schema-tagged, hash-anchored JSON document—is as much a methodological contribution as its content, and because the monograph's evidence discipline (Chapter 12) treats such artifacts, not prose, as the primary carriers of findings.

13.1. Structure of the record. `PCBA_CCDL_CONSTRUCT_COVERAGE_v0.1.json` carries, in order: a schema identifier (`project_coherence.pcba_ccdl_construct_coverage.v0.1`) that fixes the field vocabulary against silent drift; a document status (DRAFT / OPEN / NOT FROZEN) and an as-of date (2026-08-04); the source identity triple—source ID SRC-PCBA-ENGINEER-20260726, SHA-256, and page count (743)—that binds every subsequent anchor to one exact document; the three-sentence audit method of Section 2; a `formal_language_evidence` block with the three exhibits of Section 4, each carrying its own PDF and book page arrays and a one-sentence finding; a `reference_boards` array with one entry per board carrying board ID, listing pages (both numberings), outer form, the observed forms relevant to that board's verdict, a Boolean `direct_syntactic_counterexample`, a controlled-vocabulary conclusion, and a reason sentence; and finally the corpus conclusion, the two counts (four direct, one bounded), the claim boundary, and the governance labels (admission Ring 4, NOT LICENSED, disclosure HELD).

Three design choices in this structure repay attention.

Controlled verdict vocabulary. Each board's conclusion is drawn from a closed set—DIRECT EBNF COUNTEREXAMPLE or NO DIRECT COUNTEREXAMPLE IN BOUNDED SURFACE AUDIT—rather than free prose. The closed set makes the record diffable and queryable, prevents verdict inflation (there is no "conforms" value to reach for), and mirrors the closed diagnostic catalog philosophy of the DRC (Chapter 7): a finding is a coded event, not a paragraph.

Separation of observation from verdict. The per-board entries record observed forms as data (`observed_non_board_block_forms`, `observed_omitted_circuit_items`, `observed_outside_component_e`

distinct from the Boolean and the conclusion. A reviewer who disputes a verdict can check it against the recorded observations without re-deriving them; a reviewer who disputes an observation knows exactly which rendered pages to re-inspect. The record thus factors cleanly into the empirical layer (what is printed) and the logical layer (what follows by Lemma 10.9).

Both page numberings, always. Every anchor carries parallel `pdf_pages` and `book_pages` arrays. The redundancy is deliberate verification affordance, as discussed in Section 2.2.

13.2. Why machine-readable. A prose audit finding is read once and paraphrased thereafter; each paraphrase is an opportunity for strengthening drift—"no direct counterexample in the bounded surface audit" becomes "passed the audit" becomes "conforms." A machine-readable record resists this decay in two ways. First, downstream artifacts can *consume* it rather than restate it: the Gate 4 receipt counts "4 direct EBNF counterexamples" by reading this record, and any monograph table can in principle be regenerated from it. Second, it can be *diffed*: when a v0.2 audit is performed (for instance, at parser level, per Remark 10.2), the change from v0.1 will be an inspectable structural delta—which boards moved, which observations were added—rather than a rewritten narrative. The provenance discipline here follows the same separation of entities (the source PDF), activities (the audit), and derived artifacts (the record) that the W3C provenance model formalizes [**22**], without claiming conformance to that model.

13.3. Governance labels. The record closes with its own governance: admission Ring 4, license status NOT LICENSED, disclosure status HELD. In the project's evidence discipline these labels mean: the record is admitted as a working research artifact, not as canon; no claim in it is licensed for external assertion; and the artifact itself is not approved for disclosure. This monograph inherits those labels (they are restated on its title page) and adds none. The practical consequence for a reader is that every sentence of this chapter, however firmly evidenced, is a held research finding awaiting independent review—and that the record, not this prose, is the artifact of authority if the two ever diverge.

14. Claim boundary of the record

CLAIM BOUNDARY 10.25 (What the coverage record establishes, verbatim in substance). The record establishes a source-internal syntax-coverage mismatch only. It does not establish: a complete replacement grammar; parser correctness for any implementation; executable board semantics; safety effectiveness; certification; or product readiness. The record's own status labels govern: DRAFT / OPEN / NOT FROZEN, admission Ring 4, NOT LICENSED, disclosure HELD.

The boundary bears expansion on two points where a motivated reader might otherwise extrapolate.

First, the finding is not a quality verdict on the CCDL *design*. A language whose formal appendix lags its corpus can still embody sound design ideas; conversely, a fully covered grammar can formalize a poor language. The audit measures internal consistency of a document, nothing more. Second, the finding does not transfer to any *revised* source build. Every anchor in this chapter binds to the exact SHA-256 identified in Section 1; a future build that consolidates the grammar would require a fresh audit under its own hash, and nothing here prejudges its outcome.

15. Lessons for language-bearing engineering documents

The chapter closes with the general lessons the audit supports—stated as method, since (per Boundary 10.1) the audit licenses no claims beyond its source.

Grammars and corpora must be co-versioned. The root cause visible in the source is schedule divergence: reserved words, prose tables, and listings kept growing while the consolidated appendix did not. The mechanical remedy is to make the corpus part of the grammar's test suite—every published listing parsed in CI against the published grammar, with the coverage matrix regenerated on every change. Under that discipline the state documented in this chapter is unrepresentable: the build fails before the divergence ships.

Erratum clauses are confessions, not repairs. A document that instructs readers to treat omissions as errata has documented its own incompleteness. The clause has archival value—it fixes the author's intent that the prose forms are language—but it must never be given generative force in an audit or an implementation (Rule 10.13).

Audit instruments should be chosen for the robustness of their negative results. The surface-syntax method was selected because its counterexamples are indefeasible under refinement while its confirmations are visibly weak; the record then reports both with their native strength. An instrument whose positive results outrun its resolution produces conformance theater; an instrument that will not report a favorable result it cannot see produces findings that survive review.

Preserve the defect; do not silently repair it. The strongest engineering temptation, on discovering that the corpus outruns the grammar, is to quietly implement the "obvious" extended grammar and move on. The candidate refuses, for reasons developed at length in Chapter 11 for the CSB parameter fault and applying with equal force here: a silent repair destroys the evidence that the authority document requires independent review, substitutes the implementer's reconstruction for the source's authority while still trading on the source's name, and removes the regression guard that keeps the gap from being papered over in downstream claims. The coverage record—four direct counterexamples, one bounded non-counterexample, page anchors, claim boundary, held disclosure—is the durable form of the finding, and this chapter is its prose mirror.

Downstream consumers must carry the boundary with the finding. Finally, the audit illustrates how a bounded result should travel. The Gate 4 receipt (Chapter 12) does not say "the grammar is broken"; it counts "4 direct EBNF counterexamples," a number that is exactly as large as the evidence. The source-audit specification's candidate rule does not say "we will write a better grammar"; it forbids conformance claims until a matrix, rejection, and round trips exist. And this chapter, the longest prose treatment the finding will receive, closes not with the finding but with its boundary (Boundary 10.25): a source-internal syntax mismatch, in one identified build, established by a bounded surface method, held at Ring 4, licensed for nothing. When the finding travels further—into the evidence chapter, into any future disclosure—it must travel in that form: the counts, the anchors, the method, and the boundary, together or not at all.

CHAPTER 11

The Cognitive Safety Board: A Known-Fault Analysis

1. Introduction: a safety exemplar that cannot perform its function

Chapter 10 examined the source's flagship reference listings as syntax. This chapter examines the most consequential of them—the Cognitive Safety Board, CCB-1-CSB—as a specified mechanism, and reports the second preserved adverse finding of the monograph: *under the source's own declared semantics and published parameter values, the board's protective fuse can never trip.* The red stage that feeds the fuse is clamped at a saturation value of 0.1; the fuse's trip threshold is 0.9; every declared stage between them is monotone and non-amplifying; and $0.1 < 0.9$. The protective crossing is unreachable not because of an implementation bug, an unlucky input, or a numerical edge case, but as a direct arithmetic consequence of the printed design values.

The finding is developed here at full resolution: the published values with their page anchors (Section 2), a formalization of the clamp semantics and a proof of unreachability (Section 3), the distinct and subtler defect that saturation is a plateau rather than a latch and that the book defines no fail-closed state machine for the fuse (Section 4), the candidate's disposition of the board as a permanently preserved known-fault fixture that the design-rule checker must flag (Section 5), the negative-evidence philosophy that justifies preservation over repair (Section 6), the requirements that any repaired design must meet (Section 7), and the general lessons for safety-mechanism specification in cognitive-system engineering (Section 8).

One sentence of orientation before the evidence, because the subject matter invites misreading in both directions. This chapter does *not* claim that any deployed system is unsafe: the CSB is a listing in an engineering reference, the "refusal path" it models is a specified software construct, and no physical or operational system is in evidence anywhere in this monograph. Equally, this chapter does not treat the defect as a curiosity: a safety mechanism specified so that it cannot actuate is the canonical failure mode of safety engineering, and the fact that it can arise—and survive review—in a carefully produced 743-page reference is precisely the kind of evidence a design-rule discipline exists to capture.

Claim Boundary 11.1 (Scope of the chapter). Every claim in this chapter is about the internal consistency of one identified source document (source ID SRC-PCBA-ENGINEER-20260726, SHA-256 `294320B39FA521EAC086E99E2D9DB913E2B6CFF0754FF82DB1EF966424566274`): whether its published Cognitive Safety Board parameters permit its published protective function under its declared semantics. No claim is made about the safety, unsafety, or behavior of any cognitive system, model, product, or deployment. "Refusal," "fuse," and "safety" name constructs of the source's specification, not validated properties of any real system.

2. The published values

2.1. The primary listing. The Cognitive Safety Board is presented in Chapter 18 of the source, with its listing giving the red-stage and fuse values at PDF page 342 (book page 326). Three published facts carry the whole analysis:

(1) The *red stage*—the final alarm stage of the board's escalation path, whose output represents the maximal-severity signal that feeds the protective element—is clamped at a saturation value of sat = 0.1.
(2) The *fuse*—the protective element downstream of the red stage, whose actuation represents the board's terminal protective action—has trip threshold 0.9.
(3) The red stage's gain parameter is published as $\beta = \infty$: a non-finite value, in a language whose grammar and validator rules elsewhere require finite parameters.

2.2. The Appendix D repetition. The same values are repeated in the source's own Appendix D presentation of the board at PDF page 608 (book page 592)—the same pages examined for syntax in Chapter 10. The repetition matters evidentially: the value pair (sat = 0.1, threshold = 0.9) appears in two independent places in the document, produced at two different points in its structure (a tutorial chapter and a reference appendix). This forecloses the most charitable reading, a one-off typographical slip: a typo does not reproduce itself across a book's chapter text and its consolidated appendix. Whatever process produced the values, it produced them consistently.

2.3. The Appendix D note: a partial repair that sharpens the fault. At PDF page 610 (book page 594), an Appendix D note addresses one of the three facts. It states that the infinite gain $\beta = \infty$ is an idealization that validators reject, and directs the implementer to substitute a large finite value. The note thereby concedes—in the source's own voice—that the listing as printed does not pass the source's own validation rules, and it supplies a repair for the finiteness violation.

What the note does not do is the analytically decisive point. It retains the sat = 0.1 clamp, and it says nothing about the 0.9 threshold. The note is therefore a *partial* repair that leaves the load-bearing contradiction intact—indeed, it sharpens the contradiction's evidentiary status. Before the note, one could imagine that the entire parameter block was un-reviewed boilerplate. The note proves otherwise: this exact parameter block received editorial attention specific enough to catch the non-finite gain, and the attention nonetheless did not extend one line further to ask what a 0.1 output clamp means for a 0.9 downstream threshold. The clamp/threshold pair survived a targeted review of its own listing. Section 8 returns to why that pattern—review that catches type-level violations and misses function-level contradictions—is exactly what mechanical reachability checking is for.

Adverse Finding 11.2 (The published CSB values, with anchors). The source publishes, for CCB-1-CSB: red-stage saturation clamp sat = 0.1 and fuse threshold 0.9 at PDF page 342 (book page 326), repeated at PDF page 608 (book page 592); and red-stage gain $\beta = \infty$ despite finite grammar and validator rules. The Appendix D note at PDF page 610 (book page 594) directs substitution of a large finite gain while retaining the sat = 0.1 clamp, and does not reconcile the clamp with the 0.9 threshold.

3. Unreachability under the declared semantics

3.1. The declared semantics. The proof requires only three semantic commitments, all of which the source itself declares.

Definition 11.3 (Clamp semantics). A stage with saturation clamp sat $= c$ has output bounded above by c: for every input, the stage's output u satisfies $u \leq c$. Saturation is an *output plateau*: the stage's transfer curve levels off at c and remains there for all sufficiently large inputs. It is not declared to change the stage's state, to latch, or to trigger any side effect (Section 4 develops this point).

Definition 11.4 (Threshold semantics). A fuse with threshold t actuates only upon a threshold crossing: actuation requires the fuse's input to reach at least t.

Definition 11.5 (Monotone non-amplifying stage). A stage s on the declared red-stage-to-fuse path is *monotone non-amplifying* under the published parameters if for all admissible inputs $x \geq 0$, its output satisfies $s(x) \leq x$, and s is monotone non-decreasing in x. Clipping stages and cutoff stages have this property by construction (they can only reduce or pass a bound); the published path between the red-stage clamp and the fuse contains no amplifying stage.

The third commitment—no amplifier between clamp and fuse—is a fact about the published listing read under its declared stage inventory, and it is the only place where the proof touches the listing's structure rather than its two headline numbers. Its role is made explicit in the proposition so that any future challenge knows exactly what to check: if a revised build inserted a declared gain stage after the clamp, the proposition's hypothesis, not its logic, would fail.

3.2. The proposition.

Lemma 11.6 (Bound propagation). *Let $u_0 \leq c$ and let $u_i = s_i(u_{i-1})$ for $i = 1, \ldots, n$, where each s_i is monotone non-amplifying in the sense of Definition 11.5. Then $u_n \leq c$.*

Proof. By induction. The base case is the hypothesis $u_0 \leq c$. For the step, suppose $u_{i-1} \leq c$. Then $u_i = s_i(u_{i-1}) \leq u_{i-1} \leq c$, the first inequality by the non-amplifying property applied at $x = u_{i-1}$. □

Proposition 11.7 (Unreachability of the CSB fuse threshold). *Under the declared clamp semantics (Definition 11.3), with the red stage clamped at* sat $= 0.1$*, with every declared stage on the red-stage-to-fuse path monotone non-amplifying under the published parameters (Definition 11.5), and with the fuse actuating only on a threshold crossing at* 0.9 *(Definition 11.4): the fuse input never exceeds* 0.1*, and since* $0.1 < 0.9$*, the fuse never actuates. The protective crossing is unreachable for every input to the board.*

Proof. Let u_0 be the red-stage output. By the clamp, $u_0 \leq 0.1$ pointwise, for every board input and every internal condition, since the clamp bounds the stage's output unconditionally. The fuse input is u_n for the declared path $s_1, \ldots, s_n$; by Lemma 11.6 with $c = 0.1$, $u_n \leq 0.1$. Actuation requires $u_n \geq 0.9$. Since $0.1 < 0.9$, no input produces actuation. □

Corollary 11.8 (The finite-gain substitution does not repair reachability). *The repair directed by the Appendix D note—replacing $\beta = \infty$ with any large finite gain—does not affect Proposition 11.7. The gain β acts upstream of or within the red stage, whose output the clamp bounds unconditionally at* 0.1 *regardless of β; the proof uses only the clamp value and the downstream path. The note's repair addresses type-level validity (finiteness) and leaves functional validity (reachability) unchanged.*

Corollary 11.9 (The DRC necessarily flags the board). *The candidate DRC's threshold-reachability check propagates conservative upper bounds through causes, bias and gain stages,*

clipping, and cutoffs, and emits `PCBA8-SAF-001` *(*UNREACHABLE SAFETY THRESHOLD*) for any safety-critical fuse whose threshold exceeds the resolved upstream bound. On the CSB fixture the resolved bound at the fuse is* 0.1 *and the threshold is* 0.9*; the check therefore emits* `PCBA8-SAF-001` *deterministically. An unresolved bound, by the same check class, is reported as* `PCBA8-SAF-003` *(*UNRESOLVED SAFETY ANALYSIS*) and never treated as reachable.*

3.3. On the strength and modesty of the argument. The proof is elementary—deliberately so, and the elementarity is the finding. No simulation, no numerical subtlety, no semantic depth is needed: the contradiction is decidable from two printed numbers and the printed path's monotonicity, by an argument a reviewer can verify in a minute. Three observations calibrate its strength.

The result is semantic, not syntactic. Chapter 10's findings concern derivability of text; this chapter's finding would survive any grammar repair. Even under a complete grammar in which the CSB listing parses, the parsed board still has a clamp at 0.1 feeding a threshold at 0.9.

The result is conditional on the declared semantics. If "saturation" meant something other than an output plateau—a latch, an alarm event, an escalation trigger—the proof's first line would fail. Section 4 shows the source declares no such alternative semantics, and that the absence of an alternative is itself the second defect.

The result is total, not probabilistic. The unreachability holds for *every* input, not for typical inputs or with high probability. There is no operating regime, however extreme, in which the published mechanism performs its published function. In safety-engineering terms the protective function is not degraded; it is absent.

3.4. A worked illustration of bound propagation.

EXAMPLE 11.10 (Illustrative bound propagation; not from the source). The following stage chain is *entirely hypothetical and illustrative*: it is constructed for this example to show the arithmetic of Lemma 11.6 concretely, and it is not the published CSB path, whose exact stage inventory the proposition needs only qualitatively. Consider a clamped source followed by three monotone non-amplifying stages: a clipping stage $s_1(x) = \min(x, 0.8)$, an attenuating filter stage $s_2(x) = 0.7x$, and a cutoff stage $s_3(x) = x$ for $x \geq 0.05$ and 0 otherwise. With the source clamped at $c = 0.1$, the propagated bounds are:

$$u_0 \leq 0.1, \qquad u_1 \leq \min(0.1, 0.8) = 0.1, \qquad u_2 \leq 0.07, \qquad u_3 \leq 0.07.$$

A fuse at threshold 0.9 downstream of s_3 is unreachable by a margin of 0.83; indeed even a threshold of 0.08 would be unreachable in this illustration. Two features of the arithmetic generalize. First, the bound can only fall or stay flat through non-amplifying stages—the clamp is the ceiling for the entire downstream path, which is why the proof needs no knowledge of the stages beyond monotone non-amplification. Second, the conservative bound is cheap: each stage contributes one interval operation, so the DRC's reachability check is linear in path length and involves no simulation. The check that catches the fault costs less than the trace that would fail to exhibit it.

3.5. How could the values arise? Genealogy hypotheses. The audit's claims do not depend on how the contradiction entered the source, and the record deliberately asserts nothing about genesis. But a known-fault analysis is more useful to future specification authors if it maps the processes that *could* produce such a fault, because each process suggests a distinct guard. Four hypotheses are consistent with the printed evidence; all are offered strictly as hypotheses, none is asserted.

Registered Hypothesis 11.11 (Normalization mismatch). The clamp and the threshold were authored against different implicit scales—for instance, a red stage specified on a per-stage normalized scale where 0.1 is a large value, and a fuse specified on a global severity scale where 0.9 is the top decile—and the two scales were never reconciled when the stages were composed. Guard: unit and scale declarations on every port, checked at every net, which is precisely the DRC's `NET-005`/`NET-006`/`NET-009` discipline; a signal-kind/unit table would have refused the clamp-to-fuse net or forced the scales into the open.

Registered Hypothesis 11.12 (Idealization drift). The listing began as an idealized pedagogical sketch (consistent with $\beta = \infty$, an avowed idealization) in which the red stage's plateau was meant to represent "fully saturated alarm" symbolically rather than numerically, and later editorial passes concretized some values for validators (the finite-gain note) without re-deriving the ones that interact. Guard: Rule 11.22(2)—no idealization notes; the design as printed is the design as checked—plus relational checks that re-run on every edit, so that concretizing one value cannot leave its partners stale.

Registered Hypothesis 11.13 (Template reuse). The stage block was copied from another listing (a monitoring or filtering context where a 0.1 clamp is innocuous) into the safety exemplar, carrying its parameters with it; the fuse block, authored separately with a conventional high threshold, was composed downstream without an end-to-end pass. Guard: exactly the end-to-end pass that the DRC mechanizes—reachability is a whole-path property, so it is the check that copy-composition cannot satisfy by accident.

Registered Hypothesis 11.14 (Transposition or decimal error). The clamp was intended as 1.0 (no effective clamp) or the threshold as 0.09, and a single-character error produced the pair. The Appendix D repetition (Section 2) makes an isolated typographical event less plausible—the error would have to precede both occurrences—but a single upstream master value propagated to both pages remains possible. Guard: known-answer traces for the trip path (Rule 11.22(5)); any single-character corruption of either value fails the exhibited crossing trace immediately.

The four hypotheses converge on one observation worth stating plainly: *every* candidate genesis is mundane. No exotic process is needed to put a function-defeating contradiction into a safety exemplar—scale drift, pedagogical residue, copy-paste, and typos suffice, and every one of them is a process that operates in ordinary, competent documentation work. That is the strongest argument that mechanical relational checking is not optional for safety-bearing specifications: the fault classes that defeat protective functions are the same fault classes that ordinary authorship produces daily.

4. Plateau, not latch: the missing fail-closed state machine

The clamp/threshold contradiction is the loud defect. The quiet defect—likely to be missed even by a reader who catches the first—is a matter of *kind* rather than of value: the CSB's terminal safety behavior is specified as continuous signal flow, when the function it is meant to perform requires a discrete, latching, fail-closed state machine. This section develops the distinction, and then the specific hazard that follows from its absence: the source never establishes that an open fuse *preserves* the refusal path rather than *suppressing* it.

4.1. Saturation is a plateau. Under Definition 11.3, a saturated stage is simply a stage whose transfer curve has leveled off. Saturation has no memory: if the input recedes, the output recedes with it. It marks no event: nothing in the declared semantics distinguishes

"the red stage is at 0.09" from "the red stage is pinned at its clamp" except the value itself. And it is silent: no declared observer is notified that the plateau has been reached.

Now consider what the red stage is *for*. It is the maximal-severity terminus of an escalation path: reaching red is supposed to be the board's determination that the modeled condition warrants terminal protective action. Encoding that determination as a plateau has a perverse consequence quite apart from the threshold arithmetic: precisely at the moment of maximal severity, the stage *stops conveying information*. Any severity beyond the clamp is projected onto the clamp; the downstream path cannot distinguish barely-red from catastrophically-red; and if the driving condition oscillates, the red signal oscillates with it, de-asserting the alarm whenever the input dips. The mechanism's most safety-critical state is its least expressive and least stable one.

4.2. What a latch would be.

Definition 11.15 (Latching protective element). A protective element is *latching* if actuation is a transition into an absorbing state: once tripped, the element remains tripped regardless of subsequent input, until an explicitly specified, separately authorized reset transition occurs. A latching element therefore requires a state machine: a set of discrete states (at minimum *armed* and *tripped*; realistically also *fault* and *maintenance*), a transition relation naming which events move between them, and a specification of the element's effect on the protected path in every state.

Physical fuses are the canonical latch: a blown fuse stays blown. The CSB borrows the fuse's name but not its defining property—nowhere does the source declare that the CSB fuse, once tripped, stays tripped, nor what resets it, nor what the board's outputs are while it is tripped. There is no state machine in the book for the fuse at all: no state set, no transition table, no reset authority, no fault state. The fuse exists in the listing solely as a threshold comparison embedded in continuous signal flow.

Adverse Finding 11.16 (No fail-closed latch state machine is defined). The source defines the CSB's protective behavior entirely as continuous stage flow with a threshold element. It does not define a latching state machine for the fuse: no absorbing tripped state, no reset transition or authority, no fault state, and no specification of the board's behavior conditional on fuse state. In particular, the source does not establish that an open (tripped) fuse preserves the refusal path rather than suppressing it.

4.3. The open-fuse ambiguity. Finding 11.16's final sentence deserves its own subsection, because it identifies a hazard that would persist *even if the threshold were reachable*—which is why this chapter treats it as an independent defect rather than a footnote to the arithmetic.

In electrical practice, a fuse protects by *interrupting* a path: actuation opens the circuit, and the design is arranged so that the opened circuit is the safe configuration. The safety of a fuse is therefore not a property of the fuse; it is a property of the *polarity* of its placement—of which configuration, closed or open, the surrounding design makes safe. The same element that protects in one placement endangers in the mirrored placement.

Translate this to the CSB's declared purpose. The board models a refusal mechanism: its protective action is meant to *enforce* refusal when severity reaches red. Now ask the polarity question the source never asks: when the CSB fuse actuates and "opens," what happens to the refusal signal? Two architectures share the listing's surface:

(1) *Fail-closed (safe) polarity.* The fuse interrupts the *permission* path: actuation cuts off the output that permits continued operation, so an open fuse forces refusal. Fuse failure, spurious actuation, or a stuck-open fuse all err on the side of refusing.
(2) *Fail-open (dangerous) polarity.* The fuse sits *in* the refusal path: actuation interrupts the very signal that carries the refusal. An open fuse then *suppresses* refusal—the protective action disables the protection—and the mechanism's terminal act is to remove itself.

The published material does not determine which architecture the CSB instantiates. There is no fuse truth table (Section 7) mapping fuse state to path continuity, no statement of the board's outputs in the tripped state, and no hazard analysis of spurious or stuck actuation. A reader cannot prove the CSB is the dangerous polarity—but the burden in safety specification runs the other way: a protective element whose safe polarity is not established must be treated as unestablished, and an unestablished terminal element is a specification defect regardless of reachability. The candidate's DRC encodes exactly this burden allocation: an unresolved safety analysis is `PCBA8-SAF-003`, never a pass, and a fuse whose declared action cannot be shown safe is `PCBA8-SAF-002` (UNSAFE FUSE ACTION).

4.4. The plateau hazard, formalized. The claim that a plateau de-asserts under oscillation can be made exact with one small definition, which also fixes vocabulary for the hazard analysis that Rule 11.22(6) requires.

DEFINITION 11.17 (Assertion persistence). Let $r(t)$ be a stage's output over a discrete time index and let $A \subseteq \mathbb{R}$ be its assertion region (for the red stage, the clamp value or a neighborhood of it). The stage's assertion is *persistent* if $r(t) \in A$ implies $r(t') \in A$ for all $t' \geq t$ absent an authorized reset; it is *input-following* if $r(t) \in A$ holds only while the driving input keeps r in A.

REMARK 11.18 (The red stage is input-following). Under the declared clamp semantics the red stage is input-following: its membership in the assertion region at time t carries no information about membership at $t + 1$, because the clamp is a memoryless bound, not a state. Consequently, for any driving condition whose severity signal is non-monotone over time—the ordinary case for any measured or estimated quantity—the red assertion toggles with the signal. Every de-assertion interval is an interval in which the downstream protective path sees a sub-assertion value, and any downstream element that requires *sustained* assertion (an integration window, a debounce, a confirmation delay, all common in protective paths) may never see its sustain condition met even when the underlying condition is continuously hazardous but noisily measured. Persistence, where it is wanted, must therefore be supplied by an explicit latch (Definition 11.15); it cannot be an emergent property of clamped continuous flow. This is the precise content of the slogan "saturation is a plateau, not a latch": the plateau bounds amplitude, while a latch bounds *retreat*, and the two are orthogonal mechanisms.

4.5. The two defects compound. Note finally how the plateau defect and the polarity defect interact with the threshold defect. Because saturation is a plateau (no latch upstream), a reachable fuse would be armed by a signal that de-asserts under oscillation; because no fuse state machine exists (no latch downstream), even a tripped fuse has no specified persistence; and because the threshold is unreachable, none of this machinery is ever exercised, so no simulation of the published board would ever surface the downstream ambiguities. The unreachability defect *masks* the latch and polarity defects: a test campaign against the published board exercises the trip path never, and would report the board "safe"

in the emptiest possible sense—no unsafe actuation, because no actuation. This masking structure is why the candidate treats the CSB as a permanent fixture (Section 5): it is a naturally occurring specimen of three defect classes—value contradiction, missing state machine, unestablished polarity—layered so that the first hides the other two.

5. Disposition: a known-fault fixture, permanently flagged

5.1. The disposition rule.

Rule 11.19 (CSB disposition). The unmodified CSB parameters are retained in the candidate's test corpus as a *known-fault fixture*. The design-rule checker must flag the fixture: a DRC run over the CSB fixture that does not emit `PCBA8-SAF-001` is itself a checker failure. The fixture is never repaired in place; any repaired design is a separately versioned artifact under Section 7. No test result over the fixture may be represented as evidence of any safety property of any board or system.

The rule inverts the usual polarity of a test oracle, and the inversion is worth spelling out. For ordinary fixtures, green means the software accepted a valid input; for the CSB fixture, green means the software *rejected* an invalid one—the assertion is that `SAF-001` is present in the report, with the exact diagnostic identity, severity ERROR, and deterministic position that the closed catalog assigns it. The fixture thereby guards three things at once: the reachability check's existence (deleting the check breaks the fixture), its sensitivity (weakening the bound propagation until 0.1 resolves as reachable breaks the fixture), and its determinism (the diagnostic's stable identity is part of the report digest discipline of Chapter 7).

5.2. Why the flag is mandatory, not advisory. It might be objected that a reference listing from an authoritative source deserves at most a warning—that an ERROR-severity flag against the source's own exemplar is presumptuous. The candidate's position is the opposite, for a reason the two preceding sections have earned: the contradiction is total (Proposition 11.7 holds for every input), the masked defects behind it are of the highest severity class (terminal protective element with unestablished polarity), and the source's own partial repair demonstrates that the values were reviewed and retained (Section 2.3). A checker that softens a proven protective-function impossibility to a warning because the design came from an authority has re-created precisely the deference that allowed the values to survive review in the first place. Deference to sources is a bibliographic virtue and an engineering vice; the DRC is built for the engineering side.

5.3. The CSB against the full check-class architecture. Rule 11.19 names `PCBA8-SAF-001` because that is the diagnostic the headline fault deterministically produces. But the CSB fixture exercises the safety-relevant corner of the catalog more broadly, and mapping the board's defects onto the check classes of Chapter 7 shows how the architecture distributes the burden that the source's own validation carried in one undifferentiated pass.

Invariant 11.20 (Fixture diagnostic floor). Every DRC run over the unmodified CSB fixture emits `PCBA8-SAF-001` with severity ERROR, at a deterministic position in the sorted report, contributing to a stable report digest. Removal, demotion, or repositioning of the diagnostic is a checker regression by definition.

Beyond the floor, three check classes bear on the board's quiet defects. The *parameter class* (`PAR-003`) is where the $\beta = \infty$ violation lands: a non-finite value for a finite-typed parameter is an invalid parameter, independent of any note directing a substitution—the checker

reads listings, not errata. The *safety class*'s remaining codes frame the polarity and analysis gaps: a fuse whose declared action cannot be shown to preserve the protected function is `SAF-002` (UNSAFE FUSE ACTION), and a safety analysis the checker cannot complete—for instance, because the fixture declares no fuse state machine to analyze—is `SAF-003` (UNRESOLVED SAFETY ANALYSIS), which is a failure, never a silent pass. The *test-point and calibration classes* (`TP-002`, `TP-003`, `CAL-004`) encode the observability lesson: a safety-critical path without a marked, calibrated probe cannot even be argued about from traces, so unprobed safety paths are flagged before any question of their dynamics arises.

The distribution matters architecturally. The source's validators, as evidenced by the Appendix D note, enforced a single fault family (type-level finiteness) and were silent on the rest. The catalog carves the same territory into named, separately testable obligations—finiteness (`PAR`), reachability (`SAF-001`), action safety (`SAF-002`), analysis completeness (`SAF-003`), observability (`TP/CAL`)—so that each obligation has its own fixture, its own regression surface, and its own place in a report. A defect class without a code is a defect class without a guard; the CSB contributed directly to the catalog's insistence that the safety corner have three codes rather than one.

5.4. Known-fault fixtures and engineering practice. The known-fault disposition has recognizable relatives in established practice, and locating it among them clarifies what is and is not novel. It is a *negative control* in the laboratory sense: an input whose correct result is a detection, run to prove the instrument detects. It is kin to *fault injection* and to mutation-style testing—disciplines that deliberately introduce defects to measure whether a test suite notices—with the notable difference that nothing was injected: the defect is naturally occurring, harvested from a real authority document, which gives it an ecological validity that synthesized mutants lack. And it plays the role that adversarial generation plays in property-based testing [24]: it is the concrete witness that keeps a universal claim ("the checker flags unreachable thresholds") honest. The research program's planned property campaign (Chapter 12) will generate randomized boards with clamp/threshold contradictions in varied positions precisely to generalize the single natural specimen into a family; the CSB fixture remains the anchor of that family because it alone certifies that the fault class occurs outside the generator.

What *is* unusual, relative to common practice, is permanence with provenance: the fixture is retained under the source's identity, page anchors, and hash, so that the regression guard doubles as an archival record of the authority document's reviewed state. Ordinary fault injection guards a checker; a preserved, provenance-bound known fault also guards the historiography—five years from now, the claim "the published CSB could not trip" remains verifiable against the exact bytes, whatever revisions intervene.

6. The negative-evidence philosophy

Why preserve a fault when repairing it would take one line? The candidate's answer, argued here, is that for a research architecture the preserved fault is worth strictly more than the repaired board, on four grounds.

First: the fault is evidence about the source, and repair destroys evidence. The monograph's relationship to its 743-page baseline is that of an implementation to an authority document whose reliability must itself be assessed. Finding 11.2 is a data point of the highest quality about that reliability: the source's flagship safety listing contains a function-defeating value contradiction that survived targeted editorial review. Silently repairing the listing in the implementation's corpus would leave future maintainers—and future reviewers

assessing how much weight the source's other 700 pages can bear—without the specimen. An engineering record that patches its authority's faults away is laundering its authority.

Second: the fixture is a permanent regression guard for the checker. A reachability check with no failing fixture is a check that can be silently broken. The known-fault fixture is the only test that fails when the `SAF-001` path rots—and it is a *realistic* failure, drawn from a professionally produced document rather than synthesized, which makes it evidence that the check class targets faults that actually occur in the wild rather than faults invented to be caught.

Third: repair-in-place forges a provenance. The repaired board would inevitably circulate under the CSB's name—"the CSB, fixed"—attaching the source's authority to parameter choices the source never made. Who chose the new clamp value? On what hazard analysis? With what review? Repair-in-place answers none of these while appearing to inherit the source's answers. The separately versioned repair of Section 7 exists precisely to force those questions to be answered on the record, under the repairer's own name and hash.

Fourth: negative results are the currency of the claim discipline. The monograph's evidence program (Chapter 12) counts open criteria, counterexamples, and conflicts as first-class deliverables, on the view that an artifact's credibility is carried by what it admits against itself. A preserved, page-anchored, machine-checked fault in the source's safety exemplar is the strongest single exhibit that the discipline is real: it demonstrates, more convincingly than any green suite, that the project reports what it finds rather than what it hopes. The verification-not-validation lesson of the simulation literature [**23**] applies with full force: a green run over a repaired board would verify the implementation against a fixture while validating nothing about safety—whereas the flagged fault verifies the checker against reality.

6.1. Objections and responses. Three objections to the preservation policy recur in discussion and deserve answers on the record.

Objection: the fixture is unfair to the source, which marked the listing as pedagogical. Response: the source's pedagogical framing is preserved along with the fault—the fixture's documentation records the Appendix D note verbatim in substance, including its idealization language. But the note itself demonstrates that the listing was subject to validator discipline (it repairs a validator rejection), so the source's own frame treats the listing as a checkable artifact, not free prose. The audit checks it under exactly that frame. A listing exempt from checking would need to be marked exempt from checking; a listing marked "fix this value so the validators accept it" has waived the exemption.

Objection: preserving a fault in the corpus risks someone executing the faulty board as if it were good. Response: this risk is real and is engineered against three ways. The fixture fails DRC with an ERROR-severity diagnostic, and the execution pipeline (Chapter 8) refuses simulation of boards whose reports contain errors—the faulty board is not executable within the candidate at all. The fixture's identity is bound into test assertions that expect the failure, so any pathway that made it pass would break the suite. And Rule 11.19 prohibits representing fixture runs as safety evidence, closing the interpretive route. The residual risk—extraction of the listing from the source by parties outside the candidate's tooling—exists independently of the candidate's choices and is, if anything, reduced by publishing the analysis.

Objection: the fault is so elementary that preserving and formalizing it is disproportionate ceremony. Response: the elementarity is the phenomenon. The interesting fact about $0.1 < 0.9$ is not the arithmetic but its survival—through authorship, consolidation into an appendix, and a targeted editorial repair of the same parameter block. A fault class that survives that

gauntlet in a safety exemplar has empirically demonstrated that informal review does not catch it; the proportionate response to such a demonstration is exactly a formal check, a permanent fixture, and a published analysis. Ceremony is what converts an embarrassing anecdote into a guard.

Remark 11.21 (Preservation is not endorsement). Preserving the fixture asserts nothing in the fault's favor. The fixture ships flagged, its documentation names it a known fault, and Rule 11.19 forbids representing any run over it as safety evidence. Preservation is the archival act of keeping the specimen; endorsement would be the semantic act of building on it. The candidate does the first and prohibits the second.

7. Requirements on any repaired design

A repair is not a value edit; it is a design activity with obligations. The candidate fixes those obligations now, in advance of any repair being attempted, so that the bar cannot be quietly lowered when a repair becomes convenient.

Rule 11.22 (Repair requirements). Any repaired Cognitive Safety Board is a separately versioned design, under its own identity and digest, never replacing the known-fault fixture. It must supply, at minimum:

(1) **Threshold reachability.** A demonstration, by the DRC's conservative bound propagation and by at least one deterministic trace, that the protective threshold is reachable from admissible board inputs: the resolved upper bound at the fuse strictly exceeds the threshold, and a concrete input is exhibited whose trace crosses it.
(2) **Finite parameters.** Every parameter finite and validated, with no idealization notes: the design as printed is the design as checked. The $\beta = \infty$ pattern—an infinite value in the listing, a repair instruction in a note—is prohibited; whatever finite gain the design needs is the gain it declares.
(3) **Latch state transitions.** An explicit protective-element state machine per Definition 11.15: discrete states including an absorbing tripped state; a complete transition relation; an explicitly authorized reset transition; and defined behavior in every state, including fault states.
(4) **Fuse truth tables.** A complete tabulation, for every fuse state, of the continuity of every path through the protective element—resolving the polarity question of Section 4.3 on the record. The table must establish fail-closed polarity: every failure mode of the fuse (spurious trip, failure to trip, stuck state) maps to a configuration in which the refusal function is preserved or strengthened, never suppressed.
(5) **Deterministic traces.** Canonical traces (Chapter 8) for the nominal path, the trip path, the reset path, and each declared failure mode, each binding the exact DRC report, profile, and source identities, so that the design's claimed behaviors are replayable rather than narrated.
(6) **Hazard analysis.** A systematic enumeration of the board's hazard set—at minimum: failure to actuate on demand, spurious actuation, actuation suppressed by upstream oscillation (the plateau hazard of Section 4), reset abuse, and masked-defect interactions—with, for each hazard, the design feature that mitigates it and the check or trace that evidences the mitigation.

(7) **Independent safety review.** Review of the repaired design by parties independent of its authors, on the record, before any use of the design in any claim. Property-based and randomized adversarial testing of the state machine and truth tables [**24**] is expected as an instrument of that review, not as a substitute for it.

The list is ordered by dependency, not importance: reachability is listed first because without it nothing downstream is exercised (the masking structure of Section 4), but the truth-table requirement is the one that addresses the deepest defect. It is entirely possible to satisfy requirement (1) with a one-line clamp change and still ship the dangerous polarity of Section 4.3; requirements (3), (4), and (6) exist because the CSB's quiet defects demand them; requirement (7) exists because the CSB's loud defect survived non-independent review.

Remark 11.23 (No repair is scheduled). At the bound commit, no repaired CSB exists and none is scheduled ahead of the research program's earlier steps (Chapter 12). The requirements are published now so that they are commitments rather than retrospective rationalizations.

8. Lessons for safety-mechanism specification in cognitive-system engineering

The chapter closes with the general lessons the CSB analysis supports. Per Boundary 11.1, these are lessons about *specification discipline*—about how documents that specify protective mechanisms for cognitive systems should be written and checked—and carry no claim that any system is or is not safe.

Reachability is a design-rule property, and should be checked like one. The CSB's headline defect is decidable by interval arithmetic over declared parameters—the kind of property a mechanical checker verifies in microseconds and a human reviewer, reading a listing top to bottom, reliably misses because the clamp and the threshold are declared thirty lines apart and are each locally reasonable. The general rule: every protective threshold in a specification should be accompanied by a machine-checkable derivation that some admissible input reaches it. "The alarm can go off" is a theorem obligation, not an assumption.

Type-level validation is not function-level validation. The source's validators caught $\beta = \infty$ (a type-level fault: non-finite where finite is required) and missed $0.1 < 0.9$ (a function-level fault: parameters individually valid, jointly function-defeating). The pattern generalizes: schema conformance, type checking, and range validation operate per-value, while safety contradictions live in *relations between* values. A checker suite whose most semantic check is per-parameter will systematically pass exactly the faults that matter most. The DRC's reachability, feedback-gain, and polarity check classes are relational by design, and the CSB fixture is the standing demonstration of why.

Protective elements need state machines, not metaphors. "Fuse," "breaker," "interlock," and "latch" are load-bearing words in safety engineering because each names a specific discrete-state behavior with a specific failure discipline. A cognitive-system specification that borrows the vocabulary without the state machines—as the CSB borrows "fuse" for a threshold comparison in continuous flow—imports the connotation of safety while omitting its mechanism. The specification rule: any element described with protective vocabulary must be accompanied by its state set, transition relation, reset authority, and truth table, or the vocabulary must be surrendered.

Polarity must be established, not presumed. The open-fuse ambiguity of Section 4.3 is invisible to any reader who assumes that actuating a safety element is inherently safe. It is not: safety is a property of what the surrounding design does with the actuation, and the fail-closed direction must be demonstrated for each failure mode. In cognitive-system

engineering, where the protected function (a refusal, a constraint, an oversight signal) is itself a signal that can be interrupted, the presumption is especially treacherous—the same act that trips protection can sever it.

Severity plateaus discard information exactly where information matters most. Clamping a severity signal at its maximum makes the mechanism's most critical regime its least observable one. Escalation paths should preserve monotone information into their terminal stage—or, better, convert severity into discrete latched events whose persistence does not depend on the driving signal's continued cooperation.

Unreachable mechanisms produce vacuous green. The masking analysis of Section 4 is a general warning about test evidence for safety mechanisms: a mechanism that never actuates generates no unsafe-actuation findings, and an undisciplined test report will read that silence as cleanliness. Test campaigns over protective paths must therefore report *coverage of the actuation event itself*—how many runs tripped the mechanism—and treat zero as a finding, not a pass. This is the dynamic counterpart of the static reachability rule, and either alone would have caught the CSB.

Authority is not evidence. The values survived because they arrived in an authoritative package: a polished reference, a consolidated appendix, an editorial note demonstrating that review had occurred. Every layer of that authority was real, and none of it bore on whether 0.1 exceeds 0.9. A checking discipline that weights provenance over derivation will pass this fault every time; the DRC therefore weighs nothing but the board in front of it (Section 5). Causal disciplines make the same point in their own vocabulary—assumptions must be stated and checked, not inherited from the prestige of the model that carries them [**27**]—and the monograph's evidence chapter builds the general case.

Partial repairs are a distinct hazard class. The Appendix D note is, in miniature, a pattern every specification maintainer will recognize: a defect report arrives, the named defect is fixed, and the fix is verified against the named defect alone. The CSB shows the pattern's failure mode at its sharpest—the repair (finite gain) was correct, local, validated, and functionally irrelevant, and its very presence manufactures an impression that the block was reviewed and is sound. Guarding against it requires that repairs re-trigger *relational* checks over everything the touched values interact with, not merely re-validation of the touched values; in DRC terms, any parameter edit re-runs the whole report, and the report digest changes if anything anywhere moved.

Specifications should carry their own falsifiers. The deepest lesson runs beyond checking to authorship. A protective mechanism's specification should state, alongside the mechanism, the concrete demonstration that would show it working: the input that trips it, the trace it produces, the truth table its failure modes obey. The CSB specifies a mechanism and no demonstration; had the source's own conventions required an exhibited trip trace per protective element, the contradiction could not have been written down, because the exhibit would have been impossible to produce. Requiring the falsifier at authorship time (Rule 11.22(1) and (5)) moves the check from review—where it demonstrably failed—into the act of writing, where the fault cannot form. The same movement, generalized to every claim class, is the subject of the next chapter.

Claim Boundary 11.24 (What this chapter licenses). This chapter licenses exactly three claims, all Ring 4, not licensed for external assertion, disclosure held: (1) the identified source publishes the CSB values recorded in Finding 11.2 at the cited pages; (2) under the declared clamp semantics and the published parameters, the CSB fuse threshold is unreachable (Proposition 11.7), and the candidate DRC deterministically flags the fixture

with `PCBA8-SAF-001` (Corollary 11.9); (3) the source defines no fail-closed latch state machine for the fuse and does not establish open-fuse polarity (Finding 11.16). It licenses no claim that any system is safe or unsafe, no claim about repaired designs (none exist), and no claim about the source beyond the cited pages.

CHAPTER 12

Evidence, Threats to Validity, and the Research Program

1. The role of this chapter

The preceding eleven chapters specified an architecture and preserved two adverse findings. This chapter completes the monograph's evidential accounting: it states exactly what software evidence exists at the bound commit (Section 3), interprets that evidence at its correct strength (Section 4), catalogs the threats to validity that bound every claim made anywhere in this book (Section 5), lays out the dependency-ordered research program that would discharge the open obligations (Section 6), lists the claims quarantined until then (Section 7), situates the work relative to the methodological literature it leans on (Section 8), and concludes (Section 9).

The chapter's governing conviction is that an evidence section should be written with the same type discipline as the software it describes. "Evidence" is not one type. A green test suite, a hash-bound artifact manifest, a preserved counterexample, an open review criterion, and a claim record are five different types with five different licensing strengths, and treating them interchangeably—most commonly, treating green suites as if they licensed system claims—is the evidential analogue of the unit-mismatch faults the DRC exists to catch. Every number in this chapter is therefore presented with its type.

Claim Boundary 12.1 (Standing status). Everything in this monograph is DRAFT / OPEN / RING 4 / NOT LICENSED / HELD. The evidence recorded below consists of historical, source-bound software receipts, not experiments in this publication lane. Nothing in this chapter promotes any claim record, alters any status label, or asserts readiness of any kind.

2. An evidence type system

Before the receipt, the typing claim of Section 1 is made explicit, because the rest of the chapter uses it as load-bearing structure rather than as metaphor.

Definition 12.2 (Evidence types). The project's admitted evidence divides into five disjoint types:

(1) **Behavioral receipts**: green test outcomes over exact fixtures under exact profiles—suite runs, subtest results.
(2) **Identity bindings**: cryptographic digests fixing which bytes an assertion is about—artifact hashes, suite test-ID hashes, report digests.
(3) **Preserved negatives**: adverse findings held as first-class artifacts—counterexamples, source conflicts, known-fault fixtures.
(4) **Status registers**: recorded positions in a governance process—claim records with rings and license states, review criteria with open/closed states, gate passes.
(5) **Self-limits**: the record's own statements of its boundaries—method disclosures, claim boundaries, collector caveats.

Invariant 12.3 (Licensing strengths do not commute). Each evidence type licenses only its own claim form: behavioral receipts license "this software did this on this input"; identity bindings license "this assertion is about these bytes"; preserved negatives license "this defect exists in this source"; status registers license "this claim holds this status"; self-limits license "this record extends no further than this." No combination of instances of one type produces a claim of another type's form. In particular, no accumulation of behavioral receipts produces a system claim, a safety claim, or a licensing event.

The invariant sounds austere but is simply the observed failure mode of evidence sections, negated. The common degradation path runs: many green tests (type 1) are summarized as "verified" (a status word, type 4), which is paraphrased as "works" (a system claim, no type), which is marketed as "safe" (a claim the record never contained). Each step is a type error. The receipt below is presented type by type so that every number's licensing strength is visible at the point of statement, and the threats of Section 5 can be read as the formal statement of what each type cannot do.

3. The Gate-4 receipt

At the bound commit, the integrated Gate 4 receipt records the following, and exactly the following, as the project's admitted software evidence.

3.1. Artifacts and suites. The receipt binds **54 hash-bound artifacts**: specifications, audit records, claim records, review matrices, and implementation sources, each identified by SHA-256, so that the receipt's statements attach to exact bytes rather than to filenames or descriptions. Among these artifacts are the two that Chapters 10 and 11 rendered in prose—the construct-coverage record and the source-audit specification—under the hashes recorded in the front matter.

Three focused test suites are green at the receipt:

Suite	Methods	Suite test-ID SHA-256
CAS Core v8	61	`E2AF5014DE28DD0702211EF32A2757B99B869A13FAFBB0B8D1B18A50A9F8FEF5`
CAS PCBA v8	82	`D5F776C5F15758CD390470B5A542CCA110784DC5EBB9807B6BE41B2C80453470`
CAS-Ohm v1	65	`2E81A685961286D944894A22EB93345B97211B4433B6C71EAF628D84CD8EB1EF`

The combined focused run comprises **208 test methods** and **272 parameterized subtests**. The suite test-ID hashes are digests over the ordered test identifier lists, so the receipt pins not only that "the tests passed" but *which* tests existed: a silently deleted test changes the suite hash and invalidates the receipt. The 82 CAS PCBA v8 methods are the suite exercising the type system, board model, DRC catalog, simulator, trace identity, and adapter certificates of Chapters 4–9, including the known-fault fixture assertions of Chapter 11.

The three suites partition the evidence by specification, not by convenience. The CAS Core v8 suite (61 methods) exercises the semantic reference layer of Chapter 4: namespace disjointness, reference-tuple validation, definition-digest binding, and the refusal of cross-namespace identity. The CAS PCBA v8 suite (82 methods) exercises the board layer end to end: exact-type construction and revalidation of boards, components, ports, nets, test points, calibrations, and feedback declarations; the deterministic diagnostic catalog, including sort-order canonicality and report-digest stability; the fail-closed behaviors (subclass rejection, stale-linkage rejection, tampered-identity rejection); simulator trace identity and child-measurement acyclicity; adapter certificate binding and ambient-substitution rejection; and

the CSB fixture's mandatory `PCBA8-SAF-001`. The CAS-Ohm v1 suite (65 methods) exercises the parallel CAS-Ohm contract, whose eight preserved source conflicts enter the receipt separately. The partition means a regression localizes: a red method names its specification layer before any debugging begins.

Two structural properties of the suites deserve note because they are themselves tested. First, *determinism*: report and trace tests assert byte-identical digests across repeated runs, so any nondeterminism in ordering, serialization, or arithmetic surfaces as a suite failure rather than as flakiness to be shrugged at. Second, *fail-closed coverage*: a substantial fraction of the methods are negative tests—assertions that malformed, tampered, subclassed, or stale inputs are *rejected* with the exact typed error. Under Definition 12.2 these negative tests are behavioral receipts about the rejection paths, and they are the reason the candidate's "fail-closed" language is a tested property rather than a design intention.

3.2. Claim records, criteria, and preserved negatives. The receipt further records: **11 claim records**, all at admission Ring 4 and all NOT LICENSED—that is, every claim the project has articulated is registered, ringed, and unlicensed, with none promoted; **53 review criteria** of which **20 remain open**—the review matrix's accounting of what independent scrutiny would need to close, with more than a third explicitly outstanding; **4 direct EBNF counterexamples**—the Chapter 10 findings, counted from the machine-readable coverage record; and **8 CAS-Ohm source conflicts**—preserved contradictions between the CAS-Ohm v1 source material and its implementation contract, held as conflicts rather than silently resolved, in the same discipline that Chapters 10 and 11 applied to the PCBA source. The receipt also records an integrated **Gate 1 pass** for the project's foundational ceremony; Gate 1's scope is the project's own earlier gate definition and is inherited here as a historical receipt, not re-litigated.

3.3. The collector caveat. Finally, and importantly, the receipt records its own limit: the **repository-wide test collector was not clean** at the bound receipt. Collection across the full repository encountered **4 pre-existing package-path namespace collisions after 1,200 collected items**. The three focused suites ran and passed in isolation, but a repository-wide "all tests green" claim is not available and is not made: the collector defect means the universe of collectible tests is not even well-defined repository-wide at that commit. The caveat is preserved in the receipt precisely so that the focused-suite evidence cannot be quietly widened into a repository claim by later paraphrase.

3.4. Recomputing the receipt. The receipt's numbers are designed to be recomputed by a party holding only the bound commit, and the recomputation path for each class is worth stating because it defines what "auditable" means concretely here. The 54 artifact hashes are recomputed by hashing the named files; a mismatch on any one identifies the exact artifact that drifted. The suite test-ID hashes are recomputed by collecting each focused suite and hashing its ordered test identifier list; this detects test addition, deletion, and renaming without running anything. The 208/272 method and subtest counts are recomputed by the same collection. The suite outcomes are recomputed by running the suites in the pinned environment—the only step requiring execution. The counterexample count is recomputed by reading the coverage record's per-board Booleans (Chapter 10); the conflict count by reading the CAS-Ohm conflict records; the claim, ring, license, and criteria counts by reading the claim records and review matrix. The collector caveat is recomputed by attempting repository-wide collection and observing the four collisions. Nothing in the receipt requires trusting the project's assertion of it: every number is either a hash, a count over hashed

inputs, or an execution result under a hashed configuration. This recomputability, not any signature, is the receipt's whole claim to being evidence rather than narration—and it is also its ceiling, per Threat 12.11: recomputable is not the same as authorized.

REMARK 12.4 (Relation to the gate structure). Two gate references appear in the receipt and their asymmetry is intentional. The integrated Gate 1 pass is a *historical* receipt: the project's foundational ceremony completed under its own recorded definition, inherited here as a status register, not re-argued. Gate 5, by contrast, appears in this monograph only as a *future* control kernel for preregistration (program step 6), and at the bound commit its own record stands OPEN / NOT STARTED / RING 4 / NOT LICENSED / HELD / NOT FROZEN. Citing a gate as a control mechanism promotes nothing about the gate itself; the monograph uses Gate 5's kernel the way a protocol cites a registry—as infrastructure whose status travels with it.

ADVERSE FINDING 12.5 (The receipt, in one statement). At the bound commit: 54 hash-bound artifacts; three green focused suites (61, 82, and 65 methods; suite test-ID hashes as tabulated) totaling 208 methods and 272 subtests; 11 claim records, all Ring 4, all NOT LICENSED; 53 review criteria with 20 open; 4 direct EBNF counterexamples; 8 preserved CAS-Ohm source conflicts; an integrated Gate 1 pass; and a repository-wide collector that is *not* clean (4 namespace collisions after 1,200 items), so the focused suites support no repository-wide verification claim.

4. Interpreting the receipt

Each class of number in Finding 12.5 has a distinct licensing strength, and the differences do the real work.

Green suites license behavioral claims about exact fixtures. The 272 green subtests establish: the software under test, at the exact hashed sources, performed the asserted transitions on the asserted fixtures under the asserted profiles. They establish nothing about inputs outside the fixture set, nothing about the correspondence between the software's types and any cognitive reality, and nothing about safety. The correct reading of every green test is the Claim Boundary of Chapter 8: the software did the declared thing on the declared input. This is verification in the simulation-methodology sense—checking the implementation against its own specification—and is categorically distinct from validation against an intended external reality [**23**], of which the receipt contains none.

Hash bindings license identity claims. The 54 artifact hashes and three suite test-ID hashes establish which bytes the receipt speaks about. They convert the receipt's statements from descriptions (fragile under drift) into references (checkable forever). They license no quality claim whatsoever—a hash binds garbage as faithfully as gold—but they make every other claim auditable, which is their entire job. The discipline follows the provenance literature's separation of the artifact, the activity that produced it, and the derivation chain between them [**22**], implemented with digests rather than ontology conformance.

Preserved negatives license adverse claims at full strength. The 4 counterexamples and 8 conflicts are the receipt's strongest entries, in the precise sense developed in Chapter 10: adverse surface findings are robust under refinement, whereas favorable findings are bounded by instrument resolution. The project's evidence posture—negatives preserved, positives bounded—is visible in the receipt's own arithmetic: it counts its counterexamples with the same care as its passes.

Open criteria and unlicensed claims license nothing, on purpose. The 20 open criteria and the uniform NOT LICENSED status of all 11 claim records are the receipt's statement of

what has *not* happened: no independent review closure, no claim promotion. Their evidential value is honesty about position, and their practical value is that promotion, when and if it occurs, will be a visible status transition on a registered record rather than an ambient drift of language.

The collector caveat bounds the universe. The not-clean collector is the receipt's own threat-to-validity entry: it marks the exact boundary between what was tested (three focused suites) and what was not even enumerable (the repository-wide collection). Section 5 folds it into the internal-validity threat where it belongs.

5. Threats to validity

The threats are organized in six classes. Each is stated as a formal threat with its consequences and, where applicable, the program step (Section 6) that addresses it. None is hypothetical politeness; each names a live limit on what this monograph's contents can support.

5.1. Construct validity.

THREAT 12.6 (The types may not measure cognition). No experiment in the admitted evidence establishes that the selected CAS types—Noetics, Domains, Drives, Registers, ND-cells, simplex addresses—correspond to cognitive mechanisms in humans, to latent structure in machine-learned models, or to any natural kind. The circuit-shaped organization may deliver its engineering benefits (explicitness, type safety, replayability, falsifiability of the software) while being, as an ontology of cognition, wrong, partial, or unfalsified.

The threat's consequence is architectural, not rhetorical: it is why every semantic identity in the candidate is a versioned reference into a source rather than an assertion about the world (Chapter 4), why the adapter carries no automatic semantic theorem (Chapter 9), and why the claim quarantine (Section 7) holds every cognition-facing claim without exception. The construct threat cannot be discharged by more software testing—no quantity of green suites bears on it—which is why program steps 6 and 7 route it to preregistered calibration and held-out validation, the only instrument class that can touch it.

It is worth stating what discharge would even look like, because construct validity is the threat most often waved at rather than operationalized. A discharge program would require: (i) an operationalization—for each CAS type, a declared measurement procedure mapping observable behavior (human, model, or biological) to the type's values, with the procedure's own reliability characterized; (ii) discriminant evidence—demonstrations that the types are not merely relabelings of existing constructs, i.e., that the forty-cell structure captures variance that coarser or alternative structures do not; and (iii) failed falsification—preregistered predictions from typed board models that held out of sample where rival untyped models did not. Each of the three is a study family, not a test run; none exists at the bound commit; and the honest summary is that the construct threat is currently *unaddressed rather than mitigated*, which is exactly what a threat section is for.

5.2. Internal validity.

THREAT 12.7 (Bounded suites are not proofs). The DRC and simulator are evidenced by bounded regression suites—272 subtests over authored fixtures—not by proofs over all paths, exhaustive enumeration, or adversarial generation. The full CCDL parser does not exist (Chapter 10), so no evidence flows from the published listings through parsing into the checked object model. And the repository-wide collector was not clean at the receipt (4

namespace collisions after 1,200 items), so even the enumerability of the full test universe is compromised outside the focused suites.

Authored fixtures share their authors' blind spots: the faults represented are the faults the authors thought of, and the CSB specimen (Chapter 11) is a standing demonstration that professionally reviewed artifacts contain faults their authors did not think of. Property-based and randomized generation [**24**] is the standard escalation—generating boards, parameter sets, and edit sequences far outside the authored distribution and asserting invariants (report determinism, digest stability, fail-closed rejection) over the generated family—and is program step 4, together with a separately sourced mutation campaign to measure the suites' fault-detection power rather than assume it. Independent reimplementation (step 5) addresses the residual risk that specification and implementation share a common author's misreading.

The collector caveat deserves its own accounting within this threat, because its implications are easily under- and over-read. Under-read: "four collisions after 1,200 items" sounds like a 0.3% blemish, but a collection failure is not a test failure—it means an unknown set of tests beyond the 1,200th collected item was never enumerated, so the denominator of any repository-wide claim is unknown, which is qualitatively worse than a known failure count. Over-read: the collisions are package-path namespace collisions—an import-layout defect—not evidence of failing tests or defective product code, and they do not contaminate the three focused suites, whose collection is independent and whose test-ID hashes pin their contents exactly. The correct reading is scoped: the focused-suite evidence stands at full (bounded) strength; the repository-wide claim is unavailable; and the remediation (deduplicating package paths, then producing a clean full collection with its own hash) is ordinary engineering that belongs to housekeeping before step 5's hermetic reproduction, which would fail on the collision anyway. The receipt's refusal to average these two readings into a single reassuring sentence is Definition 12.2's type-5 discipline in action.

5.3. External validity.

Threat 12.8 (Nothing external is in evidence). The admitted evidence contains no live calibrated model, no physical board, no instrument chain, no held-out replication, and no multi-model evaluation. Every result is internal to the candidate's own software and fixtures. Cross-model generalization, cross-context generalization, and any correspondence to deployed systems are entirely open.

The threat is stated baldly because its most dangerous form is partial: a reader who accepts that no *safety* claim is licensed may still assume that, surely, some behavioral generalization is. None is. The candidate has never been connected to a live system in the admitted evidence lane; the calibration and test-point machinery of Chapter 7 is contract scaffolding whose *live* counterpart (instruments, uncertainty, drift, environmental validity) is program step 6's subject. Until then, external validity is not weak—it is absent by construction, and labeled so.

The design choice implicit in "absent by construction" should be defended, since a critic could ask why three years-equivalent of contract work preceded any external contact. The answer is the CSB masking lesson generalized: external evidence gathered through an unverified instrument is worse than no evidence, because it arrives bearing the instrument's faults invisibly. A calibration study run before the DRC's unit discipline existed could not distinguish a cognitive result from a unit mismatch; a model-probing result gathered before trace determinism was tested could not be replayed to check itself. The contract-first ordering is a bet—explicitly revisable at step 6—that the cheapest path to *trustworthy* external

evidence runs through verified instruments, even at the cost of a long interval during which the external column of the evidence table is honestly empty.

5.4. Numerical validity.

THREAT 12.9 (Disclosed numerical limits). The fixed decimal context controls the candidate's own arithmetic, but the binary64 Fisher–Rao implementation is a disclosed numerical pseudometric at its resolution: it is not proven that all distinct profiles are separated, and tolerance behavior near the simplex boundary is characterized only by the profile's own validation residuals. Independent numerical audit, condition analysis, and tolerance-stack analysis remain open.

The mitigations already in place are containment, not resolution: one immutable decimal context whose identity enters every report (so numerical environment drift is detectable), capped and versioned projector tolerances, and residual recording (Chapter 6). What is missing is adversarial numerics—inputs constructed to stress cancellation, near-boundary positivity, and digest-affecting rounding—which folds into program step 4's generation campaign and step 7's executor/auditor agreement requirement, under which two independent computations must agree numerically before a result is admitted.

The pseudometric disclosure merits one further sentence of interpretation, because it interacts with the identity discipline that the rest of the architecture leans on. The candidate's structural identities are exact (digests over canonical bytes), but its *geometric* statements are resolution-bounded: two simplex addresses that the binary64 Fisher–Rao implementation reports as distance zero are not thereby proven identical, and no uniqueness or separation argument anywhere in the candidate is permitted to route through the numerical metric. Where separation matters—for instance, in any future clustering or nearest-profile claim—the claim must either carry an explicit tolerance statement bounding what "distinct" means at binary64 resolution, or be computed in exact arithmetic under the decimal context. The threat, in other words, is contained by a routing rule: exact questions go to exact machinery, and the metric answers only metric questions at its stated resolution.

5.5. Causal validity.

THREAT 12.10 (No causal claims are supported). Graph connectivity, gain composition, bound propagation, prediction, and residual localization—everything the DRC and simulator compute—are structural and behavioral facts about declared models. None establishes causation in any target system. Causal claims require explicit causal models, identification assumptions, and interventions [27]; none is admitted at the bound commit.

The threat deserves emphasis because the candidate's vocabulary invites the slide: the ontology contains "Cause" and "Effect" state names, boards have directed paths, and Chapter 11 proves a reachability result that sounds causal. All of it is model-internal: Proposition 11.7 is a theorem about a specification, exactly as a type error is a theorem about a program, and neither says anything about what any external system would do. If a future program phase proposes causal claims about cognitive systems, those claims will need the full causal apparatus—models, assumptions, interventions—stated in their own records; the current architecture merely ensures that when that day comes, the structural substrate (which path feeds which) is explicit rather than folkloric.

There is nonetheless a genuine, bounded contribution the architecture makes toward eventual causal work, and stating it precisely avoids both over- and under-selling. A declared

board is, structurally, a candidate causal diagram: nodes, directed edges, and typed mechanisms, with the DRC enforcing that the diagram is complete (no unresolved endpoints), typed (no unit-incoherent edges), and honest about cycles (no undeclared feedback). What the architecture supplies is thus the *syntactic hygiene* that causal analysis presupposes—an unambiguous graph over declared variables. What it cannot supply is everything that makes such a diagram causal rather than notational: the identification of board variables with measurable quantities, the defensibility of the missing-edge assumptions, and the interventional access needed to test them [27]. The boundary between the two contributions is exactly the boundary between steps 1–5 and steps 6–7 of the program.

5.6. Safety and governance validity.

THREAT 12.11 (No safety inference; no authority). The preserved CSB finding (Chapter 11) blocks any safety-effectiveness inference from the source's safety exemplar, and the candidate makes none of its own. Hazard analysis, independent safety engineering, regulatory review, clinical review, authenticated claim authority, canon admission, disclosure approval, and product acceptance are all open. Historical AI-authored, self-hashed licensing receipts within the project record are not an authenticated reviewer or a legal authority, and nothing in this monograph treats them as one.

The last sentence is a governance point the project applies to itself. A receipt whose author and whose attestation mechanism are the same system that produced the work has the evidential status of a lab notebook: invaluable as record, worthless as authority. The receipt discipline of Section 3 is therefore deliberately *modest*: hashes and counts, which any party can recompute, rather than judgments, which the project cannot self-supply. Authority enters, if it ever does, at program step 8, from outside.

Two further governance exposures are named for completeness. *Vocabulary leakage*: the candidate's lexicon—"safety board," "fuse," "refusal path," "certification levels"—is drawn from domains where the words carry regulatory and assurance weight, and every use of them in a specification context risks being quoted out of context as an assurance statement. The mitigation is mechanical where possible (claim-language review is a named discipline at step 8; the quarantine names the prohibited inferences) and repetitive where not: this book's boundary environments exist to be quoted *with* their findings. *Status drift under summarization*: each layer of summary—receipt to chapter, chapter to abstract, abstract to citation—compresses toward capability language, because capability is what summaries are for. The countermeasure adopted here is that every summary layer carries the status string DRAFT / OPEN / RING 4 / NOT LICENSED / HELD verbatim, so that at minimum the drift must delete something visible rather than merely fail to repeat something.

6. The dependency-ordered research program

The program is eight steps. The ordering is a dependency order, not a priority order: each step consumes artifacts that earlier steps produce, and running a later step early would produce results that the missing prerequisites would immediately invalidate. The dependency structure at a glance:

Step	Activity	Consumes	Produces
1	Complete grammar + golden corpus	Ch. 10 audit record	grammar, coverage matrix
2	Round-trip parser, fail-closed	step 1	canonical textual boards
3	State/time profiles	steps 1–2	executable full corpus
4	Property + mutation campaigns	steps 1–3	hardened invariants
5	Hermetic reproduction	steps 1–4 stable	second-party checkability
6	Preregistered calibration/validation	steps 1–5	first external evidence
7	Agreement, falsifiers, replication	steps 5–6	admitted empirical results
8	Internal then external review	everything	closed criteria; authority

The table's "consumes" column is the argument for the ordering; the prose below develops it step by step: what each step is, why it sits where it sits, and what it unblocks.

Protocol 12.12 (Step 1: Complete versioned CCDL grammar and golden corpus). Specify a complete, versioned CCDL grammar covering every construct family evidenced in Chapter 10—the reserved Part-IV statement leaders, the memory and power-management registers, and the expression forms including replication fan-out—together with a golden five-board corpus fixed under exact digests.

This is first because Chapter 10 showed the published grammar covers a minority register of its own corpus: every downstream activity that touches listings (parsing, round-tripping, conformance, corpus-driven testing) is undefined until a grammar exists whose coverage is stated. The step's deliverable is as much the *coverage matrix* (Rule 10.22) as the grammar itself: a grammar without a corpus-verified matrix would recreate the source's defect at one remove. It unblocks step 2 directly and step 4's grammar-aware generation.

Protocol 12.13 (Step 2: Canonical parse/serialize round trips, fail-closed). Implement parsing and serialization such that parse-then-serialize is the identity on canonical form, every admitted board round-trips with a stable digest, and every unsupported form is rejected fail-closed with a coded diagnostic.

Second because it operationalizes step 1's grammar into checkable software and extends the hash-identity discipline (Chapter 5) to textual boards. The round-trip property is the cheap total test of parser/serializer agreement; fail-closed rejection converts the coverage boundary into an error surface rather than a silent subtraction. Unblocks: corpus-scale DRC evidence (published listings finally flow into the checked object model), and step 4's use of round-trips as a universal invariant.

Protocol 12.14 (Step 3: State and time profiles). Define separate, versioned execution profiles for the construct families the current simulator excludes: capacitor and charge dynamics, PLL and lock dynamics, memory discipline (the `charge`/`discharge`/`refresh`/`scan`/`select` family), power sequencing, stateful and stochastic feedback, and memristive history—each with declared state spaces, time semantics, and trace extensions.

Third because Chapter 10 showed the unformalized grammar registers are precisely the stateful/temporal ones: parsing them (step 2) without executable semantics would admit boards the simulator must then refuse. Profiles are *separate* by design—each is opt-in, versioned, and named in traces—so that the deterministic core's evidence is never diluted by a profile's immaturity. Unblocks: faithful execution of the full golden corpus, and meaningful simulation-level testing in step 4.

Protocol 12.15 (Step 4: Property-based campaign; sourced mutation testing). Run a property-based and randomized-generation campaign [**24**] over DRC and simulation invariants—report determinism, digest stability, sort-order canonicality, fail-closed rejection, round-trip identity, bound-propagation soundness, clamp/threshold contradiction detection in generated positions—and separately specify and source a mutation-testing campaign measuring the suites' fault-detection power.

Fourth because generation needs the grammar (step 1), the round trip (step 2), and the profiles (step 3) to define its input space and its invariants; run earlier, it could only exercise the programmatic fragment already covered by authored fixtures. This step is the direct answer to Threat 12.7 and the generalization of the CSB specimen into a fault family (Section 5.4). Mutation testing is listed separately and "sourced" deliberately: it measures the tests rather than the code, and its tooling and scope need their own record so its results cannot be confused with the property campaign's.

Protocol 12.16 (Step 5: Hermetic independent reproduction). Independently reproduce the exact source and test bundle in a hermetic toolchain: pinned dependencies, isolated environment, bit-identical artifact hashes, and suite results reproduced from the receipt's inputs by a party other than the original executor.

Fifth because reproduction is only meaningful once the thing to reproduce is stable: steps 1–4 will have churned the corpus, grammar, and suites, and reproducing a moving target wastes the independence it spends. Hermetic reproduction converts the receipt's hash bindings from self-consistency into external checkability, and it is the first step in the program that a second party performs. Unblocks: the executor/auditor discipline of step 7, for which two independent computation paths must exist.

Protocol 12.17 (Step 6: Preregistered calibration and held-out validation). Preregister, through the Gate 5 control kernel and without promoting Gate 5's own status, the first calibration and held-out validation studies: declared instruments, uncertainty budgets, environmental validity ranges, drift handling, hypotheses, falsifiers, and analysis plans fixed before data.

Sixth because this is the program's crossing from verification into validation [**23**], and everything before it exists to make the crossing honest: the grammar and profiles define what a board claims, the campaigns establish that the software does what it says, and reproduction establishes that a second party can check it. Preregistration is the instrument against the failure mode that bounded internal evidence invites—post hoc selection of favorable external results. This step is where Threat 12.6 and Threat 12.8 first become addressable at all.

Protocol 12.18 (Step 7: Agreement, falsifiers, negatives, replication). Require, for any admitted empirical result: independent executor and auditor numerical agreement; a stated falsifier for every hypothesis; publication of negative outcomes in the same records as positive ones; and independent replication before any result graduates from single-study status.

Seventh because it governs the studies that step 6 creates. The four requirements are the evidence discipline of this monograph—bounded claims, preserved negatives, checkable identities—transposed from software receipts to empirical results. Executor/auditor agreement extends the two-party structure of step 5 into the numerical domain (answering Threat 12.9); mandatory falsifiers extend the "specifications carry their own falsifiers" lesson of Chapter 11 to hypotheses; negative-outcome publication extends the preserved-conflict discipline; replication extends the reproduction discipline.

PROTOCOL 12.19 (Step 8: Internal cross-review, then commissioned external review). Complete internal cross-review of all artifacts against stable hashes—closing or explicitly waiving each of the open review criteria—then commission external formal, systems, experimental, safety, and claim-language review before any submission, disclosure, or use claim.

Last because review consumes everything: reviewing artifacts that steps 1–7 will change is review spent twice, and external reviewers' scarce attention should land on the stable, reproduced, campaign-hardened state. The five named review disciplines map onto the threat classes: formal review to construct and internal threats, systems review to internal and numerical, experimental review to external and causal, safety review to the safety threat and the Chapter 11 requirements, and claim-language review to governance—the audit of whether every public sentence carries its boundary. Only after step 8 could any status label in this book change, and this book does not predict the outcome.

REMARK 12.20 (The ordering under failure). Dependency ordering also fixes what happens when a step *fails*. A failed property campaign (step 4) returns the program to steps 1–3 with counterexamples in hand; a failed reproduction (step 5) invalidates the receipt discipline and halts the program until repaired; a falsified hypothesis at step 6–7 is a *successful* program outcome—a preserved negative at the empirical level, published under step 7's own rule. The program is ordered so that failures land early, cheaply, and on the record.

7. The claim quarantine

The following claims are quarantined: they appear in source material, project history, or plausible readings of this work, and none may be asserted, implied, or marketed on the basis of anything in this monograph. Each is held as hypothesis or prohibited inference pending the program.

(1) *Exact thought/transistor isomorphism, in any strength.* The candidate's mathematics contains projections and adapters, each explicitly non-invertible or non-semantic (Chapters 4 and 9); no isomorphism is constructed anywhere, and the analogy's utility never rested on one.

(2) *Shared four-region identity between transistor operating regions and cognitive regimes.* A four-way partition on each side is a counting coincidence until a structure-preserving correspondence with behavioral consequences is demonstrated; none is in evidence.

(3) *Overload-induced semantic involution as a validated dynamical phenomenon.* The claim is dynamical and empirical; the admitted evidence contains no dynamics beyond declared deterministic profiles and no empirical observations at all.

(4) *Fisher–Rao causal gain: any causal reading of metric quantities.* Metric quantities are descriptive geometry over declared profiles; Threat 12.10 states the missing apparatus.

(5) *Physical electrical identity for CAS quantities.* Voltage, current, resistance, charge, flux, and power as SI quantities of cognition would require a separately typed, instrumented physical profile that has never been proposed, let alone validated; the candidate's unit system deliberately types its quantities as its own.

(6) *Semantic correctness inferred from a response-region label.* A region label is a classification within a declared scheme; correctness of content is a different predicate with no admitted bridge from the first.

(7) *Safety effectiveness of any board, mechanism, or refusal path.* Blocked twice over: no safety evidence class exists in the receipt, and the source's own safety exemplar is a preserved counterexample (Chapter 11).

(8) *Certification, regulatory compliance, or clinical usability.* No certifying, regulatory, or clinical body appears anywhere in the evidence chain; the source's own disclaimer denies certification, and the candidate repeats the denial.
(9) *Complete CCDL compiler conformance.* Blocked outright by Chapter 10 until program steps 1–2 produce a grammar with a corpus-verified coverage matrix; at the bound commit conformance is not merely unproven but undefined.
(10) *Production readiness of any component of the candidate.* Twenty open review criteria, an absent parser, and unexercised state/time profiles are individually disqualifying; jointly they place the candidate's maturity where its status labels place it.
(11) *Disclosure or license authority derived from self-hashes or generated receipts.* Authority is a relation to an external authorizer; a self-computed digest can prove integrity, never authorization (Threat 12.11).
(12) *Repository-wide verification.* Blocked by the collector caveat: the claim's denominator is unenumerated at the bound commit.

The quarantine is enforceable rather than aspirational because each item is tied to a mechanism: items 1–6 are unlicensed claim records (Ring 4, NOT LICENSED); items 7–8 are blocked by the Chapter 11 findings and the absence of any safety evidence class; items 9–12 are blocked by the specific receipt facts of Section 3. A reader who finds any of these asserted anywhere under this project's name has found a defect, and is invited to treat it exactly as this book treats the CSB.

8. Related methodological work

The monograph's debts are methodological; none of the cited work validates CAS content, and the citations are confined to the methods actually borrowed.

From *simulation methodology*, the verification/validation distinction [**23**]: verification checks the model implementation against its specification, validation checks the model against its intended reality, and the two require different evidence classes and different reviewers. The entire structure of Section 4—green suites as verification, steps 6–7 as the sole route to validation—is that distinction, applied without modification.

From *provenance research*, the entity/activity/derivation factoring of W3C PROV [**22**]: every artifact in the receipt is an entity with derivations from identified activities, implemented as digest chains rather than ontology instances. The monograph claims no PROV conformance; it claims the factoring, which is what makes the receipt recomputable by outsiders.

From *graph algorithms*, the classical strongly-connected-components machinery [**20**] that the DRC's feedback analysis builds on (Chapter 7): cycle detection and condensation are the substrate on which the feedback declarations, delay requirements, and gain bounds are checked. The debt is algorithmic, and the algorithm's correctness is assumed from the literature, not re-proven here.

From *property-based testing*, the random-generation-against-invariants paradigm of QuickCheck [**24**]: the planned step-4 campaign is a direct application, and the monograph's insistence that authored fixtures under-sample fault space (Threat 12.7) is that literature's founding observation.

From *causal inference*, the requirement that causal claims rest on explicit models, identification assumptions, and interventions [**27**]: adopted here in its purely negative form—as the reason no causal claim is admitted—and reserved in its positive form for any future phase that proposes one.

A note on citation policy: this chapter cites only the five methodological anchors above, and the restraint is deliberate rather than parsimonious. The monograph's claims are about one source document, one implementation, and one receipt; a broad related-work survey—hardware description languages, statechart formalisms, information geometry, cognitive architectures, AI-safety assurance frameworks—would surround narrow claims with wide company and invite exactly the associative strengthening that the claim discipline forbids ("the work builds on X" drifts to "the work is of X's kind" drifts to "the work inherits X's validation"). Earlier and companion papers in this project's series carry the wider bibliography where it is load-bearing; here, each citation marks a method actually executed or actually scheduled, and the absence of a citation for a field marks the absence of a claim on that field.

What the monograph adds to these borrowed methods is not any single technique but a composition: source-bound adverse findings (Chapters 10–11) preserved as first-class, hash-anchored artifacts inside the same receipt that carries the green suites, under a claim-record regime in which every articulated claim is registered and none is licensed. The authors are not aware of a practice standard for cognitive-system engineering documents that requires corpus-verified grammar coverage matrices, known-fault safety fixtures, and typed evidence accounting together; this book is offered as a worked example of that composition, at Ring 4, for review.

The composition's nearest relatives, described without citation to avoid the drift just named, are: safety-case practice (structured argument with evidence nodes—but safety cases argue *for* adequacy, while this record chiefly preserves argument *against* its source); reproducibility artifacts in software research (hash-pinned environments and artifact evaluation—adopted here wholesale at the receipt layer); and registered-report publishing (preregistration with outcome-independent publication—the template for steps 6–7). The delta from each relative is the same in all three cases: the adverse findings are not appendix material but the record's center of gravity, on the thesis—argued throughout and tested by this book's own structure—that for early-stage research architectures, credibility per page is highest in the pages that count against the author.

9. Conclusion

This monograph set out to answer an engineering question: whether a circuit-shaped modeling language can make cognitive-system hypotheses more explicit, type-safe, replayable, and falsifiable without converting an analogy into a physical, causal, safety, or product claim. The answer defended across twelve chapters is a qualified yes at the software-contract level, and the qualification is the contribution.

The affirmative half is the architecture: a type system in which semantic identity is referential and versioned; a board model whose identity is its canonical digest; a design-rule checker with a closed 35-code catalog, fail-closed severities, and relational checks that catch what per-value validation misses; a narrow deterministic simulator whose every run binds its full provenance; geometry profiles that disclose their numerical limits; and an adapter that grants type-correct representation and nothing more.

The qualifying half is the evidence discipline, and it is best summarized by what this book chose to feature. Its two longest analytical chapters are devoted to faults in its own source of authority: a grammar that does not cover its own corpus, and a safety exemplar whose fuse cannot trip. Its evidence chapter leads with hash-bound counts and a collector caveat rather than a capability narrative. Its claim chapter is a quarantine. Its program defers external validation to the sixth of eight steps and external authority to the eighth. Nothing about this

composition is accidental: an architecture whose purpose is to make hypotheses falsifiable must demonstrate the falsification machinery on itself first, and the preserved findings are that demonstration.

What remains is stated in the program and will not be restated as promise. At the bound commit there are 54 artifacts, 272 green subtests, four counterexamples, eight conflicts, twenty open criteria, eleven unlicensed claims, and one collector defect; there is no parser, no live system, no external review, and no licensed claim. That is the whole of it, and the monograph ends where its discipline requires: with the boundary stated as plainly as the work.

APPENDIX A

Catalogs

This appendix collects the reference tables cited throughout the monograph. All identifiers are reproduced from the implementation's diagnostic module and the Gate 4 candidate specification; none is invented here. The governing status remains DRAFT / OPEN / RING 4 / NOT LICENSED / HELD: these catalogs describe a research candidate's software contracts, not a certified language, safety mechanism, or product.

1. The diagnostic-code catalog

Table 1 lists the deterministic `PCBA8-*` diagnostic catalog. The design-rule checker treats warnings as non-clean and never lets a warning stand in for a pass.

Table 1: The `PCBA8-*` diagnostic catalog.

Code	Class	Condition
PCBA8-MDL-001	Model	invalid board metadata
PCBA8-MDL-002	Model	duplicate reference
PCBA8-SEM-001	Semantics	unknown component kind
PCBA8-SEM-002	Semantics	unresolved semantics
PCBA8-PAR-001	Parameters	unknown parameter
PCBA8-PAR-002	Parameters	missing parameter
PCBA8-PAR-003	Parameters	invalid parameter
PCBA8-STATE-001	States	unknown state binding
PCBA8-STATE-002	States	missing state binding
PCBA8-STATE-003	States	invalid state binding
PCBA8-BIAS-001	Bias	bias-domain incoherence
PCBA8-NET-001	Nets	unknown endpoint owner
PCBA8-NET-002	Nets	unknown port
PCBA8-NET-003	Nets	invalid driver count
PCBA8-NET-004	Nets	missing net load
PCBA8-NET-005	Nets	signal-kind mismatch
PCBA8-NET-006	Nets	unit mismatch
PCBA8-NET-007	Nets	port multiply connected
PCBA8-NET-008	Nets	required port unconnected
PCBA8-NET-009	Nets	invalid signal/unit pair
PCBA8-TP-001	Test points	unknown test-point net
PCBA8-TP-002	Test points	missing safety test point
PCBA8-TP-003	Test points	safety test point not marked
PCBA8-CAL-001	Calibration	unknown calibration
PCBA8-CAL-002	Calibration	invalid calibration

Code	Class	Condition
PCBA8-CAL-003	Calibration	calibration unit mismatch
PCBA8-CAL-004	Calibration	missing test-point calibration
PCBA8-FBK-001	Feedback	undeclared feedback
PCBA8-FBK-002	Feedback	feedback without delay
PCBA8-FBK-003	Feedback	unsafe feedback gain
PCBA8-FBK-004	Feedback	invalid feedback declaration
PCBA8-SAF-001	Safety	unreachable safety threshold
PCBA8-SAF-002	Safety	unsafe fuse action
PCBA8-SAF-003	Safety	unresolved safety analysis
PCBA8-TKS-001	TKS adapter	invalid finite transition

2. Check class to code mapping

Table 2: Check classes and the codes they emit.

Check class	Codes
Structural integrity	MDL-001/002, SEM-001/002, PAR-001/002/003, STATE-001/002/003
Typed connectivity	NET-001 through NET-009
Calibration and probe integrity	TP-001/002/003, CAL-001/002/003/004, BIAS-001
Feedback analysis	FBK-001/002/003/004
Threshold reachability	SAF-001/002/003
Adapter boundary	TKS-001

3. Canonical trace schema

Table 3: Canonical trace fields.

Field group	Contents
Source identities	board and component identities; source baseline
Analysis identities	design-rule report identity; simulator profile
Context identities	component catalog identity; decimal-context identity
Signal record	typed input samples; typed outputs
Event record	component events; tick linkage
Observation record	test-point observations
Measurement lineage	measurement IDs; parent/child derivations; calibration references
Boundary	fixed Ring-4 research boundary

4. Adapter certificate fields

Table 4: Adapter certificate fields.

Field	Role
TKS kernel bytes / profile	exact identity of the finite kernel used
Board / component / transition identities	the PCBA object adapted
Tagged carrier	the exact-type tagged finite carrier
Semantic profile	the declared semantic profile
Fixed boundary	the Ring-4 research boundary

APPENDIX B

A Worked Illustrative Board

CLAIM BOUNDARY B.1. The board fragments in this appendix are *entirely hypothetical* and are constructed only to exercise the design-rule checker of Chapter 7. They are not drawn from the source corpus, they name no real cognitive mechanism, and they carry no safety, causal, or product claim. Every diagnostic below is a software outcome on an invented input. The governing status remains DRAFT / OPEN / RING 4 / NOT LICENSED / HELD.

Throughout, we write boards in an informal bracketed notation for readability; the actual candidate consumes typed Python objects, not this surface syntax. The notation is a presentation device for this appendix only and is not a claim about the CCDL grammar audited in Chapter 10.

1. A clean pass

Consider an illustrative two-stage board with one input port, one internal node, and one output port, all typed with a single compatible signal kind and unit. Each net has exactly one driver; the single test point carries a versioned calibration reference; there is no declared feedback loop; and no safety fuse is present, so no reachability obligation arises.

EXAMPLE B.2 (Clean board). *Illustrative.* Board `DEMO-CLEAN`: input `in` (signal `activation`, unit `alpha`) drives node `n1` through component `stage_a`; `n1` drives output `out` through `stage_b`; test point `tp1` on `n1` references calibration `cal_v1`. The design-rule checker resolves every endpoint, finds exactly one driver per net, matches signal kinds and units across each net, resolves the calibration reference, finds no undeclared cycle, and emits no diagnostic. The board passes; a subsequent deterministic simulation emits a canonical trace with the fields of Appendix A.

2. Representative failures

Each of the following illustrative edits to `DEMO-CLEAN` triggers one representative diagnostic. In every case the board *fails*: an error is not downgraded to a warning, and a warning is never treated as a clean result.

EXAMPLE B.3 (Two drivers on one net: `PCBA8-NET-003`). *Illustrative.* Add a second component `stage_c` whose output also drives `n1`. The net `n1` now has two drivers. The typed-connectivity check requires exactly one typed driver per net and emits `PCBA8-NET-003` (invalid driver count). The board fails.

EXAMPLE B.4 (Signal/unit mismatch: `PCBA8-NET-005` / `-006`). *Illustrative.* Retype `stage_b`'s input port to signal kind `constraint` with unit `fraction` while `n1` carries `activation` in `alpha`. The driver and load disagree under the closed admissible signal-kind/unit table, so the checker emits `PCBA8-NET-005` (signal-kind mismatch) and, for the unit disagreement, `PCBA8-NET-006` (unit mismatch). The board fails.

EXAMPLE B.5 (Unmarked safety probe: `PCBA8-TP-003`). *Illustrative.* Declare a safety-critical path through `n1` but leave `tp1` without the safety-probe marking. The calibration-and-probe check requires safety-marked paths to carry marked probes and emits `PCBA8-TP-003` (safety test point not marked). Note the boundary from Chapter 7: even a *present, marked* probe would not establish safety; the diagnostic only enforces the structural precondition.

EXAMPLE B.6 (Undeclared cycle: `PCBA8-FBK-001`). *Illustrative.* Add a component whose output feeds back into `in`'s upstream node without a feedback declaration. The feedback analysis builds the directed component graph, finds a strongly connected component, and—absent an exact feedback declaration with a unit delay, a dimensionless calibration, and a sub-unity gain bound—emits `PCBA8-FBK-001` (undeclared feedback). Had a declaration been present but without a delay, the diagnostic would instead be `PCBA8-FBK-002`.

EXAMPLE B.7 (Unreachable fuse: `PCBA8-SAF-001`). *Illustrative, echoing the source known-fault board of Chapter 11.* Insert a red stage clamped at output 0.1 upstream of a safety fuse with threshold 0.9, with only monotone non-amplifying stages between them. The threshold-reachability check propagates the conservative upper bound 0.1 along the path; since $0.1 < 0.9$, the fuse can never trip, and the checker emits `PCBA8-SAF-001` (unreachable safety threshold). This illustrative board reproduces, on invented parameters, the structural fault that Chapter 11 analyzes in the published Cognitive Safety Board.

3. A simulation narrative

EXAMPLE B.8 (Deterministic trace). *Illustrative.* Running `DEMO-CLEAN` of Example B.2 under the reference simulator: the non-delay graph is topologically ordered `in`, `stage_a`, `n1`, `stage_b`, `out`; a typed input sample enters `in`; `stage_a` fires, emitting a component event and a tick; `n1` carries the intermediate value, observed at `tp1` against `cal_v1`; `stage_b` fires; `out` produces a typed output. The result binds the board and component identities, the design-rule report identity, the catalog and decimal-context identities, the simulator profile and source baseline, the typed input and output, the two component events with tick linkage, the one test-point observation, the measurement IDs with their derivations, and the fixed Ring-4 boundary. The correct reading is only that the software performed the declared transitions for this exact fixture under this exact profile—never that any cognitive mechanism, causal relation, or safety property was established.

APPENDIX C

Glossary

This glossary defines the technical terms and identifiers used in the monograph. Chapter references are given as plain text. Every definition is scoped to this Ring-4 research candidate; no term carries a safety, causal, physical, or product claim beyond what the cited chapter states.

Adapter certificate: The immutable record binding the exact TKS v8 kernel bytes, board/component/transition identities, tagged carrier, semantic profile, and research boundary when a certified transition is adapted to a finite arrow. Attests type-correct representation only (Chapter 9).

Board: The typed, immutable, identity-bearing record of components, ports, nets, test points, calibrations, feedback declarations, and provenance metadata that constitutes a modeled circuit. Board identity includes the complete canonical structure (Chapter 5).

CCDL: The Cognitive Circuit Description Language of the 743-page source baseline. Chapter 10 audits the mismatch between its consolidated EBNF and its own five-board reference corpus.

Check class: One of the five organizing views of the design-rule checker: structural integrity, typed connectivity, calibration and probe integrity, feedback analysis, and threshold reachability (Chapter 7).

CJT: A component form appearing in the source boards (Chapter 10). Its gain parameter is defined in the source by Fisher–Rao displacement ratios; Chapter 6 and the companion CAS-Ohm treatment keep that definition distinct from a local Jacobian gain.

Cognitive Safety Board (CSB): The published safety board whose red-stage clamp of 0.1 sits upstream of a fuse threshold of 0.9, rendering the protective crossing unreachable under the declared clamp semantics (Chapter 11).

Decimal context: The single immutable, versioned arithmetic context, created from primitive constants, under which all decimal operations run; its identity enters reports and results (Chapter 4).

Design-rule checker (DRC): The deterministic checker that computes one diagnostic report from the `PCBA8-*` catalog; errors fail the board and warnings never become an implicit clean result (Chapter 7).

Fisher–Rao metric: The information-geometric metric on the open probability simplex used by the mixed-simplex geometry profile; its binary64 implementation is a disclosed numerical pseudometric at its resolution (Chapters 2 and 6).

NDCell: A Noetic-times-Domain pair; the target of the many-to-one projection from a state literal that forgets Drive and Register (Chapter 4).

Known-fault fixture: A source artifact retained unmodified as negative evidence and a regression guard, rather than silently repaired; the CSB is the principal example (Chapter 11).

Ring 4: The admission ring denoting a research candidate that is OPEN, NOT LICENSED, and not publicly disclosed. All artifacts in this monograph are Ring 4 (throughout).

Semigroupoid path: A typed sequence of finite arrows with matching adjacent objects and a typed composite but no power operation; the form to which component chains are adapted (Chapter 9).

SimplexAddress40: A strictly positive normalized profile over the forty canonical ND-cells; neither a one-hot cell nor an element of the finite TKS kernel (Chapter 4).

State literal: A Noetic-times-Domain-times-optional-Drive-times-optional-Register tuple; the richest state type, projecting many-to-one onto an NDCell (Chapter 4).

Tagged exact value: A value carrying its exact type identity, so that integer 1, Boolean *true*, and string `"true"` remain distinct throughout checking, execution, traces, and adapters (Chapter 4).

Threshold reachability: The check that propagates conservative upper bounds along an activation path and flags a safety fuse whose threshold exceeds the resolved upstream bound (`PCBA8-SAF-001`) (Chapter 7).

TKS v8 finite arrow: The certified total, finite, deterministic, stateless, non-stochastic transition admitted to the companion TKS kernel; the only component transitions eligible for adaptation (Chapter 9).

Trace: The canonical, provenance-rich record emitted by an admitted deterministic simulation, binding source, analysis, context, signal, event, observation, lineage, and boundary fields (Chapter 8; fields in Appendix A).

Bibliography

[1] S. Mac Lane and I. Moerdijk, *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*, Springer, 1994.

[2] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998.

[3] The Stacks Project Authors, "Sites and sheaves: the topology defined by a site," Tag 00ZC, https://stacks.math.columbia.edu/tag/00ZC.

[4] The Stacks Project Authors, "Sheafification," Tag 00W1, https://stacks.math.columbia.edu/tag/00W1.

[5] The Stacks Project Authors, "Representable sheaves," Tag 00WO, https://stacks.math.columbia.edu/tag/00WO.

[6] P. T. Johnstone, *Sketches of an Elephant: A Topos Theory Compendium*, Oxford University Press, 2002.

[7] S. Awodey, *Category Theory*, 2nd ed., Oxford University Press, 2010.

[8] E. Riehl, *Category Theory in Context*, Dover, 2016.

[9] N. N. Vorob'ev, "Consistent families of measures and their extensions," *Theory Probab. Appl.* 7 (1962), 147–163.

[10] S. Abramsky and A. Brandenburger, "The sheaf-theoretic structure of non-locality and contextuality," *New J. Phys.* 13 (2011), 113036.

[11] J. Hansen and R. Ghrist, "Toward a spectral theory of cellular sheaves," *J. Appl. Comput. Topol.* 3 (2019), 315–358.

[12] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed., Cambridge University Press, 2013.

[13] J.-É. Pin, "Finite semigroups and recognizable languages: an introduction," in *Semigroups, Formal Languages and Groups*, Kluwer, 1995.

[14] J. M. Howie, *Fundamentals of Semigroup Theory*, Oxford University Press, 1995.

[15] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, rev. ed., MIT Press, 2012.

[16] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," *CADE 28*, 2021.

[17] R. Hammack, *Book of Proof*, 3rd ed., 2018.

[18] IEEE Std 1076-2019, *VHDL Language Reference Manual*.

[19] D. Harel, "Statecharts: a visual formalism for complex systems," *Sci. Comput. Program.* 8 (1987), 231–274.

[20] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.* 1 (1972), 146–160.

[21] IEEE Std 754-2019, *Floating-Point Arithmetic*.

[22] W3C, *PROV-O: The PROV Ontology*, 2013.

[23] R. G. Sargent, "Verification and validation of simulation models," *J. Simulation* 7 (2013), 12–24.

[24] K. Claessen and J. Hughes, "QuickCheck: a lightweight tool for random testing of Haskell programs," *ICFP* (2000).

[25] S.-i. Amari, *Information Geometry and Its Applications*, Springer, 2016.

[26] N. Ay, J. Jost, H. V. Lê, and L. Schwachhöfer, *Information Geometry*, Springer, 2017.

[27] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed., Cambridge University Press, 2009.

[28] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, 2020.

[29] C. Lamb and S. Zacchiroli, "Reproducible builds: increasing the integrity of software supply chains," *IEEE Software* 39(2), 2022.

[30] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed., Wiley, 2020.

[31] A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014.

[32] JCGM 200:2012, *International Vocabulary of Metrology—Basic and General Concepts and Associated Terms*, 3rd ed.

[33] JCGM 100:2008, *Evaluation of Measurement Data—Guide to the Expression of Uncertainty in Measurement*.

[34] E. Buckingham, "On physically similar systems; illustrations of the use of dimensional equations," *Phys. Rev.* 4 (1914), 345–376.

[35] L. O. Chua, "Memristor—the missing circuit element," *IEEE Trans. Circuit Theory* 18 (1971), 507–519.

[36] L. O. Chua and S. M. Kang, "Memristive devices and systems," *Proc. IEEE* 64 (1976), 209–223.

[37] M. C. Kennedy and A. O'Hagan, "Bayesian calibration of computer models," *J. R. Stat. Soc. Ser. B* 63 (2001), 425–464.

[38] C. Ritz, F. Baty, J. C. Streibig, and D. Gerhard, "Dose-response analysis using R," *PLoS ONE* 10(12), 2015.

[39] A. Zou et al., "Representation engineering: a top-down approach to AI transparency," arXiv:2310.01405, 2023.

[40] N. Elhage et al., "Toy models of superposition," Anthropic, 2022.

[41] A. Vaswani et al., "Attention is all you need," *Adv. Neural Inf. Process. Syst. 30 (NIPS)*, 2017.

[42] R. A. Fisher, "On the mathematical foundations of theoretical statistics," *Philos. Trans. R. Soc. A* 222 (1922), 309–368.

[43] C. R. Rao, "Information and the accuracy attainable in the estimation of statistical parameters," *Bull. Calcutta Math. Soc.* 37 (1945), 81–91.

[44] K. Claessen, "Shrinking and showing functions," *Haskell Symposium*, 2012.

[45] B. Beizer, *Software Testing Techniques*, 2nd ed., Van Nostrand Reinhold, 1990.

[46] D. E. Knuth, *Literate Programming*, CSLI, 1992.